

Ultimate notatka

Wykład 1: Dokładność (do lab1 i lab2)

1. Czym są metody numeryczne?

Metody numeryczne są działem matematyki stosowanej. Zajmują się sposobami rozwiązywania zadań matematycznych za pomocą działań arytmetycznych.

W informatyce i naukach technicznych często pojawiają się problemy typu:

$$f(x) = 0, \quad Ax = b, \quad \int_a^b f(x) dx, \quad y' = f(x, y)$$

W praktyce rozwiązanie analityczne często:

- nie istnieje,
- jest zbyt kosztowne obliczeniowo,
- opiera się na danych obarczonych błędami,
- jest liczone na komputerze, który ma skończoną precyzję.

Celem metod numerycznych jest:

- obliczenie przybliżenia rozwiązania,
- oszacowanie błędu,
- kontrolowanie stabilności obliczeń.

2. Najważniejsze pojęcia

Dokładność

Dokładność oznacza dopuszczalny błąd wyniku. W praktyce dobiera się ją zależnie od problemu.

Przykład interpretacji:

Jeżeli wynik ma być podany z dokładnością do **0.01**, to interesuje nas błąd nie większy niż jedna setna.

Zbieżność iteracji

Zbieżność iteracji oznacza stopniowe zbliżanie się do wartości poszukiwanej.

Iteracje kończy się wtedy, gdy osiągnięta zostanie zadana dokładność.

Przykład:

Jeżeli kolejne przybliżenia rozwiązania wyglądają tak:

```
1.4  
1.41  
1.414  
1.4142
```

to widać, że wartości zbliżają się do pewnego wyniku. Można zakończyć obliczenia, gdy różnica między kolejnymi przybliżeniami jest wystarczająco mała.

Dyskretyzacja

Dyskretyzacja to zastąpienie obiektu ciągłego obiektem o skończonej liczbie węzłów.

Przykład:

Zamiast analizować funkcję na całym przedziale, wybieramy kilka punktów, np.:

```
x = 0, 1, 2, 3, 4
```

i liczymy wartości funkcji tylko w tych punktach.

Model

Model to celowo uproszczona reprezentacja rzeczywistości.

Model matematyczny

Model matematyczny to reprezentacja fragmentu rzeczywistości za pomocą symboli i operatorów matematycznych, z interpretacją odnoszącą się do danego zjawiska.

Schemat z wykładu:

Model fizyczny(doświadczenie) | teoria



Model matematyczny



Metody analityczne | Model numeryczny(metody numeryczne)

3. Plan wykładu nr 1 — dokładność

Wykład dotyczy następujących tematów:

1. miary błędu: błąd bezwzględny, błąd względny, cyfry znaczące,
2. liczby maszynowe: `float`, `double`, IEEE 754, epsilon,
3. porównywanie liczb, `Inf`, `NaN`, utrata cyfr znaczących,
4. wpływ kolejności działań: gubienie składnika, sumowanie, algorytm Kahana,
5. uwarunkowanie i stabilność algorytmów, np. na przykładzie `a2 - b2`.

4. Błąd bezwzględny

Niech:

- `x` — wartość dokładna,
- `̄x` — przybliżenie wartości dokładnej.

Błąd bezwzględny:

$$\Delta x = |x - \bar{x}|$$

Błąd bezwzględny mówi, o ile przybliżenie różni się od wartości dokładnej.

Jeżeli `x` nie jest znane, zamiast dokładnego błędu oblicza się jego oszacowanie, czyli kres górny.

Ważne

Błąd bezwzględny zależy od skali liczby.

Przykład:

Jeżeli błąd wynosi:

$$\Delta x = 0.01$$

to dla liczby bliskiej **1** może być to istotny błąd, ale dla liczby bliskiej **1000** jest to bardzo mały błąd.

Przykład

PYTHON

```
def blad_bezwzglezny(x, x_przyblizone):  
    return abs(x - x_przyblizone)  
  
# przykład  
x = 10          # wartość dokładna  
x_bar = 9.8     # wartość przybliżona  
  
delta_x = blad_bezwzglezny(x, x_bar)  
  
print("Błąd bezwzględny:", delta_x)
```

5. Błąd względny

Dla $x \neq 0$ błąd względny definiuje się jako:

$$\delta x = \frac{|x - \bar{x}|}{|x|}$$

Błąd względny mierzy dokładność niezależnie od rzędu wielkości liczby.

Przykład z wykładu

Jeżeli:

$$\Delta x = 0.01$$

to:

dla $x = 1$:

$$\delta x = 1$$

dla $x = 1000$:

$$\delta x = 0.001$$

Wniosek

W analizie numerycznej kluczowy jest **błąd względny**, ponieważ lepiej pokazuje, czy wynik jest dokładny względem skali problemu.

Przykład

PYTHON

```
def blad_wzgledny(x, x_przyblizone):  
    if x == 0:  
        raise ValueError("Błąd względny nie jest określony dla x = 0")  
  
    return abs(x - x_przyblizone) / abs(x)  
  
# przykład  
x = 1000  
x_bar = 999.99  
  
delta_x = blad_wzgledny(x, x_bar)  
  
print("Błąd względny:", delta_x)  
print("Błąd względny w procentach:", delta_x * 100, "%")
```

6. Źródła błędów w obliczeniach numerycznych

W obliczeniach numerycznych mogą pojawiać się różne błędy:

6.1. Błędy zaokrągleń

Wynikają z arytmetyki komputera, ponieważ komputer nie potrafi dokładnie zapisać wszystkich liczb rzeczywistych.

Przykład:

Liczba **0.1** nie ma skończonej reprezentacji binarnej, więc komputer przechowuje tylko jej przybliżenie.

6.2. Błędy obcięcia

Pojawiają się np. wtedy, gdy zatrzymujemy proces nieskończony.

Przykład:

Zamiast liczyć nieskończony szereg do końca, obcinamy go po określonej liczbie wyrazów.

6.3. Błędy programisty

Wynikają z niepoprawnego programu, np. złej kolejności działań, błędnego wzoru lub niewłaściwych warunków zakończenia pętli.

6.4. Błędy danych wejściowych

Dane wejściowe mogą pochodzić z pomiarów, zaokrągleń lub stałych fizycznych, które same są tylko przybliżone.

6.5. Błędy modelu

Wynikają z uproszczeń modelu matematycznego.

Model nie zawsze idealnie opisuje rzeczywistość.

6.6. Błędy metody

Niektóre metody mogą dawać duże błędy dla pewnych danych.

7. Błędy zaokrągleń

Komputer potrafi dokładnie reprezentować tylko:

- liczby całkowite z pewnego zakresu,
- liczby wymierne o skończonym rozwinięciu binarnym, również z pewnego zakresu.

Wiele ułamków dziesiętnych nie ma skończonego rozwinięcia binarnego.

Przykłady z wykładu:

$$\frac{1}{5} = 0.(0011)_2, \quad \frac{1}{10} = 0.0(0011)_2$$

Oznacza to, że liczby takie jak 0.1 nie są przechowywane dokładnie, tylko jako przybliżenia.

8. Cyfry znaczące

Niech $x \neq 0$ ma postać:

$$x = \pm 0.d_1d_2d_3 \dots \times 10^k$$

gdzie:

$$d_1 \neq 0$$

Przybliżenie $\bar{x}$ ma p cyfr znaczących, jeśli:

$$\frac{|x - \bar{x}|}{|x|} \leq \frac{1}{2} \cdot 10^{-p}$$

Liczba cyfr znaczących jest miarą błędu względnego.

Przykład

PYTHON

```
import math

def blad_wzgledny(x, x_bar):
    if x == 0:
        raise ValueError("x nie może być równe 0")
    return abs(x - x_bar) / abs(x)

def liczba_cyfr_znaczaczych(x, x_bar):
    delta = blad_wzgledny(x, x_bar)

    if delta == 0:
        return "nieskończenie wiele, bo wynik jest dokładny"

    p = -math.log10(delta)
    return int(p)

x = 123.456789
x_bar = 123.457

delta = blad_wzgledny(x, x_bar)
p = liczba_cyfr_znaczaczych(x, x_bar)

print("Błąd względny:", delta)
print("Około tyle cyfr znaczących:", p)
```

Idea:

mały błąd względny = dużo poprawnych cyfr znaczących
duży błąd względny = mało poprawnych cyfr znaczących

Interpretacja cyfr znaczących

Jeżeli:

$$\delta x \approx 10^{-p}$$

to wynik ma około **p** poprawnych cyfr znaczących.

Przykład z wykładu:

$$\delta x \approx 10^{-6}$$

oznacza około **6** poprawnych cyfr znaczących.

W obliczeniach maszynowych liczba cyfr znaczących jest ograniczona długością mantysy.

9. Typy całkowite

Dowolną liczbę całkowitą **x** można przedstawić w systemie dwójkowym:

$$x = s \sum_{i=0}^n e_i 2^i$$

gdzie:

$$s \in \{-1, 1\}$$

oraz:

$$e_i \in \{0, 1\}$$

Obliczenia na liczbach całkowitych są zwykle dokładne do momentu przepełnienia.

Wzór

liczba = znak · suma wybranych potęg liczby 2

Przykład: liczba 13

W systemie dziesiętnym:

13

Można ją zapisać jako sumę potęg dwójki:

$$13 = 8 + 4 + 1$$

czyli:

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

bo:

$$2^3 = 8$$

$$2^2 = 4$$

$$2^1 = 2$$

$$2^0 = 1$$

Zatem binarnie:

$$13 = 1101_2$$

Czytamy to tak:

$$1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13$$

Przykład: liczba -13

Tutaj wartość jest taka sama, tylko znak jest ujemny:

$$-13 = -1 \cdot (8 + 4 + 1)$$

czyli:

$$-13 = -1 \cdot (1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)$$

Na slajdzie to właśnie oznacza $s \in \{-1, 1\}$.

Jak to wygląda w Pythonie?

```
x = 13
```

```
print(bin(x))
```

PYTHON

Wynik:

```
0b1101
```

0b oznacza, że liczba jest zapisana binarnie.

Możesz też sprawdzić kilka liczb:

```
liczby = [1, 2, 3, 4, 5, 13, 25]

for x in liczby:
    print(x, "=", bin(x))
```

PYTHON

Wynik będzie np.:

```
1 = 0b1
2 = 0b10
3 = 0b11
4 = 0b100
5 = 0b101
13 = 0b1101
25 = 0b11001
```

Zakres typów całkowitych

Zakres z wykładu:

$$[-2^d, 2^d - 1]$$

Wyjątki od dokładności:

- dzielenie całkowitoliczbowe,
- przepełnienie zakresu.

Przykład dzielenia całkowitoliczbowego

Jeżeli w języku programowania wykonamy dzielenie całkowite:

```
5 / 2
```

to wynik może być:

```
2
```

a nie:

2.5

czyli tracona jest część ułamkowa.

10. Typy zmiennopozycyjne

Liczba zmiennopozycyjna ma postać:

$$x = m \cdot p^c$$

gdzie:

- **m** — mantysa,
- **c** — wykładnik,
- **p** — podstawa, zwykle **2**.

W komputerze liczba rzeczywista jest kodowana przez mantysę i wykładnik.

Reprezentacja liczby zmiennopozycyjnej

Zamiast nieskończonego rozwinięcia mantysy:

$$m = \sum_{i=1}^{\infty} e_{-i} 2^{-i}$$

```
def mantysa(e):  
    """  
    Liczy:  
    m = suma e_i * 2^(-i)  
  
    e - lista bitów mantysy, np. [1, 0, 1, 1]  
    """  
    suma = 0  
  
    for i in range(1, len(e) + 1):  
        suma += e[i - 1] * 2 ** (-i)  
  
    return suma
```

PYTHON

używa się przybliżenia obciętego do **t** bitów:

$$m_t = \sum_{i=1}^t e_{-i} 2^{-i}$$

PYTHON

```
def mantysa_obcieta(e, t):
    """
    Liczy:
    m_t = suma od i=1 do t: e_i * 2^(-i)

    e - lista bitów mantysy
    t - liczba bitów, do których obcinamy
    """
    suma = 0

    for i in range(1, t + 1):
        suma += e[i - 1] * 2 ** (-i)

    return suma
```

z ograniczeniem:

$$|m - m_t| \leq \frac{1}{2} 2^{-t}$$

PYTHON

```
def ograniczenie_bledu(t):
    """
    Liczy:
    1/2 * 2^(-t)
    """
    return 0.5 * 2 ** (-t)
```

Reprezentacja liczby:

$$x = (-1)^s \cdot 2^c \cdot m_t$$

Przykład użycia tych wyżej

PYTHON

```
e = [1, 0, 1, 1, 0, 1] # bity mantysy
t = 4

m = mantysa(e)
m_t = mantysa_obcieta(e, t)
blad_graniczny = ograniczenie_bledu(t)

print("m =", m)
print("m_t =", m_t)
print("ograniczenie błędu =", blad_graniczny)
print("|m - m_t| =", abs(m - m_t))
```

11. float i double

Z wykładu:

Typ	digits10	max_digits10	epsilon	zakres
float	6	9	około $1.19 \cdot 10^{-7}$	około $10^{-38} \dots 10^{38}$
double	15	17	około $2.22 \cdot 10^{-16}$	około $10^{-308} \dots 10^{308}$

Ważna uwaga

`setprecision(20)` dla typu `float` nie zwiększa dokładności.

To znaczy, że wypisanie większej liczby cyfr nie sprawia, że wynik staje się dokładniejszy. Program wypisuje tylko więcej cyfr przechowywanego przybliżenia.

Do wiernego wypisywania używa się zwykle `max_digits10`.

Notatka — Python

W języku Python typ `float` jest domyślnie liczbą zmiennoprzecinkową podwójnej precyzji, czyli odpowiada typowi `double` z języka C/C++.

Python nie ma osobnego podstawowego typu `float` i `double` tak jak C++. Zwykły `float` w Pythonie przechowuje około **15–17 cyfr znaczących**.

```
import sys

print(sys.float_info.dig)      # liczba cyfr znaczących
print(sys.float_info.max)     # największa wartość
print(sys.float_info.epsilon) # epsilon maszynowy
```

PYTHON

Przykład:

```
x = 0.1 + 0.2

print(x)
print(f"{x:.20f}")
```

PYTHON

Wynik:

```
0.30000000000000004
0.30000000000000004441
```

Oznacza to, że wypisanie większej liczby cyfr nie zwiększa dokładności wyniku. Program pokazuje jedynie więcej cyfr już zapisanego przybliżenia.

W Pythonie:

```
print(f"{x:.20f}")
```

PYTHON

działa podobnie jak `setprecision(20)` w C++ — zmienia tylko sposób wyświetlania liczby, ale nie zwiększa dokładności obliczeń.

Do większej dokładności można użyć modułu `decimal`:

```
from decimal import Decimal

a = Decimal("0.1")
b = Decimal("0.2")

print(a + b)
```

PYTHON

Wynik:

0.3

Czyli najważniejsze:

Python float $\approx$ C++ double
około 15-17 cyfr znaczących
większa liczba wypisanych cyfr nie oznacza większej dokładności

Można użyć typu 32-bit jako:

```
import numpy as np

def suma_pojedyncza_precyzja(a, b):
    a32 = np.float32(a)
    b32 = np.float32(b)
    return np.float32(a32 + b32)
```

PYTHON

12. Standard IEEE 754

Według slajdu z wykładu:

Typ	Rozmiar	Mantysa	Wykładnik	Przesunięcie wykładnika
Single Precision	32 bity = 4 bajty	23 bity	8 bitów	$2^7 - 1 = 127$
Double Precision	64 bity = 8 bajtów	52 bity	11 bitów	$2^{10} - 1 = 1023$

13. Reprezentacja zmiennopozycyjna i błąd względny

Niech $rd(x)$ oznacza maszynową reprezentację liczby x , czyli zaokrąglenie do najbliższej liczby maszynowej.

Dla $x \neq 0$ zachodzi model:

$$rd(x) = x(1 + \epsilon)$$

gdzie:

$$|\epsilon| \leq u$$

u to jednostka zaokrąglenia, czyli **unit roundoff**.

W arytmetyce binarnej:

$$u = \frac{1}{2} \cdot 2^{-p}$$

gdzie p jest liczbą bitów mantysy.

Wniosek

- mantysa decyduje o dokładności,
- wykładnik decyduje o zakresie.

14. Maszynowy epsilon

Maszynowy epsilon:

$$\epsilon_{mach} = \min\{\epsilon > 0 : 1 + \epsilon > 1\}$$

W praktyce dla IEEE 754 przy zaokrągleniu do najbliższej:

$$\epsilon_{mach} \approx \frac{1}{2} \cdot 2^{-p} = u$$

Typowo:

```
float  ≈ 10-7  
double ≈ 10-16
```

Przykład w Pythonie możesz dać taki:

```
epsilon = 1.0  
  
while 1.0 + epsilon > 1.0:  
    epsilon = epsilon / 2  
  
epsilon = epsilon * 2  
  
print("Maszynowy epsilon:", epsilon)  
print("1 + epsilon =", 1.0 + epsilon)  
print("1 + epsilon/2 =", 1.0 + epsilon / 2)
```

PYTHON

Wynik będzie mniej więcej:

```
Maszynowy epsilon: 2.220446049250313e-16  
1 + epsilon = 1.0000000000000002  
1 + epsilon/2 = 1.0
```

Czyli interpretacja jest taka:

epsilon to najmniejsza liczba, którą można dodać do 1,
żeby komputer jeszcze zauważył zmianę.

Dla Pythona typ `float` działa jak `double`, więc epsilon jest około:

```
2.22 · 10-16
```

Możesz też użyć gotowej wartości z Pythona:

```
import sys  
  
print(sys.float_info.epsilon)
```

PYTHON

Wynik:

```
2.220446049250313e-16
```


Wniosek

Maszynowy epsilon określa najmniejszą różnicę między liczbą 1 a najbliższą większą liczbą możliwą do zapisania w danym typie zmiennoprzecinkowym. W Pythonie dla typu `float` wynosi około $2.22 \cdot 10^{-16}$. Oznacza to, że `1 + epsilon` jest już rozróżnialne od 1, ale `1 + epsilon/2` zostaje zaokrąglone z powrotem do 1.

15. Model arytmetyki zmiennopozycyjnej

Zakładany model:

$$fl(x \circ y) = (x \circ y)(1 + \epsilon)$$

gdzie:

$$|\epsilon| \leq \epsilon_{mach}$$

oraz:

$$\circ \in \{+, -, \cdot, /\}$$

Każda operacja wprowadza błąd względny rzędu:

$$O(\epsilon_{mach})$$

Błędy mogą kumulować się w dłuższych obliczeniach.

Przykład

Dobry przykład do tego slajdu: **wielokrotne dodawanie małej liczby.**

```
suma = 0.0

for i in range(10):
    suma += 0.1

print(suma)
print(suma == 1.0)
```

PYTHON

Wynik:

```
0.9999999999999999
False
```

Matematycznie powinno być:

$$0.1 + 0.1 + \dots + 0.1 = 1.0$$

ale komputer dostaje trochę inny wynik, bo `0.1` nie jest dokładnie reprezentowane binarnie. Każde dodanie wprowadza mały błąd zaokrąglenia.

Czyli ten slajd mówi mniej więcej:

$$\text{wynik w komputerze} = \text{wynik dokładny} \cdot (1 + \text{mały błąd})$$

W kodzie można to zapisać jako notatkę:

```
# Każda operacja zmiennoprzecinkowa może wprowadzać mały błąd.  
# Przy wielu operacjach te błędy mogą się kumulować.  
  
suma = 0.0  
  
for i in range(1000000):  
    suma += 0.1  
  
dokladny_wynik = 100000.0  
  
print("Wynik obliczony:", suma)  
print("Wynik dokładny:", dokladny_wynik)  
print("Błąd:", abs(suma - dokladny_wynik))
```

PYTHON

Przykładowy wynik:

```
Wynik obliczony: 100000.00000133288  
Wynik dokładny: 100000.0  
Błąd: 0.0000013328826753422618
```

Najprostszy opis pod przykład:

W arytmetyce zmiennoprzecinkowej każda operacja może wprowadzić niewielki błąd zaokrąglenia. Przy dłuższych obliczeniach błędy te mogą się sumować, dlatego wynik wielu operacji na liczbach `float` może nie być dokładnie równy wynikowi matematycznemu.

16. Porównywanie liczb zmiennopozycyjnych

Porównywanie przez `==` jest zwykle błędem dla `float` i `double`.

Zamiast tego stosuje się test mieszany, uwzględniający tolerancję absolutną i względną:

$$|a - b| \leq \epsilon \cdot \max(1, |a|, |b|)$$

Przykład z wykładu w C++:

```
double eps = 1e-12;

bool equal = std::fabs(a-b) <= eps * std::max(1.0,
        std::max(std::fabs(a), std::fabs(b)));
```

C++

Interpretacja

- dla wartości bliskich zera dominuje część absolutna,
- dla dużych wartości dominuje część względna.

Przykład

Przykład w Pythonie do tych slajdów:

```
def prawie_rowne(a, b, eps=1e-12):
    return abs(a - b) <= eps * max(1, abs(a), abs(b))

a = 0.1 + 0.2
b = 0.3

print("a =", a)
print("b =", b)

print("Porównanie przez ==:", a == b)
print("Porównanie z tolerancją:", prawie_rowne(a, b))
```

PYTHON

Wynik:

```
a = 0.30000000000000004
b = 0.3
Porównanie przez ==: False
Porównanie z tolerancją: True
```

Czyli zamiast pisać:

```
if a == b:
    print("równe")
```

PYTHON

dla liczb `float` lepiej pisać:

PYTHON

```
if prawie_rowne(a, b):
    print("prawie równe")
```

Całość działa według wzoru ze slajdu:

PYTHON

```
abs(a - b) <= eps * max(1, abs(a), abs(b))
```

Przykład z większymi liczbami:

PYTHON

```
a = 1000000.0000001
b = 1000000.0000002

print(a == b)
print(prawie_rowne(a, b))
```

Tutaj ważna jest tolerancja względna, bo dla dużych liczb mała różnica bezwzględna może być nieistotna.

Dobry opis do notatki:

Liczb zmiennoprzecinkowych nie powinno się porównywać bezpośrednio przez `==`, ponieważ mogą zawierać małe błędy zaokrągleń. Zamiast tego sprawdza się, czy różnica między liczbami jest mniejsza od ustalonej tolerancji.

17. Dlaczego `0.1 + 0.2 != 0.3`?

Liczba `0.1` nie ma skończonej reprezentacji binarnej.

Dlatego wynik może wyglądać tak:

```
0.1 + 0.2 = 0.30000000000000004
```

Nie powinno się więc pisać:

C++

```
if (a == b)
```

lecz stosować porównywanie z tolerancją.

18. Gubienie małego składnika

Przykład z wykładu:

$$(10^{20} + 10^{-10}) - 10^{20}$$

W arytmetyce rzeczywistej wynik to:

$$10^{-10}$$

Ale w arytmetyce zmiennoprzecinkowej często:

$$fl(10^{20} + 10^{-10}) = 10^{20}$$

Dzieje się tak, ponieważ 10^{-10} jest mniejsze niż odstęp między liczbami maszynowymi w okolicy 10^{20} .

Wniosek

Skala liczb i kolejność działań wpływają na wynik.

Przykład

Tutaj najlepszy przykład w Pythonie:

```
a = 10**20
b = 10**-10

wynik = (a + b) - a

print(wynik)
```

PYTHON

Wynik:

0.0

A matematycznie powinno być:

0.0000000001

czyli:

10^{-10}

Dlaczego tak się dzieje?

```
a = 10**20
b = 10**-10

print(a + b)
print(a)
print(a + b == a)
```

Wynik:

```
1e+20
100000000000000000000
True
```

Czyli Python „zgubił” bardzo małą liczbę 10^{-10} , bo przy liczbie 10^{20} jest ona za mała, żeby zmienić zapis liczby typu `float`.

Możesz też pokazać, że **kolejność działań ma znaczenie**:

```
a = 10**20
b = 10**-10

wynik1 = (a + b) - a
wynik2 = (a - a) + b

print(wynik1)
print(wynik2)
```

Wynik:

```
0.0
1e-10
```

Opis do notatki:

W arytmetyce zmiennoprzecinkowej bardzo mały składnik dodany do bardzo dużej liczby może zostać utracony. Dzieje się tak, ponieważ odstęp między kolejnymi liczbami maszynowymi w pobliżu dużych wartości jest większy niż dodawany mały składnik. Dlatego kolejność wykonywania działań może wpływać na wynik.

Możesz jeszcze dodać wersję z dokładniejszym typem `Decimal`:

```
from decimal import Decimal, getcontext

getcontext().prec = 50

a = Decimal("1e20")
b = Decimal("1e-10")

wynik = (a + b) - a

print(wynik)
```

Wynik:

1E-10

19. Inf, NaN, overflow i underflow

Standard IEEE 754 przewiduje wartości specjalne.

Przykłady:

```
1.0 / 0.0 → ∞
0.0 / 0.0 → NaN
```

W C++:

C++

```
double x = 1.0/0.0; // inf
double y = 0.0/0.0; // nan

std::isinf(x);
std::isnan(y);
```

Overflow

Overflow występuje, gdy wynik jest poza zakresem reprezentacji.

Wtedy wynik może przejść w:

$+\infty$ lub $-\infty$

Underflow

Underflow występuje dla bardzo małych liczb.

Wynik może zostać zapisany jako:

0

albo jako liczba zdenormalizowana.

Przykład

```
import math

# wartości specjalne
x = float("inf")
y = float("nan")

print("x =", x)
print("y =", y)

print("Czy x to nieskończoność?", math.isinf(x))
print("Czy y to NaN?", math.isnan(y))
```

PYTHON

Wynik:

```
x = inf
y = nan
Czy x to nieskończoność? True
Czy y to NaN? True
```

Ważna różnica względem C++:

```
# W Pythonie to da błąd:
# print(1.0 / 0.0)
```

PYTHON

Python nie zwróci tutaj `inf`, tylko zgłosi:

```
ZeroDivisionError: float division by zero
```

Dlatego w Pythonie `inf` i `nan` najprościej tworzyć tak:


```
inf = float("inf")  
nan = float("nan")
```

Overflow — wynik za duży

```
a = 1e308  
  
wynik = a * 10  
  
print(wynik)  
print(math.isinf(wynik))
```

Wynik:

```
inf  
True
```

Czyli liczba była za duża, żeby zmieścić się w typie `float`, więc Python zapisał ją jako nieskończoność.

Underflow — wynik za mały

```
b = 1e-323  
  
wynik = b / 10  
  
print(wynik)
```

Wynik:

```
0.0
```

Czyli liczba była tak mała, że została zaokrąglona do zera.

NaN — wynik nieokreślony

```
nan = float("nan")

print(nan)
print(math.isnan(nan))
print(nan == nan)
```

PYTHON

Wynik:

```
nan
True
False
```

Ważne: **NaN** nie jest równy nawet samemu sobie, dlatego sprawdzamy go przez:

```
math.isnan(nan)
```

PYTHON

Do notatki możesz wpisać:

W Pythonie wartości specjalne można utworzyć przez `float("inf")` oraz `float("nan")`. Do ich sprawdzania służą funkcje `math.isinf()` i `math.isnan()`. Overflow oznacza wynik zbyt duży dla typu `float`, a underflow oznacza wynik tak mały, że zostaje zapisany jako `0.0` lub liczba zdenormalizowana.

20. Utrata cyfr znaczących

Przykład funkcji z wykładu:

$$f(x) = \sqrt{x^2 + 1} - 1$$

Dla małych wartości `x` mamy:

$$\sqrt{x^2 + 1} \approx 1$$

więc w wyrażeniu:

$$\sqrt{x^2 + 1} - 1$$

odejmujemy dwie prawie równe liczby. Może to powodować **utrata cyfr znaczących**, ponieważ wynik jest bardzo mały, a część dokładności zostaje utracona podczas odejmowania.

Aby uniknąć tego problemu, stosujemy racjonalizację:

$$f(x) = \sqrt{x^2 + 1} - 1$$

Mnożymy przez wyrażenie sprzężone:

$$f(x) = \frac{(\sqrt{x^2 + 1} - 1)(\sqrt{x^2 + 1} + 1)}{\sqrt{x^2 + 1} + 1}$$

Po uproszczeniu:

$$f(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

Druga postać jest **numerycznie stabilniejsza**, ponieważ nie odejmujemy już dwóch prawie równych liczb.

Przykład w Pythonie:

```
import math

def f_naiwna(x):
    return math.sqrt(x**2 + 1) - 1

def f_stabilna(x):
    return x**2 / (math.sqrt(x**2 + 1) + 1)

x = 1e-8

print("Postać zwykła:", f_naiwna(x))
print("Postać stabilna:", f_stabilna(x))
```

PYTHON

Przykładowy wynik:

```
Postać zwykła: 0.0
Postać stabilna: 5.0000000000000005e-17
```

Wniosek:

Przy odejmowaniu dwóch prawie równych liczb może dojść do utraty cyfr znaczących. Dlatego wyrażenie warto przekształcić do postaci numerycznie stabilniejszej.

Stabilniejsza postać przez racjonalizację

Zamiast liczyć:

$$f(x) = \sqrt{x^2 + 1} - 1$$

można zastosować postać:

$$f(x) = \frac{x^2}{\sqrt{x^2+1}+1}$$

Druga postać jest numerycznie stabilniejsza.

Dlaczego?

Bo nie odejmujemy dwóch liczb prawie równych. Dzięki temu zmniejszamy ryzyko utraty cyfr znaczących.

21. Propagacja błędu

Dla funkcji jednej zmiennej, korzystając z aproksymacji liniowej:

$$\Delta f \approx |f'(x)|\Delta x$$

Małe błędy danych mogą zostać:

- wzmacnione,
- osłabione,
- zachowane.

Interpretacja

Jeżeli $|f'(x)|$ jest duże, to mały błąd wejścia może dać większy błąd wyniku.

Jeżeli $|f'(x)|$ jest małe, to błąd może zostać osłabiony.

Przykład

Jasne — przykład do slajdu „**Propagacja błędu**” można zrobić na funkcji:

$$f(x) = x^2$$

Wtedy pochodna to:

$$f'(x) = 2x$$

A błąd propaguje się w przybliżeniu tak:

$$\Delta f \approx |f'(x)|\Delta x$$

Czyli:

$$\Delta f \approx |2x|\Delta x$$

Przykład w Pythonie:

```
def f(x):
    return x**2

def pochodna_f(x):
    return 2 * x

x = 10
delta_x = 0.01

x_przyblizone = x + delta_x

# dokładna zmiana wartości funkcji
delta_f_rzeczywiste = abs(f(x_przyblizone) - f(x))

# przybliżenie z propagacji błędu
delta_f_przyblizone = abs(pochodna_f(x)) * delta_x

print("x =", x)
print("Błąd danych Δx =", delta_x)

print("f(x) =", f(x))
print("f(x + Δx) =", f(x_przyblizone))

print("Rzeczywisty błąd Δf =", delta_f_rzeczywiste)
print("Przybliżony błąd Δf ≈ |f'(x)|Δx =", delta_f_przyblizone)
```

Przykładowy wynik:

```
x = 10
Błąd danych Δx = 0.01
f(x) = 100
f(x + Δx) = 100.2001
Rzeczywisty błąd Δf = 0.2001000000000066
Przybliżony błąd Δf ≈ |f'(x)|Δx = 0.2
```

Wniosek:

Mały błąd wejściowy $\Delta x = 0.01$ spowodował błąd wyniku około $\Delta f = 0.2$.
 Błąd został wzmocniony, ponieważ pochodna funkcji w punkcie $x = 10$ wynosi 20.

Możesz też dopisać krótką notatkę:

```
Jeżeli |f'(x)| > 1, błąd danych jest wzmacniany.
Jeżeli |f'(x)| < 1, błąd danych jest osłabiany.
Jeżeli |f'(x)| ≈ 1, błąd jest mniej więcej zachowany.
```

22. Stabilność i niestabilność numeryczna

Niestabilność numeryczna występuje wtedy, gdy błędy zaokrągleń lub błędy danych są wzmacniane w trakcie obliczeń.

Źródła niestabilności:

- odejmowanie liczb bliskich,
 - utrata cyfr znaczących,
 - niekorzystna kolejność działań,
 - źle dobrany wzór lub postać obliczeń,
 - kumulacja błędów w długich obliczeniach.
-

23. Stabilność algorytmu

Algorytm jest **stabilny**, jeśli nie wzmacnia nieuniknionych błędów zaokrągleń.

Jeżeli błąd końcowy jest rzędu:

$$O(\epsilon_{mach})$$

to algorytm jest stabilny.

Jeżeli błędy rosną znacząco bardziej, algorytm jest niestabilny.

24. Uwarunkowanie zadania numerycznego

Uwarunkowanie określa wrażliwość rozwiązania na zaburzenia danych wejściowych.

Zadanie jest **źle uwarunkowane**, gdy nawet niewielkie błędy danych mogą powodować duże błędy wyniku, niezależnie od algorytmu.

Ważne

Uwarunkowanie jest własnością problemu, a nie algorytmu.

25. Liczba uwarunkowania

Z propagacji błędu:

$$\Delta f \approx f'(x)\Delta x$$

Wersja względna:

$$\frac{|\Delta f|}{|f(x)|} \approx \left| \frac{x f'(x)}{f(x)} \right| \cdot \frac{|\Delta x|}{|x|}$$

Definicja liczby uwarunkowania:

$$\kappa(x) = \left| \frac{x f'(x)}{f(x)} \right|$$

Interpretacja:

względny błąd wyjścia $\approx \kappa(x) \cdot$ względny błąd wejścia

Przykład

Przykład do slajdu w Pythonie, np. dla funkcji:

$$f(x) = x^2$$

Wtedy:

$$f'(x) = 2x$$

Liczba uwarunkowania:

$$\kappa(x) = \left| \frac{x f'(x)}{f(x)} \right| = \left| \frac{x \cdot 2x}{x^2} \right| = 2$$

Czyli **błąd względny wyniku jest około 2 razy większy niż błąd względny wejścia.**

```

def f(x):
    return x**2

def df(x):
    return 2*x

def kappa(x):
    return abs(x * df(x) / f(x))

x = 10
delta_x = 0.1

x_przyblizone = x + delta_x

blad_wzgledny_wejscia = abs(delta_x) / abs(x)

blad_wzgledny_wyjscia = abs(f(x_przyblizone) - f(x)) / abs(f(x))

przewidywany_blad = kappa(x) * blad_wzgledny_wejscia

print("x =", x)
print("x przybliżone =", x_przyblizone)

print("f(x) =", f(x))
print("f(x przybliżone) =", f(x_przyblizone))

print("Liczba uwarunkowania  $\kappa(x)$  =", kappa(x))

print("Błąd względny wejścia =", blad_wzgledny_wejscia)
print("Rzeczywisty błąd względny wyjścia =", blad_wzgledny_wyjscia)
print("Przewidywany błąd względny wyjścia ≈", przewidywany_blad)

```

Przykładowy wynik:

```

x = 10
x przybliżone = 10.1
f(x) = 100
f(x przybliżone) = 102.01
Liczba uwarunkowania  $\kappa(x)$  = 2.0
Błąd względny wejścia = 0.01
Rzeczywisty błąd względny wyjścia = 0.0201
Przewidywany błąd względny wyjścia ≈ 0.02

```

Wniosek do notatki:

Liczba uwarunkowania mówi, ile razy błąd względny danych wejściowych może zostać powiększony w wyniku.

Dla funkcji ($f(x) = x^2$) mamy ($\kappa(x) = 2$), więc błąd względny wyniku jest około 2 razy większy niż błąd względny wejścia.

26. Źle uwarunkowany problem

Jeżeli:

$$\kappa(x) \gg 1$$

to mały błąd danych może powodować duży błąd wyniku.

27. Przykład liczby uwarunkowania: $f(x) = x^2$

Dana jest funkcja:

$$f(x) = x^2$$

Pochodna:

$$f'(x) = 2x$$

Liczba uwarunkowania:

$$\kappa(x) = \left| \frac{x f'(x)}{f(x)} \right|$$

Po podstawieniu:

$$\kappa(x) = \left| \frac{x \cdot 2x}{x^2} \right| = 2$$

Wniosek

Względny błąd wyniku jest około 2 razy większy niż względny błąd wejścia.

Problem jest dobrze uwarunkowany.

28. Przykład liczby uwarunkowania: $f(x) = \sin(x)$

Dana jest funkcja:

$$f(x) = \sin x$$

Pochodna:

$$f'(x) = \cos x$$

Liczba uwarunkowania:

$$\kappa(x) = \left| \frac{x \cos x}{\sin x} \right|$$

Dla małych x :

$$x \sin x \approx x, \quad \cos x \approx 1$$

więc:

$$\kappa(x) \approx \left| \frac{x}{x} \right| = 1$$

Wniosek

W pobliżu zera problem jest dobrze uwarunkowany.

Uwaga

Gdy $f(x)$ jest bardzo małe, np. blisko miejsca zerowego, liczba uwarunkowania może być bardzo duża. Wtedy problem staje się źle uwarunkowany.

29. Uwarunkowanie a stabilność

Uwarunkowanie

Uwarunkowanie jest własnością problemu:

$$w = \Phi(d)$$

Oznacza, jak bardzo wynik reaguje na zmianę danych wejściowych.

Stabilność

Stabilność jest własnością algorytmu, czyli sposobu obliczania.

Cel

Chcemy mieć stabilny algorytm dla możliwie dobrze uwarunkowanego problemu.

Przykład

Uwarunkowanie vs stabilność

Uwarunkowanie mówi, czy sam problem jest wrażliwy na małe błędy danych wejściowych.

Stabilność mówi, czy wybrany algorytm dobrze radzi sobie z błędami zaokrągleń.

Czyli:

```
uwarunkowanie = cecha problemu  
stabilność = cecha sposobu liczenia
```

Dobry przykład na podstawie wcześniejszego slajdu:

```
import math

def f_naiwna(x):
    return math.sqrt(x**2 + 1) - 1

def f_stabilna(x):
    return x**2 / (math.sqrt(x**2 + 1) + 1)

x = 1e-8

print("x =", x)
print("Algorytm naiwny:", f_naiwna(x))
print("Algorytm stabilny:", f_stabilna(x))
```

PYTHON

Przykładowy wynik:

```
x = 1e-08  
Algorytm naiwny: 0.0  
Algorytm stabilny: 5.0000000000000005e-17
```

Obie funkcje liczą matematycznie to samo:

$$\sqrt{x^2 + 1} - 1 = x^2 / (\sqrt{x^2 + 1} + 1)$$

ale pierwszy sposób jest **niestabilny numerycznie**, bo odejmuje dwie prawie równe liczby:

$$\sqrt{x^2 + 1} \approx 1$$

czyli komputer liczy coś w stylu:

```
1.0000000000000000 - 1.0000000000000000
```

i wynik wychodzi 0.0.

Drugi sposób jest **stabilniejszy**, bo nie odejmuje prawie równych liczb.

Notatka do slajdu:

Problem może być dobrze uwarunkowany, ale źle dobrany algorytm może być niestabilny. Dlatego nie wystarczy znać wzoru matematycznego — ważny jest też sposób jego obliczania.

30. Relacja między uwarunkowaniem i stabilnością

Z wykładu:

Uwarunkowanie	Algorytm	Efekt
dobrze	stabilny	wynik dokładny
dobrze	niestabilny	duży błąd
złe	stabilny	błąd wynika z natury problemu
złe	niestabilny	katastrofa numeryczna

31. Przykład: $a^2 - b^2$

Rozważamy problem:

$$\Phi(a, b) = a^2 - b^2$$

Dwa algorytmy:

Algorytm A1

$$A1 : a \cdot a - b \cdot b$$

Algorytm A2

$$A2 : (a + b)(a - b)$$

Algebraicznie oba wzory są równoważne.

Numerycznie niekoniecznie.

Dlaczego A1 może być niestabilny?

Gdy:

$$a \approx b$$

to:

$$a^2 \approx b^2$$

Wtedy w algorytmie A1 odejmujemy liczby bliskie:

$$a^2 - b^2$$

Wynik jest małą różnicą dużych liczb.

To sprzyja utracie cyfr znaczących i wzmacnianiu błędu względnego.

Dlaczego A2 może być stabilniejszy?

W algorytmie A2:

$$(a + b)(a - b)$$

mały czynnik:

$$a - b$$

występuje bezpośrednio.

Dlatego ta postać zwykle jest stabilniejsza, gdy $a \approx b$.

Wniosek

Równoważność algebraiczna nie oznacza równoważności numerycznej.

Wzór:

$$(a + b)(a - b)$$

bywa stabilniejszy niż:

$$a^2 - b^2$$

gdy $a \approx b$.

To klasyczny przykład katastrofalnej utraty cyfr znaczących.

32. Problem sumowania

Rozważamy ogólną sumę:

$$S_n = \sum_{k=1}^n a_k$$

Sumowanie jest jedną z najczęstszych operacji w obliczeniach numerycznych.

Celem analizy jest:

- porównanie różnych algorytmów sumowania,
- zbadanie wpływu precyzji, np. `float` i `double`,
- przeanalizowanie propagacji błędu numerycznego.

Przykład

```
PYTHON

import math

def suma_zwykla(liczby):
    suma = 0.0

    for x in liczby:
        suma += x

    return suma

liczby1 = [1e20, 1.0, -1e20]
liczby2 = [1e20, -1e20, 1.0]

print("Suma w kolejności 1:", suma_zwykla(liczby1))
print("Suma w kolejności 2:", suma_zwykla(liczby2))

print("Dokładniejsze sumowanie math.fsum:", math.fsum(liczby1))
```

Przykładowy wynik:

```
Suma w kolejności 1: 0.0
Suma w kolejności 2: 1.0
Dokładniejsze sumowanie math.fsum: 1.0
```

Matematycznie suma powinna wynosić:

$$10^{20} + 1 - 10^{20} = 1$$

ale w pierwszym przypadku Python najpierw liczy:

$$10^{20} + 1$$

Liczba 1 jest za mała względem 10^{20} , więc zostaje „zgubiona”. Potem:

$$10^{20} - 10^{20} = 0$$

Dlatego wynik wychodzi 0.0.

Notatka do slajdu:

Przy sumowaniu liczb zmiennoprzecinkowych kolejność działań może wpływać na wynik. Małe składniki mogą zostać utracone, gdy są dodawane do bardzo dużych liczb. Dlatego w obliczeniach numerycznych porównuje się różne algorytmy sumowania, np. zwykłe `sum()` i dokładniejsze `math.fsum()`.

33. Dlaczego sumowanie jest podchwytliwe?

Niech a_k będą liczbami zmiennoprzecinkowymi.

Możliwe trudności:

- składniki mają różne rzędy wielkości,
- składniki mogą zmieniać znak,
- składniki mogą być bardzo małe względem aktualnej sumy,
- liczba składników n może być bardzo duża.

Matematycznie suma jest dobrze określona.

Problem leży w arytmetyce zmiennoprzecinkowej.

34. Klasyczne sumowanie

Algorytm:

```
s = 0
s ← s + a_k
```

Model błędu:

$$fl(x + y) = (x + y)(1 + \epsilon)$$

gdzie:

$$|\epsilon| \leq \epsilon_{mach}$$

Po n krokach:

$$\text{błąd względny} = O(n\epsilon_{mach})$$

Wniosek

Im większe n , tym większa kumulacja błędów.

Przykład

```
def sumowanie_klasyczne(liczby):
    s = 0.0

    for a_k in liczby:
        s = s + a_k

    return s
```

PYTHON

Czyli dokładnie według algorytmu ze slajdu:

$$s = 0$$
$$s \leftarrow s + a_k$$

Przykład pokazujący kumulację błędów:


```
def sumowanie_klasyczne(liczby):
    s = 0.0

    for a_k in liczby:
        s = s + a_k

    return s

n = 1_000_000
liczby = [0.1] * n

wynik = sumowanie_klasyczne(liczby)
wynik_dokladny = n * 0.1

print("Wynik klasycznego sumowania:", wynik)
print("Wynik oczekiwany:", wynik_dokladny)
print("Błąd bezwzględny:", abs(wynik - wynik_dokladny))
```

Przykładowy wynik:

```
Wynik klasycznego sumowania: 100000.00000133288
Wynik oczekiwany: 100000.0
Błąd bezwzględny: 1.3328826753422618e-06
```

Do notatki możesz dopisać:

W klasycznym sumowaniu każda operacja dodawania może wprowadzić mały błąd zaokrąglenia. Po wielu krokach błędy mogą się kumulować, dlatego im większe `n`, tym większy może być błąd końcowy.

Można też porównać ze stabilniejszym sumowaniem w Pythonie:

```
import math

n = 1_000_000
liczby = [0.1] * n

print("sum():", sum(liczby))
print("math.fsum():", math.fsum(liczby))
```

`math.fsum()` używa dokładniejszego algorytmu sumowania niż zwykłe dodawanie po kolei.

35. Algorytm Gilla–Møllera

Idea algorytmu:

- przechowywać sumę,
- przechowywać poprawkę, czyli kompensację.

Pseudokod z wykładu:

```
s = 0
p = 0

for a_k:
    t = s + a_k
    p = p + (a_k - (t - s))
    s = t

return s + p
```

Algorytm zmniejsza utratę cyfr znaczących, ale błąd nadal rośnie z n .

36. Algorytm Kahana

Algorytm Kahana to algorytm kompensowanego sumowania.

Pseudokod z wykładu:

```
sum = 0
c = 0

for a_k:
    y = a_k - c
    t = sum + y
    c = (t - sum) - y
    sum = t

return sum
```

Algorytm kompensuje utratę małych składników.

W wykładzie podano, że błąd nie narasta liniowo z n .

37. Co można badać przy sumowaniu?

Według wykładu można badać:

- porównanie błędów dla:
 - klasycznego sumowania,
 - algorytmu Gilla–Møllera,
 - algorytmu Kahana,
- wpływ liczby składników n ,
- wpływ precyzji obliczeń,
- związek z:
 - propagacją błędów,
 - ϵ_{mach} ,
 - stabilnością algorytmów.

38. Najważniejsze wnioski z wykładu

1. Każde obliczenie numeryczne jest przybliżeniem.
2. Źródłem błędów jest reprezentacja maszynowa.
3. Algorytmy mogą być stabilne albo niestabilne.
4. Problemy mogą być dobrze albo źle uwarunkowane.
5. Sposób liczenia ma znaczenie.
6. Dwa wzory algebraicznie równoważne mogą zachowywać się inaczej numerycznie.
7. Przy liczbach zmiennoprzecinkowych nie należy bezpośrednio porównywać wartości przez $==$.
8. Błędy mogą się kumulować, szczególnie w długich obliczeniach.
9. Odejmowanie liczb bardzo bliskich może prowadzić do utraty cyfr znaczących.
10. Algorytmy kompensowane, np. Kahana, pomagają ograniczyć błędy sumowania.

39. Szybka ściągą pojęć

Pojęcie	Znaczenie
Błąd bezwzględny	Różnica między wartością dokładną a przybliżeniem
Błąd względny	Błąd odniesiony do skali wartości dokładnej
Cyfry znaczące	Miara dokładności związana z błędem względnym
Błąd zaokrąglenia	Błąd wynikający ze skończonej reprezentacji liczby w komputerze
ϵ_{mach}	Najmniejsza liczba dodatnia, dla której $1 + \epsilon > 1$

Pojęcie	Znaczenie
Mantysa	Część liczby zmiennopozycyjnej decydująca o dokładności
Wykładnik	Część liczby zmiennopozycyjnej decydująca o zakresie
Overflow	Wynik poza zakresem, np. przejście do ∞
Underflow	Wynik zbyt mały, przejście do 0 lub liczby zdenormalizowanej
NaN	Wynik nieokreślony, np. $0.0/0.0$
Inf	Nieskończoność, np. $1.0/0.0$
Stabilność algorytmu	Algorytm nie wzmacnia znacząco błędów zaokrągleń
Uwarunkowanie problemu	Wrażliwość wyniku na zaburzenia danych wejściowych
Katastrofalna utrata cyfr znaczących	Utrata dokładności przy odejmowaniu liczb bliskich
Algorytm Kahana	Metoda kompensowanego sumowania ograniczająca utratę małych składników

Wykład 2: Macierze (lab 3)

1. Definicja macierzy

Macierzą nazywamy prostokątną tablicę liczb o wymiarze $m \times n$, czyli mającą m wierszy i n kolumn.

Ogólny zapis macierzy:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix}$$

W skrócie macierz zapisujemy jako:

$$A = [a_{i,j}]$$

gdzie $a_{\{i,j\}}$ oznacza element leżący w i -tym wierszu i j -tej kolumnie.

Przykład

Macierz:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

ma wymiar 2×3 , ponieważ ma 2 wiersze i 3 kolumny.

Przykład w Pythonie

PYTHON

```
A = [  
    [1, 2, 3],  
    [4, 5, 6]  
]  
  
liczba_wierszy = len(A)  
liczba_kolumn = len(A[0])  
  
print("Macierz A:")  
print(A)  
  
print("Liczba wierszy:", liczba_wierszy)  
print("Liczba kolumn:", liczba_kolumn)  
print("Wymiar macierzy:", liczba_wierszy, "x", liczba_kolumn)
```

Wynik:

```
Macierz A:  
[[1, 2, 3], [4, 5, 6]]  
Liczba wierszy: 2  
Liczba kolumn: 3  
Wymiar macierzy: 2 x 3
```

2. Oznaczenia w macierzy

Dla macierzy:

$$A = [a_{i,j}]$$

oznaczenia są następujące:

- $a_{i,j}$ — element macierzy,
- i — numer wiersza,
- j — numer kolumny,
- $a_{1,1}, a_{2,2}, a_{3,3}, \dots$ — elementy diagonal,
- $m \times n$ — wymiar macierzy.

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 7 & 8 \\ 9 & 10 \end{bmatrix}$$

element:

$$a_{2,1} = 9$$

bo znajduje się w drugim wierszu i pierwszej kolumnie.

Przykład w Pythonie

W Pythonie indeksowanie zaczyna się od 0, więc element `a_{2,1}` matematycznie zapisujemy jako `A[1][0]`.

```
A = [  
    [7, 8],  
    [9, 10]  
]  
  
element = A[1][0]  
  
print("Element a_2,1 =", element)
```

PYTHON

Wynik:

```
Element a_2,1 = 9
```

3. Normy wektorowe

Dla wektora:

$$x = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$$

najczęściej używane normy to:

3.1. Norma euklidesowa

$$|x|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

Norma euklidesowa odpowiada zwykłej długości wektora.

Przykład

Dla wektora:

$$x = (3, 4)$$

mamy:

$$|x|_2 = \sqrt{3^2 + 4^2} = 5$$

Przykład w Pythonie

```
import math

x = [3, 4]

suma = 0

for xi in x:
    suma += xi ** 2

norma_euklidesowa = math.sqrt(suma)

print(norma_euklidesowa)
```

PYTHON

Wynik:

5.0

3.2. Norma Manhattan

$$|x|_1 = \sum_{i=1}^n |x_i|$$

Norma Manhattan to suma modułów wszystkich współrzędnych.

Przykład

Dla wektora:

$$x = (-3, 4)$$

mamy:

$$|x|_1 = |-3| + |4| = 7$$

Przykład w Pythonie

PYTHON

```
x = [-3, 4]

suma = 0

for xi in x:
    suma += abs(xi)

norma_manhattan = suma

print(norma_manhattan)
```

Wynik:

7

3.3. Norma maksimum

$$|x|_{\infty} = \max_{1 \leq i \leq n} |x_i|$$

Norma maksimum wybiera największą wartość bezwzględną spośród współrzędnych.

W wykładzie zaznaczono, że do porównań liczbowych często używa się normy maksimum, bo jest szybka i pokazuje największy błąd.

Przykład

Dla wektora:

$$x = (-3, 4, 10, -2)$$

mamy:

$$|x|_{\infty} = 10$$

Przykład w Pythonie

PYTHON

```
x = [-3, 4, 10, -2]

maksimum = abs(x[0])

for xi in x:
    if abs(xi) > maksimum:
        maksimum = abs(xi)

norma_maksimum = maksimum

print(norma_maksimum)
```

Wynik:

10

4. Odległości w $\mathbb{R}^2$ (metryki)

Dla punktów:

$$p = (x_1, x_2)$$

oraz:

$$q = (y_1, y_2)$$

w wykładzie podano kilka metryk.

4.1. Odległość euklidesowa (Metryka euklidesowa)

$$d_2(p, q) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

To standardowa odległość „po prostej”.

Przykład

Dla punktów:

$$p = (1, 2)$$

$$q = (4, 6)$$

mamy:

$$d_2(p, q) = \sqrt{(1 - 4)^2 + (2 - 6)^2} = 5$$

Przykład w Pythonie

```
import math

p = (1, 2)
q = (4, 6)

d = math.sqrt((p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)

print(d)
```

PYTHON

Wynik:

5.0

4.2. Odległość Manhattan (Metryka Manhattan)

$$d_1(p, q) = |x_1 - y_1| + |x_2 - y_2|$$

To odległość liczona jako suma przesunięć poziomych i pionowych.

Przykład

Dla punktów:

$$p = (1, 2)$$

$$q = (4, 6)$$

mamy:

$$d_1(p, q) = |1 - 4| + |2 - 6| = 7$$

Przykład w Pythonie

```
p = (1, 2)
q = (4, 6)

d = abs(p[0] - q[0]) + abs(p[1] - q[1])

print(d)
```

PYTHON

Wynik:

4.3. Metryka rzeki

W wykładzie podano metrykę rzeki, gdzie rzeka jest osią **0x**.

$$d_R(p, q) = \begin{cases} |x_2 - y_2|, & x_1 = y_1 \\ |x_1 - y_1| + |x_2| + |y_2|, & x_1 \neq y_1 \end{cases}$$

Interpretacja: jeżeli punkty są na tej samej pionowej prostej, przechodzimy bezpośrednio. W przeciwnym razie trzeba dojść do rzeki, przejść wzdłuż niej i odejść od rzeki.

Przykład

Dla:

$$p = (1, 2)$$

$$q = (4, 6)$$

ponieważ:

$$x_1 \neq y_1$$

liczymy:

$$d_R(p, q) = |1 - 4| + |2| + |6| = 11$$

Przykład w Pythonie

```
p = (1, 2)
q = (4, 6)

if p[0] == q[0]:
    d = abs(p[1] - q[1])
else:
    d = abs(p[0] - q[0]) + abs(p[1]) + abs(q[1])

print(d)
```

PYTHON

Wynik:

11

4.4. Metryka kolejowa / Metryka centrum

W wykładzie podano też metrykę kolejową, gdzie w razie potrzeby jedzie się przez punkt:

$$(0,0)$$

Definicja:

$$d_C(p, q) = \begin{cases} |p - q|_2, & p \text{ i } q \text{ leżą na jednej prostej przez } (0, 0) \\ |p|_2 + |q|_2, & \text{w przeciwnym razie} \end{cases}$$

Przykład

Dla punktów:

$$p = (1, 1)$$

$$q = (2, 2)$$

punkty leżą na tej samej prostej przechodzącej przez środek układu, więc można liczyć zwykłą odległość euklidesową.

Przykład w Pythonie

```
import math

p = (1, 1)
q = (2, 2)

def norma_euklidesowa(v):
    return math.sqrt(v[0] ** 2 + v[1] ** 2)

# Sprawdzamy, czy punkty leżą na jednej prostej przez (0,0).
# Dla punktów p=(x1,x2), q=(y1,y2) warunek współliniowości z (0,0):
# x1*y2 - x2*y1 == 0
if p[0] * q[1] - p[1] * q[0] == 0:
    roznica = (p[0] - q[0], p[1] - q[1])
    d = norma_euklidesowa(roznica)
else:
    d = norma_euklidesowa(p) + norma_euklidesowa(q)

print(d)
```

PYTHON

Wynik:

```
1.4142135623730951
```

5. Macierze szczególne

5.1. Macierz wierszowa

Macierz wierszowa ma wymiar:

$$1 \times n$$

czyli składa się z jednego wiersza.

Przykład:

$$A = [1 \quad 2 \quad 3 \quad 4]$$

Przykład w Pythonie

```
A = [[1, 2, 3, 4]]  
  
print(A)  
print("Liczba wierszy:", len(A))  
print("Liczba kolumn:", len(A[0]))
```

PYTHON

Wynik:

```
[[1, 2, 3, 4]]  
Liczba wierszy: 1  
Liczba kolumn: 4
```

5.2. Macierz kolumnowa

Macierz kolumnowa ma wymiar:

$$m \times 1$$

czyli składa się z jednej kolumny.

Przykład:

$$A = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Można ją zapisać jako transpozycję macierzy wierszowej:

$$A = [1 \quad 2 \quad 3]^T$$

Transpozycja

PYTHON

```
A = [[1, 2, 3]]

A_T = []

for i in range(len(A[0])):
    A_T.append([A[0][i]])

print("Macierz wierszowa:")
print(A)

print("Transpozycja, czyli macierz kolumnowa:")
print(A_T)
```

wynik:

```
[1  2  3]^T = [[1],
               [2],
               [3]]
```

Przykład w Pythonie

PYTHON

```
A = [
    [1],
    [2],
    [3]
]

print(A)
print("Liczba wierszy:", len(A))
print("Liczba kolumn:", len(A[0]))
```

Wynik:

```
[[1], [2], [3]]
Liczba wierszy: 3
Liczba kolumn: 1
```

5.3. Macierz zerowa

Macierz zerowa to macierz, która zawiera same zera.

Przykład:

$$A = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
m = 2
n = 3

A = []

for i in range(m):
    wiersz = []

    for j in range(n):
        wiersz.append(0)

    A.append(wiersz)

print(A)
```

Wynik:

```
[[0, 0, 0], [0, 0, 0]]
```

2 wiersze i 3 kolumny

$m \times n = 2 \times 3$

5.4. Macierz kwadratowa

Macierz kwadratowa to macierz, w której liczba wierszy i kolumn jest taka sama.

Jeżeli macierz ma wymiar:

$$n \times n$$

to mówimy, że jest macierzą stopnia n .

Przykład macierzy kwadratowej stopnia 3:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
czy_kwadratowa = len(A) == len(A[0]) # sprawdza czy ma tyle samo wierszy co kolumn  
  
print(czy_kwadratowa)
```

Wynik:

True

5.5. Macierz symetryczna

Macierz symetryczna to macierz kwadratowa, której elementy spełniają warunek:

$$a_{i,j} = a_{j,i}$$

Równoważnie:

$$A^T = A$$

Przykład:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [  
    [1, 2, 3],  
    [2, 5, 6],  
    [3, 6, 9]  
]  
  
n = len(A)  
czy_symetryczna = True  
  
for i in range(n):  
    for j in range(n):  
        if A[i][j] != A[j][i]:  
            czy_symetryczna = False  
  
print(czy_symetryczna)
```

Wynik:

True

5.6. Macierz diagonalna

Macierz diagonalna to macierz kwadratowa, w której wszystkie elementy poza diagonalą są równe zero.

Przykład:

$$A = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 7 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [
    [2, 0, 0],
    [0, 5, 0],
    [0, 0, 7]
]

n = len(A)

#-----

A = []

for i in range(n):
    wiersz = []

    for j in range(n):
        if i == j:
            wiersz.append(i + 2)
        else:
            wiersz.append(0)

    A.append(wiersz)

czy_diagonalna = True

for i in range(n):
    for j in range(n):
        if i != j and A[i][j] != 0:
            czy_diagonalna = False

print(czy_diagonalna)
```

Wynik:

True

5.7. Macierz jednostkowa

Macierz jednostkowa to szczególny przypadek macierzy diagonalnej. Na diagonalu ma same jedynki.

Przykład:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
n = 3

I = []

for i in range(n):
    wiersz = []

    for j in range(n):
        wiersz.append(0)

    I.append(wiersz)

for i in range(n):
    I[i][i] = 1

print(I)
```

Wynik:

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

5.8. Macierz górna trójkątna

Macierz górna trójkątna to macierz kwadratowa, w której elementy poniżej diagonalii są równe zero.

Przykład:

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
U = [  
    [1, 2, 3],  
    [0, 4, 5],  
    [0, 0, 6]  
]  
  
n = len(U)  
  
# -----  
n = 3  
  
U = []  
  
for i in range(n):  
    wiersz = []  
  
    for j in range(n):  
        if i <= j:  
            wiersz.append(j + 1)  
        else:  
            wiersz.append(0)  
  
    U.append(wiersz)  
  
czy_gorna_trojkatna = True  
  
for i in range(n):  
    for j in range(n):  
        if i > j and U[i][j] != 0:  
            czy_gorna_trojkatna = False  
  
print(U)  
print(czy_gorna_trojkatna)
```

Wynik:

```
[[1, 2, 3], [0, 2, 3], [0, 0, 3]]  
True
```

5.9. Macierz dolna trójkątna

Macierz dolna trójkątna to macierz kwadratowa, w której elementy powyżej diagonali są równe zero.

Przykład:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \\ 4 & 5 & 6 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
L = [  
    [1, 0, 0],  
    [2, 3, 0],  
    [4, 5, 6]  
]  
  
n = len(L)  
  
#-----  
n = 3  
  
L = []  
  
for i in range(n):  
    wiersz = []  
  
    for j in range(n):  
        if i >= j:  
            wiersz.append(i + j + 1)  
        else:  
            wiersz.append(0)  
  
    L.append(wiersz)  
  
czy_dolna_trojkatna = True  
  
for i in range(n):  
    for j in range(n):  
        if i < j and L[i][j] != 0:  
            czy_dolna_trojkatna = False  
  
print(czy_dolna_trojkatna)
```

Wynik:

True

6. Dodawanie i odejmowanie macierzy

Aby dodać lub odjąć dwie macierze, muszą mieć one takie same wymiary.

Jeżeli:

$$A = [a_{i,j}]$$

oraz:

$$B = [b_{i,j}]$$

to:

$$A \pm B = [a_{i,j} \pm b_{i,j}]$$

Dodawanie i odejmowanie wykonuje się element po elemencie.

Przykład

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$A + B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
# wymiary macierzy A
m_A = 2 # liczba wierszy
n_A = 2 # liczba kolumn

# wymiary macierzy B
m_B = 2 # liczba wierszy
n_B = 2 # liczba kolumn

# tworzenie macierzy A od zera
A = []

liczba = 1

for i in range(m_A):
    wiersz = []

    for j in range(n_A):
        wiersz.append(liczba)
        liczba += 1

    A.append(wiersz)

# tworzenie macierzy B od zera
B = []

liczba = 5

for i in range(m_B):
    wiersz = []

    for j in range(n_B):
        wiersz.append(liczba)
        liczba += 1

    B.append(wiersz)

print("Macierz A:")
print(A)

print("Macierz B:")
print(B)

# sprawdzenie, czy można dodać macierze
if len(A) == len(B) and len(A[0]) == len(B[0]):

    C = []
```

```

for i in range(len(A)):
    wiersz = []

    for j in range(len(A[0])):
        wiersz.append(A[i][j] + B[i][j])

    C.append(wiersz)

print("Macierz C = A + B:")
print(C)

else:
    print("Nie można dodać macierzy, ponieważ mają różne wymiary.")

```

PYTHON

```

A = [
    [1, 2],
    [3, 4]
]

B = [
    [5, 6],
    [7, 8]
]

C = []

for i in range(len(A)):
    wiersz = []
    for j in range(len(A[0])):
        wiersz.append(A[i][j] + B[i][j])
    C.append(wiersz)

print(C)

```

Wynik:

```
[[6, 8], [10, 12]]
```

7. Mnożenie macierzy przez liczbę rzeczywistą

Jeżeli macierz **A** mnożymy przez liczbę rzeczywistą **k**, to każdy element macierzy mnożymy przez **k**.

$$kA = [ka_{i,j}]$$

Przykład

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Dla:

$$k = 3$$

otrzymujemy:

$$3A = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

Przykład w Pythonie

```
A = [  
    [1, 2],  
    [3, 4]  
]  
  
k = 3  
  
C = []  
  
for i in range(len(A)):  
    wiersz = []  
    for j in range(len(A[0])):  
        wiersz.append(k * A[i][j])  
    C.append(wiersz)  
  
print(C)
```

PYTHON

Wynik:

```
[[3, 6], [9, 12]]
```

8. Transpozycja macierzy

Macierz transponowana do macierzy **A** to macierz:

$$A^T$$

która powstaje przez zamianę wierszy na kolumny i kolumn na wiersze.

Przykład

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Własności transpozycji

$$(A^T)^T = A$$

$$(A \pm B)^T = A^T \pm B^T$$

$$(kA)^T = kA^T$$

$$(AB)^T = B^T A^T$$

$$\det(A^T) = \det(A)$$

Przykład w Pythonie

```
A = [  
    [1, 2, 3],  
    [4, 5, 6]  
]  
  
AT = []  
  
# przechodzimy po kolumnach macierzy A  
for j in range(len(A[0])):  
    wiersz = []  
  
    # przechodzimy po wierszach macierzy A  
    for i in range(len(A)):  
        wiersz.append(A[i][j])  
  
    AT.append(wiersz)  
  
print(AT)
```

PYTHON

Wynik:

```
[[1, 4], [2, 5], [3, 6]]
```

9. Normy macierzy

Dla macierzy:

$$A = [a_{i,j}] \in \mathbb{R}^{n \times m}$$

w wykładzie podano kilka norm.

9.1. Norma Frobeniusa

$$|A|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^m a_{i,j}^2}$$

Jest to norma euklidesowa wektora otrzymanego przez „spłaszczenie” macierzy.

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

mamy:

$$|A|_F = \sqrt{1^2 + 2^2 + 3^2 + 4^2}$$

$$|A|_F = \sqrt{30}$$

Przykład w Pythonie

```
import math

A = [
    [1, 2],
    [3, 4]
]

suma = 0

for i in range(len(A)):
    for j in range(len(A[0])):
        suma += A[i][j] ** 2

norma_frobeniusa = math.sqrt(suma)

print(norma_frobeniusa)
```

PYTHON

Wynik:

9.2. Norma Manhattan jako suma modułów elementów

W wykładzie oznaczona jako:

$$|A|_{1,sum} = \sum_{i=1}^n \sum_{j=1}^m |a_{i,j}|$$

To suma wartości bezwzględnych wszystkich elementów macierzy.

Przykład w Pythonie

```
A = [  
    [1, -2],  
    [-3, 4]  
]  
  
suma = 0  
  
for i in range(len(A)):  
    for j in range(len(A[0])):  
        suma += abs(A[i][j])  
  
print(suma)
```

PYTHON

Wynik:

10

9.3. Norma maksimum elementowa

$$|A|_{max} = \max_{1 \leq i \leq n, 1 \leq j \leq m} |a_{i,j}|$$

To największy moduł elementu macierzy.

Przykład w Pythonie

PYTHON

```
A = [  
    [1, -2],  
    [-3, 4]  
]  
  
maksimum = 0  
  
for i in range(len(A)):  
    for j in range(len(A[0])):  
        if abs(A[i][j]) > maksimum:  
            maksimum = abs(A[i][j])  
  
print(maksimum)
```

Wynik:

4

9.4. Norma operatorowa $|A|_1$

W wykładzie podkreślono ważną uwagę: norma:

$$|A|_{1,sum}$$

nie jest normą operatorową indukowaną przez normę wektorową:

$$|\cdot|_1$$

Norma operatorowa ma postać:

$$|A|_1 = \max_{1 \leq j \leq m} \sum_{i=1}^n |a_{i,j}|$$

czyli jest maksymalną sumą modułów w kolumnie.

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 1 & -2 \\ 3 & 4 \end{bmatrix}$$

sumy kolumn wynoszą:

$$|1| + |3| = 4$$

$$|-2| + |4| = 6$$

więc:

$$|A|_1 = 6$$

Przykład w Pythonie

PYTHON

```
A = [  
    [1, -2],  
    [3, 4]  
]  
  
liczba_wierszy = len(A)  
liczba_kolumn = len(A[0])  
  
najwieksza_suma_kolumny = 0  
  
for j in range(liczba_kolumn):  
    suma_kolumny = 0  
    for i in range(liczba_wierszy):  
        suma_kolumny += abs(A[i][j])  
  
    if suma_kolumny > najwieksza_suma_kolumny:  
        najwieksza_suma_kolumny = suma_kolumny  
  
print(najwieksza_suma_kolumny)
```

Wynik:

6

Przykład losowe wartości w macierzy

PYTHON

```
import random

# wymiary macierzy
m = 2 # liczba wierszy
n = 2 # liczba kolumn

# tworzenie macierzy A z losowych wartości
A = []

for i in range(m):
    wiersz = []

    for j in range(n):
        liczba = random.randint(-10, 10) # osuje liczby całkowite od -10 do 10
        wiersz.append(liczba)

    A.append(wiersz)

print("Macierz A:")
print(A)

# obliczanie normy kolumnowej
norma_kolumnowa = 0

for j in range(n):
    suma_kolumny = 0

    for i in range(m):
        suma_kolumny += abs(A[i][j])

    print("Suma kolumny", j, "=", suma_kolumny)

    if suma_kolumny > norma_kolumnowa:
        norma_kolumnowa = suma_kolumny

print("Norma kolumnowa macierzy A:", norma_kolumnowa)
```

Przykładowy wynik:

```
Macierz A:
[[4, -7], [2, 5]]
Suma kolumny 0 = 6
Suma kolumny 1 = 12
Norma kolumnowa macierzy A: 12
```

10. Macierz ortogonalna

Macierz ortogonalna to macierz, dla której zachodzi:

$$A^T A = I$$

Jeżeli macierz jest kwadratowa, to:

$$A^T A = A A^T = I$$

oraz:

$$A^T = A^{-1}$$

czyli transpozycja macierzy jest jej macierzą odwrotną.

Przykład

Macierz jednostkowa jest macierzą ortogonalną:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

ponieważ:

$$I^T I = I$$

Przykład w Pythonie

PYTHON

```
A = [
    [1, 0],
    [0, 1]
]

def transponuj(M):
    wynik = []
    for j in range(len(M[0])):
        wiersz = []
        for i in range(len(M)):
            wiersz.append(M[i][j])
        wynik.append(wiersz)
    return wynik

def mnoz_macierze(A, B):
    wynik = []
    for i in range(len(A)):
        wiersz = []
        for j in range(len(B[0])):
            suma = 0
            for k in range(len(B)):
                suma += A[i][k] * B[k][j]
            wiersz.append(suma)
        wynik.append(wiersz)
    return wynik

AT = transponuj(A)
ATA = mnoz_macierze(AT, A)

print(ATA)
```

Wynik:

```
[[1, 0], [0, 1]]
```

11. Mnożenie macierzy

Iloczyn macierzy:

$$C = AB$$

istnieje wtedy, gdy liczba kolumn macierzy **A** jest równa liczbie wierszy macierzy **B**.

Jeżeli:

$$A \in \mathbb{R}^{m_1 \times n_1}$$

oraz:

$$B \in \mathbb{R}^{m_2 \times n_2}$$

to iloczyn **AB** istnieje wtedy i tylko wtedy, gdy:

$$n_1 = m_2$$

Wynik ma wymiar:

$$AB \in \mathbb{R}^{m_1 \times n_2}$$

Elementy macierzy wynikowej liczymy ze wzoru:

$$c_{i,j} = \sum_{l=1}^n a_{i,l} b_{l,j}$$

Koszt obliczeń:

$$\Theta(m_1 \cdot n_1 \cdot n_2)$$

Przykład

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Pierwszy element wyniku:

$$c_{1,1} = 1 \cdot 5 + 2 \cdot 7 = 19$$

Cały wynik:

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [
    [1, 2],
    [3, 4]
]

B = [
    [5, 6],
    [7, 8]
]

# liczba kolumn macierzy A
kolumny_A = len(A[0])

# liczba wierszy macierzy B
wiersze_B = len(B)

if kolumny_A == wiersze_B:
    C = []

    for i in range(len(A)):
        wiersz = []

        for j in range(len(B[0])):
            suma = 0

            for l in range(len(B)):
                suma += A[i][l] * B[l][j]

            wiersz.append(suma)

        C.append(wiersz)

    print("Macierz C = A * B:")
    print(C)

else:
    print("Nie można pomnożyć macierzy.")
    print("Liczba kolumn macierzy A musi być równa liczbie wierszy macierzy B.")
```

Wynik:

```
[[19, 22], [43, 50]]
```

11.1. Własności mnożenia macierzy

Mnożenie macierzy nie jest przemienne:

$$AB \neq BA$$

Mnożenie macierzy jest łączne:

$$(AB)C = A(BC)$$

Mnożenie jest rozdzielne względem dodawania:

$$A(B + C) = AB + AC$$

Macierz jednostkowa jest elementem neutralnym:

$$AI = A$$

$$IA = A$$

Przykład z wykładu: $AB \neq BA$

Dane są macierze:

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$$

Wtedy:

$$AB = \begin{bmatrix} 7 & 2 \\ 3 & 1 \end{bmatrix}$$

oraz:

$$BA = \begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$$

czyli:

$$AB \neq BA$$

Przykład w Pythonie

PYTHON

```
def mnoz_macierze(A, B):
    if len(A[0]) != len(B):
        return None

    C = []

    for i in range(len(A)):
        wiersz = []

        for j in range(len(B[0])):
            suma = 0

            for k in range(len(B)):
                suma += A[i][k] * B[k][j]

            wiersz.append(suma)

        C.append(wiersz)

    return C

A = [
    [1, 2],
    [0, 1]
]

B = [
    [1, 0],
    [3, 1]
]

AB = mnoz_macierze(A, B)
BA = mnoz_macierze(B, A)

print("AB =", AB)
print("BA =", BA)

if AB is not None and BA is not None:
    print("Czy AB == BA?", AB == BA)
else:
    print("Nie można wykonać któregoś mnożenia.")
```

Wynik:

```
AB = [[7, 2], [3, 1]]
BA = [[1, 2], [3, 7]]
Czy AB == BA? False
```

12. Przekształcenia elementarne

Elementarne przekształcenia macierzy dzielą się na przekształcenia pierwszego i drugiego rodzaju.

12.1. Przekształcenia pierwszego rodzaju

Dotyczą wierszy macierzy.

Są to:

1. przestawienie dwóch wierszy,
2. pomnożenie dowolnego wiersza przez liczbę różną od zera,
3. dodanie do wiersza krotności innego wiersza.

Przykład w Pythonie

```
A = [
    [1, 2],
    [3, 4]
]

# Zamiana dwóch wierszy
A[0], A[1] = A[1], A[0]

print(A)
```

PYTHON

Wynik:

```
[[3, 4], [1, 2]]
```

12.2. Przekształcenia drugiego rodzaju

Dotyczą kolumn macierzy.

Są to:

1. przestawienie dwóch kolumn,

2. pomnożenie dowolnej kolumny przez liczbę różną od zera,
3. dodanie do kolumny krotności innej kolumny.

Przykład w Pythonie

```
PYTHON

A = [
    [1, 2],
    [3, 4]
]

# Zamiana kolumn 0 i 1
for i in range(len(A)):
    A[i][0], A[i][1] = A[i][1], A[i][0]

print(A)
```

Wynik:

```
[[2, 1], [4, 3]]
```

13. Znaczenie przekształceń elementarnych

Przekształcenia elementarne są używane między innymi do:

- liczenia rzędu macierzy,
- liczenia wyznacznika macierzy kwadratowej,
- rozwiązywania układów równań liniowych,
- znajdowania macierzy odwrotnej.

13.1. Wpływ na rząd macierzy

Na rząd macierzy nie mają wpływu żadne operacje elementarne.

Czyli wszystkie przekształcenia elementarne można stosować przy liczeniu rzędu.

Przykład w Pythonie

Poniżej przykład pokazuje, że po dodaniu wielokrotności jednego wiersza do drugiego macierz zmienia wygląd, ale operacja jest elementarna.

```
A = [
    [1, 2],
    [3, 4]
]

# R2 := R2 - 3 * R1
for j in range(len(A[0])):
    A[1][j] = A[1][j] - 3 * A[0][j]

print(A)
```

Wynik:

```
[[1, 2], [0, -2]]
```

13.2. Wpływ operacji elementarnych na wyznacznik

Przy liczeniu wyznacznika:

1. Dodanie wielokrotności jednego wiersza do drugiego nie zmienia wyznacznika.
2. Pomnożenie wiersza lub kolumny przez k mnoży wyznacznik przez k .
3. Zamiana miejscami dwóch wierszy lub kolumn zmienia znak wyznacznika.

Przykład w Pythonie

```
def det2(A):
    return A[0][0] * A[1][1] - A[0][1] * A[1][0]

A = [
    [1, 2],
    [3, 4]
]

print("det(A) =", det2(A))

# Zamiana wierszy
B = [
    [3, 4],
    [1, 2]
]

print("det(B) =", det2(B))
```

Wynik:

$$\det(A) = -2$$

$$\det(B) = 2$$

Po zamianie wierszy znak wyznacznika się zmienił.

14. Wyznacznik macierzy

Wyznacznik to funkcja przyporządkowująca każdej macierzy kwadratowej pewną liczbę.

Wyznacznik macierzy A oznaczamy jako:

$$\det(A)$$

albo:

$$|A|$$

Jeżeli:

$$\det(A) \neq 0$$

to macierz A nazywa się **nieosobliwą**.

Jeżeli:

$$\det(A) = 0$$

to macierz A nazywa się **osobliwą**.

Własności wyznacznika

$$\det(A) = \det(A^T)$$

$$\det(AB) = \det(A) \det(B)$$

Przykład w Pythonie

PYTHON

```
def det2(A):  
    return A[0][0] * A[1][1] - A[0][1] * A[1][0]  
  
A = [  
    [1, 2],  
    [3, 4]  
]  
  
d = det2(A)  
  
print("det(A) =", d)  
  
if d != 0:  
    print("Macierz jest nieosobliwa")  
else:  
    print("Macierz jest osobliwa")
```

Wynik:

```
det(A) = -2  
Macierz jest nieosobliwa
```

Wersja dla dowolnej macierzy kwadratowej $n \times n$

PYTHON

```
def czy_kwadratowa(A):
    liczba_wierszy = len(A)

    for i in range(liczba_wierszy):
        if len(A[i]) != liczba_wierszy:
            return False

    return True

def minor(A, wiersz_usuniety, kolumna_usunieta):
    M = []

    for i in range(len(A)):
        if i != wiersz_usuniety:
            nowy_wiersz = []

            for j in range(len(A[i])):
                if j != kolumna_usunieta:
                    nowy_wiersz.append(A[i][j])

            M.append(nowy_wiersz)

    return M

def det(A):
    if not czy_kwadratowa(A):
        return None

    n = len(A)

    if n == 1:
        return A[0][0]

    if n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]

    wyznacznik = 0

    for j in range(n):
        znak = (-1) ** j
        wyznacznik += znak * A[0][j] * det(minor(A, 0, j))

    return wyznacznik
```

15. Wyznacznik metodą Laplace'a

Rozwinięcie Laplace'a pozwala obliczać wyznacznik rekurencyjnie.

Dla macierzy stopnia 1:

$$A = [a_{1,1}]$$
$$\det(A) = a_{1,1}$$

Dla macierzy stopnia większego niż 1:

$$\det(A) = \sum_{i=1}^n (-1)^{i+j} a_{i,j} M_{i,j}$$

gdzie $M_{i,j}$ to wyznacznik macierzy powstałej z A przez skreślenie i -tego wiersza i j -tej kolumny.

W wykładzie ta definicja jest oparta o rozwinięcie wzdłuż j -tej kolumny.

Przypadek macierzy 2×2

Dla:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}$$

mamy:

$$\det(A) = a_{1,1}a_{2,2} - a_{1,2}a_{2,1}$$

Przykład w Pythonie

PYTHON

```
def det2(A):  
    return A[0][0] * A[1][1] - A[0][1] * A[1][0]  
  
A = [  
    [2, 1],  
    [5, 3]  
]  
  
print(det2(A))
```

Wynik:

1

15.1. Wyznacznik macierzy 3 × 3

Dla macierzy:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix}$$

wyznacznik ma postać:

$$\det(A) = a_{1,1}a_{2,2}a_{3,3} + a_{1,2}a_{2,3}a_{3,1} + a_{1,3}a_{2,1}a_{3,2} - a_{1,3}a_{2,2}a_{3,1} - a_{1,1}a_{2,3}a_{3,2} - a_{1,2}a_{2,1}a_{3,3}$$

Przykład w Pythonie

PYTHON

```
def det3(A):  
    a = A[0][0]  
    b = A[0][1]  
    c = A[0][2]  
  
    d = A[1][0]  
    e = A[1][1]  
    f = A[1][2]  
  
    g = A[2][0]  
    h = A[2][1]  
    i = A[2][2]  
  
    wyznacznik = (  
        a * e * i  
        + b * f * g  
        + c * d * h  
        - c * e * g  
        - b * d * i  
        - a * f * h  
    )  
  
    return wyznacznik  
  
A = [  
    [1, 2, 1],  
    [2, 3, 1],  
    [1, 1, 1]  
]  
  
print(det3(A))
```

Wynik:

15.2. Algorytm rekurencyjny Laplace'a

Algorytm z wykładu:

```
def det(A):
    n = size(A)

    if n == 1:
        return A[1][1]

    if n == 2:
        return A[1][1] * A[2][2] - A[1][2] * A[2][1]

    s = 0

    for j = 1..n:
        M = minor(A, row=1, col=j)
        s += (-1)^(1+j) * A[1][j] * det(M)

    return s
```

Wada: koszt rośnie bardzo szybko, więc dla dużych macierzy ta metoda jest niepraktyczna.

Przykład w Pythonie

PYTHON

```
def minor(A, wiersz_do_usuniecia, kolumna_do_usuniecia):
    M = []

    for i in range(len(A)):
        if i == wiersz_do_usuniecia:
            continue

        nowy_wiersz = []

        for j in range(len(A)):
            if j == kolumna_do_usuniecia:
                continue

            nowy_wiersz.append(A[i][j])

        M.append(nowy_wiersz)

    return M

def det_laplace(A):
    n = len(A)

    if n == 1:
        return A[0][0]

    if n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]

    suma = 0

    # rozwinięcie wzdłuż pierwszego wiersza
    for j in range(n):
        znak = (-1) ** j
        suma += znak * A[0][j] * det_laplace(minor(A, 0, j))

    return suma

A = [
    [1, 2, 1],
    [2, 3, 1],
    [1, 1, 1]
]

print(det_laplace(A))
```

Wynik:

16. Macierz odwrotna

Dla macierzy kwadratowej A macierzą odwrotną jest macierz:

$$A^{-1}$$

spełniająca warunek:

$$A \cdot A^{-1} = A^{-1} \cdot A = I$$

gdzie I to macierz jednostkowa.

Warunkiem koniecznym i wystarczającym istnienia macierzy odwrotnej jest:

$$\det(A) \neq 0$$

Przykład w Pythonie

PYTHON

```
def det2(A):  
    return A[0][0] * A[1][1] - A[0][1] * A[1][0]  
  
A = [  
    [2, 1],  
    [5, 3]  
]  
  
if det2(A) != 0:  
    print("Macierz odwrotna istnieje")  
else:  
    print("Macierz odwrotna nie istnieje")
```

Wynik:

```
Macierz odwrotna istnieje
```

16.1. Macierz odwrotna dla macierzy 2×2

Dla macierzy:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

jeżeli:

$$\det(A) \neq 0$$

to:

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$$

wyznacznik wynosi:

$$\det(A) = 2 \cdot 3 - 1 \cdot 5 = 1$$

więc:

$$A^{-1} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
def inverse2(A):  
    a = A[0][0]  
    b = A[0][1]  
    c = A[1][0]  
    d = A[1][1]  
  
    det = a * d - b * c  
  
    if det == 0:  
        raise ValueError("Macierz osobliwa, brak macierzy odwrotnej")  
  
    return [  
        [d / det, -b / det],  
        [-c / det, a / det]  
    ]  
  
A = [  
    [2, 1],  
    [5, 3]  
]  
  
A_inv = inverse2(A)  
  
print(A_inv)
```

Wynik:

```
[[3.0, -1.0], [-5.0, 2.0]]
```

16.2. Odwracanie macierzy przez dopełnienia

Niech A będzie nieosobliwą macierzą kwadratową, czyli:

$$\det(A) \neq 0$$

Dopełnienie algebraiczne elementu $a_{i,j}$:

$$A_{i,j} = (-1)^{i+j} M_{i,j}$$

gdzie $M_{i,j}$ to wyznacznik macierzy powstałej przez usunięcie i -tego wiersza i j -tej kolumny.

Macierz odwrotną można otrzymać ze wzoru:

$$A^{-1} = \frac{1}{\det(A)} (A^D)^T$$

gdzie:

- A^D — macierz dopełnień algebraicznych,
- $(A^D)^T$ — macierz dołączona.

```
inverse_cofactor(A):
    n = size(A)
    d = det(A)
    if d == 0: error("singular")
    C = zeros(n,n) # macierz dopełnień
    for i = 1..n:
        for j = 1..n:
            M = minor(A, row=i, col=j)
            C[i][j] = (-1)^(i+j) * det(M)
    Adj = transpose(C) # macierz dołączona
    return (1/d) * Adj
```

Przykład w Pythonie

PYTHON

```
def minor(A, wiersz_do_usuniecia, kolumna_do_usuniecia):
    M = []

    for i in range(len(A)):
        if i == wiersz_do_usuniecia:
            continue

        nowy_wiersz = []

        for j in range(len(A)):
            if j == kolumna_do_usuniecia:
                continue

            nowy_wiersz.append(A[i][j])

        M.append(nowy_wiersz)

    return M

def det_laplace(A):
    n = len(A)

    if n == 1:
        return A[0][0]

    if n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]

    suma = 0

    for j in range(n):
        suma += ((-1) ** j) * A[0][j] * det_laplace(minor(A, 0, j))

    return suma

def transponuj(A):
    AT = []

    for j in range(len(A[0])):
        wiersz = []
        for i in range(len(A)):
            wiersz.append(A[i][j])
        AT.append(wiersz)

    return AT

def inverse_cofactor(A):
```

```

n = len(A)
d = det_laplace(A)

if d == 0:
    raise ValueError("Macierz osobliwa")

C = []

for i in range(n):
    wiersz = []
    for j in range(n):
        M = minor(A, i, j)
        dopelnienie = ((-1) ** (i + j)) * det_laplace(M)
        wiersz.append(dopelnienie)
    C.append(wiersz)

Adj = transponuj(C)

A_inv = []

for i in range(n):
    wiersz = []
    for j in range(n):
        wiersz.append(Adj[i][j] / d)
    A_inv.append(wiersz)

return A_inv

A = [
    [2, 1],
    [5, 3]
]

print(inverse_cofactor(A))

```

Wynik:

```
[[3.0, -1.0], [-5.0, 2.0]]
```

Uwaga z wykładu: ta metoda jest w praktyce bardzo droga obliczeniowo, ponieważ wymaga liczenia wyznaczników minorów.

16.3. Zależności dla macierzy odwrotnej

W wykładzie podano następujące zależności:

$$(A^{-1})^{-1} = A$$

$$(kA)^{-1} = \frac{1}{k} A^{-1}$$

gdzie:

$$k \neq 0$$

$$(A^T)^{-1} = (A^{-1})^T$$

$$(AB)^{-1} = B^{-1}A^{-1}$$

$$\det(A^{-1}) = \frac{1}{\det(A)}$$

Przykład w Pythonie

```
def det2(A):
    return A[0][0] * A[1][1] - A[0][1] * A[1][0]

A = [
    [2, 1],
    [5, 3]
]

det_A = det2(A)

print("det(A) =", det_A)
print("det(A^-1) =", 1 / det_A)
```

PYTHON

Wynik:

```
det(A) = 1
det(A^-1) = 1.0
```

17. Metoda Gaussa i metoda Gaussa-Jordana

17.1. Eliminacja Gaussa

Eliminacja Gaussa polega na sprowadzeniu macierzy A do postaci trójkątnej górnej.

Następnie układ równań można rozwiązać przez podstawianie wsteczne.

W wykładzie podano też, że prowadzi to do rozkładu:

$$A = LU$$

Przykład w Pythonie

Poniżej sprowadzamy macierz do postaci trójkątnej górnej.

```

A = [
    [1.0, 2.0, 1.0],
    [2.0, 3.0, 1.0],
    [1.0, 1.0, 1.0]
]

n = len(A)

for k in range(n - 1):
    for i in range(k + 1, n):
        m = A[i][k] / A[k][k]

        for j in range(k, n):
            A[i][j] = A[i][j] - m * A[k][j]

print(A)

```

Wynik:

```
[[1.0, 2.0, 1.0], [0.0, -1.0, -1.0], [0.0, 0.0, 1.0]]
```

17.2. Metoda Gaussa-Jordana

Metoda Gaussa-Jordana sprowadza macierz do postaci jednostkowej.

Eliminowane są elementy zarówno poniżej, jak i powyżej pivota.

Można dzięki niej bezpośrednio wyznaczać:

- macierz odwrotną,
- wyznacznik.

Schemat z wykładu

Dla:

$$k = 1, \dots, n$$

1. wybieramy pivot,
2. normalizujemy wiersz:

$$R_k := \frac{1}{a_{k,k}} R_k$$

3. dla wszystkich:

$$i \neq k$$

wykonujemy:

$$R_i := R_i - a_{i,k}R_k$$

Efekt końcowy:

$$A \rightarrow I$$

Przykład w Pythonie

```
PYTHON

A = [
    [2.0, 1.0],
    [5.0, 3.0]
]

n = len(A)

for k in range(n):
    pivot = A[k][k]

    # Normalizacja wiersza z pivotem
    for j in range(n):
        A[k][j] = A[k][j] / pivot

    # Zerowanie pozostałych elementów w kolumnie k
    for i in range(n):
        if i != k:
            wspolczynnik = A[i][k]
            for j in range(n):
                A[i][j] = A[i][j] - wspolczynnik * A[k][j]

print(A)
```

Wynik:

```
[[1.0, 0.0], [0.0, 1.0]]
```

18. Operacje elementarne a wyznacznik w metodzie Gaussa

Dopuszczalne operacje wierszowe:

1. zamiana wierszy:

$$R_i \leftrightarrow R_j$$

2. mnożenie wiersza przez skalar:

λ

3. dodanie wielokrotności wiersza do innego wiersza.

Wpływ na wyznacznik:

- zamiana wierszy zmienia znak wyznacznika,
- mnożenie wiersza przez λ mnoży wyznacznik przez λ ,
- dodanie wielokrotności wiersza nie zmienia wyznacznika.

Po sprowadzeniu do postaci trójkątnej:

$$\det(A) = (-1)^p \prod_{i=1}^n u_{i,i}$$

gdzie p to liczba zamian wierszy.

Przykład z wykładu

Macierz:

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

po eliminacji trójkątnej:

$$U = \begin{bmatrix} 1 & 2 & 1 \\ 0 & -1 & -1 \\ 0 & 0 & 1 \end{bmatrix}$$

Wyznacznik:

$$\det(A) = 1 \cdot (-1) \cdot 1 = -1$$


```
A = [  
    [1.0, 2.0, 1.0],  
    [2.0, 3.0, 1.0],  
    [1.0, 1.0, 1.0]  
]  
  
n = len(A)  
  
for k in range(n - 1):  
    for i in range(k + 1, n):  
        m = A[i][k] / A[k][k]  
  
        for j in range(k, n):  
            A[i][j] = A[i][j] - m * A[k][j]  
  
det = 1  
  
for i in range(n):  
    det *= A[i][i]  
  
print("Macierz U:")  
print(A)  
print("det(A) =", det)
```

Wynik:

```
Macierz U:  
[[1.0, 2.0, 1.0], [0.0, -1.0, -1.0], [0.0, 0.0, 1.0]]  
det(A) = -1.0
```

19. Eliminacja Gaussa bez pivotingu

Dla:

$$k = 1, \dots, n - 1$$

wykonuje się kroki:

1. przyjmujemy pivot:

$$a_{k,k}$$

2. dla:

$$i = k + 1, \dots, n$$

liczymy:

$$m_{i,k} = \frac{a_{i,k}}{a_{k,k}}$$

3. aktualizujemy wiersze:

$$R_i := R_i - m_{i,k}R_k$$

Wymaganie:

$$a_{k,k} \neq 0$$

Problem: metoda może być niestabilna numerycznie przy małych pivotach.

Przykład w Pythonie

```
PYTHON
A = [
    [2.0, 1.0],
    [4.0, 3.0]
]

n = len(A)

for k in range(n - 1):
    if A[k][k] == 0:
        raise ValueError("Pivot jest równy zero")

    for i in range(k + 1, n):
        m = A[i][k] / A[k][k]

        for j in range(k, n):
            A[i][j] = A[i][j] - m * A[k][j]

print(A)
```

Wynik:

```
[[2.0, 1.0], [0.0, 1.0]]
```

20. Eliminacja Gaussa z pivotingiem częściowym

W pivotingu częściowym dla każdej kolumny **k** wybiera się wiersz **p**, taki że:

$$|a_{p,k}| = \max_{i=k, \dots, n} |a_{i,k}|$$

Następnie zamienia się wiersze:

$$R_k \leftrightarrow R_p$$

i wykonuje standardową eliminację.

Zalety pivotingu

- poprawia stabilność numeryczną,
- ogranicza propagację błędów zaokrągleń.

Przykład w Pythonie

```
PYTHON

A = [
    [0.0001, 1.0],
    [1.0, 1.0]
]

n = len(A)

for k in range(n - 1):
    # Wybór wiersza z największym elementem w kolumnie k
    p = k

    for i in range(k + 1, n):
        if abs(A[i][k]) > abs(A[p][k]):
            p = i

    # Zamiana wierszy
    A[k], A[p] = A[p], A[k]

    # Eliminacja
    for i in range(k + 1, n):
        m = A[i][k] / A[k][k]

        for j in range(k, n):
            A[i][j] = A[i][j] - m * A[k][j]

print(A)
```

Wynik:

```
[[1.0, 1.0], [0.0, 0.9999]]
```

21. Metoda Gaussa-Jordana – schemat postępowania

Metoda Gaussa-Jordana sprowadza macierz do postaci jednostkowej.

Dla:

$$k = 1, \dots, n$$

wykonuje się następujące kroki:

1. Wybór pivota, opcjonalnie z pivotingiem.
2. Normalizacja wiersza:

$$R_k := \frac{1}{a_{k,k}} R_k$$

czyli cały wiersz z pivotem dzielimy przez wartość pivota, aby na diagonalu otrzymać 1.

3. Dla wszystkich:

$$i \neq k$$

wykonujemy:

$$R_i := R_i - a_{i,k} R_k$$

czyli zerujemy wszystkie pozostałe elementy w kolumnie pivota.

Efekt końcowy:

$$A \rightarrow I$$

czyli macierz A zostaje sprowadzona do macierzy jednostkowej.

Przykład

Weźmy macierz:

$$A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$$

Po zastosowaniu metody Gaussa-Jordana macierz zostanie sprowadzona do postaci:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [
    [2.0, 1.0],
    [5.0, 3.0]
]

n = len(A)

for k in range(n):
    # 1. Wybór pivota
    pivot = A[k][k]

    if pivot == 0:
        raise ValueError("Pivot jest równy zero")

    # 2. Normalizacja wiersza:
    #  $R_k := (1 / a_{kk}) * R_k$ 
    for j in range(n):
        A[k][j] = A[k][j] / pivot

    # 3. Zerowanie pozostałych elementów w kolumnie k
    #  $R_i := R_i - a_{ik} * R_k$ 
    for i in range(n):
        if i != k:
            współczynnik = A[i][k]

            for j in range(n):
                A[i][j] = A[i][j] - współczynnik * A[k][j]

print(A)
```

Wynik:

```
[[1.0, 0.0], [0.0, 1.0]]
```

Wersja z pivotingiem częściowym

Pivoting częściowy polega na tym, że w kolumnie **k** szukamy wiersza z największą wartością bezwzględną elementu w tej kolumnie, a potem zamieniamy ten wiersz z wierszem **k**.

```
A = [  
    [0.0001, 1.0],  
    [1.0, 1.0]  
]  
  
n = len(A)  
  
for k in range(n):  
    # Wybór pivotu z pivotingiem częściowym  
    p = k  
    największy = abs(A[k][k])  
  
    for i in range(k + 1, n):  
        if abs(A[i][k]) > największy:  
            największy = abs(A[i][k])  
            p = i  
  
    # Zamiana wierszy, jeśli znaleziono lepszy pivot  
    if p != k:  
        A[k], A[p] = A[p], A[k]  
  
    pivot = A[k][k]  
  
    if pivot == 0:  
        raise ValueError("Pivot jest równy zero")  
  
    # Normalizacja wiersza z pivotem  
    for j in range(n):  
        A[k][j] = A[k][j] / pivot  
  
    # Zerowanie pozostałych elementów w kolumnie k  
    for i in range(n):  
        if i != k:  
            współczynnik = A[i][k]  
  
            for j in range(n):  
                A[i][j] = A[i][j] - współczynnik * A[k][j]  
  
print(A)
```

Wynik:

```
[[1.0, 0.0], [0.0, 1.0]]
```

Ważne

Metoda Gaussa-Jordana eliminuje elementy zarówno poniżej, jak i powyżej pivotu. Dzięki temu na końcu otrzymujemy bezpośrednio macierz jednostkową.

Przy wyznaczaniu macierzy odwrotnej stosuje się macierz rozszerzoną:

$$[A \mid I]$$

i wykonuje przekształcenia:

$$[A \mid I] \rightarrow [I \mid A^{-1}]$$

22. Wyznaczanie macierzy odwrotnej metodą Gaussa-Jordana

Tworzymy macierz rozszerzoną:

$$[A \mid I]$$

Następnie stosujemy metodę Gaussa-Jordana:

$$[A \mid I] \rightarrow [I \mid A^{-1}]$$

Warunek:

$$\det(A) \neq 0$$

Złożoność obliczeniowa:

$$O(n^3)$$

Przykład z wykładu

Dla:

$$A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$$

tworzymy macierz rozszerzoną:

$$\begin{bmatrix} 2 & 1 & 1 & 0 \\ 5 & 3 & 0 & 1 \end{bmatrix}$$

Po eliminacji:

$$\begin{bmatrix} 1 & 0 & 3 & -1 \\ 0 & 1 & -5 & 2 \end{bmatrix}$$

czyli:

$$A^{-1} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [
    [2.0, 1.0],
    [5.0, 3.0]
]

n = len(A)

# Tworzymy macierz rozszerzoną [A | I]
AI = []

for i in range(n):
    wiersz = []

    for j in range(n):
        wiersz.append(A[i][j])

    for j in range(n):
        if i == j:
            wiersz.append(1.0)
        else:
            wiersz.append(0.0)

    AI.append(wiersz)

# Gauss-Jordan
for k in range(n):
    pivot = AI[k][k]

    if pivot == 0:
        raise ValueError("Pivot równy zero")

    # Normalizacja wiersza
    for j in range(2 * n):
        AI[k][j] = AI[k][j] / pivot

    # Zerowanie kolumny
    for i in range(n):
        if i != k:
            wspolczynnik = AI[i][k]

            for j in range(2 * n):
                AI[i][j] = AI[i][j] - wspolczynnik * AI[k][j]

# Odczytujemy prawą część jako A^-1
A_inv = []

for i in range(n):
    A_inv.append(AI[i][n:])
```



```
print(A_inv)
```

Wynik:

```
[[3.0, -1.0], [-5.0, 2.0]]
```

23. Uwagi praktyczne z wykładu

W wykładzie podano kilka ważnych uwag praktycznych:

1. Metoda Gaussa-Jordana jest rzadko używana do rozwiązywania układów równań, bo jest mniej efektywna niż metoda oparta o rozkład LU.
2. Metoda Gaussa-Jordana jest bardzo wygodna do:
 - obliczania macierzy odwrotnej,
 - analizy rzędu macierzy.
3. W praktyce zawsze stosuje się pivoting.

Przykład w Pythonie

Poniższy fragment pokazuje sam wybór pivota, czyli najważniejszy element pivotingu częściowego:

```
PYTHON

A = [
    [0.001, 2.0],
    [5.0, 3.0]
]

k = 0
p = k

for i in range(k + 1, len(A)):
    if abs(A[i][k]) > abs(A[p][k]):
        p = i

print("Najlepszy wiersz na pivot:", p)
print("Wartość pivota:", A[p][k])
```

Wynik:

Najlepszy wiersz na pivot: 1
Wartość pivota: 5.0

24. Najważniejsze rzeczy do zapamiętania na kolosa

24.1. Macierz

Macierz to prostokątna tablica liczb o wymiarze:

$$m \times n$$

Python

```
A = [  
    [1, 2],  
    [3, 4]  
]  
  
print(len(A), "x", len(A[0]))
```

PYTHON

24.2. Element macierzy

Element:

$$a_{i,j}$$

leży w **i**-tym wierszu i **j**-tej kolumnie.

Python

```
A = [  
    [1, 2],  
    [3, 4]  
]  
  
print(A[1][0])
```

PYTHON

Wynik:

3

24.3. Dodawanie macierzy

Macierze można dodać tylko wtedy, gdy mają takie same wymiary.

$$A + B = [a_{i,j} + b_{i,j}]$$

Python

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

C = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]

print(C)
```

PYTHON

24.4. Transpozycja

Transpozycja zamienia wiersze na kolumny.

$$(A^T)^T = A$$

Python

```
A = [[1, 2, 3], [4, 5, 6]]

AT = [[A[i][j] for i in range(len(A))] for j in range(len(A[0]))]

print(AT)
```

PYTHON

24.5. Mnożenie macierzy

Iloczyn AB istnieje, gdy liczba kolumn A jest równa liczbie wierszy B .

$$c_{i,j} = \sum_{l=1}^n a_{i,l} b_{l,j}$$

Python

PYTHON

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

C = []

for i in range(len(A)):
    wiersz = []
    for j in range(len(B[0])):
        suma = 0
        for k in range(len(B)):
            suma += A[i][k] * B[k][j]
        wiersz.append(suma)
    C.append(wiersz)

print(C)
```

24.6. Mnożenie macierzy nie jest przemienne

Zwykle:

$$AB \neq BA$$

Python

PYTHON

```
A = [[1, 2], [0, 1]]
B = [[1, 0], [3, 1]]

def matmul(A, B):
    return [
        [
            sum(A[i][k] * B[k][j] for k in range(len(B)))
            for j in range(len(B[0]))
        ]
        for i in range(len(A))
    ]

print(matmul(A, B))
print(matmul(B, A))
```

24.7. Wyznacznik macierzy 2 × 2

Dla:

$$A = \begin{bmatrix} a & b & c & d \end{bmatrix}$$

mamy:

$$\det(A) = ad - bc$$

Python

```
A = [[2, 1], [5, 3]]

det = A[0][0] * A[1][1] - A[0][1] * A[1][0]

print(det)
```

PYTHON

24.8. Macierz odwrotna

Macierz odwrotna istnieje wtedy, gdy:

$$\det(A) \neq 0$$

oraz spełnia:

$$AA^{-1} = A^{-1}A = I$$

Python

```
A = [[2, 1], [5, 3]]

det = A[0][0] * A[1][1] - A[0][1] * A[1][0]

if det != 0:
    print("Macierz odwrotna istnieje")
else:
    print("Macierz odwrotna nie istnieje")
```

PYTHON

24.9. Gauss bez pivotingu

W eliminacji Gaussa używamy pivota:

$$a_{k,k}$$

i mnożnika:

$$m_{i,k} = \frac{a_{i,k}}{a_{k,k}}$$

Potem wykonujemy:

$$R_i := R_i - m_{i,k} R_k$$

Python

```
A = [[2.0, 1.0], [4.0, 3.0]]

m = A[1][0] / A[0][0]

for j in range(2):
    A[1][j] = A[1][j] - m * A[0][j]

print(A)
```

PYTHON

24.10. Pivoting częściowy

Wybieramy największy element w danej kolumnie jako pivot:

$$|a_{p,k}| = \max_{i=k, \dots, n} |a_{i,k}|$$

Python

```
A = [[0.001, 2.0], [5.0, 3.0]]

k = 0
p = k

for i in range(k + 1, len(A)):
    if abs(A[i][k]) > abs(A[p][k]):
        p = i

A[k], A[p] = A[p], A[k]

print(A)
```

PYTHON

Wykład 3: Układy równań liniowych — metody bezpośrednie (lab 4)

1. Układ równań liniowych

Układ m równań liniowych z n niewiadomymi ma postać:

$$a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + \dots + a_{1,n}x_n = b_1$$

$$a_{2,1}x_1 + a_{2,2}x_2 + a_{2,3}x_3 + \dots + a_{2,n}x_n = b_2$$

...

$$a_{m,1}x_1 + a_{m,2}x_2 + a_{m,3}x_3 + \dots + a_{m,n}x_n = b_m$$

Można go też zapisać krócej:

$$\sum_{j=1}^n a_{i,j}x_j = b_i$$

dla:

$$i = 1, 2, \dots, m$$

Przykład

Układ:

$$2x_1 + x_2 = 5$$

$$x_1 + 3x_2 = 7$$

ma 2 równania i 2 niewiadome.

Przykład w Pythonie

PYTHON

```
# Przykładowy układ:  
# 2x1 + 1x2 = 5  
# 1x1 + 3x2 = 7  
  
A = [  
    [2, 1],  
    [1, 3]  
]  
  
b = [5, 7]  
  
print("Macierz współczynników A:")  
print(A)  
  
print("Wektor wyrazów wolnych b:")  
print(b)
```

2. Zapis macierzowy układu równań

Układ równań liniowych można zapisać w postaci macierzowej:

$$Ax = b$$

gdzie:

- **A** — macierz główna układu, złożona ze współczynników,

- $\mathbf{x}$ — wektor niewiadomych,
- $\mathbf{b}$ — wektor wyrazów wolnych.

Macierz współczynników:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix}$$

Wektor niewiadomych:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Wektor wyrazów wolnych:

$$\mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Cały układ:

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [  
    [2, 1],  
    [1, 3]  
]  
  
x = [1, 2]  
  
b = []  
  
for i in range(len(A)):  
    suma = 0  
  
    for j in range(len(A[0])):  
        suma = suma + A[i][j] * x[j]  
  
    b.append(suma)  
  
print("Dla x =", x)  
print("Ax =", b)
```

Wynik:

```
Dla x = [1, 2]  
Ax = [4, 7]
```

3. Macierz rozszerzona

Macierz rozszerzona powstaje przez dołączenie do macierzy głównej **A** wektora wyrazów wolnych **b** jako ostatniej kolumny.

Oznaczamy ją jako:

$$A_b$$

Macierz rozszerzona ma rozmiar:

$$m \times (n + 1)$$

Jeżeli:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix}$$

oraz:

$$b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

to:

$$A_b = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} & b_1 \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} & b_m \end{bmatrix}$$

Przykład

Dla:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

oraz:

$$b = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

macierz rozszerzona to:

$$A_b = \begin{bmatrix} 2 & 1 & 5 \\ 1 & 3 & 7 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [
    [2, 1],
    [1, 3]
]

b = [5, 7]

Ab = []

for i in range(len(A)):
    wiersz = []

    for j in range(len(A[0])):
        wiersz.append(A[i][j])

    wiersz.append(b[i])

    Ab.append(wiersz)

print(Ab)
```

Wynik:

```
[[2, 1, 5], [1, 3, 7]]
```

4. Rząd macierzy

Macierz można traktować jako zbiór wektorów kolumnowych albo wierszowych.

Największa liczba niezależnych liniowo wektorów kolumnowych w macierzy A jest równa największej liczbie niezależnych liniowo wektorów wierszowych w A .

Ta liczba nazywa się **rzędem macierzy**.

Oznaczenie:

$$\text{rank}(A) = r$$

Przykład

Jeżeli jeden wiersz macierzy jest wielokrotnością innego wiersza, to wiersze są liniowo zależne.

Dla macierzy:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

drugi wiersz jest dwa razy większy od pierwszego, więc rząd nie wynosi 2.

Przykład w Pythonie

Prosty przykład sprawdzający zależność dwóch wierszy w macierzy 2 x 2:

```
PYTHON
A = [
    [1, 2],
    [2, 4]
]

if A[0][0] != 0:
    wspolczynnik = A[1][0] / A[0][0]

    zalezne = True

    for j in range(len(A[0])):
        if A[1][j] != wspolczynnik * A[0][j]:
            zalezne = False

    if zalezne:
        print("Wiersze są liniowo zależne")
    else:
        print("Wiersze nie są liniowo zależne")
else:
    print("Nie sprawdzam tym sposobem, bo pierwszy element jest równy 0")
```

5. Twierdzenie Kroneckera-Capellego

Twierdzenie Kroneckera-Capellego pozwala określić liczbę rozwiązań układu równań liniowych na podstawie rzędów macierzy.

5.1. Układ oznaczony

Jeżeli:

$$\text{rank}(A) = \text{rank}(A_b) = n$$

to układ ma dokładnie jedno rozwiązanie.

Taki układ nazywa się **oznaczony**.

5.2. Układ nieoznaczony

Jeżeli:

$$\text{rank}(A) = \text{rank}(A_b) < n$$

to układ ma nieskończenie wiele rozwiązań.

Taki układ nazywa się **nieoznaczony**.

5.3. Układ sprzeczny

Jeżeli:

$$\text{rank}(A) < \text{rank}(A_b)$$

to układ nie ma rozwiązań.

Taki układ nazywa się **sprzeczny**.

Przykład w Pythonie

```
rank_A = 2
rank_Ab = 2
n = 2

if rank_A == rank_Ab and rank_A == n:
    print("Układ oznaczony – dokładnie jedno rozwiązanie")
elif rank_A == rank_Ab and rank_A < n:
    print("Układ nieoznaczony – nieskończenie wiele rozwiązań")
elif rank_A < rank_Ab:
    print("Układ sprzeczny – brak rozwiązań")
```

PYTHON

6. Typy układów równań liniowych

6.1. Układ jednorodny

Jeżeli wszystkie wyrazy wolne są równe 0, to układ nazywa się **jednorodnym**.

Taki układ ma zawsze rozwiązanie.

Czyli:

$$b = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
b = [0, 0, 0]

czy_jednorodny = True

for i in range(len(b)):
    if b[i] != 0:
        czy_jednorodny = False

if czy_jednorodny:
    print("Układ jest jednorodny")
else:
    print("Układ nie jest jednorodny")
```

6.2. Układ kwadratowy

Jeżeli liczba wierszy jest równa liczbie kolumn w macierzy głównej, czyli:

$$m = n$$

to układ nazywa się **kwadratowym**.

Przykład w Pythonie

PYTHON

```
A = [
    [2, 1],
    [1, 3]
]

m = len(A)
n = len(A[0])

if m == n:
    print("Układ jest kwadratowy")
else:
    print("Układ nie jest kwadratowy")
```

7. Metody rozwiązywania układów równań liniowych

W wykładzie podano dwa główne typy metod:

1. metody bezpośrednie,
2. metody iteracyjne.

7.1. Metody bezpośrednie

Metody bezpośrednie dają dokładne rozwiązanie po skończonej liczbie przekształceń układu wejściowego, pomijając błędy zaokrągleń.

Cechy metod bezpośrednich:

- są efektywne dla układów o macierzach pełnych,
- mocno obciążają pamięć,
- ze względu na błędy zaokrągleń mogą być niestabilne.

Do metod bezpośrednich należą między innymi:

- użycie macierzy odwrotnej,
- wzory Cramera,
- układ równań z macierzą trójkątną,
- metoda eliminacji Gaussa,
- rozkłady trójkątne macierzy,
- metoda Gaussa-Jordana,
- metoda Doolittle'a,
- metoda Crouta,
- metoda Cholesky'ego.

7.2. Metody iteracyjne

Metody iteracyjne tworzą ciąg wektorów zbieżny do szukanego rozwiązania.

Cechy metod iteracyjnych:

- liczba kroków nie jest z góry znana,
- dobrze sprawdzają się dla macierzy rzadkich o dużych rozmiarach,
- obciążenie pamięci nie jest zbyt duże,
- mogą wystąpić problemy ze zbieżnością rozwiązania.

Do metod iteracyjnych należą między innymi:

- metoda Jacobiego,
 - metoda Gaussa-Seidela,
 - metoda Czebyszewa.
-

8. Macierze pełne i rzadkie

W praktyce macierze współczynników dzielą się na dwie grupy.

8.1. Macierze pełne

Macierze pełne mają dużo elementów niezerowych.

W wykładzie podano, że nieduże macierze pełne mogą mieć stopień mniejszy np. od 30.

Przykład

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Przykład w Pythonie

```
PYTHON
A = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

liczba_niezerowych = 0

for i in range(len(A)):
    for j in range(len(A[0])):
        if A[i][j] != 0:
            liczba_niezerowych = liczba_niezerowych + 1

print("Liczba elementów niezerowych:", liczba_niezerowych)
```

8.2. Macierze rzadkie

Macierze rzadkie mają mało elementów niezerowych.

W wykładzie podano, że takie macierze często są bardzo duże, np. stopnia 100 lub większego, a elementy niezerowe mogą leżeć blisko głównej diagonal.

Przykład

$$A = \begin{bmatrix} 4 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 0 & 1 & 4 & 1 \\ 0 & 0 & 1 & 4 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [  
    [4, 1, 0, 0],  
    [1, 4, 1, 0],  
    [0, 1, 4, 1],  
    [0, 0, 1, 4]  
]  
  
liczba_zer = 0  
liczba_niezerowych = 0  
  
for i in range(len(A)):  
    for j in range(len(A[0])):  
        if A[i][j] == 0:  
            liczba_zer = liczba_zer + 1  
        else:  
            liczba_niezerowych = liczba_niezerowych + 1  
  
print("Zera:", liczba_zer)  
print("Elementy niezerowe:", liczba_niezerowych)
```

9. Rozwiązywanie układu za pomocą macierzy odwrotnej

Dla macierzy odwrotnej zachodzi:

$$AA^{-1} = A^{-1}A = I$$

Układ:

$$Ax = b$$

można rozwiązać, jeśli znamy macierz odwrotną:

$$A^{-1}Ax = A^{-1}b$$

czyli:

$$Ix = A^{-1}b$$

ostatecznie:

$$x = A^{-1}b$$

Przykład

Dla:

$$A^{-1} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$$

oraz:

$$b = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

liczymy:

$$x = A^{-1}b$$

Przykład w Pythonie

PYTHON

```
A_inv = [  
    [3, -1],  
    [-5, 2]  
]  
  
b = [5, 7]  
  
x = []  
  
for i in range(len(A_inv)):  
    suma = 0  
  
    for j in range(len(A_inv[0])):  
        suma = suma + A_inv[i][j] * b[j]  
  
    x.append(suma)  
  
print(x)
```

Wynik:

```
[8, -11]
```

10. Wzory Cramera

Wzory Cramera są metodą bezpośrednią rozwiązywania układów równań liniowych.

Jeżeli:

$$\det(A) \neq 0$$

to:

$$x_k = \frac{\det(A_k)}{\det(A)}$$

dla:

$$k = 1, 2, \dots, n$$

gdzie A_k jest macierzą powstałą przez zastąpienie k -tej kolumny macierzy A wektorem wyrazów wolnych b .

Przykład dla układu 2×2

Układ:

$$2x_1 + x_2 = 5$$

$$x_1 + 3x_2 = 7$$

Macierz główna:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

Wektor wyrazów wolnych:

$$b = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

Macierz A_1 :

$$A_1 = \begin{bmatrix} 5 & 1 \\ 7 & 3 \end{bmatrix}$$

Macierz A_2 :

$$A_2 = \begin{bmatrix} 2 & 5 \\ 1 & 7 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
def det2(A):  
    return A[0][0] * A[1][1] - A[0][1] * A[1][0]  
  
A = [  
    [2, 1],  
    [1, 3]  
]  
  
b = [5, 7]  
  
A1 = [  
    [b[0], A[0][1]],  
    [b[1], A[1][1]]  
]  
  
A2 = [  
    [A[0][0], b[0]],  
    [A[1][0], b[1]]  
]  
  
det_A = det2(A)  
det_A1 = det2(A1)  
det_A2 = det2(A2)  
  
if det_A != 0:  
    x1 = det_A1 / det_A  
    x2 = det_A2 / det_A  
  
    print("x1 =", x1)  
    print("x2 =", x2)  
else:  
    print("Nie można użyć wzorów Cramera, bo det(A) = 0")
```

Wynik:

```
x1 = 1.6  
x2 = 1.8
```

11. Wzory Cramera — przykład 3×3

Dla układu:

$$a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 = b_1$$

$$a_{2,1}x_1 + a_{2,2}x_2 + a_{2,3}x_3 = b_2$$

$$a_{3,1}x_1 + a_{3,2}x_2 + a_{3,3}x_3 = b_3$$

obliczamy:

$$\det(A) = \det \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix}$$

Następnie:

$$x_1 = \frac{1}{\det(A)} \det \begin{bmatrix} b_1 & a_{1,2} & a_{1,3} \\ b_2 & a_{2,2} & a_{2,3} \\ b_3 & a_{3,2} & a_{3,3} \end{bmatrix} \frac{\det(A_1)}{\det(A)}$$

$$x_2 = \frac{1}{\det(A)} \det \begin{bmatrix} a_{1,1} & b_1 & a_{1,3} \\ a_{2,1} & b_2 & a_{2,3} \\ a_{3,1} & b_3 & a_{3,3} \end{bmatrix} \frac{\det(A_2)}{\det(A)}$$

$$x_3 = \frac{1}{\det(A)} \det \begin{bmatrix} a_{1,1} & a_{1,2} & b_1 \\ a_{2,1} & a_{2,2} & b_2 \\ a_{3,1} & a_{3,2} & b_3 \end{bmatrix} \frac{\det(A_3)}{\det(A)}$$

Przykład w Pythonie

PYTHON

```
def det3(A):
    wynik = (
        A[0][0] * A[1][1] * A[2][2]
        + A[0][1] * A[1][2] * A[2][0]
        + A[0][2] * A[1][0] * A[2][1]
        - A[0][2] * A[1][1] * A[2][0]
        - A[0][1] * A[1][0] * A[2][2]
        - A[0][0] * A[1][2] * A[2][1]
    )

    return wynik
```

```
A = [
    [1, 1, 1],
    [2, 1, 3],
    [1, -1, 1]
]
```

```
b = [6, 13, 2]
```

```
A1 = [
    [b[0], A[0][1], A[0][2]],
    [b[1], A[1][1], A[1][2]],
    [b[2], A[2][1], A[2][2]]
]
```

```
A2 = [
    [A[0][0], b[0], A[0][2]],
    [A[1][0], b[1], A[1][2]],
    [A[2][0], b[2], A[2][2]]
]
```

```
A3 = [
    [A[0][0], A[0][1], b[0]],
    [A[1][0], A[1][1], b[1]],
    [A[2][0], A[2][1], b[2]]
]
```

```
det_A = det3(A)
```

```
if det_A != 0:
    x1 = det3(A1) / det_A
    x2 = det3(A2) / det_A
    x3 = det3(A3) / det_A
```

```
print("x1 =", x1)
print("x2 =", x2)
print("x3 =", x3)
```

```
else:  
    print("Nie można użyć wzorów Cramera")
```

12. Układ z macierzą trójkątną górną

Jeżeli macierz układu równań liniowych jest macierzą trójkątną, to układ rozwiązuje się szczególnie łatwo.

Dla macierzy trójkątnej górnej układ ma postać:

$$a_{1,1}x_1 + a_{1,2}x_2 + \dots + a_{1,n}x_n = b_1$$

$$a_{2,2}x_2 + \dots + a_{2,n}x_n = b_2$$

...

$$a_{n,n}x_n = b_n$$

Aby istniało jednoznaczne rozwiązanie, wszystkie elementy na głównej przekątnej muszą być różne od zera.

Rozwiązanie zaczynamy od ostatniej niewiadomej:

$$x_n = \frac{b_n}{a_{n,n}}$$

Następnie:

$$x_i = \frac{b_i - \sum_{k=i+1}^n a_{i,k}x_k}{a_{i,i}}$$

dla:

$$i = n - 1, n - 2, \dots, 1$$

Metoda ta nazywa się **podstawianiem w tył**.

Przykład

Układ:

$$2x_1 + x_2 - x_3 = 1$$

$$3x_2 + 2x_3 = 12$$

$$4x_3 = 8$$

Z ostatniego równania:

$$x_3 = 2$$

Przykład w Pythonie

PYTHON

```
U = [  
    [2.0, 1.0, -1.0],  
    [0.0, 3.0, 2.0],  
    [0.0, 0.0, 4.0]  
]  
  
b = [1.0, 12.0, 8.0]  
  
n = len(U)  
x = [0.0, 0.0, 0.0]  
  
for i in range(n - 1, -1, -1):  
    suma = 0  
  
    for k in range(i + 1, n):  
        suma = suma + U[i][k] * x[k]  
  
    x[i] = (b[i] - suma) / U[i][i]  
  
print(x)
```

13. Układ z macierzą trójkątną dolną

Dla macierzy trójkątnej dolnej wykonuje się **podstawianie w przód**.

Najpierw liczymy:

$$x_1 = \frac{b_1}{a_{1,1}}$$

Następnie:

$$x_i = \frac{b_i - \sum_{k=1}^{i-1} a_{i,k} x_k}{a_{i,i}}$$

dla:

$$i = 2, 3, \dots, n$$

Przykład

Układ:

$$2x_1 = 4$$

$$3x_1 + x_2 = 7$$

$$x_1 - x_2 + 2x_3 = 3$$


```
L = [  
    [2.0, 0.0, 0.0],  
    [3.0, 1.0, 0.0],  
    [1.0, -1.0, 2.0]  
]  
  
b = [4.0, 7.0, 3.0]  
  
n = len(L)  
x = [0.0, 0.0, 0.0]  
  
for i in range(n):  
    suma = 0  
  
    for k in range(0, i):  
        suma = suma + L[i][k] * x[k]  
  
    x[i] = (b[i] - suma) / L[i][i]  
  
print(x)
```

14. Eliminacja Gaussa

Eliminacja Gaussa polega na sprowadzeniu układu równań do postaci trójkątnej, a następnie rozwiązaniu go przez podstawianie wstecz.

Dla układu 3×3 :

$$a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 = b_1$$

$$a_{2,1}x_1 + a_{2,2}x_2 + a_{2,3}x_3 = b_2$$

$$a_{3,1}x_1 + a_{3,2}x_2 + a_{3,3}x_3 = b_3$$

celem jest wyzerowanie elementów pod główną przekątną.

Idea kroku eliminacji

W każdym kroku odejmujemy od jednego wiersza odpowiednią wielokrotność innego wiersza.

Dla pierwszej kolumny:

$$R_i := R_i - mR_1$$

gdzie:

$$m = \frac{a_{i,1}}{a_{1,1}}$$

Przykład w Pythonie

PYTHON

```
A = [
    [2.0, 1.0, -1.0],
    [-3.0, -1.0, 2.0],
    [-2.0, 1.0, 2.0]
]

b = [8.0, -11.0, -3.0]

n = len(A)

# Eliminacja Gaussa
for k in range(n - 1):
    for i in range(k + 1, n):
        m = A[i][k] / A[k][k]

        for j in range(k, n):
            A[i][j] = A[i][j] - m * A[k][j]

        b[i] = b[i] - m * b[k]

print("Macierz po eliminacji:")
print(A)

print("Wektor b po eliminacji:")
print(b)
```

15. Wzory ogólne w eliminacji Gaussa

Współczynniki macierzy i wyrazy wolne w każdym kroku eliminacji można obliczać ze wzorów:

$$a_{i,j}^{(k)} = a_{i,j}^{(k-1)} \frac{a_{i,k}^{(k-1)} a_{k,j}^{(k-1)}}{a_{k,k}^{(k-1)}}$$

oraz:

$$b_i^{(k)} = b_i^{(k-1)} \frac{a_{i,k}^{(k-1)} b_k^{(k-1)}}{a_{k,k}^{(k-1)}}$$

dla:

$$i, j = k + 1, k + 2, \dots, n$$

Po otrzymaniu układu trójkątnego rozwiązujemy go przez podstawianie wstecz:

$$x_i = \frac{b_i^{(i-1)} - \sum_{j=i+1}^n a_{i,j}^{(i-1)} x_j}{a_{i,i}^{(i-1)}}$$

dla:

$$i = n, n-1, \dots, 1$$

Przykład w Pythonie

PYTHON

```
def eliminacja_gaussa(A, b):
    n = len(A)

    for k in range(n - 1):
        for i in range(k + 1, n):
            m = A[i][k] / A[k][k]

            for j in range(k, n):
                A[i][j] = A[i][j] - m * A[k][j]

            b[i] = b[i] - m * b[k]

    x = [0.0 for i in range(n)]

    for i in range(n - 1, -1, -1):
        suma = 0

        for j in range(i + 1, n):
            suma = suma + A[i][j] * x[j]

        x[i] = (b[i] - suma) / A[i][i]

    return x

A = [
    [2.0, 1.0, -1.0],
    [-3.0, -1.0, 2.0],
    [-2.0, 1.0, 2.0]
]

b = [8.0, -11.0, -3.0]

x = eliminacja_gaussa(A, b)

print(x)
```

16. Wybór elementu podstawowego

Eliminacja Gaussa w podstawowej formie nie jest niezawodna.

Jeżeli element podstawowy, czyli pivot, jest równy zero, algorytm wymagałby dzielenia przez zero.

Element podstawowy to element macierzy, za pomocą którego dokonujemy eliminacji zmiennej z dalszych równań.

Jeżeli:

$$a_{k,k}^{(k)} = 0$$

należy zamienić wiersze, aby uzyskać niezerowy pivot.

W praktyce często wybiera się wiersz, w którym element w danej kolumnie ma największą wartość bezwzględną.

Przykład

Macierz rozszerzona:

$$\left[\begin{array}{ccc|c} 0 & 2 & 2 & 1 \\ 3 & 3 & 0 & 3 \\ 1 & 0 & 1 & 2 \end{array} \right]$$

Tu pierwszy pivot byłby równy:

$$a_{1,1} = 0$$

więc trzeba zamienić wiersze.

Przykład w Pythonie

PYTHON

```
Ab = [  
    [0.0, 2.0, 2.0, 1.0],  
    [3.0, 3.0, 0.0, 3.0],  
    [1.0, 0.0, 1.0, 2.0]  
]  
  
k = 0  
p = k  
najwiekszy = abs(Ab[k][k])  
  
for i in range(k + 1, len(Ab)):  
    if abs(Ab[i][k]) > najwiekszy:  
        najwiekszy = abs(Ab[i][k])  
        p = i  
  
Ab[k], Ab[p] = Ab[p], Ab[k]  
  
print(Ab)
```

Wynik:

```
[[3.0, 3.0, 0.0, 3.0], [0.0, 2.0, 2.0, 1.0], [1.0, 0.0, 1.0, 2.0]]
```

17. Częściowy wybór elementu głównego

Częściowy wybór elementu głównego polega na tym, że w i -tym kroku eliminacji Gaussa patrzymy na elementy w i -tej kolumnie i wybieramy wiersz z największą wartością bezwzględną.

Po zamianie wierszy można wykonać kolejny krok eliminacji.

W wykładzie po zamianie wierszy otrzymano macierz:

$$\left[\begin{array}{ccc|c} 3 & 3 & 0 & 3 \\ 0 & 2 & 2 & 1 \\ 0 & -1 & 1 & 1 \end{array} \right]$$

Po kolejnym kroku:

$$\left[\begin{array}{ccc|c} 3 & 3 & 0 & 3 \\ 0 & 2 & 2 & 1 \\ 0 & 0 & 2 & \frac{3}{2} \end{array} \right]$$

Końcowe rozwiązanie:

$$x_3 = \frac{3}{4}$$

$$x_2 = -\frac{1}{4}$$

$$x_1 = \frac{5}{4}$$

Przykład w Pythonie

PYTHON

```
A = [
    [0.0, 2.0, 2.0],
    [3.0, 3.0, 0.0],
    [1.0, 0.0, 1.0]
]

b = [1.0, 3.0, 2.0]

n = len(A)

for k in range(n - 1):
    p = k
    najwiekszy = abs(A[k][k])

    for i in range(k + 1, n):
        if abs(A[i][k]) > najwiekszy:
            najwiekszy = abs(A[i][k])
            p = i

    if p != k:
        A[k], A[p] = A[p], A[k]
        b[k], b[p] = b[p], b[k]

    for i in range(k + 1, n):
        m = A[i][k] / A[k][k]

        for j in range(k, n):
            A[i][j] = A[i][j] - m * A[k][j]

        b[i] = b[i] - m * b[k]

x = [0.0, 0.0, 0.0]

for i in range(n - 1, -1, -1):
    suma = 0

    for j in range(i + 1, n):
        suma = suma + A[i][j] * x[j]

    x[i] = (b[i] - suma) / A[i][i]

print(x)
```

Wynik:

```
[1.25, -0.25, 0.75]
```

18. Pełny wybór elementu głównego

Pełny wybór elementu głównego polega na wyszukaniu największego co do modułu współczynnika nie tylko w kolumnie pod napotkanym zerem, ale w całej podmacierzy „w dół i w prawo”.

Może to poprawić dokładność, ale jest bardziej czasochłonne.

Przykład w Pythonie

PYTHON

```
A = [
    [0.0, 2.0, 2.0],
    [3.0, 3.0, 0.0],
    [1.0, 0.0, 1.0]
]

k = 0

najwiekszy = abs(A[k][k])
wiersz_pivota = k
kolumna_pivota = k

for i in range(k, len(A)):
    for j in range(k, len(A[0])):
        if abs(A[i][j]) > najwiekszy:
            najwiekszy = abs(A[i][j])
            wiersz_pivota = i
            kolumna_pivota = j

print("Największy element:", najwiekszy)
print("Wiersz:", wiersz_pivota)
print("Kolumna:", kolumna_pivota)
```

19. Rozkład LU

Jeżeli macierz **A** można przedstawić jako iloczyn macierzy trójkątnej dolnej **L** i trójkątnej górnej **U**, to:

$$A = LU$$

Jeżeli macierz **A** jest nieosobliwa, to:

$$A^{-1} = (LU)^{-1} = U^{-1}L^{-1}$$

Rozwiązanie układu:

$$Ax = b$$

można sprowadzić do dwóch układów trójkątnych:

$$Ly = b$$

oraz:

$$Ux = y$$

Najpierw rozwiązujemy $Ly = b$ przez podstawianie w przód, a potem $Ux = y$ przez podstawianie wstecz.

Przykład w Pythonie

PYTHON

```
L = [  
    [1.0, 0.0, 0.0],  
    [2.0, 1.0, 0.0],  
    [1.0, -1.0, 1.0]  
]  
  
U = [  
    [2.0, 1.0, -1.0],  
    [0.0, 3.0, 2.0],  
    [0.0, 0.0, 4.0]  
]  
  
b = [1.0, 12.0, 8.0]  
  
n = len(L)  
  
# Rozwiązujemy Ly = b  
y = [0.0, 0.0, 0.0]  
  
for i in range(n):  
    suma = 0  
  
    for k in range(0, i):  
        suma = suma + L[i][k] * y[k]  
  
    y[i] = (b[i] - suma) / L[i][i]  
  
# Rozwiązujemy Ux = y  
x = [0.0, 0.0, 0.0]  
  
for i in range(n - 1, -1, -1):  
    suma = 0  
  
    for k in range(i + 1, n):  
        suma = suma + U[i][k] * x[k]  
  
    x[i] = (y[i] - suma) / U[i][i]  
  
print("y =", y)  
print("x =", x)
```

20. Eliminacja Gaussa a rozkład LU

Jednym ze sposobów uzyskania rozkładu LU jest eliminacja Gaussa.

W wyniku eliminacji Gaussa otrzymujemy macierz górnotrójkątną **U**.

Macierz dolnotrójkątną **L** wyznacza się tak, że współczynniki użyte do eliminacji wpisuje się w odpowiednie miejsca macierzy **L**.

Macierz **L** ma na diagonalu wartości **1**.

Przykład

Jeżeli w eliminacji używamy mnożnika:

$$m = \frac{a_{i,k}}{a_{k,k}}$$

to ten mnożnik trafia do macierzy **L**.

```
A = [
    [2.0, 1.0],
    [4.0, 3.0]
]

n = len(A)

L = [
    [1.0, 0.0],
    [0.0, 1.0]
]

U = [
    [A[0][0], A[0][1]],
    [A[1][0], A[1][1]]
]

for k in range(n - 1):
    for i in range(k + 1, n):
        m = U[i][k] / U[k][k]

        L[i][k] = m

        for j in range(k, n):
            U[i][j] = U[i][j] - m * U[k][j]

print("L =", L)
print("U =", U)
```

21. Macierz permutacji

Jeżeli eliminacja Gaussa wymaga zamiany wierszy, to zamiast rozkładu:

$$A = LU$$

otrzymujemy:

$$PA = LU$$

gdzie P jest macierzą permutacji.

Przykład z wykładu:

$$PA = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} a_{3,1} & a_{3,2} & a_{3,3} \\ a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \end{bmatrix}$$

Macierz permutacji ma własność:

$$P^T P = I$$

stąd:

$$P^T = P^{-1}$$

oraz:

$$A = P^T LU$$

Przykład w Pythonie

```
PYTHON

P = [
    [0, 0, 1],
    [1, 0, 0],
    [0, 1, 0]
]

A = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

PA = []

for i in range(len(P)):
    wiersz = []

    for j in range(len(A[0])):
        suma = 0

        for k in range(len(A)):
            suma = suma + P[i][k] * A[k][j]

        wiersz.append(suma)

    PA.append(wiersz)

print(PA)
```

Wynik:

```
[[7, 8, 9], [1, 2, 3], [4, 5, 6]]
```

22. Metoda Doolittle'a

W metodzie Doolittle'a szukamy rozkładu:

$$A = LU$$

przy czym macierz L ma na diagonalu same jedynki.

Dla macierzy 3×3 :

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ l_{2,1} & 1 & 0 \\ l_{3,1} & l_{3,2} & 1 \end{bmatrix} \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} \\ 0 & u_{2,2} & u_{2,3} \\ 0 & 0 & u_{3,3} \end{bmatrix}$$

Wzory ogólne z wykładu:

$$u_{i,j} = a_{i,j} \sum_{k=1}^{i-1} l_{i,k} u_{k,j}$$

dla:

$$j = i, i+1, \dots, n$$

oraz:

$$l_{j,i} = \frac{a_{j,i} - \sum_{k=1}^{i-1} l_{j,k} u_{k,i}}{u_{i,i}}$$

dla:

$$j = i+1, i+2, \dots, n$$

Metoda Doolittle'a staje się niezawodna dopiero w połączeniu z wyborem elementu podstawowego.

Przykład w Pythonie

PYTHON

```
A = [  
    [2.0, 1.0, 1.0],  
    [4.0, -6.0, 0.0],  
    [-2.0, 7.0, 2.0]  
]  
  
n = len(A)  
  
L = []  
U = []  
  
for i in range(n):  
    wiersz_L = []  
    wiersz_U = []  
  
    for j in range(n):  
        if i == j:  
            wiersz_L.append(1.0)  
        else:  
            wiersz_L.append(0.0)  
  
        wiersz_U.append(0.0)  
  
    L.append(wiersz_L)  
    U.append(wiersz_U)  
  
for i in range(n):  
    # Liczymy wiersz U  
    for j in range(i, n):  
        suma = 0  
  
        for k in range(i):  
            suma = suma + L[i][k] * U[k][j]  
  
        U[i][j] = A[i][j] - suma  
  
    # Liczymy kolumnę L  
    for j in range(i + 1, n):  
        suma = 0  
  
        for k in range(i):  
            suma = suma + L[j][k] * U[k][i]  
  
        L[j][i] = (A[j][i] - suma) / U[i][i]  
  
print("L =", L)  
print("U =", U)
```

23. Metoda Crouta

W metodzie Crouta przyjmuje się, że macierz U ma na głównej przekątnej same jedynki.

Czyli dla macierzy 3×3 :

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} = \begin{bmatrix} l_{1,1} & 0 & 0 \\ l_{2,1} & l_{2,2} & 0 \\ l_{3,1} & l_{3,2} & l_{3,3} \end{bmatrix} \begin{bmatrix} 1 & u_{1,2} & u_{1,3} \\ 0 & 1 & u_{2,3} \\ 0 & 0 & 1 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [  
    [2.0, 1.0, 1.0],  
    [4.0, -6.0, 0.0],  
    [-2.0, 7.0, 2.0]  
]  
  
n = len(A)  
  
L = []  
U = []  
  
for i in range(n):  
    wiersz_L = []  
    wiersz_U = []  
  
    for j in range(n):  
        wiersz_L.append(0.0)  
  
        if i == j:  
            wiersz_U.append(1.0)  
        else:  
            wiersz_U.append(0.0)  
  
    L.append(wiersz_L)  
    U.append(wiersz_U)  
  
for j in range(n):  
    for i in range(j, n):  
        suma = 0  
  
        for k in range(j):  
            suma = suma + L[i][k] * U[k][j]  
  
        L[i][j] = A[i][j] - suma  
  
    for i in range(j + 1, n):  
        suma = 0  
  
        for k in range(j):  
            suma = suma + L[j][k] * U[k][i]  
  
        U[j][i] = (A[j][i] - suma) / L[j][j]  
  
print("L =", L)  
print("U =", U)
```


24. Rozkład Cholesky'ego

Rozkład Cholesky'ego, nazywany też rozkładem Banachiewicza, stosuje się dla macierzy symetrycznych i dodatnio określonych.

Macierz musi spełniać warunek symetrii:

$$a_{i,j} = a_{j,i}$$

oraz warunek dodatniej określoności:

$$x^T A x > 0$$

dla każdego x .

Wtedy można zapisać:

$$A = L L^T$$

gdzie L jest macierzą trójkątną dolną.

Wzory z wykładu:

$$l_{i,i} = \sqrt{a_{i,i} - \sum_{k=1}^{i-1} l_{i,k}^2}$$

oraz:

$$l_{j,i} = \frac{1}{l_{i,i}} \left(a_{j,i} - \sum_{k=1}^{i-1} l_{j,k} l_{i,k} \right)$$

dla:

$$j = i + 1, i + 2, \dots, n$$

Wykład podaje, że ilość operacji potrzebna do znalezienia rozkładu Cholesky'ego jest o połowę mniejsza w porównaniu z LU. Metoda jest też stabilna numerycznie i nie wymaga wyboru elementu podstawowego.

```
import math

A = [
    [4.0, 2.0],
    [2.0, 3.0]
]

n = len(A)

L = []

for i in range(n):
    wiersz = []

    for j in range(n):
        wiersz.append(0.0)

    L.append(wiersz)

for i in range(n):
    suma = 0

    for k in range(i):
        suma = suma + L[i][k] ** 2

    L[i][i] = math.sqrt(A[i][i] - suma)

    for j in range(i + 1, n):
        suma = 0

        for k in range(i):
            suma = suma + L[j][k] * L[i][k]

        L[j][i] = (A[j][i] - suma) / L[i][i]

print(L)
```

25. Najważniejsze rzeczy do zapamiętania na kolosa

25.1. Zapis układu

Układ równań liniowych zapisujemy jako:

$$Ax = b$$

Python

PYTHON

```
A = [  
    [2, 1],  
    [1, 3]  
]  
  
b = [5, 7]  
  
print(A)  
print(b)
```

25.2. Macierz rozszerzona

Macierz rozszerzona to macierz główna z dołączonym wektorem wyrazów wolnych:

$$A_b = \begin{bmatrix} a_{1,1} & a_{1,2} & b_1 & a_{2,1} & a_{2,2} & b_2 \end{bmatrix}$$

Python

PYTHON

```
A = [  
    [2, 1],  
    [1, 3]  
]  
  
b = [5, 7]  
  
Ab = []  
  
for i in range(len(A)):  
    wiersz = []  
  
    for j in range(len(A[0])):  
        wiersz.append(A[i][j])  
  
    wiersz.append(b[i])  
    Ab.append(wiersz)  
  
print(Ab)
```

25.3. Twierdzenie Kroneckera-Capellego

Jeżeli:

$$\text{rank}(A) = \text{rank}(A_b) = n$$

układ ma dokładnie jedno rozwiązanie.

Jeżeli:

$$\text{rank}(A) = \text{rank}(A_b) < n$$

układ ma nieskończenie wiele rozwiązań.

Jeżeli:

$$\text{rank}(A) < \text{rank}(A_b)$$

układ nie ma rozwiązań.

25.4. Wzory Cramera

Dla:

$$\det(A) \neq 0$$

można liczyć:

$$x_k = \frac{\det(A_k)}{\det(A)}$$

25.5. Podstawianie wstecz

Dla macierzy trójkątnej górnej:

$$x_i = \frac{b_i - \sum_{k=i+1}^n a_{i,k} x_k}{a_{i,i}}$$

25.6. Podstawianie w przód

Dla macierzy trójkątnej dolnej:

$$x_i = \frac{b_i - \sum_{k=1}^{i-1} a_{i,k} x_k}{a_{i,i}}$$

25.7. Eliminacja Gaussa

Eliminacja Gaussa sprowadza układ do postaci trójkątnej, a potem stosuje się podstawianie wstecz.

25.8. Pivot

Pivot to element podstawowy, przez który dzielimy podczas eliminacji.

Jeżeli pivot jest równy zero albo bardzo mały, należy zamienić wiersze.

25.9. Rozkład LU

Jeżeli:

$$A = LU$$

to rozwiązujemy dwa układy:

$$Ly = b$$

oraz:

$$Ux = y$$

25.10. Rozkład Cholesky'ego

Jeżeli macierz jest symetryczna i dodatnio określona, można zastosować rozkład:

$$A = LL^T$$

Wykład 4: Układy równań liniowych II — metody iteracyjne (lab 5, 6)

1. Wprowadzenie do metod iteracyjnych

Metody iteracyjne są alternatywą dla metod bezpośrednich w rozwiązywaniu układów równań liniowych.

W metodach iteracyjnych nie wyznacza się rozwiązania od razu w skończonej liczbie przekształceń, tylko zaczyna się od pewnego przybliżenia początkowego i stopniowo je poprawia.

Układ równań ma postać:

$$Ax = b$$

gdzie:

- **A** — macierz współczynników,
- **x** — wektor niewiadomych,
- **b** — wektor wyrazów wolnych.

Metody iteracyjne generują ciąg przybliżeń:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

który powinien zbiegać do szukanego rozwiązania **x**.

Kiedy metody iteracyjne są szczególnie przydatne?

Metody iteracyjne stosuje się szczególnie wtedy, gdy:

- układ równań jest zbyt duży dla metod bezpośrednich,
- macierz układu jest rzadka i dobrze uwarunkowana,
- potrzebne jest szybkie przybliżone rozwiązanie,
- zasoby pamięciowe są ograniczone.

Przykład w Pythonie

```
# Przykład układu Ax = b

A = [
    [4, -2],
    [-2, 5]
]

b = [0, 2]

# Przybliżenie początkowe x^(0)
x = [0, 0]

print("Macierz A:")
print(A)

print("Wektor b:")
print(b)

print("Przybliżenie początkowe x:")
print(x)
```

PYTHON

2. Przykłady metod iteracyjnych

W wykładzie podano następujące przykłady metod iteracyjnych:

- metoda Jacobiego,

- metoda Gaussa-Seidla,
- metoda sukcesywnych nadrelaksacji **SOR**,
- metoda gradientów sprzężonych dla macierzy symetrycznych i dodatnio określonych.

3. Ogólny schemat metod iteracyjnych

Ogólny schemat metody iteracyjnej wygląda tak:

1. Wybieramy początkowe przybliżenie rozwiązania:

$$x^{(0)}$$

2. Powtarzamy kolejne iteracje, aż zostanie spełnione kryterium zbieżności.
3. W każdej iteracji poprawiamy aktualne przybliżenie.
4. Sprawdzamy warunek stopu, np. różnicę między kolejnymi przybliżeniami.
5. Kończymy, gdy rozwiązanie jest wystarczająco dokładne.

Przykład w Pythonie

```
PYTHON

x_poprzednie = [0.0, 0.0]
x_nowe = [0.1, 0.2]

epsilon = 0.001

# Liczymy największą różnicę między kolejnymi przybliżeniami
blad = abs(x_nowe[0] - x_poprzednie[0])

for i in range(1, len(x_nowe)):
    roznica = abs(x_nowe[i] - x_poprzednie[i])

    if roznica > blad:
        blad = roznica

if blad < epsilon:
    print("Warunek stopu spełniony")
else:
    print("Trzeba wykonać kolejną iterację")

print("Błąd =", blad)
```

4. Postać iteracyjna układu równań

W większości metod iteracyjnych układ:

$$Ax = b$$

przekształca się do postaci:

$$x = Wx + Z$$

gdzie:

- **W** — macierz iteracji,
- **Z** — wektor,
- **x** — szukany wektor niewiadomych.

Mając dane przybliżenie początkowe:

$$x^{(0)}$$

kolejne przybliżenia liczymy ze wzoru:

$$x^{(k)} = Wx^{(k-1)} + Z$$

dla:

$$k = 1, 2, 3, \dots$$

Przykład w Pythonie

PYTHON

```
W = [
    [0.0, 0.5],
    [0.2, 0.0]
]

Z = [0.0, 0.4]

x_poprzednie = [0.0, 0.0]
x_nowe = []

for i in range(len(W)):
    suma = 0

    for j in range(len(W[0])):
        suma = suma + W[i][j] * x_poprzednie[j]

    x_nowe.append(suma + Z[i])

print("x^(1) =", x_nowe)
```

5. Metoda Jacobiego

5.1. Idea metody Jacobiego

Metoda Jacobiego polega na tym, że każdą niewiadomą w nowej iteracji obliczamy tylko na podstawie wartości ze starej iteracji.

To znaczy, że przy liczeniu:

$$x_i^{(k)}$$

korzystamy z wartości:

$$x_j^{(k-1)}$$

czyli ze starego wektora przybliżeń.

Wszystkie nowe wartości są obliczane niezależnie, a aktualizacja następuje dopiero po zakończeniu całej iteracji.

Ważna intuicja

Metoda Jacobiego działa według zasady:

najpierw policz wszystkie nowe wartości, potem zaktualizuj wektor

Dzięki temu metoda jest łatwa do zrównoleglenia.

Przykład w Pythonie

PYTHON

```
# Przykład pokazujący ideę:
# nowe wartości liczymy do osobnej listy,
# nie nadpisujemy od razu starego x.

x_stare = [0.0, 0.0]
x_nowe = [0.0, 0.0]

# przykład wzorów iteracyjnych:
# x1 = 0.5 * x2
# x2 = 0.4 + 0.2 * x1

x_nowe[0] = 0.5 * x_stare[1]
x_nowe[1] = 0.4 + 0.2 * x_stare[0]

# Dopiero po policzeniu wszystkich wartości aktualizujemy x
x_stare = x_nowe

print(x_stare)
```

5.2. Rozkład macierzy na **D** + **R**

Dany jest układ:

$$Ax = b$$

W metodzie Jacobiego macierz główną **A** rozbijamy na sumę dwóch macierzy:

$$A = D + R$$

gdzie:

- **D** — macierz diagonalna,
- **R** — reszta elementów macierzy **A**.

Macierz **D** zawiera tylko elementy z głównej przekątnej:

$$D = \begin{bmatrix} a_{1,1} & 0 & \dots & 0 \\ 0 & a_{2,2} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{n,n} \end{bmatrix}$$

Macierz **R** zawiera pozostałe elementy macierzy **A**:

$$R = \begin{bmatrix} 0 & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & 0 & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \dots & 0 \end{bmatrix}$$

Przykład

Dla macierzy:

$$A = \begin{bmatrix} 4 & -2 \\ -2 & 5 \end{bmatrix}$$

mamy:

$$D = \begin{bmatrix} 4 & 0 \\ 0 & 5 \end{bmatrix}$$

oraz:

$$R = \begin{bmatrix} 0 & -2 \\ -2 & 0 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
A = [  
    [4, -2],  
    [-2, 5]  
]  
  
n = len(A)  
  
D = []  
R = []  
  
for i in range(n):  
    wiersz_D = []  
    wiersz_R = []  
  
    for j in range(n):  
        if i == j:  
            wiersz_D.append(A[i][j])  
            wiersz_R.append(0)  
        else:  
            wiersz_D.append(0)  
            wiersz_R.append(A[i][j])  
  
    D.append(wiersz_D)  
    R.append(wiersz_R)  
  
print("D =", D)  
print("R =", R)
```

5.3. Przekształcenie układu do postaci iteracyjnej

Zaczynamy od:

$$Ax = b$$

Ponieważ:

$$A = D + R$$

to:

$$(D + R)x = b$$

czyli:

$$Dx + Rx = b$$

Przenosimy składnik Rx na prawą stronę:

$$Dx = -Rx + b$$

Mnożymy obustronnie przez:

$$D^{-1}$$

i dostajemy:

$$D^{-1}Dx = -D^{-1}Rx + D^{-1}b$$

czyli:

$$x = -D^{-1}Rx + D^{-1}b$$

Wprowadzamy oznaczenia:

$$W = -D^{-1}R$$

oraz:

$$Z = D^{-1}b$$

Ostatecznie:

$$x = Wx + Z$$

a iteracyjnie:

$$x^{(k)} = Wx^{(k-1)} + Z$$

Przykład w Pythonie

PYTHON

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
b = [0.0, 2.0]  
  
n = len(A)  
  
W = []  
Z = []  
  
for i in range(n):  
    wiersz_W = []  
  
    for j in range(n):  
        if i == j:  
            wiersz_W.append(0.0)  
        else:  
            wiersz_W.append(-A[i][j] / A[i][i])  
  
    W.append(wiersz_W)  
    Z.append(b[i] / A[i][i])  
  
print("W =", W)  
print("Z =", Z)
```

Wynik:

```
W = [[0.0, 0.5], [0.4, 0.0]]  
Z = [0.0, 0.4]
```

Dla przykładu:

PYTHON

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
b = [0.0, 2.0]
```

układ równań to:

$$\begin{aligned}4x_1 - 2x_2 &= 0 \\ -2x_1 + 5x_2 &= 2\end{aligned}$$

Przekształcamy każde równanie tak, żeby po lewej zostało jedno x .

Z pierwszego równania:

$$4x_1 = 2x_2$$

$$x_1 = 0.5x_2$$

Z drugiego równania:

$$5x_2 = 2 + 2x_1$$

$$x_2 = 0.4 + 0.4x_1$$

Czyli:

$$x_1 = 0 \cdot x_1 + 0.5x_2 + 0$$

$$x_2 = 0.4x_1 + 0 \cdot x_2 + 0.4$$

Stąd:

$$W = \begin{bmatrix} 0 & 0.5 \\ 0.4 & 0 \end{bmatrix}$$

oraz:

$$Z = \begin{bmatrix} 0 \\ 0.4 \end{bmatrix}$$

Twój kod właśnie to liczy:

```
wiersz_W.append(-A[i][j] / A[i][i])
```

PYTHON

bo dla elementów poza przekątną mamy:

$$w_{ij} = -\frac{a_{ij}}{a_{ii}}$$

a dla z :

```
Z.append(b[i] / A[i][i])
```

PYTHON

czyli:

$$z_i = \frac{b_i}{a_{ii}}$$

Transpozycja byłaby potrzebna tylko wtedy, gdyby chcieć zapisywać wektor x jako **wektor wierszowy** i mnożyć z innej strony. Ale w metodach numerycznych standardowo przyjmuje się, że:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

czyli jest to wektor kolumnowy. Dlatego zapis:

$$x^{(k+1)} = Wx^{(k)} + Z$$

jest poprawny bez transponowania.

5.4. Odwrotność macierzy diagonalnej

Macierz odwrotna do macierzy diagonalnej  też jest macierzą diagonalną.

Jeżeli:

$$D = \begin{bmatrix} a_{1,1} & 0 & \dots & 0 \\ 0 & a_{2,2} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{n,n} \end{bmatrix}$$

to:

$$D^{-1} = \begin{bmatrix} \frac{1}{a_{1,1}} & 0 & \dots & 0 \\ 0 & \frac{1}{a_{2,2}} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \frac{1}{a_{n,n}} \end{bmatrix}$$

Przykład w Pythonie

```
D = [
    [4.0, 0.0],
    [0.0, 5.0]
]

n = len(D)
D_inv = []

for i in range(n):
    wiersz = []

    for j in range(n):
        if i == j:
            wiersz.append(1 / D[i][j])
        else:
            wiersz.append(0.0)

    D_inv.append(wiersz)

print(D_inv)
```

PYTHON

5.5. Macierz W

Dla metody Jacobiego:

$$W = -D^{-1}R$$

Elementy macierzy W są następujące:

$$w_{i,j} = -\frac{a_{i,j}}{a_{i,i}}$$

dla:

$$i, j = 1, 2, \dots, n$$

oraz:

$$j \neq i$$

Na diagonalu macierzy W są zera:

$$w_{i,i} = 0$$

Przykład w Pythonie

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
n = len(A)  
W = []  
  
for i in range(n):  
    wiersz = []  
  
    for j in range(n):  
        if i == j:  
            wiersz.append(0.0)  
        else:  
            wiersz.append(-A[i][j] / A[i][i])  
  
    W.append(wiersz)  
  
print(W)
```

PYTHON

5.6. Wektor z

Dla metody Jacobiego:

$$Z = D^{-1}b$$

Elementy wektora z są następujące:

$$Z_i = \frac{b_i}{a_{i,i}}$$

dla:

$$i = 1, 2, \dots, n$$

Przykład w Pythonie

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
b = [0.0, 2.0]  
  
Z = []  
  
for i in range(len(A)):  
    Z.append(b[i] / A[i][i])  
  
print(Z)
```

PYTHON

5.7. Wzór dla poszczególnych niewiadomych w metodzie Jacobiego

W praktyce często używa się wzoru:

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k-1)} \right)$$

dla:

$$i = 1, 2, \dots, n$$

Najważniejsze jest to, że każda nowa wartość:

$$x_i^{(k)}$$

jest liczona tylko na podstawie starego przybliżenia:

$$x^{(k-1)}$$

Przykład w Pythonie

PYTHON

```
A = [
    [4.0, -2.0],
    [-2.0, 5.0]
]

b = [0.0, 2.0]

x_stare = [0.0, 0.0]
x_nowe = []

n = len(A)

for i in range(n):
    suma = 0

    for j in range(n):
        if j != i:
            suma = suma + A[i][j] * x_stare[j]

    wartosc = (b[i] - suma) / A[i][i]
    x_nowe.append(wartosc)

print(x_nowe)
```

5.8. Algorytm metody Jacobiego

Dane:

- macierz **A**,
- wektor **b**,
- przybliżenie początkowe $x^{(0)}$,
- dokładność **epsilon**.

Kroki:

1. Wybierz przybliżenie początkowe:

$$x^{(0)}$$

2. Dla każdej iteracji oblicz nowy wektor:

$$x^{(k)}$$

używając tylko wartości z poprzedniej iteracji:

$$x^{(k-1)}$$

3. Porównaj:

$$x^{(k)}$$

oraz:

$$x^{(k-1)}$$

4. Jeżeli różnica jest mała, zakończ.

5. W przeciwnym razie wykonaj kolejną iterację.

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
b = [0.0, 2.0]  
  
n = len(A)  
  
x_stare = [0.0, 0.0]  
epsilon = 0.001  
maks_iteracji = 100  
  
for iteracja in range(maks_iteracji):  
    x_nowe = []  
  
    for i in range(n):  
        suma = 0  
  
        for j in range(n):  
            if j != i:  
                suma = suma + A[i][j] * x_stare[j]  
  
        wartosc = (b[i] - suma) / A[i][i]  
        x_nowe.append(wartosc)  
  
    blad = abs(x_nowe[0] - x_stare[0])  
  
    for i in range(1, n):  
        roznica = abs(x_nowe[i] - x_stare[i])  
  
        if roznica > blad:  
            blad = roznica  
  
    x_stare = x_nowe  
  
    if blad < epsilon:  
        break  
  
print("Liczba iteracji:", iteracja + 1)  
print("Przybliżone rozwiązanie:", x_stare)
```

6. Przykład metody Jacobiego z wykładu

W wykładzie rozważany jest układ:

$$4x_1 - 2x_2 = 0$$

$$-2x_1 + 5x_2 - x_3 = 2$$

$$-x_2 + 4x_3 + 2x_4 = 3$$

$$2x_3 + 3x_4 = -2$$

Rozwiązanie dokładne wynosi:

$$x = \begin{bmatrix} 0.5 \\ 1 \\ 2 \\ -2 \end{bmatrix}$$

Przekształcamy układ do postaci:

$$x = Wx + Z$$

Z każdego równania wyznaczamy odpowiednią niewiadomą:

$$x_1 = \frac{1}{2}x_2$$

$$x_2 = \frac{2}{5} + \frac{1}{5}x_1 + \frac{1}{5}x_3$$

$$x_3 = \frac{3}{4} + \frac{1}{4}x_2 - \frac{2}{4}x_4$$

$$x_4 = -\frac{2}{3} - \frac{2}{3}x_3$$

Macierz iteracji i wektor **Z**:

$$W = \begin{bmatrix} 0 & \frac{2}{4} & 0 & 0 \\ \frac{2}{5} & 0 & \frac{1}{5} & 0 \\ 0 & \frac{1}{4} & 0 & -\frac{2}{4} \\ 0 & 0 & -\frac{2}{3} & 0 \end{bmatrix}$$

$$Z = \left[0 \quad \frac{2}{5} \quad \frac{3}{4} \quad -\frac{2}{3} \right]$$

Dla przybliżenia początkowego:

$$x^{(0)} = [0 \ 0 \ 0 \ 0]$$

wykonujemy kolejne iteracje:

$$x_1^{(k)} = \frac{1}{2}x_2^{(k-1)}$$

$$x_2^{(k)} = \frac{2}{5} + \frac{1}{5}x_1^{(k-1)} + \frac{1}{5}x_3^{(k-1)}$$

$$x_3^{(k)} = \frac{3}{4} + \frac{1}{4}x_2^{(k-1)} - \frac{2}{4}x_4^{(k-1)}$$

$$x_4^{(k)} = -\frac{2}{3} - \frac{2}{3}x_3^{(k-1)}$$

Wyniki iteracji z wykładu

Dla dokładności:

$$\epsilon = 10^{-3}$$

w wykładzie iteracje kończą się na wartości:

$$x^{(16)} \approx [0.4994 \ 0.9992 \ 1.9986 \ -1.9986]$$

Błąd w tabeli dla iteracji 16 wynosi:

$$0.0008$$

Przykład w Pythonie

```
PYTHON

x_stare = [0.0, 0.0, 0.0, 0.0]

epsilon = 0.001
maks_iteracji = 100

for iteracja in range(maks_iteracji):
    x_nowe = [0.0, 0.0, 0.0, 0.0]

    x_nowe[0] = 0.5 * x_stare[1]
    x_nowe[1] = 2 / 5 + (1 / 5) * x_stare[0] + (1 / 5) * x_stare[2]
    x_nowe[2] = 3 / 4 + (1 / 4) * x_stare[1] - (2 / 4) * x_stare[3]
    x_nowe[3] = -2 / 3 - (2 / 3) * x_stare[2]

    blad = abs(x_nowe[0] - x_stare[0])

    for i in range(1, len(x_nowe)):
        roznica = abs(x_nowe[i] - x_stare[i])

        if roznica > blad:
            blad = roznica

    x_stare = x_nowe

    if blad < epsilon:
        break

print("Iteracja:", iteracja + 1)
print("x =", x_stare)
print("blad =", blad)
```

7. Warunek stopu

Obliczenia można zakończyć, gdy spełniony jest warunek stopu.

W wykładzie podano warunek:

$$\frac{|x^{(k)} - x^{(k-1)}|}{|x^{(k)}|} \leq \epsilon$$

Oznacza to, że porównujemy różnicę kolejnych przybliżeń z wielkością aktualnego przybliżenia.

W przykładzie z wykładu:

$$\frac{1.9986 - 1.9979}{1.9986} \approx 0.00035 \leq 10^{-3}$$

Przykład w Pythonie

PYTHON

```
x_poprzednie = [0.4986, 0.9982, 1.9979, -1.9979]
x_aktualne = [0.4994, 0.9992, 1.9986, -1.9986]

# Norma maksimum licznika
licznik = abs(x_aktualne[0] - x_poprzednie[0])

for i in range(1, len(x_aktualne)):
    roznica = abs(x_aktualne[i] - x_poprzednie[i])

    if roznica > licznik:
        licznik = roznica

# Norma maksimum mianownika
mianownik = abs(x_aktualne[0])

for i in range(1, len(x_aktualne)):
    wartosc = abs(x_aktualne[i])

    if wartosc > mianownik:
        mianownik = wartosc

iloraz = licznik / mianownik

print("Iloraz =", iloraz)

if iloraz <= 0.001:
    print("Można zakończyć obliczenia")
else:
    print("Trzeba iterować dalej")
```

8. Metoda Jacobiego — podsumowanie

Prosty algorytm Jacobiego wymaga, aby:

$$a_{i,i} \neq 0$$

dla każdego:

$$i = 1, 2, \dots, n$$

Jeżeli ten warunek nie jest spełniony, a układ jest nieosobliwy, to układ można przekształcić tak, aby ten warunek był spełniony.

Przyspieszenie zbieżności otrzymuje się przez wybór elementów największych co do wartości bezwzględnej na przekątnej.

Przykład w Pythonie

```
PYTHON

A = [
    [0.0, 2.0],
    [4.0, 5.0]
]

czy_diagonala_niezerowa = True

for i in range(len(A)):
    if A[i][i] == 0:
        czy_diagonala_niezerowa = False

if czy_diagonala_niezerowa:
    print("Można bezpośrednio zastosować prostą metodę Jacobiego")
else:
    print("Trzeba przekształcić układ, bo na diagonalu jest zero")
```

9. Metoda Gaussa-Seidla

9.1. Idea metody Gaussa-Seidla

Metoda Gaussa-Seidla jest metodą iteracyjną rozwiązywania układów równań liniowych.

Podobnie jak metoda Jacobiego służy do rozwiązania układu:

$$Ax = b$$

Różnica polega na sposobie aktualizacji wartości niewiadomych.

W metodzie Gaussa-Seidla korzystamy z najnowszych dostępnych wartości.

To znaczy:

- dla wcześniejszych niewiadomych w danej iteracji używamy już nowych wartości,

- dla późniejszych niewiadomych używamy jeszcze starych wartości.

Intuicja

Metoda Gaussa-Seidla działa według zasady:

licz i od razu poprawiaj

Przykład w Pythonie

PYTHON

```
# W metodzie Gaussa-Seidla nadpisujemy wartości od razu.  
  
x = [0.0, 0.0]  
  
# Najpierw liczymy x1 i od razu zapisujemy nową wartość  
x[0] = 0.5 * x[1]  
  
# Potem liczymy x2, korzystając już z nowego x1  
x[1] = 0.4 + 0.2 * x[0]  
  
print(x)
```

9.2. Wzór metody Gaussa-Seidla

Ogólny wzór metody Gaussa-Seidla:

$$x_i^{(k)} = \frac{1}{a_{i,i}} \left(b_i - \sum_{j=1}^{i-1} a_{i,j} x_j^{(k)} - \sum_{j=i+1}^n a_{i,j} x_j^{(k-1)} \right)$$

dla:

$$i = 1, 2, \dots, n$$

Pierwsza suma używa już nowych wartości:

$$x_j^{(k)}$$

Druga suma używa starych wartości:

$$x_j^{(k-1)}$$

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
b = [0.0, 2.0]  
  
x = [0.0, 0.0]  
n = len(A)  
  
for i in range(n):  
    suma = 0  
  
    for j in range(n):  
        if j != i:  
            suma = suma + A[i][j] * x[j]  
  
    x[i] = (b[i] - suma) / A[i][i]  
  
print(x)
```

9.3. Algorytm metody Gaussa-Seidla

Kroki:

1. Wybierz przybliżenie początkowe:

$$x^{(0)}$$

2. Dla każdej iteracji przechodź po niewiadomych:

$$i = 1, 2, \dots, n$$

3. Oblicz:

$$x_i^{(k)}$$

korzystając z nowych i starych wartości.

4. Sprawdź warunek stopu.
5. Jeśli warunek stopu jest spełniony, zakończ.

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
b = [0.0, 2.0]  
  
n = len(A)  
x = [0.0, 0.0]  
  
epsilon = 0.001  
maks_iteracji = 100  
  
for iteracja in range(maks_iteracji):  
    x_stare = []  
  
    for i in range(n):  
        x_stare.append(x[i])  
  
    for i in range(n):  
        suma = 0  
  
        for j in range(n):  
            if j != i:  
                suma = suma + A[i][j] * x[j]  
  
        x[i] = (b[i] - suma) / A[i][i]  
  
    blad = abs(x[0] - x_stare[0])  
  
    for i in range(1, n):  
        roznica = abs(x[i] - x_stare[i])  
  
        if roznica > blad:  
            blad = roznica  
  
    if blad < epsilon:  
        break  
  
    print("Liczba iteracji:", iteracja + 1)  
    print("x =", x)
```

9.4. Metoda Gaussa-Seidla w zapisie macierzowym

Dla rozkładu:

$$A = D + L + U$$

gdzie:

- D — macierz diagonalna,
- L — macierz trójkątna dolna,
- U — macierz trójkątna górna,

metoda Gaussa-Seidla ma postać:

$$(L + D)x^{(k)} = -Ux^{(k-1)} + b$$

Po przekształceniu:

$$x^{(k)} = (L + D)^{-1}Ux^{(k-1)} + (L + D)^{-1}b$$

dla:

$$k = 1, 2, \dots$$

Wymagane jest:

$$a_{i,i} \neq 0$$

dla wszystkich:

$$i = 1, 2, \dots, n$$

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
n = len(A)  
  
D = []  
L = []  
U = []  
  
for i in range(n):  
    wiersz_D = []  
    wiersz_L = []  
    wiersz_U = []  
  
    for j in range(n):  
        if i == j:  
            wiersz_D.append(A[i][j])  
            wiersz_L.append(0.0)  
            wiersz_U.append(0.0)  
        elif i > j:  
            wiersz_D.append(0.0)  
            wiersz_L.append(A[i][j])  
            wiersz_U.append(0.0)  
        else:  
            wiersz_D.append(0.0)  
            wiersz_L.append(0.0)  
            wiersz_U.append(A[i][j])  
  
    D.append(wiersz_D)  
    L.append(wiersz_L)  
    U.append(wiersz_U)  
  
print("D =", D)  
print("L =", L)  
print("U =", U)
```

10. Przykład metody Gaussa-Seidla z wykładu

W wykładzie metoda Gaussa-Seidla została zastosowana do tego samego układu:

$$4x_1 - 2x_2 = 0$$

$$-2x_1 + 5x_2 - x_3 = 2$$

$$-x_2 + 4x_3 + 2x_4 = 3$$

$$2x_3 + 3x_4 = -2$$

Kolejne iteracje mają postać:

$$\begin{aligned}x_1^{(k)} &= \frac{1}{2}x_2^{(k-1)} \\x_2^{(k)} &= \frac{2 + 2x_1^{(k)} + x_3^{(k-1)}}{5} \\x_3^{(k)} &= \frac{3 + x_2^{(k)} - 2x_4^{(k-1)}}{4} \\x_4^{(k)} &= \frac{-2 - 2x_3^{(k)}}{3}\end{aligned}$$

Dla:

$$\epsilon = 10^{-3}$$

zakończenie obliczeń nastąpiło po 9 iteracjach.

Obliczenia iteracyjne kończymy wtedy, gdy kolejne przybliżenia różnią się już bardzo mało.

Stosujemy warunek:

$$\frac{|x^{(k)} - x^{(k-1)}|}{|x^{(k)}|} \leq \epsilon$$

W tym przykładzie:

$$\epsilon = 10^{-3}$$

Korzystamy z normy maksimum, czyli:

$$|x|_{\max} = \max_i |x_i|$$

Oznacza to, że bierzemy największą wartość bezwzględną spośród elementów wektora.

Dla ostatnich iteracji z wykładu:

$$\frac{|x^{(k)} - x^{(k-1)}|}{|x^{(k)}|} = \frac{1.9997 - 1.9992}{1.9997} \approx 0.00025$$

Ponieważ:

$$0.00025 \leq 10^{-3}$$

to obliczenia można zakończyć po 9 iteracjach.

W wykładzie ostatnie przybliżenie z tabeli to:

$$x \approx \begin{bmatrix} 0.4995 \\ 0.9996 \\ 1.9995 \\ -1.9997 \end{bmatrix}$$

Przykład w Pythonie

PYTHON

```
```python
def norma_maksimum_wektora(x):
 maksimum = abs(x[0])

 for xi in x:
 if abs(xi) > maksimum:
 maksimum = abs(xi)

 return maksimum

def roznica_wektorow(x_nowe, x_stare):
 roznica = []

 for i in range(len(x_nowe)):
 roznica.append(x_nowe[i] - x_stare[i])

 return roznica

def kryterium_stopu(x_nowe, x_stare, epsilon):
 roznica = roznica_wektorow(x_nowe, x_stare)

 licznik = norma_maksimum_wektora(roznica)
 mianownik = norma_maksimum_wektora(x_nowe)

 if mianownik == 0:
 return False

 blad = licznik / mianownik

 print("Błąd względny =", blad)

 return blad <= epsilon

x_stare = [0.4994, 0.9994, 1.9992, -1.9995]
x_nowe = [0.4995, 0.9996, 1.9995, -1.9997]

epsilon = 10**(-3)

if kryterium_stopu(x_nowe, x_stare, epsilon):
 print("Kończymy obliczenia")
else:
 print("Wykonujemy kolejną iterację")
```

Wynik będzie w stylu:



Błąd względny = 0.00015002250337550562  
Kończymy obliczenia

## Inny przykład

```
```python
x = [0.0, 0.0, 0.0, 0.0]

epsilon = 0.001
maks_iteracji = 100

for iteracja in range(maks_iteracji):
    x_stare = []

    for i in range(len(x)):
        x_stare.append(x[i])

    x[0] = 0.5 * x[1]
    x[1] = (2 + 2 * x[0] + x[2]) / 5
    x[2] = (3 + x[1] - 2 * x[3]) / 4
    x[3] = (-2 - 2 * x[2]) / 3

    blad = abs(x[0] - x_stare[0])

    for i in range(1, len(x)):
        roznica = abs(x[i] - x_stare[i])

        if roznica > blad:
            blad = roznica

    if blad < epsilon:
        break

print("Iteracja:", iteracja + 1)
print("x =", x)
print("blad =", blad)
```

11. Jacobi a Gauss-Seidel

W wykładzie porównano metody Jacobiego i Gaussa-Seidla.

Cecha	Jacobi	Gauss-Seidel
Dane w iteracji	tylko stare wartości	nowe + stare wartości
Aktualizacja	jednoczesna	sekwencyjna

Cecha	Jacobi	Gauss-Seidel
Szybkość zbieżności	wolniejsza	zwykle szybsza
Równoległość	bardzo dobra	ograniczona
Złożoność implementacji	prosta	trochę trudniejsza

Intuicja

Metoda Jacobiego:

najpierw policz wszystko, potem zaktualizuj

Metoda Gaussa-Seidla:

licz i od razu poprawiaj

Wniosek

Metoda Gaussa-Seidla zwykle zbiega szybciej, ale metoda Jacobiego łatwiej się zrównolegla.

12. Warunek zbieżności

Kluczowe pytanie metod iteracyjnych brzmi:

czy iteracje prowadzą do rozwiązania?

Dla równania:

$$x^{(k)} = Wx^{(k-1)} + Z$$

zbieżność zależy od macierzy W .

Warunek zbieżności:

$$\rho(W) < 1$$

gdzie:

$$\rho(W)$$

oznacza największą wartość własną.

W praktyce używa się łatwiejszego warunku:

$$|W| < 1$$

Jeżeli:

$$|W| < 1$$

to błędy maleją i metoda powinna być zbieżna.

Jeżeli:

$$|W| > 1$$

to błędy rosną i metoda może być rozbieżna.

Przykład w Pythonie

PYTHON

```
W = [  
    [0.0, 0.5],  
    [0.4, 0.0]  
]  
  
# Liczymy normę wierszową  
norma = 0  
  
for i in range(len(W)):  
    suma_wiersza = 0  
  
    for j in range(len(W[0])):  
        suma_wiersza = suma_wiersza + abs(W[i][j])  
  
    if suma_wiersza > norma:  
        norma = suma_wiersza  
  
print("Norma W =", norma)  
  
if norma < 1:  
    print("Metoda powinna być zbieżna")  
else:  
    print("Metoda może nie być zbieżna")
```

13. Normy macierzy

Do badania zbieżności można liczyć różne normy macierzy.

13.1. Norma wierszowa

$$|W|_{\infty} = \max_i \sum_{j=1}^n |w_{i,j}|$$

Czyli liczymy sumę modułów w każdym wierszu i wybieramy największą.

Przykład w Pythonie

```
PYTHON

W = [
    [0.0, 0.5],
    [0.4, 0.0]
]

norma_wierszowa = 0

for i in range(len(W)):
    suma_wiersza = 0

    for j in range(len(W[0])):
        suma_wiersza = suma_wiersza + abs(W[i][j])

    if suma_wiersza > norma_wierszowa:
        norma_wierszowa = suma_wiersza

print(norma_wierszowa)
```

13.2. Norma kolumnowa

$$|W|_1 = \max_j \sum_{i=1}^n |w_{i,j}|$$

Czyli liczymy sumę modułów w każdej kolumnie i wybieramy największą.

Przykład w Pythonie

PYTHON

```
W = [
    [0.0, 0.5],
    [0.4, 0.0]
]

norma_kolumnowa = 0

for j in range(len(W[0])):
    suma_kolumny = 0

    for i in range(len(W)):
        suma_kolumny = suma_kolumny + abs(W[i][j])

    if suma_kolumny > norma_kolumnowa:
        norma_kolumnowa = suma_kolumny

print(norma_kolumnowa)
```

13.3. Norma Frobeniusa

$$|W|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n w_{i,j}^2}$$

Przykład w Pythonie

PYTHON

```
import math

W = [
    [0.0, 0.5],
    [0.4, 0.0]
]

suma = 0

for i in range(len(W)):
    for j in range(len(W[0])):
        suma = suma + W[i][j] ** 2

norma_frobeniusa = math.sqrt(suma)

print(norma_frobeniusa)
```

14. Sprawdzanie zbieżności — algorytm

Aby sprawdzić zbieżność metody iteracyjnej dla układu:

$$Ax = b$$

można wykonać następujące kroki:

1. Wyznacz macierz iteracji W :

$$w_{i,j} = \begin{cases} 0, & i = j \\ -\frac{a_{i,j}}{a_{i,i}}, & i \neq j \end{cases}$$

2. Oblicz normę macierzy:

$$|W|$$

3. Jeżeli:

$$|W| < 1$$

to metoda iteracyjna powinna być zbieżna.

4. W przeciwnym razie układ jest źle uwarunkowany dla tej metody albo należy zmienić metodę lub przekształcić układ.

```
A = [  
    [4.0, -2.0],  
    [-2.0, 5.0]  
]  
  
n = len(A)  
  
W = []  
  
for i in range(n):  
    wiersz = []  
  
    for j in range(n):  
        if i == j:  
            wiersz.append(0.0)  
        else:  
            wiersz.append(-A[i][j] / A[i][i])  
  
    W.append(wiersz)  
  
norma = 0  
  
for i in range(n):  
    suma_wiersza = 0  
  
    for j in range(n):  
        suma_wiersza = suma_wiersza + abs(W[i][j])  
  
    if suma_wiersza > norma:  
        norma = suma_wiersza  
  
print("W =", W)  
print("Norma =", norma)  
  
if norma < 1:  
    print("Warunek zbieżności jest spełniony")  
else:  
    print("Warunek zbieżności nie jest spełniony")
```

15. Zbieżność a uwarunkowanie

W wykładzie podkreślono różnicę między zbieżnością a uwarunkowaniem.

Pojęcie	Dotyczy	Pytanie	Zależne od	Warunek
Zbieżność	metody	czy działa?	macierzy W	$\ W\ < 1$
Uwarunkowanie	układu $Ax = b$	czy jest stabilny?	macierzy A	liczba uwarunkowania

Można mieć:

- zbieżność i złe uwarunkowanie,
- dobre uwarunkowanie i brak zbieżności.

Przykład w Pythonie

PYTHON

```

norma_W = 0.75
liczba_uwarunkowania = 1000

if norma_W < 1:
    print("Metoda iteracyjna powinna być zbieżna")
else:
    print("Metoda może być rozbieżna")

if liczba_uwarunkowania > 100:
    print("Układ może być źle uwarunkowany")
else:
    print("Układ nie wygląda na źle uwarunkowany")

```

16. Jak sprawdzić układ równań?

Dany jest układ:

$$Ax = b$$

Wykład podaje dwa kroki sprawdzania.

Krok 1. Czy metoda zadziała?

Sprawdzamy:

$$|W| < 1$$

Jeżeli tak, to metoda iteracyjna powinna być zbieżna.

Krok 2. Czy wynik będzie wiarygodny?

Sprawdzamy uwarunkowanie układu przez liczbę uwarunkowania:

$$\kappa(A) = |A| \cdot |A^{-1}|$$

Wniosek

Zbieżność oznacza, że algorytm działa.

Uwarunkowanie oznacza, czy wynik ma sens i czy błędy nie będą mocno wzmacniane.

Przykład w Pythonie

PYTHON

```
norma_W = 0.75
kappa_A = 20

if norma_W < 1:
    print("Krok 1: metoda powinna działać")
else:
    print("Krok 1: metoda może nie działać")

if kappa_A < 100:
    print("Krok 2: wynik powinien być w miarę wiarygodny")
else:
    print("Krok 2: wynik może być mało wiarygodny")
```

17. Najważniejsze rzeczy do zapamiętania na kolosa

17.1. Metody iteracyjne

Metody iteracyjne zaczynają od przybliżenia początkowego:

$$x^{(0)}$$

i tworzą ciąg:

$$x^{(0)}, x^{(1)}, x^{(2)}, \dots$$

który powinien zbiegać do rozwiązania.

17.2. Postać iteracyjna

Najważniejsza postać:

$$x = Wx + Z$$

oraz:

$$x^{(k)} = Wx^{(k-1)} + Z$$

17.3. Metoda Jacobiego

W metodzie Jacobiego każdą nową wartość liczymy tylko ze starych wartości:

$$x_i^{(k)} = \frac{1}{a_{i,i}} \left(b_i - \sum_{j=1, j \neq i}^n a_{i,j} x_j^{(k-1)} \right)$$

17.4. Metoda Gaussa-Seidla

W metodzie Gaussa-Seidla korzystamy z nowych i starych wartości:

$$x_i^{(k)} = \frac{1}{a_{i,i}} \left(b_i - \sum_{j=1}^{i-1} a_{i,j} x_j^{(k)} - \sum_{j=i+1}^n a_{i,j} x_j^{(k-1)} \right)$$

17.5. Warunek stopu

Przykładowy warunek stopu:

$$\frac{|x^{(k)} - x^{(k-1)}|}{|x^{(k)}|} \leq \epsilon$$

17.6. Warunek zbieżności

Metoda powinna być zbieżna, gdy:

$$|W| < 1$$

Dokładniejszy warunek teoretyczny:

$$\rho(W) < 1$$

17.7. Różnica między Jacobim a Gauss-Seidlem

Jacobi:

najpierw policz wszystko, potem zaktualizuj

Gauss-Seidel:

17.8. Zbieżność i uwarunkowanie

Zbieżność dotyczy metody.

Uwarunkowanie dotyczy układu:

$$Ax = b$$

Wykład 5: Równania nieliniowe (lab 7)

1. Wprowadzenie do równań nieliniowych

Równania nieliniowe są podstawą wielu zagadnień naukowych i inżynierskich.

Często nie da się ich rozwiązać prostym wzorem analitycznym, dlatego stosuje się metody numeryczne.

Równanie nieliniowe może mieć postać:

$$f(x) = 0$$

Szukamy takiej wartości:

$$x^*$$

dla której:

$$f(x^*) = 0$$

Wartość x^* nazywamy **miejszem zerowym funkcji** albo **pierwiastkiem równania**.

Przykład

Dla funkcji:

$$f(x) = x^2 - 4$$

miejscami zerowymi są:

$$x = -2$$

oraz:

$$x = 2$$

bo:

$$(-2)^2 - 4 = 0$$

oraz:

$$2^2 - 4 = 0$$

Przykład w Pythonie

```
def f(x):  
    return x ** 2 - 4  
  
x = 2  
  
wartosc = f(x)  
  
if wartosc == 0:  
    print("x jest miejscem zerowym")  
else:  
    print("x nie jest miejscem zerowym")  
  
print("f(x) =", wartosc)
```

PYTHON

2. Charakterystyka równań nieliniowych

Równania nieliniowe charakteryzują się tym, że zależność między zmiennymi nie jest liniowa.

Oznacza to, że zmiana jednej zmiennej nie powoduje proporcjonalnej zmiany drugiej zmiennej.

Przykłady równań nieliniowych z wykładu

Równanie wielomianowe:

$$x^3 - 6x^2 + 11x - 6 = 0$$

Równanie trygonometryczne:

$$\sin(x) + x^2 - 1 = 0$$

Równanie eksponencjalne:

$$e^x - x - 2 = 0$$

Równanie logarytmiczne:

$$\ln(x) + x^2 - 3 = 0$$

Równanie z funkcją specjalną:

$$J_0(x) + x^2 - 2 = 0$$

gdzie $J_0(x)$ jest funkcją Bessela pierwszego rodzaju zerowego rzędu.

Przykład w Pythonie

```
import math

def f1(x):
    return x ** 3 - 6 * x ** 2 + 11 * x - 6

def f2(x):
    return math.sin(x) + x ** 2 - 1

def f3(x):
    return math.exp(x) - x - 2

def f4(x):
    return math.log(x) + x ** 2 - 3

x = 1.5

print("f1(x) =", f1(x))
print("f2(x) =", f2(x))
print("f3(x) =", f3(x))
print("f4(x) =", f4(x))
```

PYTHON

3. Problemy przy rozwiązywaniu równań nieliniowych

Przy rozwiązywaniu równań nieliniowych mogą pojawić się problemy:

1. Dobór punktu startowego może wpływać na zbieżność metody.
2. Metoda może nie zbiegać do rozwiązania.
3. Metoda może zbiegać bardzo wolno.
4. Jeżeli funkcja ma wiele pierwiastków, metoda może dojść do różnych rozwiązań zależnie od punktu startowego.

Przykład

Funkcja:

$$f(x) = x^3 - 6x^2 + 11x - 6$$

ma kilka miejsc zerowych.

Dla różnych punktów startowych metoda numeryczna może znaleźć różne pierwiastki.

Przykład w Pythonie

```
def f(x):  
    return x ** 3 - 6 * x ** 2 + 11 * x - 6  
  
punkty = [0, 1, 2, 3, 4]  
  
for x in punkty:  
    print("x =", x, "f(x) =", f(x))
```

PYTHON

4. Podstawowe metody rozwiązywania równań nieliniowych

W wykładzie wymieniono następujące metody:

1. **Metoda bisekcji** — metoda podziału przedziału na pół.
2. **Metoda Newtona**, czyli metoda stycznych — wykorzystuje pochodną funkcji.
3. **Metoda siecznych** — podobna do metody Newtona, ale nie wymaga liczenia pochodnej.
4. **Metoda regula falsi** — metoda fałszywej pozycji, łącząca cechy metody siecznych i metod przedziałowych.

5. Ważne twierdzenia

5.1. Twierdzenie Darboux

Jeżeli funkcja:

$$f(x)$$

jest ciągła w przedziale domkniętym:

$$[a, b]$$

oraz η jest liczbą z przedziału:

$$[f(a), f(b)]$$

to istnieje takie:

$$c \in [a, b]$$

że:

$$u = f(c)$$

Sens twierdzenia

Funkcja ciągła przyjmuje wszystkie wartości pośrednie między wartościami na końcach przedziału.

Przykład w Pythonie

```
PYTHON

def f(x):
    return x ** 2

a = 1
b = 3

fa = f(a)
fb = f(b)

u = 4

if fa <= u <= fb:
    print("Wartość u leży między f(a) i f(b)")
    print("Dla tej funkcji istnieje c takie, że f(c) = u")
else:
    print("Nie sprawdzamy tym sposobem")
```

5.2. Twierdzenie Bolzano-Cauchy'ego

Jeżeli funkcja:

$$f(x)$$

jest ciągła w przedziale domkniętym:

$$[a, b]$$

oraz:

$$f(a) \cdot f(b) < 0$$

to między punktami **a** i **b** znajduje się co najmniej jeden pierwiastek równania:

$$f(x) = 0$$

Sens twierdzenia

Jeżeli funkcja na końcach przedziału ma różne znaki, to gdzieś pomiędzy musi przeciąć oś .

Przykład

Dla funkcji:

$$f(x) = x^2 - 4$$

na przedziale:

$$[1, 5]$$

mamy:

$$f(1) = -3$$

oraz:

$$f(5) = 21$$

czyli:

$$f(1) \cdot f(5) < 0$$

więc w przedziale istnieje pierwiastek.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x ** 2 - 4  
  
a = 1  
b = 5  
  
fa = f(a)  
fb = f(b)  
  
if fa * fb < 0:  
    print("W przedziale istnieje co najmniej jeden pierwiastek")  
else:  
    print("Nie można stwierdzić istnienia pierwiastka tym warunkiem")  
  
print("f(a) =", fa)  
print("f(b) =", fb)
```


5.3. Przedział izolacji pierwiastka

Jeżeli spełnione są założenia twierdzenia Bolzano-Cauchy'ego i dodatkowo znak pochodnej jest stały w przedziale:

$$\operatorname{sgn}(f'(x)) = \operatorname{const}$$

dla:

$$x \in [a, b]$$

to przedział ten jest **przedziałem izolacji pierwiastka** równania:

$$f(x) = 0$$

Sens

Przedział izolacji zawiera dokładnie jeden pierwiastek.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x ** 2 - 4  
  
def fp(x):  
    return 2 * x  
  
a = 1  
b = 5  
  
fa = f(a)  
fb = f(b)  
  
# Sprawdzamy znak pochodnej w kilku punktach przedziału  
punkty = [1, 2, 3, 4, 5]  
  
znak_dodatni = True  
  
for x in punkty:  
    if fp(x) <= 0:  
        znak_dodatni = False  
  
if fa * fb < 0 and znak_dodatni:  
    print("Przedział może być przedziałem izolacji pierwiastka")  
else:  
    print("Warunki nie są spełnione")
```

6. Funkcja signum

Funkcja signum, oznaczana jako:

$$\operatorname{sgn}(x)$$

jest zdefiniowana następująco:

$$\operatorname{sgn}(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases}$$

Czyli:

- zwraca **-1** dla wartości ujemnych,
- zwraca **0** dla zera,
- zwraca **1** dla wartości dodatnich.

Przykład w Pythonie

```
def sgn(x):  
    if x < 0:  
        return -1  
    elif x == 0:  
        return 0  
    else:  
        return 1  
  
wartosci = [-5, 0, 3]  
  
for x in wartosci:  
    print("x =", x, "sgn(x) =", sgn(x))
```

PYTHON

7. Pochodne funkcji

7.1. Charakterystyka pierwszej pochodnej

Pierwsza pochodna funkcji dostarcza informacji o tym, czy funkcja rośnie, czy maleje.

Jeżeli:

$$f'(x) > 0$$

to funkcja rośnie w danym przedziale.

Jeżeli:

$$f'(x) < 0$$

to funkcja maleje w danym przedziale.

Miejsca zerowe pochodnej mogą wskazywać potencjalne punkty ekstremalne funkcji.

Jeżeli funkcja najpierw rośnie, a potem maleje, to może mieć maksimum lokalne.

Jeżeli funkcja najpierw maleje, a potem rośnie, to może mieć minimum lokalne.

Przykład

Dla funkcji:

$$f(x) = x^2$$

pochodna wynosi:

$$f'(x) = 2x$$

Dla:

$$x > 0$$

funkcja rośnie, bo:

$$f'(x) > 0$$

Dla:

$$x < 0$$

funkcja maleje, bo:

$$f'(x) < 0$$

Przykład w Pythonie

```
def fp(x):  
    return 2 * x  
  
punkty = [-2, 0, 2]  
  
for x in punkty:  
    wartosc = fp(x)  
  
    if wartosc > 0:  
        print("x =", x, "funkcja rośnie")  
    elif wartosc < 0:  
        print("x =", x, "funkcja maleje")  
    else:  
        print("x =", x, "możliwe ekstremum")
```

PYTHON

7.2. Charakterystyka drugiej pochodnej

Druga pochodna funkcji informuje o krzywiznie funkcji.

Jeżeli:

$$f''(x) > 0$$

to funkcja jest wypukła w danym przedziale.

Jeżeli:

$$f''(x) < 0$$

to funkcja jest wklęsła w danym przedziale.

Punkty, w których:

$$f''(x) = 0$$

mogą oznaczać punkty przegięcia funkcji.

Przykład

Dla funkcji:

$$f(x) = x^2$$

druga pochodna wynosi:

$$f''(x) = 2$$

czyli funkcja jest wypukła.

Przykład w Pythonie

```
def fpp(x):  
    return 2  
  
x = 1  
wartosc = fpp(x)  
  
if wartosc > 0:  
    print("Funkcja jest wypukła")  
elif wartosc < 0:  
    print("Funkcja jest wklęsła")  
else:  
    print("Możliwy punkt przegięcia")
```

PYTHON

8. Metoda bisekcji

8.1. Idea metody bisekcji

Metoda bisekcji służy do znajdowania pierwiastka równania:

$$f(x) = 0$$

w przedziale:

$$[a, b]$$

Metoda wymaga, aby funkcja była ciągła i zmieniała znak na końcach przedziału:

$$f(a) \cdot f(b) < 0$$

W każdej iteracji dzielimy przedział na pół i wybieramy tę połowę, w której nadal występuje zmiana znaku.

Punkt środkowy

Punkt środkowy przedziału można liczyć jako:

$$x = \frac{a + b}{2}$$

ale w obliczeniach numerycznych lepiej liczyć:

$$x = a + \frac{b - a}{2}$$

Przykład w Pythonie

```
def f(x):  
    return x ** 3 - 3 * x ** 2 - 2 * x + 5  
  
a = 1.0  
b = 2.0  
  
x = a + (b - a) / 2  
  
print("Punkt środkowy:", x)  
print("f(a) =", f(a))  
print("f(b) =", f(b))  
print("f(x) =", f(x))
```

PYTHON

8.2. Algorytm bisekcji

Kroki metody bisekcji:

1. Wybierz przedział:

$$[a, b]$$

2. Sprawdź, czy:

$$f(a) \cdot f(b) < 0$$

3. Oblicz punkt środkowy:

$$x = a + \frac{b - a}{2}$$

4. Sprawdź znak:

$$f(x)$$

5. Wybierz nowy przedział, w którym funkcja zmienia znak.
6. Powtarzaj aż do spełnienia warunku stopu.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x ** 3 - 3 * x ** 2 - 2 * x + 5  
  
def sgn(x):  
    if x < 0:  
        return -1  
    elif x == 0:  
        return 0  
    else:  
        return 1  
  
a = 1.0  
b = 2.0  
epsilon = 0.001  
  
if sgn(f(a)) == sgn(f(b)):  
    print("Brak zmiany znaku na końcach przedziału")  
else:  
    for i in range(100):  
        x = a + (b - a) / 2  
  
        if abs(b - a) < epsilon:  
            break  
  
        if sgn(f(a)) != sgn(f(x)):  
            b = x  
        else:  
            a = x  
  
    print("Przybliżony pierwiastek:", x)  
    print("Liczba iteracji:", i + 1)  
    print("f(x) =", f(x))
```

Obliczanie kolejnych przybliżeń w metodzie bisekcji

W metodzie bisekcji rozpoczynamy od przedziału izolacji pierwiastka:

$$[a, b]$$

Zakładamy, że pierwsze dwie wartości ciągu to:

$$x_1 = a$$

oraz:

$$x_2 = b$$

Dla każdego kolejnego kroku iteracji:

$$i = 3, 4, \dots$$

nową wartość:

$$x_i$$

obliczamy jako średnią dwóch wcześniejszych punktów ograniczających aktualny przedział izolacji:

$$x_i = \frac{x_{i-1} + x_k}{2}$$

gdzie:

$$k \in i - 3, i - 2$$

Wybór wartości k jest taki, aby spełnione były warunki:

$$|x_i - x_{i-1}| = |x_i - x_k|$$

oraz:

$$f(x_{i-1}) \cdot f(x_k) < 0$$

Drugi warunek oznacza, że wartości funkcji na końcach wybranego przedziału mają przeciwne znaki, więc w tym przedziale znajduje się pierwiastek równania:

$$f(x) = 0$$

Prostsza interpretacja

W praktyce oznacza to, że w każdym kroku:

1. liczymy środek aktualnego przedziału,
2. sprawdzamy, po której stronie występuje zmiana znaku funkcji,
3. zostawiamy tylko tę połowę przedziału, w której nadal znajduje się pierwiastek.


```
def f(x):  
    return x ** 3 - 3 * x ** 2 - 2 * x + 5  
  
def sgn(x):  
    if x < 0:  
        return -1  
    elif x == 0:  
        return 0  
    else:  
        return 1  
  
# Początkowy przedział izolacji  
x1 = 1.0  
x2 = 2.0  
  
# Lista przechowuje kolejne wartości x_i  
x = [x1, x2]  
  
liczba_iteracji = 5  
  
for i in range(3, liczba_iteracji + 3):  
    # Aktualne końce przedziału  
    lewy = x[len(x) - 2]  
    prawy = x[len(x) - 1]  
  
    # Nowe przybliżenie jako środek przedziału  
    xi = lewy + (prawy - lewy) / 2  
  
    # Sprawdzamy, w której połowie jest zmiana znaku  
    if sgn(f(lewy)) != sgn(f(xi)):  
        # Pierwiastek jest między lewym końcem i środkiem  
        x.append(xi)  
    else:  
        # Pierwiastek jest między środkiem i prawym końcem  
        x[len(x) - 2] = xi  
  
    print("Iteracja:", i - 2)  
    print("Aktualne przybliżenie:", xi)  
    print("f(xi) =", f(xi))
```

8.3. Kryteria zakończenia metody bisekcji

W wykładzie podano następujące kryteria zakończenia iteracji:

1. Zadana liczba kroków.
2. Dostatecznie mały błąd.
3. Wartość funkcji dostatecznie bliska zeru.

Uwaga

Równość:

$$f(x) = 0$$

nie powinna być głównym kryterium zakończenia obliczeń, ponieważ przez błędy zaokrągleń uzyskanie dokładnego zera jest mało prawdopodobne.

Przykład w Pythonie

```
def f(x):  
    return x ** 3 - 3 * x ** 2 - 2 * x + 5  
  
a = 1.0  
b = 2.0  
epsilon = 0.001  
  
x = a + (b - a) / 2  
blad = abs(b - a)  
  
if blad < epsilon:  
    print("Przedział jest wystarczająco mały")  
  
if abs(f(x)) < epsilon:  
    print("Wartość funkcji jest bliska zeru")  
  
print("x =", x)  
print("błąd przedziału =", blad)  
print("f(x) =", f(x))
```

PYTHON

8.4. Błąd metody bisekcji

Dokładność przybliżenia w i -tym kroku można oszacować jako:

$$|x_i - x^*| < \frac{b - a}{2^{i-2}}$$

Oznacza to, że w każdej iteracji przedział izolacji pierwiastka zmniejsza się o połowę.

Przykład

Po 12 krokach metody bisekcji dla przedziału:

$$[1, 2]$$

mamy:

$$|x_{12} - x^*| < \frac{2-1}{2^{12-2}}$$

czyli:

$$|x_{12} - x^*| < \frac{1}{2^{10}}$$

Przykład w Pythonie

```
a = 1
b = 2
i = 12

blad = (b - a) / (2 ** (i - 2))

print("Oszacowanie błędu:", blad)
```

PYTHON

8.5. Wskazówki praktyczne dla bisekcji

Z wykładu:

1. Metoda znajduje jedno miejsce zerowe, a nie wszystkie miejsca zerowe w przedziale.
2. Nie należy kończyć obliczeń warunkiem:

$$f(x) = 0$$

3. Punkt środkowy lepiej liczyć ze wzoru:

$$a + \frac{b-a}{2}$$

zamiast:

$$\frac{a+b}{2}$$

4. Zmianę znaku lepiej badać przez:

$$\operatorname{sgn}(f(x_i)) \neq \operatorname{sgn}(f(x_j))$$

zamiast:

$$f(x_i) \cdot f(x_j) < 0$$

bo unikamy zbędnego mnożenia.

Przykład w Pythonie

PYTHON

```
def sgn(x):
    if x < 0:
        return -1
    elif x == 0:
        return 0
    else:
        return 1

def f(x):
    return x ** 3 - 3 * x ** 2 - 2 * x + 5

a = 1.0
b = 2.0

x = a + (b - a) / 2

if sgn(f(a)) != sgn(f(x)):
    print("Pierwiastek jest w przedziale [a, x]")
else:
    print("Pierwiastek jest w przedziale [x, b]")
```

8.6. Przykład metody bisekcji z wykładu

Rozważamy funkcję:

$$f(x) = x^3 - 3x^2 - 2x + 5$$

Szukamy pierwiastka w przedziale:

$$[1, 2]$$

Krok 1

Początkowy przedział:

$$[a_1, b_1] = [1, 2]$$

Punkt środkowy:

$$x_1 = \frac{1 + 2}{2} = 1.5$$

Wartości funkcji:

$$f(1) = 1$$

$$f(2) = -3$$

$$f(1.5) = -1.375$$

Ponieważ występuje zmiana znaku między 1 i 1.5, przechodzimy do przedziału:

$$[1, 1.5]$$

Krok 2

Nowy przedział:

$$[1, 1.5]$$

Punkt środkowy:

$$x_2 = 1.25$$

Wartość:

$$f(1.25) = -0.234$$

Przechodzimy do przedziału:

$$[1, 1.25]$$

Krok 3

Nowy przedział:

$$[1, 1.25]$$

Punkt środkowy:

$$x_3 = 1.125$$

Wartość:

$$f(1.125) = 0.376$$

Przechodzimy do przedziału:

$$[1.125, 1.25]$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x ** 3 - 3 * x ** 2 - 2 * x + 5  
  
def sgn(x):  
    if x < 0:  
        return -1  
    elif x == 0:  
        return 0  
    else:  
        return 1  
  
a = 1.0  
b = 2.0  
  
for i in range(5):  
    x = a + (b - a) / 2  
  
    print("Krok:", i + 1)  
    print("a =", a, "b =", b, "x =", x, "f(x) =", f(x))  
  
    if sgn(f(a)) != sgn(f(x)):  
        b = x  
    else:  
        a = x
```

9. Metoda stycznych Newtona

9.1. Idea metody Newtona

Metoda stycznych, czyli metoda Newtona, polega na przybliżaniu pierwiastka równania:

$$f(x) = 0$$

za pomocą miejsc zerowych stycznych do wykresu funkcji.

Wzór iteracyjny:

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$$

dla:

$$i = 2, 3, 4, \dots$$

Interpretacja

W każdym kroku budujemy styczną do wykresu funkcji w aktualnym punkcie, a miejsce przecięcia tej stycznej z osią **OX** traktujemy jako kolejne przybliżenie pierwiastka.

Przykład w Pythonie

```
PYTHON

def f(x):
    return x ** 2 - 4 * x + 3

def fp(x):
    return 2 * x - 4

x = 1.5

for i in range(5):
    if fp(x) == 0:
        print("Pochodna równa zero – nie można wykonać kroku")
        break

    x_nowe = x - f(x) / fp(x)

    print("Iteracja:", i + 1, "x =", x_nowe, "błąd =", abs(x_nowe - x))

    x = x_nowe
```

9.2. Wybór pierwszego przybliżenia w metodzie Newtona

Pierwsze przybliżenie:

$$x_1$$

często wybiera się spośród końców przedziału:

$$[a, b]$$

W wykładzie podano kryteria:

Jeżeli dla:

$$x \in [a, b]$$

zachodzi:

$$f'(x) \cdot f''(x) < 0$$

to wybieramy:

$$x_1 = a$$

Jeżeli:

$$f'(x) \cdot f''(x) > 0$$

to wybieramy:

$$x_1 = b$$

Przykład w Pythonie

PYTHON

```
def fp(x):
    return 2 * x - 4

def fpp(x):
    return 2

a = 0
b = 2

# Sprawdzamy znak iloczynu pochodnych w punkcie środkowym
x = a + (b - a) / 2

iloczyn = fp(x) * fpp(x)

if iloczyn < 0:
    x1 = a
elif iloczyn > 0:
    x1 = b
else:
    x1 = x

print("Wybrane pierwsze przybliżenie:", x1)
```

9.3. Błąd i zbieżność metody Newtona

Błąd w każdym kroku można oszacować wzorem:

$$\Delta \approx |x_i - x_{i-1}|$$

Metoda stycznych wykazuje szybką zbieżność, dokładniej zbieżność kwadratową.

Współczynnik zbieżności wynosi:

2

Metoda może być rozbieżna, jeżeli:

- pierwsze przybliżenie jest zbyt daleko od pierwiastka,
- funkcja nie jest dostatecznie gładka w otoczeniu pierwiastka.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x ** 2 - 4 * x + 3  
  
def fp(x):  
    return 2 * x - 4  
  
x = 1.5  
epsilon = 0.001  
  
for i in range(20):  
    if fp(x) == 0:  
        print("Pochodna równa zero")  
        break  
  
    x_nowe = x - f(x) / fp(x)  
    blad = abs(x_nowe - x)  
  
    print("Iteracja:", i + 1, "x =", x_nowe, "błąd =", blad)  
  
    x = x_nowe  
  
    if blad < epsilon:  
        break
```

9.4. Warunek zakończenia metody Newtona

Obliczenia kończymy, gdy:

$$|x_i - x_{i-1}| < \epsilon$$

gdzie:

$$\epsilon$$

jest wcześniej ustalonym progiem błędu.

Przykład w Pythonie

PYTHON

```
x_poprzednie = 0.9996951220
x_aktualne = 0.9999999535

epsilon = 0.001

blad = abs(x_aktualne - x_poprzednie)

if blad < epsilon:
    print("Warunek stopu spełniony")
else:
    print("Trzeba liczyć dalej")

print("Błąd =", blad)
```

9.5. Potencjalne problemy metody Newtona

Podczas stosowania metody stycznych mogą pojawić się problemy:

1. Iteracje mogą być rozbieżne przy złym wyborze punktu startowego.
2. Może nastąpić zatrzymanie postępu, gdy:

$$f'(x_{i-1})$$

jest bliskie zeru.

3. Mogą pojawić się trudności z oszacowaniem globalnej dokładności przybliżenia.

Przykład z wykładu

Dla funkcji:

$$f(x) = x^2 - 4x + 3$$

i punktu startowego:

$$x_1 = 2$$

pochodna wynosi:

$$f'(x) = 2x - 4$$

czyli:

$$f'(2) = 0$$

Nie można wtedy wykonać kroku Newtona, bo wystąpiłoby dzielenie przez zero.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x ** 2 - 4 * x + 3  
  
def fp(x):  
    return 2 * x - 4  
  
x = 2  
  
if fp(x) == 0:  
    print("Nie można zastosować metody Newtona, bo pochodna jest równa zero")  
else:  
    x_nowe = x - f(x) / fp(x)  
    print(x_nowe)
```

9.6. Przykład metody Newtona z wykładu

Funkcja:

$$f(x) = x^2 - 4x + 3$$

Pochodna:

$$f'(x) = 2x - 4$$

Dla punktu startowego:

$$x_0 = 1.5$$

mamy:

$$f(x_0) = -0.75$$

oraz:

$$f'(x_0) = -1$$

Pierwszy krok:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

czyli:

$$x_1 = 1.5 - \frac{-0.75}{-1} = 0.75$$

Następnie:

$$x_2 = 0.975$$

W tabeli z wykładu kolejne wartości zbliżają się do:

$$x = 1$$

Przykład w Pythonie

```
def f(x):  
    return x ** 2 - 4 * x + 3  
  
def fp(x):  
    return 2 * x - 4  
  
x = 1.5  
  
for i in range(5):  
    if fp(x) == 0:  
        print("Pochodna równa zero")  
        break  
  
    x_nowe = x - f(x) / fp(x)  
    blad = abs(x_nowe - x)  
  
    print("Iteracja:", i + 1)  
    print("x =", x_nowe)  
    print("błąd =", blad)  
  
    x = x_nowe
```

PYTHON

10. Metoda siecznych

10.1. Idea metody siecznych

Metoda siecznych służy do przybliżania pierwiastka równania:

$$f(x) = 0$$

Nie wymaga obliczania pochodnej funkcji.

Zamiast stycznej wykorzystuje prostą przechodzącą przez dwa punkty wykresu:

$$(x_i, f(x_i))$$

oraz:

$$(x_{i-1}, f(x_{i-1}))$$

Wzór na kolejne przybliżenie:

$$x_{i+1} = x_i - f(x_i) \cdot \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}$$

dla:

$$i = 2, 3, 4, \dots$$

Błąd przybliżenia można oszacować jako:

$$\Delta \approx |x_{i+1} - x_i|$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x ** 2 - 4 * x + 3  
  
x_poprzedni = 0.0  
x_aktualny = 2.0  
  
for i in range(5):  
    f_poprzedni = f(x_poprzedni)  
    f_aktualny = f(x_aktualny)  
  
    mianownik = f_aktualny - f_poprzedni  
  
    if mianownik == 0:  
        print("Nie można wykonać kroku, mianownik jest równy zero")  
        break  
  
    x_nowy = x_aktualny - f_aktualny * (x_aktualny - x_poprzedni) / mianownik  
  
    blad = abs(x_nowy - x_aktualny)  
  
    print("Iteracja:", i + 1, "x =", x_nowy, "błąd =", blad)  
  
    x_poprzedni = x_aktualny  
    x_aktualny = x_nowy
```

10.2. Wybór punktów startowych w metodzie siecznych

W wykładzie opisano wybór punktów startowych zależnie od znaków pierwszej i drugiej pochodnej.

Jeżeli:

$$f'(x) > 0$$

oraz:

$$f''(x) > 0$$

albo:

$$f'(x) < 0$$

oraz:

$$f''(x) < 0$$

to kolejne przybliżenia są z niedomiarem:

$$x_i < x_{i+1} < x_{i+2} < \dots < x^*$$

Jeżeli:

$$f'(x) > 0$$

oraz:

$$f''(x) < 0$$

albo:

$$f'(x) < 0$$

oraz:

$$f''(x) > 0$$

to kolejne przybliżenia są z nadmiarem:

$$x_i > x_{i+1} > x_{i+2} > \dots > x^*$$

Dla:

$$x \in [a, b]$$

punkty startowe można dobrać na podstawie iloczynu:

$$f'(x) \cdot f''(x)$$

Jeżeli:

$$f'(x) \cdot f''(x) < 0$$

to:

$$x_2 = a$$

oraz:

$$x_1 = b$$

Jeżeli:

$$f'(x) \cdot f''(x) > 0$$

to:

$$x_2 = b$$

oraz:

$$x_1 = a$$

Przykład w Pythonie

PYTHON

```
def fp(x):  
    return 2 * x - 4  
  
def fpp(x):  
    return 2  
  
a = 0  
b = 2  
  
x = a + (b - a) / 2  
  
iloczyn = fp(x) * fpp(x)  
  
if iloczyn < 0:  
    x2 = a  
    x1 = b  
elif iloczyn > 0:  
    x2 = b  
    x1 = a  
else:  
    x1 = a  
    x2 = b  
  
print("x1 =", x1)  
print("x2 =", x2)
```

10.3. Przykład metody siecznych z wykładu

Analizujemy funkcję:

$$f(x) = x^2 - 4x + 3$$

w przedziale:

$$[0, 2]$$

Punkty startowe:

$$x_0 = 0$$

oraz:

$$x_1 = 2$$

Krok 1

Obliczamy:

$$x_2 = 2 - f(2) \cdot \frac{2 - 0}{f(2) - f(0)}$$

Z wykładu:

$$x_2 = 1.5$$

Błąd:

$$\Delta \approx |1.5 - 2| = 0.5$$

Krok 2

Używamy:

$$x_1 = 2$$

oraz:

$$x_2 = 1.5$$

Obliczamy:

$$x_3 = 0$$

Błąd:

$$\Delta \approx |0 - 1.5| = 1.5$$

Dalsze iteracje z wykładu prowadzą do:

$$x = 1$$


```
def f(x):  
    return x ** 2 - 4 * x + 3  
  
x0 = 0.0  
x1 = 2.0  
  
for i in range(10):  
    f0 = f(x0)  
    f1 = f(x1)  
  
    mianownik = f1 - f0  
  
    if mianownik == 0:  
        print("Mianownik równy zero")  
        break  
  
    x2 = x1 - f1 * (x1 - x0) / mianownik  
  
    blad = abs(x2 - x1)  
  
    print("Iteracja:", i + 1, "x =", x2, "błąd =", blad)  
  
    x0 = x1  
    x1 = x2
```

10.4. Podsumowanie metody siecznych

Z wykładu:

- metoda siecznych nie wymaga liczenia pochodnych,
- metoda może nie być zbieżna przy źle dobranym przedziale,
- w ogólnym przypadku wymaga więcej iteracji niż metoda stycznych,
- współczynnik zbieżności wynosi około:

$$1.62$$

Metoda Newtona ma zwykle mniejszą liczbę iteracji, ale wymaga pochodnej.

```
czy_pochodna_dostepna = False

if czy_pochodna_dostepna:
    print("Można rozważyć metodę Newtona")
else:
    print("Można rozważyć metodę siecznych")
```

11. Metoda regula falsi

11.1. Idea metody regula falsi

Metoda **regula falsi**, czyli metoda fałszywej pozycji, jest metodą numeryczną rozwiązywania równań nieliniowych.

Łączy cechy:

- metody siecznych,
- metod przedziałowych.

Podobnie jak metoda siecznych, konstruuje prostą łączącą dwa punkty wykresu funkcji.

Jednak w przeciwieństwie do metody siecznych zachowuje przedział izolacji pierwiastka.

Zasada działania

Wybieramy przedział:

$$[a, b]$$

taki, że:

$$f(a)$$

oraz:

$$f(b)$$

mają przeciwne znaki.

Następnie rysujemy cięciwę między punktami:

$$(a, f(a))$$

oraz:

$$(b, f(b))$$

Punkt przecięcia tej cięciwy z osią **ox** jest nowym przybliżeniem pierwiastka.

Potem aktualizujemy jeden z końców przedziału tak, aby pierwiastek nadal pozostawał w przedziale.

Własności z wykładu

Metoda regula falsi:

- jest zawsze zbieżna, jeśli dobrze wybrano przedział początkowy,
- zachowuje przedział izolacji pierwiastka,
- ma wolną zbieżność,
- jest metodą liniową, czyli:

$$p = 1$$

Przykład w Pythonie

PYTHON

```
def f(x):
    return x ** 3 - 3 * x ** 2 - 2 * x + 5

def sgn(x):
    if x < 0:
        return -1
    elif x == 0:
        return 0
    else:
        return 1

a = 1.0
b = 2.0
epsilon = 0.001

if sgn(f(a)) == sgn(f(b)):
    print("Brak zmiany znaku – nie można zastosować metody")
else:
    x = a

    for i in range(100):
        fa = f(a)
        fb = f(b)

        mianownik = fb - fa

        if mianownik == 0:
            print("Mianownik równy zero")
            break

        x_nowe = a - fa * (b - a) / mianownik

        if abs(x_nowe - x) < epsilon:
            x = x_nowe
            break

    x = x_nowe

    if sgn(f(a)) != sgn(f(x)):
        b = x
    else:
        a = x

    print("Przybliżony pierwiastek:", x)
    print("f(x) =", f(x))
```

12. Porównanie metod

Metoda	Najważniejsza cecha	Zaleta	Wada
Bisekcja	dzieli przedział na pół	niezawodna przy zmianie znaku	wolna zbieżność
Newton	używa stycznej i pochodnej	szybka zbieżność	wymaga pochodnej i dobrego startu
Siecznych	używa dwóch punktów	nie wymaga pochodnej	może nie być zbieżna
Regula fałsi	zachowuje przedział izolacji	zbieżna przy dobrym przedziale	wolna zbieżność

13. Najważniejsze rzeczy do zapamiętania na kolosa

13.1. Miejsce zerowe

Miejsce zerowe funkcji to taka wartość:

$$x^*$$

że:

$$f(x^*) = 0$$

13.2. Twierdzenie Bolzano-Cauchy'ego

Jeżeli:

$$f(a) \cdot f(b) < 0$$

i funkcja jest ciągła na:

$$[a, b]$$

to w tym przedziale istnieje co najmniej jeden pierwiastek.

13.3. Funkcja signum

$$\operatorname{sgn}(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases}$$

13.4. Metoda bisekcji

Punkt środkowy najlepiej liczyć jako:

$$x = a + \frac{b - a}{2}$$

W każdym kroku przedział zmniejsza się o połowę.

13.5. Błąd bisekcji

$$|x_i - x^*| < \frac{b - a}{2^{i-2}}$$

13.6. Metoda Newtona

$$x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$$

Wymaga pochodnej.

Nie działa dobrze, gdy:

$$f'(x_{i-1}) = 0$$

albo gdy pochodna jest bliska zeru.

13.7. Metoda siecznych

$$x_{i+1} = x_i - f(x_i) \cdot \frac{x_i - x_{i-1}}{f(x_i) - f(x_{i-1})}$$

Nie wymaga pochodnej.

13.8. Metoda regula falsi

Metoda regula falsi zachowuje przedział izolacji pierwiastka i dlatego ma gwarancję zbieżności przy dobrze dobranym przedziale początkowym.

Wykład 6: Interpolacja (lab 8)

1. Co to jest interpolacja?

Interpolacja to proces znajdowania takiej funkcji, która przechodzi przez zadany zbiór punktów danych.

W matematyce i informatyce interpolacja jest podstawową techniką wykorzystywaną do estymacji wartości między znanymi punktami danych.

Jeżeli mamy dane punkty:

$$(x_i, y_i)$$

gdzie:

$$y_i = f(x_i)$$

dla:

$$i = 0, 1, 2, \dots, n$$

to szukamy funkcji interpolacyjnej:

$$W(x)$$

takiej, że:

$$W(x_i) = y_i$$

dla każdego węzła interpolacji.

Punkty:

$$(x_i, y_i)$$

nazywa się **węzłami interpolacji**.

Przykład

Dane są punkty:

$$(0, 1), (1, 3), (2, 2)$$

Interpolacja polega na znalezieniu funkcji, która przechodzi przez te punkty.

Przykład w Pythonie

PYTHON

```
punkty = [  
    [0, 1],  
    [1, 3],  
    [2, 2]  
]  
  
for punkt in punkty:  
    x = punkt[0]  
    y = punkt[1]  
    print("x =", x, "y =", y)
```

2. Do czego służy interpolacja?

Interpolacja ma szerokie zastosowanie, między innymi w:

- przetwarzaniu sygnałów i obrazów,
- aproksymacji funkcji w analizie numerycznej,
- symulacjach komputerowych,
- grafice komputerowej,
- inżynierii i naukach przyrodniczych do modelowania zjawisk.

Interpolacja pozwala oszacować wartości funkcji między punktami, które już znamy.

Przykład

Jeżeli znamy temperaturę o godzinie 10:00 i 12:00, to za pomocą interpolacji możemy oszacować temperaturę o godzinie 11:00.

Przykład w Pythonie

PYTHON

```
# Znanе dane:
# godzina 10 -> temperatura 20
# godzina 12 -> temperatura 24

x0 = 10
y0 = 20

x1 = 12
y1 = 24

x = 11

# Interpolacja liniowa
y = y0 + (y1 - y0) / (x1 - x0) * (x - x0)

print("Przybliżona temperatura o godzinie", x, "wynosi", y)
```

3. Rodzaje interpolacji

Wyróżnia się kilka podstawowych rodzajów interpolacji:

1. **Interpolacja wielomianowa**, np. metoda Lagrange'a i Newtona.
2. **Interpolacja funkcji sklepanych**, czyli splajny liniowe, kwadratowe i sześciennne.
3. **Interpolacja za pomocą krzywych Béziera**.
4. **Interpolacja Hermite'a**.

4. Funkcja interpolacyjna

Funkcja interpolacyjna:

$$W(x)$$

jest konstruowana tak, aby dokładnie przechodziła przez dane punkty.

Warunek interpolacji:

$$W(x_i) = y_i$$

dla:

$$i = 0, 1, 2, \dots, n$$

Celem interpolacji jest:

- przybliżenie wartości funkcji w punktach poza danymi węzłami,
- oszacowanie błędu wartości przybliżonych.

Przykład

Jeżeli:

$$W(0) = 1$$

oraz:

$$W(1) = 3$$

to funkcja interpolacyjna musi przechodzić przez punkty:

$$(0, 1)$$

oraz:

$$(1, 3)$$

Przykład w Pythonie

PYTHON

```
def W(x):  
    # Prosta przechodząca przez punkty (0,1) i (1,3)  
    return 1 + 2 * x  
  
punkty = [  
    [0, 1],  
    [1, 3]  
]  
  
for punkt in punkty:  
    x = punkt[0]  
    y = punkt[1]  
  
    if W(x) == y:  
        print("W(", x, ") =", W(x), "zgadza się z y =", y)  
    else:  
        print("Punkt nie leży na funkcji")
```

5. Konstrukcja funkcji interpolacyjnej

Funkcja interpolacyjna jest konstruowana jako kombinacja liniowa funkcji bazowych:

$$W(x) = \sum_{i=0}^n a_i \varphi_i(x)$$

czyli:

$$W(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \cdots + a_n\varphi_n(x)$$

gdzie:

- $\varphi_i(x)$
— funkcje bazowe,
- a_i
— współczynniki wyznaczone na podstawie danych węzłów interpolacji.

Zapis macierzowy

Wprowadzamy macierz bazową:

$$\Phi(x) = [\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x)]$$

oraz wektor współczynników:

$$A = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix}$$

Wtedy:

$$W(x) = \Phi(x) \cdot A$$

Po podstawieniu $n + 1$ węzłów tworzy się układ $n + 1$ równań z $n + 1$ niewiadomymi:

$$W(x_0) = a_0\varphi_0(x_0) + a_1\varphi_1(x_0) + \cdots + a_n\varphi_n(x_0) = y_0$$

$$W(x_1) = a_0\varphi_0(x_1) + a_1\varphi_1(x_1) + \cdots + a_n\varphi_n(x_1) = y_1$$

...

$$W(x_n) = a_0\varphi_0(x_n) + a_1\varphi_1(x_n) + \cdots + a_n\varphi_n(x_n) = y_n$$

W zapisie macierzowym:

$$X \cdot A = Y$$

czyli:

$$\begin{bmatrix} \varphi_0(x_0) & \varphi_1(x_0) & \cdots & \varphi_n(x_0) \\ \varphi_0(x_1) & \varphi_1(x_1) & \cdots & \varphi_n(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_0(x_n) & \varphi_1(x_n) & \cdots & \varphi_n(x_n) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

gdzie:

- X — macierz główna układu,
- A — wektor współczynników,
- Y — wektor wartości funkcji.

Jeżeli macierz X jest nieosobliwa, czyli:

$$\det(X) \neq 0$$

to współczynniki można zapisać jako:

$$A = X^{-1} \cdot Y$$

a wielomian interpolacyjny:

$$W(x) = \Phi(x) \cdot X^{-1} \cdot Y$$

Przykład w Pythonie

PYTHON

```
# Przykład dla bazy:
# phi0(x) = 1
# phi1(x) = x
# Szukamy W(x) = a0 + a1*x dla punktów (0,1), (1,3)

x0 = 0
y0 = 1

x1 = 1
y1 = 3

# Układ:
# a0 + a1*x0 = y0
# a0 + a1*x1 = y1

a1 = (y1 - y0) / (x1 - x0)
a0 = y0 - a1 * x0

print("a0 =", a0)
print("a1 =", a1)

def W(x):
    return a0 + a1 * x

print("W(0.5) =", W(0.5))
```

6. Tablicowanie a interpolacja

Interpolację można rozumieć jako proces odwrotny do tablicowania funkcji.

Tablicowanie

Tablicowanie polega na stworzeniu tablicy wartości dla danej funkcji.

Używa się go wtedy, gdy znana jest analityczna postać funkcji.

Czyli mamy wzór funkcji, np.:

$$f(x) = x^2$$

i liczymy wartości dla kolejnych argumentów.

Interpolacja

Interpolacja polega na wyznaczeniu analitycznej formy funkcji na podstawie zestawu jej wartości.

Czyli mamy punkty, np.:

$$(0, 0), (1, 1), (2, 4)$$

i szukamy funkcji, która przez nie przechodzi.

Przykład w Pythonie

```
def f(x):  
    return x * x  
  
# Tablicowanie funkcji  
wartosci = []  
  
for x in range(0, 4):  
    y = f(x)  
    wartosci.append([x, y])  
  
print("Tablica wartości:")  
  
for para in wartosci:  
    print(para)
```

PYTHON

7. Poszukiwanie funkcji interpolacyjnej

W procesie interpolacji zazwyczaj dąży się do znalezienia funkcji interpolacyjnej o wcześniej założonej formie.

Przykłady takich funkcji:

- wielomiany algebraiczne,
- funkcje trygonometryczne,
- inne postacie funkcji dopasowane do charakteru danych.

Najczęściej w praktyce używa się wielomianów algebraicznych, ponieważ są łatwe do definiowania i obliczania.

Przykład w Pythonie

PYTHON

```
# Przykład różnych możliwych postaci funkcji przybliżającej

def wielomian(x):
    return 1 + 2 * x - x * x

def trygonometryczna(x):
    import math
    return math.sin(x)

x = 0.5

print("Wielomian:", wielomian(x))
print("Funkcja trygonometryczna:", trygonometryczna(x))
```

8. Interpolacja wielomianowa

Interpolacja wielomianowa jest powszechnie stosowaną metodą interpolacji.

Opiera się na bazie jednomianów:

$$\varphi_0(x) = 1$$

$$\varphi_1(x) = x$$

$$\varphi_2(x) = x^2$$

...

$$\varphi_n(x) = x^n$$

Wielomian interpolacyjny ma postać:

$$W_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

Po podstawieniu wartości w węzłach otrzymujemy układ równań:

$$W_n(x_0) = a_0 + a_1x_0 + \dots + a_nx_0^n = y_0$$

$$W_n(x_1) = a_0 + a_1x_1 + \dots + a_nx_1^n = y_1$$

...

$$W_n(x_n) = a_0 + a_1x_n + \dots + a_nx_n^n = y_n$$

Warunek interpolacji wymaga, aby:

$$W_n(x_i) = y_i$$

dla:

$$i = 0, 1, \dots, n$$

Układ równań ma jednoznaczne rozwiązanie, gdy wszystkie węzły:

$$x_i$$

są różne.

Przykład w Pythonie

PYTHON

```
# Wielomian W(x) = a0 + a1*x + a2*x^2
# Pokazujemy obliczanie wartości dla znanych współczynników

a0 = 1
a1 = 2
a2 = -1

def W(x):
    return a0 + a1 * x + a2 * x * x

punkty_x = [0, 1, 2, 3]

for x in punkty_x:
    print("x =", x, "W(x) =", W(x))
```

9. Macierz Vandermonde'a

Dla interpolacji wielomianowej macierz główna układu ma postać macierzy Vandermonde'a:

$$X = \begin{bmatrix} 1 & x_0 & \dots & x_0^n \\ 1 & x_1 & \dots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \dots & x_n^n \end{bmatrix}$$

Wyznacznik macierzy Vandermonde'a:

$$D = \det(X) = \prod_{0 \leq j < i \leq n} (x_i - x_j)$$

Jeżeli wszystkie węzły są różne, to:

$$D \neq 0$$

Macierz odwrotna do bazy wielomianowej jest czasami nazywana macierzą Lagrange'a.

Uwaga numeryczna

Ta metoda interpolacji jest matematycznie elegancka, ale może nie być efektywna numerycznie.

Macierz  jest pełna i może być źle uwarunkowana, co prowadzi do ryzyka dużych błędów w obliczeniach numerycznych.

Przykład z wykładu

Dla punktów:

$$x_0 = 2$$

$$x_1 = 3$$

$$x_2 = 4$$

wyznacznik Vandermonde'a:

$$D = \begin{vmatrix} 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 4 & 16 \end{vmatrix}$$

Korzystając ze wzoru:

$$D = (x_1 - x_0)(x_2 - x_0)(x_2 - x_1)$$

otrzymujemy:

$$D = (3 - 2)(4 - 2)(4 - 3) = 2$$

Przykład w Pythonie

```
x = [2, 3, 4]

D = 1

for i in range(len(x)):
    for j in range(i):
        D = D * (x[i] - x[j])

print("Wyznacznik Vandermonde'a =", D)
```

PYTHON

10. Obliczanie współczynników wielomianu

Wartości współczynników:

$$a_i$$

można obliczać ze wzoru wynikającego z twierdzenia Cramera:

$$a_i = \frac{1}{D} \sum_{j=0}^n y_j X_{j+1, i+1}$$

gdzie:

- D — wyznacznik macierzy głównej układu,
- $X_{j+1, i+1}$ — dopełnienia algebraiczne elementów z odpowiedniej kolumny macierzy głównej.

Przykład w Pythonie

Poniżej prosty przykład dla wielomianu liniowego przechodzącego przez dwa punkty.

```
PYTHON

# Punkty: (0,1), (1,3)
# Szukamy W(x)=a0+a1*x

x0 = 0
y0 = 1

x1 = 1
y1 = 3

D = x1 - x0

if D != 0:
    a1 = (y1 - y0) / D
    a0 = y0 - a1 * x0

    print("a0 =", a0)
    print("a1 =", a1)
else:
    print("Nie można wyznaczyć wielomianu, bo węzły są takie same")
```

11. Funkcje interpolacyjne i przybliżanie funkcji

Funkcje odgrywają kluczową rolę w modelowaniu matematycznym, ponieważ służą jako narzędzia do opisu relacji między różnymi zmiennymi.

W obliczeniach numerycznych pojawia się problem przybliżonego przedstawiania funkcji. Chodzi o to, aby możliwe było obliczenie jej wartości dla dowolnych argumentów z przedziału:

$$\langle a, b \rangle$$

za pomocą ograniczonej liczby operacji arytmetycznych i logicznych.

Zamiast pracować bezpośrednio na funkcji oryginalnej:

$$f$$

wybiera się funkcję przybliżającą:

$$\tilde{f}$$

która ma reprezentować oryginalną funkcję.

Wybór funkcji przybliżającej:

$$\tilde{f}$$

może zależeć od wielu czynników. Ważne jest, aby taka funkcja umożliwiała proste obliczenie jej wartości.

Dlatego często wybiera się wielomiany algebraiczne jako funkcje przybliżające, ponieważ są łatwe do definiowania i obliczania.

Wielomian algebraiczny ma postać:

$$W_n(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \dots + a_nx^n$$

Taki wielomian jest często stosowany jako funkcja przybliżająca, ponieważ można go zdefiniować za pomocą skończonej liczby współczynników:

$$a_0, a_1, a_2, \dots, a_n$$

oraz łatwo obliczać jego wartości.

Interpolacja za pomocą wielomianów umożliwia przybliżenie dowolnej funkcji. Jeżeli argument:

$$x$$

nie jest węzłem interpolacji, to wartość:

$$W_n(x)$$

reprezentuje estymację wartości:

$$y$$

czyli przybliżoną wartość funkcji w tym punkcie.

Błąd przybliżenia to różnica między funkcją oryginalną a funkcją przybliżającą. Można go oceniać przez porównanie wartości funkcji oryginalnej i wielomianu interpolacyjnego w wybranych punktach poza węzłami interpolacji.

Czyli porównujemy:

$$f(x)$$

oraz:

$$W_n(x)$$

a błąd można zapisać jako:

$$|f(x) - W_n(x)|$$

Przykład

Założmy, że funkcją oryginalną jest:

$$f(x) = x^2$$

a funkcją przybliżającą jest wielomian:

$$W_1(x) = 2x - 1$$

Dla punktu:

$$x = 1.5$$

możemy porównać wartości:

$$f(1.5)$$

oraz:

$$W_1(1.5)$$

Różnica między tymi wartościami jest błędem przybliżenia.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
def W(x):  
    return 2 * x - 1  
  
x = 1.5  
  
wartosc_f = f(x)  
wartosc_W = W(x)  
  
blad = abs(wartosc_f - wartosc_W)  
  
print("f(x) =", wartosc_f)  
print("W(x) =", wartosc_W)  
print("Błąd przybliżenia =", blad)
```

12. Przykład interpolacji funkcji $\sin(\pi x)$

W wykładzie podano przykład interpolacji funkcji:

$$\sin(\pi x)$$

w przedziale:

$$[-1, 1]$$

dla pięciu węzłów interpolacji.

Dane węzły:

$$x_0 = -1, \quad y_0 = 0$$

$$x_1 = -0.5, \quad y_1 = -1$$

$$x_2 = 0, \quad y_2 = 0$$

$$x_3 = 0.5, \quad y_3 = 1$$

$$x_4 = 1, \quad y_4 = 0$$

Macierz Vandermonde'a:

$$X = \begin{bmatrix} 1 & x_0 & x_0^2 & x_0^3 & x_0^4 \\ 1 & x_1 & x_1^2 & x_1^3 & x_1^4 \\ 1 & x_2 & x_2^2 & x_2^3 & x_2^4 \\ 1 & x_3 & x_3^2 & x_3^3 & x_3^4 \\ 1 & x_4 & x_4^2 & x_4^3 & x_4^4 \end{bmatrix}$$

Podstawiając wartości węzłów:

$$X = \begin{bmatrix} 1 & -1 & 1 & -1 & 1 \\ 1 & -0.5 & 0.25 & -0.125 & 0.0625 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0.5 & 0.25 & 0.125 & 0.0625 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Wyznacznik $D = \det(X)$ obliczamy jako:

$$D = (x_1 - x_0)(x_2 - x_0)(x_3 - x_0)(x_4 - x_0) \cdot (x_2 - x_1)(x_3 - x_1)(x_4 - x_1) \cdot (x_3 - x_2)(x_4 - x_2) \cdot (x_4 - x_3)$$

Co daje:

$$D = \frac{9}{32}$$

Po obliczeniu współczynników:

$$a_0 = 0$$

$$a_1 = \frac{8}{3}$$

$$a_2 = 0$$

$$a_3 = -\frac{8}{3}$$

$$a_4 = 0$$

Ostatecznie wielomian interpolacyjny:

$$W_3(x) = \frac{8}{3}x - \frac{8}{3}x^3$$

Przykład w Pythonie

PYTHON

```
def W(x):
    return (8 / 3) * x - (8 / 3) * x * x * x

wezly = [-1, -0.5, 0, 0.5, 1]

for x in wezly:
    print("x =", x, "W(x) =", W(x))
```

13. Interpolacja Lagrange'a

13.1. Funkcje bazowe Lagrange'a

Dla $n + 1$ węzłów:

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

funkcje bazowe są zdefiniowane jako:

$$\varphi_i(x) = \prod_{j=0, j \neq i}^n (x - x_j)$$

gdzie dla każdego i funkcja:

$$\varphi_i(x)$$

pomija czynnik:

$$(x - x_i)$$

i jest wielomianem stopnia n .

Wielomian interpolacyjny Lagrange'a można zapisać w uproszczonej formie:

$$W(x) = \sum_{i=0}^n y_i \left(\prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} \right)$$

Ta postać daje bezpośrednią metodę obliczania wartości wielomianu interpolacyjnego w dowolnym punkcie x .

Przykład

Dla dwóch punktów:

$$(x_0, y_0)$$

oraz:

$$(x_1, y_1)$$

wielomian Lagrange'a:

$$W(x) = y_0 \frac{x - x_1}{x_0 - x_1} + y_1 \frac{x - x_0}{x_1 - x_0}$$

Przykład w Pythonie

PYTHON

```
x_wezly = [0, 1, 2]
y_wezly = [1, 3, 2]

x = 1.5

W = 0

for i in range(len(x_wezly)):
    L = 1

    for j in range(len(x_wezly)):
        if j != i:
            L = L * (x - x_wezly[j]) / (x_wezly[i] - x_wezly[j])

    W = W + y_wezly[i] * L

print("W(", x, ") =", W)
```

14. Interpolacja Newtona

Interpolacja Newtona wykorzystuje **ilorazy różnicowe**.

Wielomian interpolacyjny Newtona:

$$p(x) = \sum_{k=0}^n f[x_0, x_1, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j)$$

czyli można też zapisać:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots$$

gdzie współczynniki:

$$a_k$$

są ilorazami różnicowymi:

$$a_k = f[x_0, x_1, \dots, x_k]$$

Iloraz różnicowy rzędu zerowego

$$f[x_i] = f(x_i)$$

Iloraz różnicowy rzędu pierwszego

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$

Iloraz różnicowy rzędu k

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Ilorazy różnicowe można obliczać tablicą trójkątną.

Dla $i = 0, \dots, n$

Przykład w Pythonie — tablica ilorazów różnicowych

PYTHON

```
x = [0.0, 1.0, 2.0]
y = [1.0, 3.0, 2.0]

n = len(x)

# Tworzymy tablicę ilorazów różnicowych
tablica = []

for i in range(n):
    wiersz = []

    for j in range(n):
        wiersz.append(0.0)

    tablica.append(wiersz)

# Pierwsza kolumna to wartości funkcji
for i in range(n):
    tablica[i][0] = y[i]

# Kolejne kolumny
for j in range(1, n):
    for i in range(n - j):
        licznik = tablica[i + 1][j - 1] - tablica[i][j - 1]
        mianownik = x[i + j] - x[i]
        tablica[i][j] = licznik / mianownik

print("Tablica ilorazów różnicowych:")
for wiersz in tablica:
    print(wiersz)
```

14.1. Współczynniki Newtona w jednej tablicy

Ze wskazówek do laboratorium: współczynniki wielomianu Newtona można wyznaczać metodą ilorazów różnicowych, ale nie trzeba przechowywać całej tablicy trójkątnej.

Można użyć jednej tablicy i nadpisywać wartości.

Na początku tablica współczynników zawiera wartości:

$$a_i = y_i$$

Potem kolejne współczynniki wyznaczamy, idąc od końca tablicy.

Schemat aktualizacji:

$$a_i = \frac{a_i - a_{i-1}}{x_i - x_{i-j}}$$

Złożoność takiego algorytmu wynosi:

$$O(n^2)$$

Przykład w Pythonie

```
x = [0.0, 1.0, 2.0]
y = [1.0, 3.0, 2.0]

n = len(x)

# Kopiujemy y do tablicy współczynników a
a = []

for i in range(n):
    a.append(y[i])

# Obliczamy ilorazy różnicowe w jednej tablicy
for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
        licznik = a[i] - a[i - 1]
        mianownik = x[i] - x[i - j]
        a[i] = licznik / mianownik

print("Współczynniki Newtona:")
for wspolczynnik in a:
    print(wspolczynnik)
```

PYTHON

14.2. Obliczanie wartości wielomianu Newtona

Wielomian Newtona ma postać:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots$$

Wartość tego wielomianu można liczyć bezpośrednio, ale wygodniej i szybciej użyć schematu Hornera dla postaci Newtona.

Schemat Hornera dla postaci Newtona:

$$P(x) = a_n + (x - x_{n-1})(a_{n-1} + (x - x_{n-2})(\dots))$$

W praktyce iterujemy od końca:

```
result = result * (X - x_i) + a_i
```

Złożoność obliczania wartości wielomianu tym sposobem:

$$O(n)$$

Przykład w Pythonie

```
x_wezly = [0.0, 1.0, 2.0]
a = [1.0, 2.0, -1.5]

X = 1.5

# Zaczynamy od ostatniego współczynnika
result = a[len(a) - 1]

for i in range(len(a) - 2, -1, -1):
    result = result * (X - x_wezly[i]) + a[i]

print("P(", X, ") =", result)
```

PYTHON

15. Aproksymacja funkcji za pomocą szeregów Maclaurina

Ze wskazówek do laboratorium wynika, że w praktyce często zamiast dokładnej funkcji można liczyć jej przybliżenie za pomocą skończonej sumy szeregu Maclaurina.

Nie jest to interpolacja w ścisłym sensie, ale jest powiązane z tematem przybliżania funkcji wartościami obliczanymi numerycznie.

15.1. Aproksymacja funkcji e^x

Funkcja eksponencjalna ma rozwinięcie Maclaurina:

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

W praktyce stosuje się sumę skończoną:

$$e^x \approx \sum_{k=0}^n \frac{x^k}{k!}$$

Wskazówki implementacyjne

Obliczanie silni w każdej iteracji jest kosztowne.

Lepiej użyć zależności między kolejnymi wyrazami szeregu:

$$\frac{x^k}{k!} = \frac{x^{k-1}}{(k-1)!} \cdot \frac{x}{k}$$

Dzięki temu nie trzeba osobno liczyć:

$$x^k$$

ani:

$$k!$$

Nie trzeba też używać funkcji `pow()`.

Przykład w Pythonie

```
import math

x = 1.0
n = 10

suma = 1.0
wyraz = 1.0

for k in range(1, n + 1):
    wyraz = wyraz * x / k
    suma = suma + wyraz

print("Przybliżenie e^x =", suma)
print("Wartość biblioteczna =", math.exp(x))
print("Błąd =", abs(suma - math.exp(x)))
```

PYTHON

15.2. Aproksymacja funkcji $\sin x$

Rozwinięcie Maclaurina funkcji sinus:

$$\sin x = \sum_{k=1}^{\infty} (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

Aproksymacja skończona:

$$\sin x \approx \sum_{k=1}^n (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

Wskazówki implementacyjne

Dla dużych wartości x szereg może zbiegać wolno.

Warto redukować argument:

$$x \mapsto x \bmod 2\pi$$

Współczynnik:

$$(-1)^{k-1}$$

można aktualizować zmienną, bez używania `pow()`.

Przykład w Pythonie

```
import math

x = 1.0
n = 10

# Redukcja argumentu
x = x % (2 * math.pi)

suma = 0.0

# Pierwszy wyraz szeregu to x
wyraz = x
znak = 1

for k in range(1, n + 1):
    suma = suma + znak * wyraz

    # Przechodzimy do kolejnego wyrazu:
    # x^(2k+1)/(2k+1)! z x^(2k-1)/(2k-1)!
    mianownik = (2 * k) * (2 * k + 1)
    wyraz = wyraz * x * x / mianownik

    znak = -znak

print("Przybliżenie sin(x) =", suma)
print("Wartość biblioteczna =", math.sin(x))
print("Błąd =", abs(suma - math.sin(x)))
```

PYTHON

15.3. Wnioski praktyczne z aproksymacji Maclaurina

Rozwinięcia Maclaurina są proste, ale mogą być numerycznie niestabilne.

Biblioteki standardowe są zwykle lepiej zoptymalizowane i dokładniejsze.

W praktyce warto:

- unikać kosztownego liczenia silni w każdej iteracji,
- unikać `pow()` tam, gdzie wystarczy mnożenie,
- porównywać wynik z funkcją biblioteczną,
- mierzyć czas wykonania programu,
- pamiętać, że dla dużych argumentów niektóre szeregi zbiegać mogą wolno.

Przykład w Pythonie — bardzo prosty pomiar czasu

PYTHON

```
import time
import math

x = 1.0
n = 100000

start = time.time()

suma = 1.0
wyras = 1.0

for k in range(1, n + 1):
    wyras = wyras * x / k
    suma = suma + wyras

koniec = time.time()

print("Wynik =", suma)
print("math.exp(x) =", math.exp(x))
print("Czas =", koniec - start)
```

16. Funkcje interpolacyjne i przybliżanie funkcji

Funkcje odgrywają ważną rolę w modelowaniu matematycznym, ponieważ opisują relacje między zmiennymi.

W obliczeniach numerycznych często pojawia się problem przybliżonego przedstawienia funkcji tak, aby można było obliczać jej wartości dla dowolnych argumentów z przedziału:

$$[a, b]$$

za pomocą ograniczonej liczby operacji arytmetycznych i logicznych.

Wielomiany algebraiczne są często wybierane jako funkcje przybliżające, ponieważ:

- łatwo je zdefiniować,
- łatwo obliczać ich wartości,
- mają skończoną liczbę współczynników.

Przykładowy wielomian:

$$W_n(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \dots + a_nx^n$$

Przykład w Pythonie

PYTHON

```
# Obliczanie wartości wielomianu zwykłą metodą

a = [1, 2, -1, 0.5]
x = 2

wynik = 0
potega = 1

for i in range(len(a)):
    wynik = wynik + a[i] * potega
    potega = potega * x

print("Wartość wielomianu =", wynik)
```

17. Błąd interpolacji

Wielomian interpolacyjny:

$$W_n(x)$$

dla argumentów x , które nie są węzłami, reprezentuje przybliżoną wartość funkcji.

Błąd przybliżenia to różnica między funkcją oryginalną a funkcją przybliżającą.

Dla wielomianu interpolacyjnego stopnia n , przybliżającego funkcję:

$$f(x)$$

na podstawie $n + 1$ węzłów, istnieje taka liczba:

$$\xi \in (a, b)$$

że reszta interpolacji ma postać:

$$r(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \cdot p_n(x)$$

gdzie:

$$p_n(x) = (x - x_0)(x - x_1) \dots (x - x_n)$$

Sens

Błąd zależy od:

- pochodnej rzędu **n+1**,
- rozmieszczenia węzłów,
- punktu, w którym liczymy wartość.

Przykład w Pythonie

```
# Liczymy p_n(x) = (x-x0)(x-x1)...(x-xn)

wezly = [0, 1, 2]
x = 1.5

p = 1

for xi in wezly:
    p = p * (x - xi)

print("p_n(x) =", p)
```

PYTHON

18. Zbieżność wielomianów interpolacyjnych

18.1. Twierdzenie Fabera

Dla dowolnego ciągu układów węzłów:

$$a \leq x_0 < x_1 < \dots < x_n \leq b$$

istnieje taka funkcja ciągła w:

$$[a, b]$$

że ciąg wielomianów interpolacyjnych zbudowanych dla tych węzłów nie jest do niej zbieżny.

Sens

Nie zawsze zwiększanie liczby węzłów poprawia interpolację.

18.2. Drugie twierdzenie o zbieżności

Jeżeli f jest funkcją ciągłą w:

$$[a, b]$$

to istnieje taki ciąg układów węzłów:

$$a \leq x_0 < x_1 < \dots < x_n \leq b$$

że zbudowane dla nich wielomiany interpolacyjne tworzą ciąg zbieżny do f .

Sens

Można dobrać takie węzły, aby interpolacja była zbieżna.

Przykład w Pythonie

```
liczby_wezlow = [3, 5, 10, 20]

for n in liczby_wezlow:
    print("Liczba węzłów:", n)
```

PYTHON

19. Efekt Rungego

Efekt Rungego oznacza, że zwiększenie liczby węzłów interpolacji lub stopnia wielomianu interpolacyjnego nie zawsze prowadzi do lepszego przybliżenia funkcji.

Problem pojawia się szczególnie, gdy:

- interpolowana funkcja jest przybliżana wielomianem wysokiego stopnia,
- węzły interpolacyjne są równoodległe,
- patrzymy na końce przedziału interpolacji.

Wielomiany wysokiego stopnia mogą wykazywać oscylacje między punktami węzłowymi, co prowadzi do dużego wzrostu błędu na krańcach przedziału.

Rozwiązanie problemu

Aby ograniczyć efekt Rungego, można użyć:

1. interpolacji kawałkowej, np. funkcji sklejanych,
2. węzłów Czebyszewa zamiast równoodległych punktów.

Przykład w Pythonie

PYTHON

```
# Przykład tworzenia równoodległych węzłów w przedziale [-1,1]

a = -1
b = 1
n = 5

wezly = []

for i in range(n):
    x = a + i * (b - a) / (n - 1)
    wezly.append(x)

print("Równoodległe węzły:")
for x in wezly:
    print(x)
```

20. Interpolacja funkcjami sklejanymi

Interpolacja funkcjami sklejanymi, czyli **spline interpolation**, to metoda przybliżania funkcji za pomocą kawałkami wielomianów niskiego stopnia.

Wielomiany te są sklepane tak, aby całość była gładka.

Funkcja sklejana stopnia k :

- na każdym podprzedziale jest wielomianem stopnia co najwyżej k ,
- ma ciągłe pochodne do rzędu:

$$k - 1$$

w punktach sklejenia.

Sens

Zamiast jednego wielomianu wysokiego stopnia używa się kilku wielomianów niskiego stopnia na mniejszych podprzedziałach.

Dzięki temu można zmniejszyć oscylacje.

Przykład w Pythonie

PYTHON

```
# Dane punkty
x = [0, 1, 2, 3]
y = [1, 3, 2, 5]

# Sprawdzamy, do którego przedziału należy punkt xp
xp = 1.5

indeks = 0

for i in range(len(x) - 1):
    if x[i] <= xp <= x[i + 1]:
        indeks = i

print("Punkt", xp, "leży w przedziale [", x[indeks], ",", x[indeks + 1], "]")
```

21. Funkcje sklejane liniowe

Najprostszy typ funkcji sklejanych to funkcje sklejane liniowe.

Dla każdego podprzedziału:

$$[x_i, x_{i+1}]$$

interpolacja liniowa jest dana wzorem:

$$S(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i}(x - x_i)$$

Przykład z wykładu

Dane pomiarowe:

$$(0, 1), (1, 3), (2, 2), (3, 5)$$

Na kolejnych przedziałach:

$$S_0(x) = 1 + 2x, \quad x \in [0, 1]$$

$$S_1(x) = 4 - x, \quad x \in [1, 2]$$

$$S_2(x) = 3x - 4, \quad x \in [2, 3]$$

Zatem funkcja sklejana:

$$S(x) = \begin{cases} 1 + 2x, & x \in [0, 1] \\ 4 - x, & x \in [1, 2] \\ 3x - 4, & x \in [2, 3] \end{cases}$$

Przykład w Pythonie

PYTHON

```
x = [0, 1, 2, 3]
y = [1, 3, 2, 5]

xp = 1.5

wartosc = None

for i in range(len(x) - 1):
    if x[i] <= xp <= x[i + 1]:
        wartosc = y[i] + (y[i + 1] - y[i]) / (x[i + 1] - x[i]) * (xp - x[i])

print("S(", xp, ") =", wartosc)
```

22. Funkcje sklepane kubiczne

Funkcje sklepane kubiczne są funkcjami sklepanymi stopnia 3.

Są często używane w praktyce, ponieważ dają dobrą równowagę między złożonością obliczeniową a jakością interpolacji.

Na przedziale:

$$[x_i, x_{i+1}]$$

funkcja ma postać:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

Warunki sklejenia zapewniają gładkość przejść między wielomianami.

Warunki brzegowe i warunki sklejenia

Dla węzłów zewnętrznych:

$$s_0(x_0) = f(x_0)$$

oraz:

$$s_{n-1}(x_n) = f(x_n)$$

Warunek naturalności:

$$s_0''(x_0) = 0$$

oraz:

$$s_{n-1}''(x_n) = 0$$

Dla każdego węzła wewnętrznego:

$$x_i$$

dla:

$$i = 1, 2, \dots, n - 1$$

mamy warunki:

$$s_{i-1}(x_i) = s_i(x_i) = f(x_i)$$

$$s'_{i-1}(x_i) = s'_i(x_i)$$

$$s''_{i-1}(x_i) = s''_i(x_i)$$

Przykład w Pythonie

```
# Obliczamy wartość przykładowego wielomianu kubicznego:  
# S_i(x) = ai + bi*(x-xi) + ci*(x-xi)^2 + di*(x-xi)^3
```

PYTHON

```
ai = 1  
bi = 2  
ci = -1  
di = 0.5
```

```
xi = 0  
x = 0.5
```

```
h = x - xi
```

```
S = ai + bi * h + ci * h * h + di * h * h * h
```

```
print("S(x) =", S)
```

23. Przykład naturalnego splajnu kubicznego

Dla danych:

$$(0, 0), (1, 1), (2, 0)$$

szukamy naturalnego splajnu kubicznego w postaci:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

Na przedziałach:

$$[0, 1]$$

oraz:

$$[1, 2]$$

przyjmujemy:

$$S_0(x) = a_0 + b_0x + c_0x^2 + d_0x^3$$

$$S_1(x) = a_1 + b_1(x-1) + c_1(x-1)^2 + d_1(x-1)^3$$

Warunki interpolacji:

$$S_0(0) = 0$$

$$S_0(1) = 1$$

$$S_1(1) = 1$$

$$S_1(2) = 0$$

Warunki naturalności:

$$S_0''(0) = 0$$

$$S_1''(2) = 0$$

Warunki sklejenia w punkcie:

$$x = 1$$

są następujące:

$$S_0'(1) = S_1'(1)$$

$$S_0''(1) = S_1''(1)$$

Z warunków dostajemy:

$$a_0 = 0$$

$$a_1 = 1$$

$$c_0 = 0$$

oraz układ:

$$b_0 + d_0 = 1$$

$$b_1 + c_1 + d_1 = -1$$

$$b_0 + 3d_0 = b_1$$

$$6d_0 = 2c_1$$

$$2c_1 + 6d_1 = 0$$

Po rozwiązaniu:

$$b_0 = \frac{3}{2}$$

$$d_0 = -\frac{1}{2}$$

$$b_1 = 0$$

$$c_1 = -\frac{3}{2}$$

$$d_1 = \frac{1}{2}$$

Zatem:

$$S_0(x) = \frac{3}{2}x\frac{1}{2}x^3, \quad x \in [0, 1]$$

$$S_1(x) = 1\frac{3}{2}(x-1)^2 + \frac{1}{2}(x-1)^3, \quad x \in [1, 2]$$

Ostatecznie:

$$S(x) = \begin{cases} \frac{3}{2}x - \frac{1}{2}x^3, & x \in [0, 1] \\ 1 - \frac{3}{2}(x-1)^2 + \frac{1}{2}(x-1)^3, & x \in [1, 2] \end{cases}$$

Splajn kubiczny daje funkcję gładką: ciągłe są:

$$S(x)$$

$$S'(x)$$

oraz:

$$S''(x)$$

w punkcie sklejenia.

Przykład w Pythonie

```
def S(x):
    if 0 <= x <= 1:
        return (3 / 2) * x - (1 / 2) * x * x * x
    elif 1 < x <= 2:
        h = x - 1
        return 1 - (3 / 2) * h * h + (1 / 2) * h * h * h
    else:
        return None

punkty = [0, 0.5, 1, 1.5, 2]

for x in punkty:
    print("x =", x, "S(x) =", S(x))
```

PYTHON

24. Krzywe Béziera

Krzywe Béziera są szeroko stosowane w:

- grafice komputerowej,
- animacji,
- projektowaniu CAD,
- modelowaniu gładkich i łatwo kontrolowanych kształtów.

Krzywa Béziera stopnia n jest kombinacją liniową punktów kontrolnych:

$$P_0, P_1, \dots, P_n$$

z wykorzystaniem wielomianów bazowych Bernsteina:

$$B(t) = \sum_{i=0}^n P_i B_{i,n}(t)$$

gdzie:

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

dla:

$$t \in [0, 1]$$

Właściwości krzywych Béziera

1. Krzywa zaczyna się w:

$$P_0$$

2. Krzywa kończy się w:

$$P_n$$

3. Tylko punkty kontrolne:

$$P_0$$

oraz:

$$P_n$$

leżą na krzywej.

4. Pozostałe punkty kontrolne wpływają na kształt krzywej.
5. Krzywa jest zawsze zawarta w otoczce wypukłej swoich punktów kontrolnych.
6. Do wyznaczania punktów na krzywej można użyć algorytmu de Casteljau.

Przykład w Pythonie — algorytm de Casteljau

PYTHON

```
# Punkty kontrolne 2D
punkty = [
    [0.0, 0.0],
    [1.0, 2.0],
    [2.0, 0.0]
]

t = 0.5

# Kopiujemy punkty do roboczej listy
robocze = []

for punkt in punkty:
    robocze.append([punkt[0], punkt[1]])

n = len(robocze)

for r in range(1, n):
    for i in range(n - r):
        robocze[i][0] = (1 - t) * robocze[i][0] + t * robocze[i + 1][0]
        robocze[i][1] = (1 - t) * robocze[i][1] + t * robocze[i + 1][1]

print("Punkt na krzywej dla t =", t, "to", robocze[0])
```

25. Wizualizacja interpolacji

Ze wskazówek do laboratorium: przy interpolacji warto obliczyć wartości wielomianu w wielu punktach i narysować wykres.

Na wykresie dobrze jest zaznaczyć:

- punkty wejściowe,
- wielomian interpolacyjny.

Dzięki temu widać, czy wielomian dobrze przechodzi przez punkty oraz czy nie pojawiają się duże oscylacje.


```
import matplotlib.pyplot as plt

x_wezly = [0.0, 1.0, 2.0]
y_wezly = [1.0, 3.0, 2.0]

# Obliczamy współczynniki Newtona w jednej tablicy
n = len(x_wezly)
a = []

for i in range(n):
    a.append(y_wezly[i])

for j in range(1, n):
    for i in range(n - 1, j - 1, -1):
        a[i] = (a[i] - a[i - 1]) / (x_wezly[i] - x_wezly[i - j])

# Funkcja do obliczania wartości wielomianu Newtona schematem Hornera
def P(X):
    result = a[len(a) - 1]

    for i in range(len(a) - 2, -1, -1):
        result = result * (X - x_wezly[i]) + a[i]

    return result

# Punkty do wykresu
x_wykres = []
y_wykres = []

start = 0.0
koniec = 2.0
liczba_punktow = 100

for i in range(liczba_punktow):
    X = start + i * (koniec - start) / (liczba_punktow - 1)
    x_wykres.append(X)
    y_wykres.append(P(X))

plt.plot(x_wykres, y_wykres, label="wielomian interpolacyjny")
plt.scatter(x_wezly, y_wezly, label="punkty wejściowe")
plt.legend()
plt.grid(True)
plt.show()
```

Interpolacja jest użyteczna, ale wiąże się z pewnymi problemami.

Najważniejsze problemy:

1. **Efekt Rungego** przy interpolacji wielomianowej na równoodległych węzłach.
2. **Wybór odpowiedniej metody interpolacji** dla danego problemu.
3. **Złożoność obliczeniowa** niektórych metod interpolacyjnych.
4. **Błędy numeryczne** wynikające ze złego uwarunkowania macierzy.

Przykład w Pythonie

```
metoda = "wielomian wysokiego stopnia"
wezly_rownoodlegle = True

if metoda == "wielomian wysokiego stopnia" and wezly_rownoodlegle:
    print("Może pojawić się efekt Rungego")
else:
    print("Ryzyko efektu Rungego jest mniejsze")
```

PYTHON

27. Najważniejsze rzeczy do zapamiętania na kolosa

27.1. Definicja interpolacji

Interpolacja polega na znalezieniu funkcji:

$$W(x)$$

takiej, że:

$$W(x_i) = y_i$$

dla danych węzłów:

$$(x_i, y_i)$$

27.2. Funkcja interpolacyjna

Funkcję interpolacyjną zapisujemy jako:

$$W(x) = \sum_{i=0}^n a_i \varphi_i(x)$$

27.3. Interpolacja wielomianowa

Wielomian interpolacyjny ma postać:

$$W_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

27.4. Macierz Vandermonde'a

Dla bazy jednomianowej powstaje macierz:

$$X = \begin{bmatrix} 1 & x_0 & \dots & x_0^n & 1 & x_1 & \dots & x_1^n & \vdots & \vdots & \ddots & \vdots & 1 & x_n & \dots & x_n^n \end{bmatrix}$$

Jej wyznacznik:

$$D = \prod_{0 \leq j < i \leq n} (x_i - x_j)$$

27.5. Interpolacja Lagrange'a

Wzór Lagrange'a:

$$W(x) = \sum_{i=0}^n y_i \left(\prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} \right)$$

27.6. Interpolacja Newtona

Wzór Newtona:

$$p(x) = \sum_{k=0}^n f[x_0, x_1, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j)$$

27.7. Ilorazy różnicowe

Rząd zerowy:

$$f[x_i] = f(x_i)$$

Rząd pierwszy:

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$

Rząd **k**:

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

27.8. Współczynniki Newtona

Współczynniki Newtona można zapisać jako:

$$a_k = f[x_0, x_1, \dots, x_k]$$

Można je liczyć w jednej tablicy, nadpisując wartości od końca.

Złożoność:

$$O(n^2)$$

27.9. Schemat Hornera dla postaci Newtona

Wartość wielomianu Newtona można liczyć od końca:

```
result = result * (X - x_i) + a_i
```

Złożoność:

$$O(n)$$

27.10. Rozwinięcie Maclaurina dla e^x

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$$

W praktyce:

$$e^x \approx \sum_{k=0}^n \frac{x^k}{k!}$$

27.11. Rozwinięcie Maclaurina dla $\sin x$

$$\sin x = \sum_{k=1}^{\infty} (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

W praktyce:

$$\sin x \approx \sum_{k=1}^n (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

27.12. Błąd interpolacji

Reszta interpolacji:

$$r(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} p_n(x)$$

gdzie:

$$p_n(x) = (x - x_0)(x - x_1) \dots (x - x_n)$$

27.13. Efekt Rungego

Efekt Rungego pojawia się głównie przy:

- wielomianach wysokiego stopnia,
 - równoodległych węzłach,
 - końcach przedziału interpolacji.
-

27.14. Funkcje sklejane

Funkcje sklejane to wielomiany niskiego stopnia na podprzedziałach, sklejone tak, aby całość była gładka.

27.15. Liniowa funkcja sklejana

$$S(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} (x - x_i)$$

27.16. Kubiczna funkcja sklejana

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

27.17. Krzywa Béziera

$$B(t) = \sum_{i=0}^n P_i B_{i,n}(t)$$

gdzie:

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

Wykład 7: Aproksymacja (lab 9)

1. Zagadnienie aproksymacji

Aproksymacja polega na zastąpieniu funkcji:

$$f$$

inną funkcją:

$$f^*$$

albo na znalezieniu funkcji:

$$f^*$$

na podstawie pewnego znanego ciągu wartości funkcji:

$$f$$

Wartości te mogą być obarczone błędem, ponieważ często pochodzą z pomiarów empirycznych.

Funkcja aproksymująca:

$$f^*$$

powinna mieć taką własność, że łatwo wykonuje się na niej operacje matematyczne, na przykład:

- różniczkowanie,
- całkowanie,
- obliczanie wartości.

Dlatego jako funkcje aproksymujące stosuje się między innymi:

- wielomiany algebraiczne,
- funkcje wymierne,
- wielomiany trygonometryczne.

Różnica między interpolacją a aproksymacją

W interpolacji funkcja musi dokładnie przechodzić przez dane punkty.

W aproksymacji funkcja nie musi przechodzić dokładnie przez punkty. Ma tylko możliwie dobrze przybliżać dane.

Przykład

Dane są punkty pomiarowe:

$$(1, 1), (3, 12), (5, 25), (7, 38)$$

Szukamy funkcji, np. prostej:

$$F(x) = ax + b$$

która najlepiej przybliża te dane.

Przykład w Pythonie

```
punkty = [  
    [1, 1],  
    [3, 12],  
    [5, 25],  
    [7, 38]  
]  
  
for punkt in punkty:  
    x = punkt[0]  
    y = punkt[1]  
    print("x =", x, "y =", y)
```

PYTHON

2. Źródła błędów w aproksymacji

W aproksymacji występują dwa główne źródła błędów.

2.1. Błędy danych wejściowych

Są to błędy pomiarów.

Dane wejściowe mogą pochodzić z doświadczeń, pomiarów lub obserwacji, więc mogą być niedokładne.

2.2. Błędy modelu

Są to błędy wynikające z wyboru konkretnego modelu, czyli klasy funkcji, którą dopasowujemy do danych.

Przykład: próbujemy dopasować prostą do danych, które w rzeczywistości układają się bardziej jak parabola.

Ważny wniosek

Aproksymację można traktować jako problem dostosowania modelu matematycznego do danych wejściowych i znanych faktów.

Przykład w Pythonie

PYTHON

```
# Dane pomiarowe mogą być niedokładne.
# Model liniowy może nie pasować idealnie.

x = [1, 2, 3]
y_pomiar = [2.1, 3.9, 6.2]

# Przykładowy model: F(x) = 2x
def F(x):
    return 2 * x

for i in range(len(x)):
    blad = abs(y_pomiar[i] - F(x[i]))
    print("x =", x[i], "pomiar =", y_pomiar[i], "model =", F(x[i]), "błąd =",
          blad)
```

3. Aproksymacja w postaci ogólnej

W aproksymacji liniowej względem współczynników funkcję przybliża się funkcją postaci:

$$f^*(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \dots + a_k\varphi_k(x)$$

gdzie:

- $\varphi_0, \varphi_1, \dots, \varphi_k$
— znane funkcje bazowe,
- $a_0, a_1, \dots, a_k$

- współczynniki, które trzeba dobrać,
- współczynniki dobiera się tak, aby zminimalizować błąd.

Jeżeli:

$$\varphi_i(x) = x^i$$

to funkcja:

$$f^*(x)$$

jest wielomianem stopnia k .

Wtedy układ:

$$1, x, x^2, \dots, x^k$$

nazywamy bazą zbioru wszystkich wielomianów stopnia k .

Przykład

Dla wielomianu stopnia 2 mamy:

$$f^*(x) = a_0 + a_1x + a_2x^2$$

czyli funkcje bazowe to:

$$\varphi_0(x) = 1$$

$$\varphi_1(x) = x$$

$$\varphi_2(x) = x^2$$

Przykład w Pythonie

PYTHON

```
# Funkcja aproksymująca:
# f*(x) = a0 + a1*x + a2*x^2

a0 = 1
a1 = 2
a2 = -0.5

def f_aproksymujaca(x):
    return a0 + a1 * x + a2 * x * x

punkty_x = [0, 1, 2, 3]

for x in punkty_x:
    print("x =", x, "f*(x) =", f_aproksymujaca(x))
```

4. Aproksymacja dla punktów danych

Mając dany zbiór punktów:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

szukamy funkcji:

$$f(x)$$

z danej klasy, która w punktach:

$$x_1, x_2, \dots, x_n$$

najlepiej przybliża wartości:

$$y_i$$

Podobne zagadnienie można sformułować dla funkcji. Dla danej funkcji:

$$g(x)$$

szukamy funkcji:

$$f(x)$$

która ją przybliża.

Trzeba wtedy określić miarę jakości przybliżenia, czyli odległość między:

$$y_1, y_2, \dots, y_n$$

a:

$$f(x_1), f(x_2), \dots, f(x_n)$$

albo między funkcjami:

$$g(x)$$

oraz:

$$f(x)$$

Przykład w Pythonie

PYTHON

```
x = [1, 2, 3]
y = [2, 4, 5]

def f_model(x):
    return 1.5 * x + 0.5

for i in range(len(x)):
    wartosc_modelu = f_model(x[i])
    blad = y[i] - wartosc_modelu
    print("x =", x[i], "y =", y[i], "model =", wartosc_modelu, "różnica =", blad)
```

5. Metryka

Metryka to miara odległości w zbiorze.

Przestrzeń metryczna to para:

$$(X, d)$$

gdzie:

- X
— zbiór,
- d
— funkcja określająca odległość między elementami zbioru.

Metryka:

$$d(x, y)$$

spełnia warunki:

1. Odległość jest równa zero wtedy i tylko wtedy, gdy punkty są takie same:

$$d(x, y) = 0 \Leftrightarrow x = y$$

2. Odległość jest symetryczna:

$$d(x, y) = d(y, x)$$

3. Spełniony jest warunek trójkąta:

$$d(x, y) + d(y, z) \geq d(x, z)$$

Wartość:

$$d(x, y)$$

reprezentuje odległość między punktami x i y .

Przykład w Pythonie

```
def odleglosc_1D(x, y):  
    return abs(x - y)
```

```
x = 2  
y = 7
```

```
d = odleglosc_1D(x, y)
```

```
print("Odległość =", d)
```

PYTHON

6. Norma funkcji

Niech F będzie rodziną funkcji rzeczywistych, ciągłych i ograniczonych, określonych na przedziale:

$$K = [a, b]$$

albo na zbiorze:

$$K = x_1, x_2, \dots, x_n$$

Norma funkcji to odwzorowanie:

$$|\cdot| : F \rightarrow [0, 1)$$

które funkcji:

$$f \in F$$

przypisuje nieujemną liczbę:

$$|f|$$

Norma spełnia warunki:

1. Norma jest równa zero tylko dla funkcji zerowej:

$$|f| = 0 \Leftrightarrow f \equiv 0$$

2. Norma jest jednorodna:

$$|\lambda f| = |\lambda| |f|$$

3. Spełnia warunek trójkąta:

$$|f| + |g| \geq |f + g|$$

Norma określa metrykę w rodzinie funkcji:

$$d_{|\cdot|}(f, g) = |f - g|$$

Przykład w Pythonie

Prosty przykład dla funkcji określonej na skończonym zbiorze punktów

PYTHON

```
def f(x):  
    return x * x  
  
def g(x):  
    return x + 1  
  
punkty = [0, 1, 2]  
  
for x in punkty:  
    roznica = abs(f(x) - g(x))  
    print("x =", x, "|f(x)-g(x)| =", roznica)
```

7. Norma jednostajna

Norma jednostajna jest zdefiniowana wzorem:

$$|f| = \sup_{x \in K} |f(x)|$$

gdzie:

sup

oznacza supremum, czyli najmniejsze ograniczenie górne.

Dla skończonego zbioru punktów można rozumieć ją jako największą wartość bezwzględną funkcji.

Norma różnicy funkcji

Dla dwóch funkcji:

f

oraz:

g

możemy liczyć:

$$|f - g| = \sup_{x \in K} |f(x) - g(x)|$$

Czyli szukamy największej różnicy między funkcjami na danym zbiorze lub przedziale.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
def g(x):  
    return x + 1  
  
punkty = [-2, -1, 0, 1, 2]  
  
najwieksza_roznica = abs(f(punkty[0]) - g(punkty[0]))  
  
for i in range(1, len(punkty)):  
    roznica = abs(f(punkty[i]) - g(punkty[i]))  
  
    if roznica > najwieksza_roznica:  
        najwieksza_roznica = roznica  
  
print("Norma jednostajna różnicy =", najwieksza_roznica)
```

7.1. Przykład normy jednostajnej z wykładu

Na przedziale:

$$K = [-5, 5]$$

zdefiniowano funkcje:

$$f(x) = \frac{x}{x^2 + 1} - \frac{10x^2}{10x^2 + 1}$$

oraz:

$$g(x) = \frac{x}{10}$$

Interesuje nas maksimum ich różnicy:

$$h(x) = f(x) - g(x)$$

W wykładzie podano:

$$h(x) = \frac{9x}{100x^2 + 10}$$

Pochodna funkcji:

$$h'(x) = \frac{-9(10x^2 - 1)}{10(10x^2 + 1)^2}$$

Dla:

$$x > 0$$

pochodna zeruje się w punkcie:

$$x_0 = \sqrt{\frac{1}{10}} \approx 0.3162277660168379$$

W tym punkcie funkcja **h** osiąga maksimum.

Maksymalna różnica między funkcjami wynosi:

$$|f - g| = h(x_0) = \frac{9}{2 \cdot 10^{3/2}} \approx 0.142302494707577$$

Przykład w Pythonie

PYTHON

```
def h(x):
    return 9 * x / (100 * x * x + 10)

x0 = (1 / 10) ** 0.5

wartosc = h(x0)

print("x0 =", x0)
print("h(x0) =", wartosc)
```

8. Norma L2

Norma **L2**, nazywana także normą kwadratową, jest zdefiniowana wzorem:

$$|f|_2 = \sqrt{\int_a^b f^2(x) dx}$$

Można ją też zapisać jako:

$$|f| = \sqrt{\int_a^b f^2(x) dx}$$

Norma **L2** mierzy błąd w sensie średniokwadratowym.

Przykład w Pythonie — przybliżenie całki metodą prostokątów

PYTHON

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 1000  
  
dx = (b - a) / n  
  
calka = 0.0  
  
for i in range(n):  
    x = a + i * dx  
    calka = calka + f(x) * dx  
  
norma_L2 = calka ** 0.5  
  
print("Przybliżona norma L2 =", norma_L2)
```

8.1. Przykład normy L2 z wykładu

Dla funkcji:

$$h(x) = \frac{9x}{100x^2 + 10}$$

na przedziale:

$$[-5, 5]$$

norma **L2** wynosi:

$$|h| = \sqrt{\int_{-5}^5 h^2(x) dx}$$

Z wykładu:

$$|h| \approx 0.1923634359500439$$

Przykład w Pythonie — numeryczne oszacowanie

PYTHON

```
def h(x):  
    return 9 * x / (100 * x * x + 10)  
  
a = -5.0  
b = 5.0  
n = 100000  
  
dx = (b - a) / n  
  
calka = 0.0  
  
for i in range(n):  
    x = a + i * dx  
    calka = calka + h(x) * h(x) * dx  
  
norma = calka ** 0.5  
  
print("Przybliżona norma L2 =", norma)
```

9. Norma L1

Norma **L1** dla funkcji:

f

jest zdefiniowana wzorem:

$$|f|_1 = \int_a^b |f(x)| dx$$

Norma ta jest dobrze zdefiniowana, jeżeli całka jest zbieżna.

Przykład w Pythonie — przybliżenie całki

PYTHON

```
def f(x):  
    return x  
  
a = -1.0  
b = 1.0  
n = 1000  
  
dx = (b - a) / n  
  
calka = 0.0  
  
for i in range(n):  
    x = a + i * dx  
    calka = calka + abs(f(x)) * dx  
  
print("Przybliżona norma L1 =", calka)
```

9.1. Przykład normy L1 z wykładu

Dla funkcji:

$$h(x) = \frac{9x}{100x^2 + 10}$$

na przedziale:

$$[0, 5]$$

z wykładu:

$$\int_0^5 h(x) dx = \frac{9 \ln 210}{200} \approx 0.2486453822609302$$

Na przedziale:

$$[-5, 5]$$

norma **L1** wynosi:

$$|h|_1 = \int_{-5}^5 |h(x)| dx \approx 0.4972907645218605$$

Przykład w Pythonie

PYTHON

```
def h(x):  
    return 9 * x / (100 * x * x + 10)  
  
a = -5.0  
b = 5.0  
n = 100000  
  
dx = (b - a) / n  
  
calka = 0.0  
  
for i in range(n):  
    x = a + i * dx  
    calka = calka + abs(h(x)) * dx  
  
print("Przybliżona norma L1 =", calka)
```

10. Normy funkcji na zbiorach skończonych lub ciągach

Normy można definiować także dla funkcji określonych na zbiorach skończonych lub ciągach.

Niech:

$$K = x_1, x_2, \dots, x_n$$

oraz:

$$a_i = f(x_i)$$

Wtedy można określić:

Norma jednostajna

$$|f| = \sup\{|a_1|, |a_2|, \dots\}$$

Norma L2

$$|f|_2 = \sqrt{\sum_{i=1}^{\infty} a_i^2}$$

Norma L1

$$|f|_1 = \sum_{i=1}^{\infty} |a_i|$$

Dla skończonego zbioru punktów sumy kończą się na ostatnim elemencie.

Przykład w Pythonie

PYTHON

```
a = [1, -2, 3, -4]

# Norma jednostajna
norma_jednostajna = abs(a[0])

for i in range(1, len(a)):
    if abs(a[i]) > norma_jednostajna:
        norma_jednostajna = abs(a[i])

# Norma L2
suma_kwadratow = 0

for i in range(len(a)):
    suma_kwadratow = suma_kwadratow + a[i] * a[i]

norma_L2 = suma_kwadratow ** 0.5

# Norma L1
norma_L1 = 0

for i in range(len(a)):
    norma_L1 = norma_L1 + abs(a[i])

print("Norma jednostajna =", norma_jednostajna)
print("Norma L2 =", norma_L2)
print("Norma L1 =", norma_L1)
```

11. Szeregi potęgowe

Szereg potęgowy zdefiniowany dla pewnego punktu:

$$x_0$$

ma postać:

$$f(x) = \sum_{k=0}^{\infty} a_k (x - x_0)^k$$

gdzie:

$$x \in (x_0 - r, x_0 + r)$$

a:

$$r$$

jest promieniem zbieżności szeregu.

Dla:

$$x$$

spoza tego przedziału szereg jest rozbieżny.

Przykład w Pythonie

PYTHON

```
# Przykład obliczania skończonej części szeregu:
# a0 + a1*(x-x0) + a2*(x-x0)^2

a = [1, 2, 3]
x0 = 0
x = 2

wynik = 0
potega = 1

for k in range(len(a)):
    wynik = wynik + a[k] * potega
    potega = potega * (x - x0)

print("Wartość przybliżenia =", wynik)
```

12. Forma szeregu potęgowego i reszta szeregu

Szereg potęgowy można zapisać jako sumę skończoną oraz resztę:

$$f(x) = \sum_{k=0}^{n-1} a_k (x - x_0)^k + R_n$$

gdzie:

$$R_n$$

jest resztą szeregu.

W praktyce do aproksymacji używa się skończonej liczby składników:

$$f(x) \approx \sum_{k=0}^{n-1} a_k (x - x_0)^k$$

Przykład w Pythonie

PYTHON

```
# Im więcej wyrazów szeregu, tym zwykle lepsze przybliżenie,  
# o ile x leży w przedziale zbieżności.
```

```
a = [1, 1, 0.5, 1 / 6]
```

```
x0 = 0
```

```
x = 1
```

```
wynik = 0
```

```
potega = 1
```

```
for k in range(len(a)):
```

```
    wynik = wynik + a[k] * potega
```

```
    potega = potega * (x - x0)
```

```
print("Przybliżenie =", wynik)
```

13. Wzór Taylora i Maclaurina

Jeżeli:

$$f(x) = \sum_{k=0}^{n-1} a_k (x - x_0)^k + R_n$$

oraz:

$$x \in (x_0 - r, x_0 + r)$$

to można przybliżyć:

$$f(x) \approx \sum_{k=0}^{n-1} a_k (x - x_0)^k$$

gdzie:

$$a_k = \frac{f^{(k)}(x_0)}{k!}$$

Taki wzór nazywamy **wzorem Taylora**.

Dla:

$$x_0 = 0$$

wzór Taylora nazywamy **wzorem Maclaurina**:

$$f(x) \approx \sum_{k=0}^{n-1} a_k x^k$$

Przykład w Pythonie — przybliżenie funkcji e^x

PYTHON

```
# e^x ≈ 1 + x + x^2/2! + x^3/3! + ...

x = 1.0
n = 10

suma = 1.0
wyraz = 1.0

for k in range(1, n):
    wyraz = wyraz * x / k
    suma = suma + wyraz

print("Przybliżenie e^x =", suma)
```

14. Aproksymacja funkcji $\sin x$ wzorem Maclaurina

Przybliżenie funkcji:

$$\sin x$$

wzorem Maclaurina ma postać:

$$\sin x \approx \sum_{k=1}^n (-1)^{k-1} \frac{x^{2k-1}}{(2k-1)!}$$

czyli:

$$\sin x \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + (-1)^{n-1} \frac{x^{2n-1}}{(2n-1)!}$$

Przykład w Pythonie

PYTHON

```
x = 1.0
n = 10

suma = 0.0
wyraz = x
znak = 1

for k in range(1, n + 1):
    suma = suma + znak * wyraz

    mianownik = (2 * k) * (2 * k + 1)
    wyraz = wyraz * x * x / mianownik

    znak = -znak

print("Przybliżenie sin(x) =", suma)
```

15. Wielomiany Czebyszewa

Wielomiany Czebyszewa definiuje się rekurencyjnie:

$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$$

dla:

$$k \geq 2$$

W przedziale:

$$[-1, 1]$$

wielomiany Czebyszewa można zapisać jako:

$$T_k(x) = \cos(k \arccos x)$$

dla:

$$k = 0, 1, 2, \dots$$

Przykład

Pierwsze wielomiany:

$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_2(x) = 2x^2 - 1$$

$$T_3(x) = 4x^3 - 3x$$

Przykład w Pythonie

PYTHON

```
def T(k, x):
    if k == 0:
        return 1
    elif k == 1:
        return x
    else:
        T_poprzedni_2 = 1
        T_poprzedni_1 = x

        for i in range(2, k + 1):
            T_aktualny = 2 * x * T_poprzedni_1 - T_poprzedni_2
            T_poprzedni_2 = T_poprzedni_1
            T_poprzedni_1 = T_aktualny

        return T_poprzedni_1

x = 0.5

for k in range(5):
    print("T_", k, "(", x, ") =", T(k, x))
```

16. Aproksymacja za pomocą wielomianów Czebyszewa

Funkcję:

$$f(x)$$

można przybliżać sumami wielomianów Czebyszewa:

$$f(x) \approx \frac{c_0}{2} + \sum_{k=1}^n c_k T_k(x)$$

gdzie:

$$c_k = \frac{2}{\pi} \int_{-1}^1 \frac{f(x) T_k(x)}{\sqrt{1-x^2}} dx$$

Wielomiany Czebyszewa są użyteczne w aproksymacji, ponieważ pomagają ograniczać błędy przybliżenia.

Przykład w Pythonie — obliczanie sumy dla danych współczynników

PYTHON

```
def T(k, x):
    if k == 0:
        return 1
    elif k == 1:
        return x
    else:
        T0 = 1
        T1 = x

        for i in range(2, k + 1):
            T2 = 2 * x * T1 - T0
            T0 = T1
            T1 = T2

        return T1

# Przykładowe współczynniki c0, c1, c2
c = [1.0, 0.5, -0.25]

x = 0.3

wynik = c[0] / 2

for k in range(1, len(c)):
    wynik = wynik + c[k] * T(k, x)

print("Przybliżenie =", wynik)
```

17. Przykład funkcji signum dla wielomianów Czebyszewa

Funkcja signum:

$$\operatorname{sgn}(x)$$

jest określona jako:

$$\operatorname{sgn}(x) = \begin{cases} 1, & x > 0 \\ -1, & x < 0 \\ 0, & x = 0 \end{cases}$$

Na dziedzinie:

$$(-1, 1)$$

można zapisać:

$$f(x) = \begin{cases} 1, & x \in (0, 1) \\ -1, & x \in (-1, 0) \\ 0, & x = 0 \end{cases}$$

Współczynniki:

$$c_k$$

dla funkcji signum wynoszą:

$$c_k = \begin{cases} 0, & k = 2i \\ (-1)^{k+1} \frac{4}{\pi k}, & k = 2i + 1 \end{cases}$$

dla:

$$k = 0, 1, \dots$$

Przykład w Pythonie

PYTHON

```
import math

def c(k):
    if k % 2 == 0:
        return 0
    else:
        return ((-1) ** (k + 1)) * 4 / (math.pi * k)

for k in range(1, 8):
    print("k =", k, "c_k =", c(k))
```

18. Szeregi trygonometryczne Fouriera

Szereg trygonometryczny Fouriera dla funkcji okresowej ma postać:

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos(kx) + b_k \sin(kx))$$

gdzie:

$$x \in [-\pi, \pi]$$

pod warunkiem zbieżności szeregu.

Funkcja:

$$f(x)$$

jest okresowa z okresem:

$$2\pi$$

Aproksymację funkcji otrzymujemy, biorąc początkowe składniki sumy:

$$f(x) \approx \frac{a_0}{2} + \sum_{k=1}^n (a_k \cos(kx) + b_k \sin(kx))$$

Przykład w Pythonie

```
import math

x = 1.0

# Przykładowe współczynniki
a0 = 0.0
a = [0.0, 0.0, 0.0]
b = [1.0, 0.5, 0.25]

wynik = a0 / 2

for k in range(1, 4):
    wynik = wynik + a[k - 1] * math.cos(k * x) + b[k - 1] * math.sin(k * x)

print("Przybliżenie Fouriera =", wynik)
```

PYTHON

19. Przykład funkcji signum w szeregu Fouriera

Dla funkcji signum na dziedzinie:

$$(-\pi, \pi)$$

mamy:

$$f(x) = \begin{cases} 1, & x \in (0, \pi) \\ -1, & x \in (-\pi, 0) \\ 0, & x = 0 \end{cases}$$

Współczynniki szeregu Fouriera są takie, że:

$$a_k = 0$$

oraz:

$$b_k = \begin{cases} 0, & k = 2i \\ \frac{4}{\pi k}, & k = 2i + 1 \end{cases}$$

gdzie:

$$k = 0, 1, 2, \dots$$

Aproksymacja funkcji signum szeregiem trygonometrycznym:

$$f(x) \approx \sum_{k=0}^n b_{2k+1} \sin((2k+1)x)$$

gdzie:

$$s(k, x) = b_k \sin(kx)$$

czyli:

$$f(x) \approx \sum_{k=0}^n s(2k+1, x)$$

Przykład w Pythonie

```
import math

def przyblizenie_sgn(x, n):
    wynik = 0.0

    for k in range(n + 1):
        indeks = 2 * k + 1
        b = 4 / (math.pi * indeks)
        wynik = wynik + b * math.sin(indeks * x)

    return wynik

x = 1.0
n = 10

print("Przybliżenie signum =", przyblizenie_sgn(x, n))
```

PYTHON

20. Aproksymacja średniokwadratowa

Aproksymacja średniokwadratowa, czyli **metoda najmniejszych kwadratów**, służy do przybliżania funkcji z użyciem normy **L2**.

Dla zbioru punktów:

$$(x_i, y_i)$$

gdzie:

$$y_i = f(x_i)$$

szukamy funkcji:

$$F(x)$$

takiej, aby wyrażenie:

$$|F - f|^2 = \sum_{i=1}^n w(x_i)(F(x_i) - y_i)^2$$

osiągało minimum.

W dalszych rozważaniach przyjmuje się dla uproszczenia:

$$w(x) = 1$$

czyli:

$$|F - f|^2 = \sum_{i=1}^n (F(x_i) - y_i)^2$$

Przykład w Pythonie

PYTHON

```
x = [1, 2, 3]
y = [2, 4, 5]

def F(x):
    return 1.5 * x + 0.5

blad_kwadratowy = 0

for i in range(len(x)):
    roznica = F(x[i]) - y[i]
    blad_kwadratowy = blad_kwadratowy + roznica * roznica

print("Suma kwadratów błędów =", blad_kwadratowy)
```

21. Aproksymacja liniowa

Najprostszym przypadkiem aproksymacji średniokwadratowej jest aproksymacja liniowa.

Szukamy funkcji:

$$F(x) = ax + b$$

która minimalizuje funkcję błędu:

$$h(a, b) = \sum_{i=1}^n (ax_i + b - y_i)^2$$

Po obliczeniu pochodnych cząstkowych i przekształceniach otrzymujemy:

$$a = \frac{nA - BC}{nD - B^2}$$

oraz:

$$b = \frac{CD - AB}{nD - B^2}$$

gdzie:

$$A = \sum_{i=1}^n x_i y_i$$

$$B = \sum_{i=1}^n x_i$$

$$C = \sum_{i=1}^n y_i$$

$$D = \sum_{i=1}^n x_i^2$$

Przykład z wykładu

Dane:

x_i	y_i
1	1
3	12
5	25
7	38

Wynik z wykładu:

$$a = 6.2$$

$$b = -5.8$$

czyli prosta aproksymująca:

$$F(x) = 6.2x - 5.8$$

```
x = [1, 3, 5, 7]
y = [1, 12, 25, 38]

n = len(x)

A = 0
B = 0
C = 0
D = 0

for i in range(n):
    A = A + x[i] * y[i]
    B = B + x[i]
    C = C + y[i]
    D = D + x[i] * x[i]

mianownik = n * D - B * B

if mianownik != 0:
    a = (n * A - B * C) / mianownik
    b = (C * D - A * B) / mianownik

    print("a =", a)
    print("b =", b)
else:
    print("Nie można obliczyć współczynników")
```

21.1. Układ równań dla aproksymacji liniowej

Dla prostej:

$$F(x) = ax + b$$

układ równań można zapisać jako:

$$nb + a \sum_{i=1}^n x_i = \sum_{i=1}^n y_i$$
$$b \sum_{i=1}^n x_i + a \sum_{i=1}^n x_i^2 = \sum_{i=1}^n x_i y_i$$

W postaci macierzowej:

$$\begin{bmatrix} n & \sum_{i=1}^n x_i \\ \sum_{i=1}^n x_i & \sum_{i=1}^n x_i^2 \end{bmatrix} \begin{bmatrix} b \\ a \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^n y_i & \sum_{i=1}^n x_i y_i \end{bmatrix}$$


```
x = [1, 3, 5, 7]
y = [1, 12, 25, 38]

n = len(x)

suma_x = 0
suma_y = 0
suma_x2 = 0
suma_xy = 0

for i in range(n):
    suma_x = suma_x + x[i]
    suma_y = suma_y + y[i]
    suma_x2 = suma_x2 + x[i] * x[i]
    suma_xy = suma_xy + x[i] * y[i]

macierz = [
    [n, suma_x],
    [suma_x, suma_x2]
]

prawa_strona = [suma_y, suma_xy]

print("Macierz układu:")
print(macierz)

print("Prawa strona:")
print(prawa_strona)
```

22. Aproksymacja wielomianowa

W aproksymacji wielomianowej przyjmujemy funkcję aproksymującą:

$$F(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \dots + a_m\varphi_m(x)$$

Dla uproszczenia często przyjmuje się:

$$\varphi_k(x) = x^k$$

Wtedy:

$$F(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m$$

Zadaniem jest minimalizacja funkcji błędu:

$$h(a_0, a_1, \dots, a_m) = \sum_{i=1}^n \left(\sum_{j=0}^m a_j x_i^j y_i \right)^2$$

Dla każdego współczynnika dostajemy układ równań normalnych.

Przykład w Pythonie — obliczenie błędu dla wielomianu drugiego stopnia

```
PYTHON

x = [0, 0.5, 1.0, 1.5, 2.0]
y = [2.00, 2.48, 2.84, 3.00, 2.91]

a = -67 / 175
b = 2159 / 1750
c = 6953 / 3500

blad = 0

for i in range(len(x)):
    F = a * x[i] * x[i] + b * x[i] + c
    roznica = F - y[i]
    blad = blad + roznica * roznica

print("Suma kwadratów błędów =", blad)
```

23. Przykład aproksymacji wielomianem drugiego stopnia

Dane z wykładu:

x_i	y_i
0	2.00
0.5	2.48
1.0	2.84
1.5	3.00
2.0	2.91

Szukamy wielomianu:

$$F(x) = ax^2 + bx + c$$

Równania wynikające z minimalizacji błędu:

$$a \sum_{i=1}^n x_i^4 + b \sum_{i=1}^n x_i^3 + c \sum_{i=1}^n x_i^2 \sum_{i=1}^n x_i^2 y_i$$

$$a \sum_{i=1}^n x_i^3 + b \sum_{i=1}^n x_i^2 + c \sum_{i=1}^n x_i \sum_{i=1}^n x_i y_i$$

$$a \sum_{i=1}^n x_i^2 + b \sum_{i=1}^n x_i + nc \sum_{i=1}^n y_i$$

Po rozwiązaniu układu z wykładu:

$$a = -\frac{67}{175}$$

$$b = \frac{2159}{1750}$$

$$c = \frac{6953}{3500}$$

Wielomian aproksymujący:

$$F(x) = -\frac{67}{175}x^2 + \frac{2159}{1750}x + \frac{6953}{3500}$$

Przykład w Pythonie

```
x = [0, 0.5, 1.0, 1.5, 2.0]

a = -67 / 175
b = 2159 / 1750
c = 6953 / 3500

for i in range(len(x)):
    F = a * x[i] * x[i] + b * x[i] + c
    print("x =", x[i], "F(x) =", F)
```

PYTHON

24. Najważniejsze rzeczy do zapamiętania na kolosa

24.1. Aproksymacja

Aproksymacja polega na zastąpieniu funkcji:

f

prostsza funkcją:

f^*

która dobrze ją przybliża.

24.2. Źródła błędów

W aproksymacji występują:

- błędy danych wejściowych,
 - błędy modelu.
-

24.3. Ogólna postać aproksymacji liniowej względem współczynników

$$f^*(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \cdots + a_k\varphi_k(x)$$

24.4. Metryka

Metryka określa odległość między elementami zbioru.

Warunek trójkąta:

$$d(x, y) + d(y, z) \geq d(x, z)$$

24.5. Norma funkcji

Norma funkcji określa jej „wielkość”.

Metrykę między funkcjami można zapisać jako:

$$d_{|\cdot|}(f, g) = |f - g|$$

24.6. Norma jednostajna

$$|f| = \sup_{x \in K} |f(x)|$$

24.7. Norma L2

$$|f|_2 = \sqrt{\int_a^b f^2(x), dx}$$

24.8. Norma L1

$$|f|_1 = \int_a^b |f(x)|, dx$$

24.9. Szereg potęgowy

$$f(x) = \sum_{k=0}^{\infty} a_k (x - x_0)^k$$

24.10. Wzór Taylora

$$f(x) \approx \sum_{k=0}^{n-1} a_k (x - x_0)^k$$

gdzie:

$$a_k = \frac{f^{(k)}(x_0)}{k!}$$

24.11. Wzór Maclaurina

Dla:

$$x_0 = 0$$

wzór Taylora nazywa się wzorem Maclaurina:

$$f(x) \approx \sum_{k=0}^{n-1} a_k x^k$$

24.12. Maclaurin dla

$$\sin x$$

$$\sin x \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

24.13. Wielomiany Czebyszewa

$$T_0(x) = 1$$

$$T_1(x) = x$$

$$T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$$

24.14. Szereg Fouriera

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos(kx) + b_k \sin(kx))$$

24.15. Aproksymacja średniokwadratowa

Szukamy funkcji:

$$F(x)$$

takiej, aby suma kwadratów błędów była minimalna:

$$\sum_{i=1}^n (F(x_i) - y_i)^2$$

24.16. Aproksymacja liniowa

Dla:

$$F(x) = ax + b$$

minimalizujemy:

$$h(a, b) = \sum_{i=1}^n (ax_i + b - y_i)^2$$

24.17. Współczynniki aproksymacji liniowej

$$a = \frac{nA - BC}{nD - B^2}$$

$$b = \frac{CD - AB}{nD - B^2}$$

gdzie:

$$A = \sum_{i=1}^n x_i y_i$$

$$B = \sum_{i=1}^n x_i$$

$$C = \sum_{i=1}^n y_i$$

$$D = \sum_{i=1}^n x_i^2$$

Wykład 8: Różniczkowanie numeryczne (lab 10)

1. Wstęp do różniczkowania numerycznego

Różniczkowanie numeryczne jest metodą obliczania przybliżonych wartości pochodnych funkcji na podstawie wartości tej funkcji w skończonej liczbie punktów.

Stosuje się je wtedy, gdy trudno albo niemożliwe jest uzyskanie pochodnej analitycznie.

Może się tak zdarzyć, gdy:

- funkcja jest bardzo skomplikowana,
- funkcja pochodzi z danych pomiarowych,
- mamy tylko wartości funkcji w wybranych punktach,
- model opisuje złożone zjawisko fizyczne, ekonomiczne albo biologiczne.

Zamiast liczyć dokładną pochodną ze wzoru, przybliżamy ją za pomocą wartości funkcji w punktach położonych blisko badanego punktu.

Przykład

Jeżeli znamy wartości funkcji:

$$f(x)$$

w punktach:

$$x$$

oraz:

$$x + h$$

to możemy przybliżyć pochodną wzorem:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.01  
  
pochodna = (f(x + h) - f(x)) / h  
  
print("Przybliżona pochodna =", pochodna)
```

Wynik:

```
Przybliżona pochodna = 4.0099999999999891
```

2. Zastosowania różniczkowania numerycznego

Różniczkowanie numeryczne ma wiele zastosowań praktycznych.

2.1. Inżynieria i fizyka

W inżynierii i fizyce różniczkowanie numeryczne jest używane między innymi do analizy dynamiki układów fizycznych.

Stosuje się je np. w:

- mechanice płynów,
- symulacjach komputerowych,
- analizie prędkości i przyspieszenia,
- obliczaniu gradientów ciśnienia i prędkości.

Przykładowo, w mechanice płynów pochodną ciśnienia:

$$p$$

względem współrzędnej:

$$x$$

można przybliżyć wzorem różnicy w przód:

$$\frac{dp}{dx} \approx \frac{p(x+h) - p(x)}{h}$$

gdzie:

h

oznacza mały krok przestrzenny.

Przykład w Pythonie

```
def p(x):  
    return 2 * x * x + 3  
  
x = 1.0  
h = 0.01  
  
dp_dx = (p(x + h) - p(x)) / h  
  
print("Przybliżona pochodna ciśnienia =", dp_dx)
```

PYTHON

2.2. Ekonomia i finanse

W ekonomii i finansach różniczkowanie numeryczne pozwala obliczać stopy zmian.

Może być używane np. przy modelowaniu opcji finansowych za pomocą równania Blacka-Scholesa.

Przykład w Pythonie

```
# Przykładowa funkcja wartości pewnego wskaźnika finansowego  
  
def wartosc(t):  
    return 100 + 5 * t + 0.2 * t * t  
  
t = 10.0  
h = 0.01  
  
tempo_zmian = (wartosc(t + h) - wartosc(t)) / h  
  
print("Przybliżone tempo zmian =", tempo_zmian)
```

PYTHON

2.3. Biologia i medycyna

W biologii i medycynie różniczkowanie numeryczne stosuje się do modelowania zmian w czasie.

Można w ten sposób obliczać np.:

- szybkość zmian stężenia leku w organizmie,
- tempo rozprzestrzeniania się substancji chemicznych,
- zmiany liczebności populacji.

Przykład w Pythonie

PYTHON

```
def stezenie(t):  
    return 10 / (1 + t)  
  
t = 2.0  
h = 0.01  
  
szybkosc_zmiany = (stezenie(t + h) - stezenie(t)) / h  
  
print("Przybliżona szybkość zmiany stężenia =", szybkosc_zmiany)
```

2.4. Informatyka

W informatyce różniczkowanie numeryczne występuje m.in. w algorytmach uczenia maszynowego.

Pochodne funkcji kosztu są potrzebne do aktualizacji parametrów modelu, np. w metodzie spadku gradientu.

Przykład w Pythonie

PYTHON

```
def funkcja_kosztu(w):  
    return (w - 3) * (w - 3)  
  
w = 0.0  
h = 0.001  
  
gradient = (funkcja_kosztu(w + h) - funkcja_kosztu(w)) / h  
  
print("Przybliżony gradient =", gradient)
```

3. Metoda różnic skończonych

Różnice skończone to metoda przybliżania pochodnych funkcji przez wykorzystanie wartości funkcji w skończonej liczbie punktów.

Metoda różnic skończonych może być wyprowadzona:

- z ilorazu różnicowego,
- z rozwinięcia funkcji w szereg Taylora.

Jest szeroko stosowana w numerycznym rozwiązywaniu równań różniczkowych, szczególnie wtedy, gdy rozwiązanie analityczne jest trudne albo niemożliwe.

Przykład

Dla funkcji:

$$f(x) = x^2$$

pochodna dokładna wynosi:

$$f'(x) = 2x$$

Ale numerycznie możemy ją przybliżyć na przykład wzorem:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Przykład w Pythonie

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.1  
  
pochodna_numeryczna = (f(x + h) - f(x)) / h  
pochodna_dokladna = 2 * x  
  
print("Pochodna numeryczna =", pochodna_numeryczna)  
print("Pochodna dokładna =", pochodna_dokladna)  
print("Błąd =", abs(pochodna_numeryczna - pochodna_dokladna))
```

PYTHON

4. Podstawowe wzory różnic skończonych

Dla funkcji:

$$f$$

określonej w punkcie:

$$x$$

oraz małego przyrostu:

$$h$$

pochodną można przybliżyć na kilka sposobów.

4.1. Różnica w przód (zwykła) - metoda Newtona

Różnica w przód, nazywana też zwykłą, ma postać:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Korzysta z wartości funkcji w punkcie:

$$x$$

oraz w punkcie po prawej stronie:

$$x+h$$

Przykład w Pythonie

```
def f(x):  
    return x * x
```

```
x = 2.0  
h = 0.01
```

```
pochodna = (f(x + h) - f(x)) / h
```

```
print(pochodna)
```

PYTHON

4.2. Różnica wsteczna

Różnica wsteczna ma postać:

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

Korzysta z wartości funkcji w punkcie:

$$x$$

oraz w punkcie po lewej stronie:

$$x - h$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.01  
  
pochodna = (f(x) - f(x - h)) / h  
  
print(pochodna)
```

4.3. Różnica centralna

Różnica centralna ma postać:

$$f'(x) \approx \frac{f(x + h) - f(x - h)}{2h}$$

Korzysta z wartości funkcji po obu stronach punktu:

$$x$$

czyli w punktach:

$$x - h$$

oraz:

$$x + h$$

Na slajdzie z porównaniem różnic skończonych pokazano, że różnica centralna jest symetryczna względem punktu:

$$x_0$$

i zwykle lepiej przybliża prawdziwą pochodną niż proste różnice jednostronne.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.1  
  
pochodna = (f(x + h) - f(x - h)) / (2 * h)  
  
print(pochodna)
```

Wynik:

4.0000000000000036

5. Przykład różnicy w przód (metoda Newtona)

Chcemy obliczyć pochodną funkcji:

$$f(x) = x^2$$

w punkcie:

$$x = 2$$

dla:

$$h = 0.01$$

Stosujemy wzór:

$$f'(2) \approx \frac{f(2.01) - f(2)}{0.01}$$

Obliczamy:

$$f(2.01) = 2.01^2 = 4.0401$$

oraz:

$$f(2) = 4$$

Zatem:

$$f'(2) \approx \frac{4.0401 - 4}{0.01} = 4.01$$

Pochodna dokładna funkcji:

$$f(x) = x^2$$

wynosi:

$$f'(x) = 2x$$

więc:

$$f'(2) = 4$$

Otrzymane przybliżenie jest bliskie wartości dokładnej.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.01  
  
pochodna_numeryczna = (f(x + h) - f(x)) / h  
pochodna_dokladna = 2 * x  
  
print("Pochodna numeryczna =", pochodna_numeryczna)  
print("Pochodna dokładna =", pochodna_dokladna)  
print("Błąd =", abs(pochodna_numeryczna - pochodna_dokladna))
```

6. Przykład różnicy centralnej

Chcemy obliczyć pochodną funkcji:

$$f(x) = x^2$$

w punkcie:

$$x = 2$$

dla:

$$h = 0.1$$

Stosujemy wzór różnicy centralnej:

$$f'(2) \approx \frac{f(2.1) - f(1.9)}{0.2}$$

Ponieważ:

$$f(2.1) = 2.1^2 = 4.41$$

oraz:

$$f(1.9) = 1.9^2 = 3.61$$

to:

$$f'(2) \approx \frac{4.41 - 3.61}{0.2} = 4.0$$

Wartość dokładna:

$$f'(2) = 4$$

czyli w tym przykładzie wynik jest dokładny.

Przykład w Pythonie

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.1  
  
pochodna_numeryczna = (f(x + h) - f(x - h)) / (2 * h)  
pochodna_dokladna = 2 * x  
  
print("Pochodna numeryczna =", pochodna_numeryczna)  
print("Pochodna dokładna =", pochodna_dokladna)
```

PYTHON

7. Wyprowadzenie metod różnic skończonych ze wzoru Taylora

Rozwinięcie funkcji analitycznej:

$$f(x)$$

w otoczeniu punktu:

$$x$$

w szereg Taylora ma postać:

$$f(x + h) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \dots$$

Wprowadzamy operator różniczkowania:

$$D^k f(x) = f^{(k)}(x)$$

Wtedy można zapisać:

$$f(x+h) = \left(1 + \frac{hD}{1!} + \frac{h^2 D^2}{2!} + \dots\right) f(x)$$

czyli:

$$f(x+h) = e^{hD} f(x)$$

Definiujemy operator różnicy zwykłej:

$$\Delta f(x) = f(x+h) - f(x)$$

oraz operator różnicy wstecznej:

$$\nabla f(x) = f(x) - f(x-h)$$

Przykład w Pythonie

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.1  
  
delta = f(x + h) - f(x)  
nabla = f(x) - f(x - h)  
  
print("Delta f(x) =", delta)  
print("Nabla f(x) =", nabla)
```

PYTHON

8. Równość operatorów i logarytmowanie

Z zależności operatorowych otrzymujemy:

$$e^{hD} = 1 + \Delta$$

oraz:

$$1 - \nabla = e^{-hD}$$

Po logarytmowaniu:

$$\ln(1 + \Delta) = hD$$

czyli:

$$D = \frac{1}{h} \ln(1 + \Delta)$$

Dla różnicy wstecznej:

$$\ln(1 - \nabla) = -hD$$

czyli:

$$D = -\frac{1}{h}\ln(1 - \nabla)$$

To pozwala wyprowadzać wzory na pochodne za pomocą operatorów różnic skończonych.

Przykład w Pythonie

PYTHON

```
# Ten przykład pokazuje tylko ideę operatora różnicy.  
# Nie liczymy logarytmu operatora, tylko samą różnicę.  
  
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.1  
  
Delta = f(x + h) - f(x)  
  
przyblizenie = Delta / h  
  
print("Przybliżenie pochodnej z operatora Delta =", przyblizenie)
```

9. Wzory na pochodne dla różnicy zwykłej

Dla różnicy zwykłej:

$$D^k = \frac{1}{h^k}(\ln(1 + \Delta))^k$$

Rozwijamy logarytm:

$$\ln(1 + \Delta) = \Delta - \frac{\Delta^2}{2} + \frac{\Delta^3}{3} - \frac{\Delta^4}{4} + \dots$$

Stąd można wyprowadzić wzory na pochodne funkcji wyrażone za pomocą różnic zwykłych.

Dla pierwszej pochodnej ($k = 1$):

$$f^{(1)}(x) = \frac{1}{h} \left(\Delta f(x) - \frac{1}{2} \Delta^2 f(x) + \frac{1}{3} \Delta^3 f(x) - \frac{1}{4} \Delta^4 f(x) + \dots \right)$$

Dla drugiej pochodnej ($k = 2$):

$$f^{(2)}(x) = \frac{1}{h^2} \left(\Delta^2 f(x) - \Delta^3 f(x) + \frac{11}{12} \Delta^4 f(x) - \frac{10}{12} \Delta^5 f(x) + \dots \right)$$

Dla trzeciej pochodnej ($k = 3$):

$$f^{(3)}(x) = \frac{1}{h^3} \left(\Delta^3 f(x) - \frac{3}{2} \Delta^4 f(x) + \frac{7}{4} \Delta^5 f(x) - \frac{45}{24} \Delta^6 f(x) + \dots \right)$$

Przykład w Pythonie — różnice zwykłe

PYTHON

```
def f(x):  
    return x * x * x  
  
x = 1.0  
h = 0.1  
  
f0 = f(x)  
f1 = f(x + h)  
f2 = f(x + 2 * h)  
f3 = f(x + 3 * h)  
  
Delta1 = f1 - f0  
Delta2 = f2 - 2 * f1 + f0  
Delta3 = f3 - 3 * f2 + 3 * f1 - f0  
  
print("Delta f(x) =", Delta1)  
print("Delta^2 f(x) =", Delta2)  
print("Delta^3 f(x) =", Delta3)
```

10. Wzory jednostronne dla pochodnych

Z rozwinięć operatorowych można otrzymać wzory z różną liczbą punktów.

10.1. Dwupunktowa różnica zwykła

$$f'(x) \approx \frac{f(x_i + h) - f(x_i)}{h}$$

albo dla siatki:

$$f'(x) \approx \frac{f(x_{i+1}) - f(x_i)}{h}$$

Błąd tej metody jest rzędu:

$$O(h)$$

Przykład w Pythonie

```
def f(x):  
    return x * x
```

```
xi = 2.0  
h = 0.01
```

```
wynik = (f(xi + h) - f(xi)) / h
```

```
print(wynik)
```

PYTHON

10.2. Trzypunktowa różnica zwykła

Dokładniejszy wzór jednostronny w przód:

$$f'(x) \approx \frac{-3f(x_i) + 4f(x_{i+1}) - f(x_{i+2})}{2h}$$

Błąd tej metody jest rzędu:

$$O(h^2)$$

Przykład w Pythonie

```
def f(x):  
    return x * x
```

```
xi = 2.0  
h = 0.1
```

```
wynik = (-3 * f(xi) + 4 * f(xi + h) - f(xi + 2 * h)) / (2 * h)
```

```
print(wynik)
```

PYTHON

10.3. Druga pochodna z różnic zwykłych

Dla drugiej pochodnej można użyć wzoru:

$$f''(x) \approx \frac{f(x_i) - 2f(x_{i+1}) + f(x_{i+2}))}{h^2}$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
xi = 2.0  
h = 0.1  
  
wynik = (f(xi) - 2 * f(xi + h) + f(xi + 2 * h)) / (h * h)  
  
print(wynik)
```

10.4. Druga pochodna z większą liczbą punktów

W wykładzie podano też wzór:

$$f''(x_i) \approx \frac{2f(x_i) - 5f(x_{i+1}) + 4f(x_{i+2}) - 3f(x_{i+3})}{h^2}$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
xi = 2.0  
h = 0.1  
  
wynik = (  
    2 * f(xi)  
    - 5 * f(xi + h)  
    + 4 * f(xi + 2 * h)  
    - 3 * f(xi + 3 * h)  
) / (h * h)  
  
print(wynik)
```

11. Wzory na pochodne dla różnicy wstecznej

Dla różnicy wstecznej:

$$D^k = \frac{1}{h^k} (-\ln(1 - \nabla))^k$$

Ponieważ:

$$\ln(1 - \nabla) = - \left(\nabla + \frac{\nabla^2}{2} + \frac{\nabla^3}{3} + \frac{\nabla^4}{4} + \dots \right)$$

to:

$$D^k = \frac{1}{h^k} \left(\nabla + \frac{\nabla^2}{2} + \frac{\nabla^3}{3} + \frac{\nabla^4}{4} + \dots \right)^k$$

Dla pierwszej pochodnej (k=1):

$$f^{(1)}(x) = \frac{1}{h} \left(\nabla f(x) + \frac{1}{2} \nabla^2 f(x) + \frac{1}{3} \nabla^3 f(x) + \dots \right)$$

Dla drugiej pochodnej (k=2):

$$f^{(2)}(x) = \frac{1}{h^2} \left(\nabla^2 f(x) + \nabla^3 f(x) + \frac{11}{12} \nabla^4 f(x) + \dots \right)$$

Dla trzeciej pochodnej (k=3):

$$f^{(3)}(x) = \frac{1}{h^3} \left(\nabla^3 f(x) + \frac{3}{2} \nabla^4 f(x) + \frac{34}{24} \nabla^5 f(x) + \dots \right)$$

Przykład w Pythonie — różnice wsteczne

PYTHON

```
def f(x):  
    return x * x * x  
  
x = 1.0  
h = 0.1  
  
f0 = f(x)  
f1 = f(x - h)  
f2 = f(x - 2 * h)  
f3 = f(x - 3 * h)  
  
Nabla1 = f0 - f1  
Nabla2 = f0 - 2 * f1 + f2  
Nabla3 = f0 - 3 * f1 + 3 * f2 - f3  
  
print("Nabla f(x) =", Nabla1)  
print("Nabla^2 f(x) =", Nabla2)  
print("Nabla^3 f(x) =", Nabla3)
```

12. Podsumowanie wzorów jednostronnych

12.1. Dwupunktowe różnice zwykłe

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Błąd:

$$O(h)$$

12.2. Trzypunktowe różnice zwykłe

$$f'(x) \approx \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h}$$

Błąd:

$$O(h^2)$$

12.3. Dwupunktowe różnice wsteczne

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

Błąd:

$$O(h)$$

12.4. Trzypunktowe różnice wsteczne

$$f'(x) \approx \frac{3f(x) - 4f(x-h) + f(x-2h)}{2h}$$

Błąd:

$$O(h^2)$$

Przykład w Pythonie — porównanie wzorów

PYTHON

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.1  
  
roznica_przod_2 = (f(x + h) - f(x)) / h  
  
roznica_przod_3 = (-3 * f(x) + 4 * f(x + h) - f(x + 2 * h)) / (2 * h)  
  
roznica_wstecz_2 = (f(x) - f(x - h)) / h  
  
roznica_wstecz_3 = (3 * f(x) - 4 * f(x - h) + f(x - 2 * h)) / (2 * h)  
  
print("Dwupunktowa w przód =", roznica_przod_2)  
print("Trzypunktowa w przód =", roznica_przod_3)  
print("Dwupunktowa wsteczna =", roznica_wstecz_2)  
print("Trzypunktowa wsteczna =", roznica_wstecz_3)
```

13. Różnice centralne

Wzory różniczkowania numerycznego dla różnicy zwykłej i wstecznej korzystają z punktów leżących tylko po jednej stronie punktu:

$$x_0$$

Różnice centralne korzystają z wartości funkcji po obu stronach punktu:

$$x_0$$

Są to wzory symetryczne.

Operator różnicy centralnej dla pierwszej pochodnej:

$$\delta f(x) = \frac{f(x + h) - f(x - h)}{2h}$$

Dla drugiej pochodnej:

$$\delta^2 f(x) = \frac{f(x + h) - 2f(x) + f(x - h)}{h^2}$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
x = 2.0  
h = 0.1  
  
pierwsza = (f(x + h) - f(x - h)) / (2 * h)  
druga = (f(x + h) - 2 * f(x) + f(x - h)) / (h * h)  
  
print("Pierwsza pochodna =", pierwsza)  
print("Druga pochodna =", druga)
```

14. Wyprowadzenie różnic centralnych z szeregu Taylora

Rozwijamy funkcję w punktach:

$$x + h$$

oraz:

$$x - h$$

Wzory Taylora:

$$f(x + h) = f(x) + f'(x)h + \frac{f''(x)}{2}h^2 + \frac{f'''(x)}{6}h^3 + O(h^4)$$

oraz:

$$f(x - h) = f(x) - f'(x)h + \frac{f''(x)}{2}h^2 - \frac{f'''(x)}{6}h^3 + O(h^4)$$

Po odjęciu tych wzorów dostajemy wzór na pierwszą pochodną:

$$\frac{f(x + h) - f(x - h)}{2h} = f'(x) + O(h^2)$$

czyli:

$$\delta f(x) = f'(x) + O(h^2)$$

Po odpowiednim złożeniu wzorów dostajemy drugą pochodną:

$$\frac{f(x + h) - 2f(x) + f(x - h))}{h^2} = f''(x) + O(h^2)$$

czyli:

$$\delta^2 f(x) = f''(x) + O(h^2)$$

Przykład w Pythonie

PYTHON

```
def f(x):
    return x * x * x

x = 2.0
h = 0.01

pochodna_centralna = (f(x + h) - f(x - h)) / (2 * h)

# Dokładna pochodna funkcji x^3 to 3x^2
pochodna_dokladna = 3 * x * x

print("Pochodna centralna =", pochodna_centralna)
print("Pochodna dokładna =", pochodna_dokladna)
print("Błąd =", abs(pochodna_centralna - pochodna_dokladna))
```

15. Podsumowanie wzorów centralnych

15.1. Dwupunktowe różnice centralne

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Błąd:

$$O(h^2)$$

15.2. Czteropunktowe różnice centralne

$$f'(x) \approx \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h}$$

Błąd:

$$O(h^4)$$

Czteropunktowy wzór centralny jest dokładniejszy, ponieważ ma błąd rzędu:

$$O(h^4)$$

zamiast:

$$O(h^2)$$

```
def f(x):  
    return x * x * x  
  
x = 2.0  
h = 0.1  
  
pochodna_2pkt = (f(x + h) - f(x - h)) / (2 * h)  
  
pochodna_4pkt = (  
    f(x - 2 * h)  
    - 8 * f(x - h)  
    + 8 * f(x + h)  
    - f(x + 2 * h)  
) / (12 * h)  
  
pochodna_dokladna = 3 * x * x  
  
print("Dwupunktowa centralna =", pochodna_2pkt)  
print("Czteropunktowa centralna =", pochodna_4pkt)  
print("Dokładna =", pochodna_dokladna)
```

16. Błędy metody różnic skończonych

Błąd metody różnic skończonych zależy od:

- wartości kroku:

$$h$$

- wyższych pochodnych funkcji:

$$f$$

- błędów zaokrągleń w komputerze.

Dla różnicy centralnej błąd jest zwykle rzędu:

$$O(h^2)$$

Oznacza to, że zmniejszanie:

$$h$$

początkowo poprawia wynik.

Ale zbyt małe:

h

może powodować problemy numeryczne związane z precyzją arytmetyki komputerowej.

Przykład w Pythonie

PYTHON

```
import math

def f(x):
    return math.exp(x)

x = 0.0

kroki = [1.0, 0.1, 0.01, 0.001, 0.0001]

for h in kroki:
    pochodna = (f(x + h) - f(x - h)) / (2 * h)
    blad = abs(pochodna - 1.0)

    print("h =", h, "pochodna =", pochodna, "błąd =", blad)
```

16.1. Przykład błędu dla funkcji e^x

Rozważamy funkcję:

$$f(x) = e^x$$

Chcemy obliczyć pochodną w punkcie:

$$x = 0$$

korzystając z dwupunktowych różnic centralnych:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2)$$

Dla:

$$x = 0$$

mamy:

$$f'(0) = \frac{e^h - e^{-h}}{2h} + O(h^2)$$

Podczas obliczeń komputer wprowadza błąd zaokrąglenia:

$$e^h \leftrightarrow e^h + R_1$$

oraz:

$$e^{-h} \leftrightarrow e^{-h} + R_2$$

Wtedy otrzymujemy:

$$f'(0) = \frac{e^h + R_1 - e^{-h} - R_2}{2h} + O(h^2)$$

czyli:

$$f'(0) = \frac{e^h - e^{-h}}{2h} + \frac{R_1 - R_2}{2h} + O(h^2)$$

Gdy zmniejszamy:

$$h$$

to błąd obcięcia:

$$O(h^2)$$

maleje, ale błąd zaokrąglenia:

$$\frac{R_1 - R_2}{2h}$$

rośnie.

Wniosek

Nie zawsze warto wybierać bardzo małe:

$$h$$

bo może to zwiększyć błąd zaokrągleń.

17. Różniczkowanie funkcji aproksymującej

Często funkcja:

$$f$$

nie jest znana dokładnie.

Możemy mieć tylko jej przybliżone wartości w punktach, np. z pomiarów.

W takim przypadku używa się różnic skończonych do aproksymowania pochodnej na podstawie tych wartości.

Przykład

Mamy dane:

x	$f(x)$
1.0	1.0
1.1	1.21
1.2	1.44

Możemy przybliżyć pochodną w punkcie:

$$x = 1.1$$

za pomocą różnicy centralnej:

$$f'(1.1) \approx \frac{f(1.2) - f(1.0)}{0.2}$$

Przykład w Pythonie

```
x = [1.0, 1.1, 1.2]
y = [1.0, 1.21, 1.44]

h = x[1] - x[0]

pochodna = (y[2] - y[0]) / (2 * h)

print("Przybliżona pochodna =", pochodna)
```

PYTHON

18. Różniczkowanie za pomocą wielomianów Lagrange'a

Różnice skończone można połączyć z interpolacją wielomianową Lagrange'a.

Pozwala to obliczać pochodne w punktach między znanymi wartościami funkcji.

Ogólnie można zapisać:

$$f'(x) \approx \frac{\sum_{k=0}^n f(x_k) l'_k(x)}{\sum_{k=0}^n l_k(x)}$$

gdzie:

$$l_k(x)$$

to wielomiany interpolacyjne Lagrange'a.

Ponieważ dla bazy Lagrange'a zachodzi:

$$\sum_{k=0}^n l_k(x) = 1$$

to w praktyce często zostaje:

$$f'(x) \approx \sum_{k=0}^n f(x_k) l'_k(x)$$

Przykład w Pythonie — idea

PYTHON

```
# Ten przykład pokazuje sam sens:
# najpierw można zbudować wielomian interpolacyjny,
# a potem różniczkować go numerycznie.

def W(x):
    # Przykładowy wielomian interpolacyjny
    return x * x

x = 2.0
h = 0.001

pochodna = (W(x + h) - W(x - h)) / (2 * h)

print("Przybliżona pochodna wielomianu =", pochodna)
```

18.1. Wielomian Lagrange'a przez trzy punkty

Zapisujemy wielomian przechodzący przez trzy punkty:

$$(x_i, y_i)$$

$$(x_{i+1}, y_{i+1})$$

$$(x_{i+2}, y_{i+2})$$

Wielomian ma postać:

$$f(x) = \frac{(x - x_{i+1})(x - x_{i+2})}{(x_i - x_{i+1})(x_i - x_{i+2})} y_i + \frac{(x - x_i)(x - x_{i+2})}{(x_{i+1} - x_i)(x_{i+1} - x_{i+2})} y_{i+1} + \frac{(x - x_i)(x - x_{i+1})}{(x_{i+2} - x_i)(x_{i+2} - x_{i+1})} y_{i+2}$$

Po zróżniczkowaniu i podstawieniu:

$$x = x_{i+1}$$

można otrzymać wzór na pochodną w punkcie środkowym.

Jeżeli punkty są równomiernie rozłożone, czyli:

$$x_{i+2} - x_{i+1} = x_{i+1} - x_i = h$$

to dostajemy różnicę centralną:

$$f'(x_{i+1}) = \frac{y_{i+2} - y_i}{2h}$$

Zalety tego podejścia

1. Punkty nie muszą być równomiernie rozłożone.
2. Można policzyć pochodną w dowolnym punkcie między:

$$x_i$$

a:

$$x_{i+2}$$

Przykład w Pythonie

PYTHON

```
# Punkty równomiernie rozłożone:
# x0 = 1.0, x1 = 1.1, x2 = 1.2

x0 = 1.0
x1 = 1.1
x2 = 1.2

y0 = x0 * x0
y1 = x1 * x1
y2 = x2 * x2

h = x1 - x0

pochodna_w_x1 = (y2 - y0) / (2 * h)

print("Pochodna w x1 =", pochodna_w_x1)
```

19. Ekstrapolacja Richardsona

Ekstrapolacja Richardsona to metoda poprawiania dokładności przybliżenia.

Zakładamy, że mamy przybliżenie:

$$D(h)$$

pewnej wartości dokładnej:

$$L$$

i że błąd zależy od potęg:

$$h^2, h^4, h^6, \dots$$

Dla różnicy centralnej z szeregu Taylora otrzymujemy:

$$L = D(h) + a_2 h^2 + a_4 h^4 + a_6 h^6 + \dots$$

gdzie:

- L
— dokładna pierwsza pochodna,
- $D(h)$
— przybliżenie pochodnej,
- $a_2 h^2 + a_4 h^4 + a_6 h^6 + \dots$
— błąd przybliżenia.

Przykład

Dla różnicy centralnej:

$$D(h) = \frac{f(x+h) - f(x-h)}{2h}$$

błąd jest rzędu:

$$O(h^2)$$

Ekstrapolacja Richardsona pozwala zbudować przybliżenie o błędzie:

$$O(h^4)$$

19.1. Pierwszy krok ekstrapolacji Richardsona

Zapisujemy:

$$L = D(h) + a_2 h^2 + a_4 h^4 + a_6 h^6 + \dots$$

Następnie zamieniamy:

$$h$$

na:

$$\frac{h}{2}$$

i mnożymy równanie przez 4:

$$4L = 4D\left(\frac{h}{2}\right) + a_2h^2 + \frac{a_4h^4}{4} + \frac{a_6h^6}{16} + \dots$$

Po odjęciu pierwszego równania od drugiego i podzieleniu przez 3 otrzymujemy:

$$L = \frac{4}{3}D\left(\frac{h}{2}\right) - \frac{1}{3}D(h) - \frac{a_4h^4}{4} - \frac{a_6h^6}{16} - \dots$$

Pierwszy krok ekstrapolacji Richardsona:

$$D^{(1)}(h) = \frac{4D\left(\frac{h}{2}\right)D(h)}{3}$$

Wynik ma błąd rzędu:

$$O(h^4)$$

zamiast:

$$O(h^2)$$

Przykład w Pythonie

```
import math

def f(x):
    return math.exp(x)

def D(x, h):
    return (f(x + h) - f(x - h)) / (2 * h)

x = 0.0
h = 0.1

D_h = D(x, h)
D_h2 = D(x, h / 2)

richardson = (4 * D_h2 - D_h) / 3

print("D(h) =", D_h)
print("D(h/2) =", D_h2)
print("Richardson =", richardson)
print("Wartość dokładna =", 1.0)
```

PYTHON

20. Ogólny wzór ekstrapolacji Richardsona

Oznaczamy:

$$D^{(1)}(h) = \frac{4D\left(\frac{h}{2}\right) - D(h)}{3}$$

Po pierwszym kroku:

$$L = D^{(1)}(h) + b_4 h^4 + b_6 h^6 + \dots$$

Ogólnie:

$$D^{(k)}(h) = \frac{4^k D^{(k-1)}\left(\frac{h}{2}\right) - D^{(k-1)}(h)}{4^k - 1}$$

21. Schemat ekstrapolacji Richardsona

Wybieramy krok początkowy:

$$h$$

oraz liczbę kroków ekstrapolacji:

$$M$$

Obliczamy:

$$D(n, 0) = D\left(\frac{h}{2^n}\right)$$

dla:

$$0 \leq n \leq M$$

Następnie dla:

$$k = 1, 2, \dots, M$$

oraz:

$$n = k, k + 1, \dots, M$$

stosujemy wzór:

$$D(n, k) := D(n, k - 1) + \frac{D(n, k - 1) - D(n - 1, k - 1)}{4^k - 1}$$

Równoważnie:

$$D(n, k) = \frac{4^k D(n, k - 1) - D(n - 1, k - 1)}{4^k - 1}$$

Otrzymujemy trójkątną tablicę przybliżeń:

$$\begin{array}{cccc}
 D(0,0) & & & \\
 D(1,0) & D(1,1) & & \\
 D(2,0) & D(2,1) & D(2,2) & \\
 \vdots & \vdots & \vdots & \ddots \\
 D(M,0) & D(M,1) & D(M,2) & \dots D(M,M)
 \end{array}$$

Przykład w Pythonie

PYTHON

```
import math

def f(x):
    return math.exp(x)

def D_pochodna(x, h):
    return (f(x + h) - f(x - h)) / (2 * h)

x = 0.0
h = 1.0
M = 4

# Tworzymy tablicę wypełnioną zerami
D = []

for i in range(M + 1):
    wiersz = []

    for j in range(M + 1):
        wiersz.append(0.0)

    D.append(wiersz)

# Pierwsza kolumna
for n in range(M + 1):
    krok = h / (2 ** n)
    D[n][0] = D_pochodna(x, krok)

# Kolejne kolumny
for k in range(1, M + 1):
    for n in range(k, M + 1):
        czynnik = 4 ** k
        D[n][k] = (czynnik * D[n][k - 1] - D[n - 1][k - 1]) / (czynnik - 1)

# Wypisanie tablicy
for i in range(M + 1):
    for j in range(i + 1):
        print(D[i][j], end=" ")
    print()
```

22. Przykład ekstrapolacji Richardsona z wykładu

W wykładzie podano przykład dla funkcji:

$$f(x) = \arctan x$$

w punkcie:

$$x = \sqrt{2}$$

Pochodna funkcji:

$$f'(x) = \frac{1}{x^2 + 1}$$

więc:

$$f'(\sqrt{2}) = \frac{1}{3}$$

W tabeli z wykładu kolejne przybliżenia dążą do wartości:

$$0.3333337$$

czyli do:

$$\frac{1}{3}$$

```
import math

def f(x):
    return math.atan(x)

def D_pochodna(x, h):
    return (f(x + h) - f(x - h)) / (2 * h)

x = math.sqrt(2)
h = 1.0
M = 4

D = []

for i in range(M + 1):
    wiersz = []

    for j in range(M + 1):
        wiersz.append(0.0)

    D.append(wiersz)

for n in range(M + 1):
    krok = h / (2 ** n)
    D[n][0] = D_pochodna(x, krok)

for k in range(1, M + 1):
    for n in range(k, M + 1):
        czynnik = 4 ** k
        D[n][k] = (czynnik * D[n][k - 1] - D[n - 1][k - 1]) / (czynnik - 1)

print("Tablica Richardsona:")

for i in range(M + 1):
    for j in range(i + 1):
        print(round(D[i][j], 7), end=" ")
    print()

print("Wartość dokładna =", 1 / 3)
```

23. Najważniejsze rzeczy do zapamiętania na kolosa

23.1. Różniczkowanie numeryczne

Różniczkowanie numeryczne polega na przybliżaniu pochodnych funkcji za pomocą wartości tej funkcji w skończonej liczbie punktów.

23.2. Różnica w przód

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Błąd:

$$O(h)$$

23.3. Różnica wsteczna

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

Błąd:

$$O(h)$$

23.4. Różnica centralna

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Błąd:

$$O(h^2)$$

23.5. Druga pochodna centralna

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

Błąd:

$$O(h^2)$$

23.6. Czteropunktowa różnica centralna

$$f'(x) \approx \frac{f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)}{12h}$$

Błąd:

$$O(h^4)$$

23.7. Problem z bardzo małym krokiem

Zmniejszanie kroku:

$$h$$

zmniejsza błąd obcięcia, ale może zwiększać błąd zaokrągleń.

Dlatego bardzo małe:

$$h$$

nie zawsze daje najlepszy wynik.

23.8. Różniczkowanie funkcji aproksymującej

Jeżeli znamy tylko wartości funkcji w punktach, możemy różniczkować funkcję aproksymującą albo interpolującą, np. wielomian Lagrange'a.

23.9. Różniczkowanie z Lagrange'a

Dla trzech równomiernie rozłożonych punktów:

$$x_i, x_{i+1}, x_{i+2}$$

otrzymujemy wzór:

$$f'(x_{i+1}) = \frac{y_{i+2} - y_i}{2h}$$

23.10. Ekstrapolacja Richardsona

Jeżeli:

$$L = D(h) + a_2 h^2 + a_4 h^4 + \dots$$

to pierwszy krok Richardsona daje:

$$D^{(1)}(h) = \frac{4D\left(\frac{h}{2}\right) - D(h)}{3}$$

23.11. Ogólny schemat Richardsona

$$D(n, k) = D(n, k-1) + \frac{D(n, k-1) - D(n-1, k-1)}{4^k - 1}$$

Wykład 9: Całkowanie numeryczne (lab 11)

1. Cel całkowania numerycznego

Całkowanie numeryczne, nazywane również **kwadraturą numeryczną**, służy do znajdowania przybliżonej wartości całek oznaczonych.

Stosuje się je szczególnie wtedy, gdy:

- rozwiązanie analityczne jest trudne,
- rozwiązanie analityczne jest niemożliwe do uzyskania,
- funkcja podcałkowa jest skomplikowana,
- znamy tylko wartości funkcji w wybranych punktach.

Głównym celem całkowania numerycznego jest obliczenie pola pod wykresem funkcji.

Całka oznaczona ma postać:

$$I = \int_a^b f(x) dx$$

W wykładzie zakładamy, że funkcja:

$$f$$

jest przynajmniej ciągła w domkniętym przedziale:

$$[a, b]$$

Oznacza to, że funkcja jest też ograniczona w tym przedziale.

Przykład

Dla funkcji:

$$f(x) = x^2$$

całka:

$$\int_0^1 x^2 dx$$

oznacza pole pod wykresem funkcji x^2 na przedziale:

$$[0, 1]$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
  
print("Przedział całkowania:")  
print("a =", a)  
print("b =", b)  
  
print("Przykładowa wartość funkcji f(0.5) =", f(0.5))
```

2. Podstawowe definicje

2.1. Całkowanie numeryczne

Całkowanie numeryczne to metoda obliczania przybliżonej wartości całki za pomocą dyskretnych sum.

Zamiast liczyć dokładną wartość całki, zastępujemy ją sumą prostszych elementów, np. prostokątów, trapezów albo pól pod parabolami.

Przykład

Pole pod krzywą można przybliżyć sumą prostokątów:

$$\int_a^b f(x) dx \approx h \sum_{i=0}^{n-1} f(x_i)$$

gdzie:

$$h$$

jest szerokością podprzedziału.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
h = (b - a) / n  
  
suma = 0.0  
  
for i in range(n):  
    x = a + i * h  
    suma = suma + f(x)  
  
calka = h * suma  
  
print("Przybliżona całka =", calka)
```

2.2. Kwadratura

Kwadratura to tradycyjny termin używany w całkowaniu numerycznym.

Najczęściej odnosi się do obliczania całek jednowymiarowych.

Dwu- i wielowymiarowe całkowania nazywane są czasami **kubaturami**, chociaż nazwa kwadratura bywa stosowana również w wyższych wymiarach.

2.3. Błąd całkowania numerycznego

Błąd całkowania numerycznego to różnica między wartością przybliżoną a dokładną wartością całki.

Można zapisać:

$$\text{błąd} = |I - I_{\text{przybl}}|$$

gdzie:

-

I

— dokładna wartość całki,

- $I_{prz\text{ybl}}$
— wartość obliczona metodą numeryczną.

Przykład w Pythonie

```
wartosc_dokladna = 1 / 3
wartosc_przyblizona = 0.328125

blad = abs(wartosc_dokladna - wartosc_przyblizona)

print("Błąd =", blad)
```

PYTHON

3. Zastosowania całkowania numerycznego

Całkowanie numeryczne jest stosowane w wielu dziedzinach, między innymi w:

- fizyce,
- inżynierii,
- ekonomii,
- finansach,
- biologii,
- statystyce,
- symulacjach komputerowych.

Jest potrzebne tam, gdzie nie można uzyskać rozwiązania analitycznego, na przykład w:

- dynamice płynów,
- optyce,
- metodach elementów skończonych,
- obliczaniu wartości oczekiwanych,
- obliczaniu prawdopodobieństw,
- obliczaniu parametrów statystycznych.

Przykład w Pythonie

PYTHON

```
zastosowania = [  
    "fizyka",  
    "inżynieria",  
    "ekonomia",  
    "finanse",  
    "statystyka",  
    "symulacje komputerowe"  
]  
  
for zastosowanie in zastosowania:  
    print(zastosowanie)
```

4. Funkcja pierwotna

Funkcja:

$$F(x)$$

jest funkcją pierwotną funkcji:

$$f(x)$$

jeżeli spełnia warunek:

$$F'(x) = f(x)$$

Jeżeli:

$$F(x)$$

jest funkcją pierwotną funkcji:

$$f(x)$$

to również:

$$F(x) + C$$

jest funkcją pierwotną tej samej funkcji, gdzie:

$$C$$

jest dowolną stałą.

Przykład

Dla funkcji:

$$f(x) = 2x$$

funkcją pierwotną jest:

$$F(x) = x^2$$

bo:

$$F'(x) = 2x$$

Przykład w Pythonie

```
def f(x):  
    return 2 * x  
  
def F(x):  
    return x * x  
  
x = 3  
  
print("f(x) =", f(x))  
print("F(x) =", F(x))
```

PYTHON

5. Całka nieoznaczona

Klasa funkcji pierwotnych jest całką nieoznaczoną funkcji:

$$f(x)$$

Zapis:

$$\int f(x) dx = F(x) + C$$

gdzie:

- $F(x)$
— funkcja pierwotna,
- C
— dowolna stała.

Przykład

$$\int 2x dx = x^2 + C$$

Przykład w Pythonie

PYTHON

```
# Python nie liczy tutaj symbolicznie całki.  
# Pokazujemy tylko sprawdzenie wartości funkcji pierwotnej.  
  
def F(x, C):  
    return x * x + C  
  
x = 2  
C = 5  
  
print("F(x) + C =", F(x, C))
```

6. Funkcje elementarne

Funkcjami elementarnymi są:

- funkcje stałe,
- funkcje potęgowe,
- funkcje wykładnicze,
- funkcje logarytmiczne,
- funkcje trygonometryczne,
- funkcje cyklometryczne,
- funkcje otrzymane z nich za pomocą skończonej liczby działań arytmetycznych,
- funkcje otrzymane przez składanie funkcji.

Przykład

Funkcją elementarną jest na przykład:

$$f(x) = e^x + \sin(x) + x^2$$

bo składa się z funkcji elementarnych.

Przykład w Pythonie

PYTHON

```
import math

def f(x):
    return math.exp(x) + math.sin(x) + x * x

x = 1.0

print("f(x) =", f(x))
```

7. Przykłady funkcji pierwotnych, które nie są elementarne

W wykładzie podano przykłady całek, których funkcje pierwotne nie są funkcjami elementarnymi, mimo że funkcje podcałkowe są elementarne.

Przykłady:

$$F(x) = \int e^{-x^2} dx$$

$$G(x) = \int \frac{\sin x}{x} dx$$

$$H(x) = \int \frac{e^x}{\sqrt{x}} dx$$

Oznacza to, że nie zawsze da się zapisać funkcję pierwotną za pomocą prostych znanych funkcji elementarnych.

W takich sytuacjach stosuje się metody numeryczne.

Przykład w Pythonie

PYTHON

```
import math

def f(x):
    return math.exp(-x * x)

x = 1.0

print("Wartość funkcji podcałkowej e^(-x^2) dla x=1:", f(x))
```


8. Całka oznaczona

Całka oznaczona funkcji:

$$f(x)$$

w granicach od:

$$a$$

do:

$$b$$

jest liczbą:

$$\int_a^b f(x)dx = F(x)\Big|_a^b = F(b) - F(a)$$

gdzie:

$$F(x)$$

jest dowolną funkcją pierwotną funkcji:

$$f(x)$$

Zauważmy też, że:

$$\int_a^x f(t)dt = F(x)$$

jest pewną funkcją pierwotną funkcji:

$$f(x)$$

Przykład

Dla:

$$f(x) = 2x$$

funkcja pierwotna to:

$$F(x) = x^2$$

Zatem:

$$\int_0^3 2x \, dx = F(3) - F(0)$$

czyli:

$$\int_0^3 2x \, dx = 9 - 0 = 9$$

Przykład w Pythonie

PYTHON

```
def F(x):  
    return x * x  
  
a = 0  
b = 3  
  
calka = F(b) - F(a)  
  
print("Wartość całki =", calka)
```

9. Rozwinięcie funkcji podcałkowej w szereg potęgowy

Problem obliczenia całki można czasami rozwiązać przez rozwinięcie funkcji podcałkowej w szereg potęgowy.

W wykładzie podano przykład funkcji:

$$e^{-x^2}$$

Korzystamy z rozwinięcia funkcji wykładniczej:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

Dla funkcji:

$$e^{-x^2}$$

otrzymujemy:

$$e^{-x^2} = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{n!}$$

```
import math

x = 0.5
m = 5

suma = 0.0

silnia = 1.0
potega_x2 = 1.0
znak = 1.0

for n in range(m + 1):
    if n > 0:
        silnia = silnia * n
        potega_x2 = potega_x2 * x * x
        znak = -znak

    wyraz = znak * potega_x2 / silnia
    suma = suma + wyraz

print("Przybliżenie e^(-x^2) =", suma)
print("Wartość biblioteczna =", math.exp(-x * x))
```

9.1. Obliczanie całki z rozwinięcia funkcji

Skoro:

$$e^{-x^2} = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{n!}$$

to:

$$\int e^{-x^2} dx = \sum_{n=0}^{\infty} \int \frac{(-1)^n x^{2n}}{n!} dx$$

Po scałkowaniu wyraz po wyrazie:

$$\int e^{-x^2} dx = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)n!}$$

W praktyce stosujemy sumę skończoną:

$$\int e^{-x^2} dx \approx \sum_{n=0}^m \frac{(-1)^n x^{2n+1}}{(2n+1)n!}$$

Liczba:

musi być dostatecznie duża, aby błąd był dostatecznie mały.

Przykład w Pythonie

PYTHON

```
import math

def F_przyblizone(x, m):
    suma = 0.0

    silnia = 1.0
    potega = x
    znak = 1.0

    for n in range(m + 1):
        if n > 0:
            silnia = silnia * n
            potega = potega * x * x
            znak = -znak

        wyraz = znak * potega / ((2 * n + 1) * silnia)
        suma = suma + wyraz

    return suma

x = 1.0

print("m = 2:", F_przyblizone(x, 2))
print("m = 3:", F_przyblizone(x, 3))
```

9.2. Funkcja z wykładu

W wykładzie zdefiniowano funkcję:

$$F(x) = \int_0^x e^{-t^2} dt$$

Następnie porównano wartości tej funkcji dla:

$$x = 0.1, 0.2, \dots, 1.0$$

obliczone programem Maxima oraz ze wzoru szeregowego dla:

$$m = 2$$

i:

$$m = 3$$

Dla:

$$x = 1.0$$

w tabeli z wykładu podano wartości:

$$F(1.0) \approx 0.7468$$

dla programu Maxima,

$$F(1.0) \approx 0.7667$$

dla:

$$m = 2$$

oraz:

$$F(1.0) \approx 0.7429$$

dla:

$$m = 3$$

Przykład w Pythonie

PYTHON

```
def F_przyblizone(x, m):
    suma = 0.0

    silnia = 1.0
    potega = x
    znak = 1.0

    for n in range(m + 1):
        if n > 0:
            silnia = silnia * n
            potega = potega * x * x
            znak = -znak

        wyraz = znak * potega / ((2 * n + 1) * silnia)
        suma = suma + wyraz

    return suma

x = 1.0

print("F(1.0), m=2 =", F_przyblizone(x, 2))
print("F(1.0), m=3 =", F_przyblizone(x, 3))
```

10. Kwadratury interpolacyjne

Metody Newtona-Cotesa to zbiór metod numerycznego całkowania, nazywanego też kwadraturą.

Dana jest funkcja:

$$f(x)$$

ciągła i ograniczona w przedziale:

$$[a, b]$$

Przedział dzielimy na skończoną liczbę podprzedziałów:

$$a = x_0 < x_1 < x_2 < \dots < x_i < x_{i+1} < \dots < x_n = b$$

gdzie:

$$i = 0, 1, \dots, n$$

Zwykle punkty są rozmieszczone równomiernie:

$$h = x_{i+1} - x_i = \text{const}$$

Wtedy:

$$h = \frac{b - a}{n}$$

Całkę można zapisać jako sumę całek po podprzedziałach:

$$\int_{x_0=a}^{x_n=b} f(x) dx = \sum_{i=0}^{n-1} \int_{x_i}^{x_{i+1}} f(x) dx$$

Przykład w Pythonie

```
a = 0.0
b = 1.0
n = 4

h = (b - a) / n

wezly = []

for i in range(n + 1):
    x = a + i * h
    wezly.append(x)

print("Węzły:")

for x in wezly:
    print(x)
```

PYTHON

10.1. Interpolacyjne przybliżenie całki

Poszczególne składniki sumy oznaczamy jako:

$$\sigma_i = \int_{x_i}^{x_{i+1}} f(x) dx$$

Istotą kwadratur interpolacyjnych jest przybliżenie funkcji:

$$f(x)$$

w przedziale:

$$[x_i, x_{i+1}]$$

wielomianem interpolacyjnym:

$$W(x)$$

Wtedy:

$$\sigma_i \approx \int_{x_i}^{x_{i+1}} W(x) dx$$

gdzie:

$$W(x)$$

jest wielomianem interpolacyjnym.

Przykład w Pythonie

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
h = (b - a) / n  
  
for i in range(n):  
    xi = a + i * h  
    xi1 = xi + h  
  
    print("Podprzedział:", xi, xi1)
```

PYTHON

11. Interpolacja Lagrange'a w całkowaniu numerycznym

Jeżeli mamy równoodległe węzły interpolacji:

$$a = x_0 < x_1 < x_2 < \dots < x_{n-1} < x_n = b$$

oraz znamy wartości:

$$f(x_i) = y_i$$

to całkę:

$$\int_a^b f(x)dx$$

można przybliżyć przez całkę z wielomianu interpolacyjnego Lagrange'a:

$$\int_a^b f(x)dx \approx \int_a^b L_n(x)dx$$

gdzie:

$$L_n(x)$$

jest wielomianem interpolacyjnym Lagrange'a stopnia co najwyżej:

$$n$$

i spełnia warunki:

$$L_n(x_0) = y(x_0)$$

$$L_n(x_1) = y(x_1)$$

...

$$L_n(x_n) = y(x_n)$$

Przykład w Pythonie

```
x_wezly = [0.0, 0.5, 1.0]
y_wezly = [0.0, 0.25, 1.0]

for i in range(len(x_wezly)):
    print("x =", x_wezly[i], "y =", y_wezly[i])
```

PYTHON

11.1. Zmienna pomocnicza

Niech:

$$h_n = \frac{b-a}{n}$$

będzie długością kroku między węzłami interpolacji.

Wprowadzamy zmienną pomocniczą:

$$x = a + th$$

Wtedy dla funkcji bazowej Lagrange'a:

$$\lambda_i(x)$$

można zapisać:

$$\lambda_i(x) = \lambda_i(a + th) = \prod_{j=0, j \neq i}^n \frac{t - j}{i - j} = g(t)$$

Przykład w Pythonie

```
a = 0.0
h = 0.25
t = 2.0

x = a + t * h

print("x =", x)
```

PYTHON

11.2. Przybliżenie całki przez wielomiany Lagrange'a

Mamy:

$$\int_a^b L_n(x) dx = \int_a^b \sum_{i=0}^n f(x_i) \cdot \lambda_i(x) dx$$

czyli:

$$\int_a^b L_n(x) dx = \sum_{i=0}^n f(x_i) \int_a^b \lambda_i(x) dx$$

Korzystając z podstawienia:

$$x = a + t \cdot h$$

oraz:

$$f(x_i) = f(a + i \cdot h)$$

otrzymujemy wzór:

$$\int_a^b f(x) dx = \sum_{i=0}^n f(x_i) \cdot h \cdot \int_0^n \prod_{j=0, j \neq i}^n \frac{t - j}{i - j} dt$$

To prowadzi do wzorów Newtona-Cotesa.

12. Rodzaje wzorów Newtona-Cotesa

W wykładzie wyróżniono dwa główne rodzaje wzorów Newtona-Cotesa:

1. **wzory otwarte,**
 2. **wzory zamknięte.**
-

12.1. Zamknięte wzory Newtona-Cotesa

Zamknięte wzory Newtona-Cotesa uwzględniają wartości funkcji we wszystkich punktach, włącznie ze skrajnymi punktami przedziału.

Dla zamkniętego wzoru Newtona-Cotesa rzędu:

$$n$$

mamy:

$$\int_a^b f(x)dx \approx \sum_{i=0}^n w_i f(x_i)$$

gdzie:

$$x_i = h \cdot i + x_0$$

oraz:

$$h = \frac{x_n - x_0}{n}$$

Liczby:

$$w_i$$

to wagi uzyskane z wielomianów bazowych Lagrange'a.

Przykładami wzorów zamkniętych są:

- metoda trapezów,
- metoda Simpsona.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
x0 = 0.0  
xn = 1.0  
n = 2  
  
h = (xn - x0) / n  
  
for i in range(n + 1):  
    xi = h * i + x0  
    print("x_", i, "=", xi, "f(x_i) =", f(xi))
```

12.2. Otwarte wzory Newtona-Cotesa

Otwarte wzory Newtona-Cotesa pomijają wartości funkcji w skrajnych punktach przedziału.

Dla otwartego wzoru Newtona-Cotesa rzędu:

$$n$$

mamy:

$$\int_a^b f(x)dx \approx \sum_{i=1}^{n-1} w_i f(x_i)$$

Wagi są wyznaczone podobnie jak w przypadku wzoru zamkniętego.

Przykładem wzoru otwartego jest metoda prostokątów.

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
h = (b - a) / n  
  
for i in range(n):  
    x_srodek = a + i * h + h / 2  
    print("Środek podprzedziału:", x_srodek, "f =", f(x_srodek))
```

13. Metoda prostokątów

Metoda prostokątów jest najprostszą metodą kwadratury.

Pole pod wykresem funkcji przybliża się za pomocą sumy pól prostokątów.

W wykładzie przyjęto, że funkcję podcałkową:

$$f(x)$$

na odcinku:

$$[x_i, x_{i+1}]$$

przybliżamy stałą wartością:

$$W(x) = f(x_i)$$

albo w praktyce dla węzłów równoodległych często:

$$y_i = f\left(x_i + \frac{h}{2}\right)$$

czyli wartością w środku podprzedziału.

Dla jednego podprzedziału:

$$\sigma_i = \int_{x_i}^{x_{i+1}} f(x)dx \approx \int_{x_i}^{x_{i+1}} y_i dx$$

Ponieważ:

$$\int_{x_i}^{x_{i+1}} y_i dx = [y_i x]_{x_i}^{x_{i+1}} = y_i(x_{i+1} - x_i) = y_i h$$

to:

$$\sigma_i \approx y_i h$$

Dla całego przedziału:

$$\int_a^b f(x) dx = h \sum_{i=0}^{n-1} y_i$$

Przybliżona wartość całki jest sumą pól prostokątów o podstawie:

$$h$$

i wysokości:

$$y_i$$

Przykład w Pythonie

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
h = (b - a) / n  
  
suma = 0.0  
  
for i in range(n):  
    x_srodek = a + i * h + h / 2  
    y = f(x_srodek)  
    suma = suma + y  
  
calka = h * suma  
  
print("Metoda prostokątów =", calka)
```

PYTHON

13.1. Przykład metody prostokątów z wykładu

Dana jest funkcja:

$$f(x) = -0.1x^2 + 2x$$

Liczymy całkę oznaczoną w przedziale:

$$[0, 15]$$

metodą prostokątów.

Przedział dzielimy na odcinki długości:

$$h = 3$$

W wykładzie podano, że metoda prostokątów daje wynik:

113.625

Dokładny wynik wynosi:

112.5

Na wykresie z wykładu prostokąty pokazują przybliżenie pola pod parabolą.

Przykład w Pythonie

```
PYTHON

def f(x):
    return -0.1 * x * x + 2 * x

a = 0.0
b = 15.0
h = 3.0

n = int((b - a) / h)

suma = 0.0

for i in range(n):
    x_srodek = a + i * h + h / 2
    y = f(x_srodek)
    suma = suma + y

calka = h * suma

print("Wartość metodą prostokątów =", calka)
print("Wartość dokładna z wykładu =", 112.5)
print("Błąd =", abs(calka - 112.5))
```

Wynik:

```
Wartość metodą prostokątów = 113.625
Wartość dokładna z wykładu = 112.5
Błąd = 1.125
```

14. Metoda trapezów

Metoda trapezów polega na przybliżeniu funkcji podcałkowej prostą przechodzącą przez dwa punkty:

$$(x_i, f(x_i))$$

oraz:

$$(x_{i+1}, f(x_{i+1}))$$

Zamiast prostokąta używamy trapezu.

Dla jednego podprzedziału:

$$[x_i, x_{i+1}]$$

pole trapezu wynosi:

$$\sigma_i = \frac{1}{2}h(y_{i+1} + y_i)$$

gdzie:

$$y_i = f(x_i)$$

oraz:

$$y_{i+1} = f(x_{i+1})$$

Po zsumowaniu pól trapezów dla całego przedziału:

$$\int_a^b f(x)dx = \frac{1}{2}h \sum_{i=0}^{n-1} (y_{i+1} + y_i)$$

Można to zapisać także jako:

$$\int_a^b f(x)dx = h \left[\frac{1}{2}(y_0 + y_n) + \sum_{i=1}^{n-1} y_i \right]$$

Przybliżona wartość całki jest sumą pól trapezów.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
h = (b - a) / n  
  
suma = 0.0  
  
for i in range(1, n):  
    x = a + i * h  
    suma = suma + f(x)  
  
calka = h * (0.5 * (f(a) + f(b)) + suma)  
  
print("Metoda trapezów =", calka)
```

14.1. Błąd metody trapezów

Wzór trapezów jest dokładny, jeżeli funkcja:

$$f$$

jest wielomianem stopnia co najwyżej pierwszego.

W innych przypadkach pojawia się błąd przybliżenia.

W wykładzie podano błąd w postaci:

$$\delta = \frac{1}{12} |f''(\xi)| (b - a)^3$$

gdzie:

$$\xi \in (a, b)$$

oraz:

$$h = \frac{b - a}{n}$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
h = (b - a) / n  
  
suma = 0.0  
  
for i in range(1, n):  
    x = a + i * h  
    suma = suma + f(x)  
  
calka = h * (0.5 * (f(a) + f(b)) + suma)  
  
dokladna = 1 / 3  
blad = abs(dokladna - calka)  
  
print("Metoda trapezów =", calka)  
print("Wartość dokładna =", dokladna)  
print("Błąd =", blad)
```

14.2. Przykład dla funkcji z wykładu

Dla funkcji:

$$f(x) = -0.1x^2 + 2x$$

na przedziale:

$$[0, 15]$$

i kroku:

$$h = 3$$

można policzyć przybliżenie metodą trapezów.

Na wykresie z wykładu trapezy lepiej dopasowują się do kształtu paraboli niż prostokąty.

Przykład w Pythonie

PYTHON

```
def f(x):  
    return -0.1 * x * x + 2 * x  
  
a = 0.0  
b = 15.0  
h = 3.0  
  
n = int((b - a) / h)  
  
suma = 0.0  
  
for i in range(1, n):  
    x = a + i * h  
    suma = suma + f(x)  
  
calka = h * (0.5 * (f(a) + f(b)) + suma)  
  
print("Metoda trapezów =", calka)  
print("Dokładny wynik z wykładu =", 112.5)  
print("Błąd =", abs(calka - 112.5))
```

15. Metoda Simpsona

Metoda Simpsona przybliża pole pod wykresem funkcji za pomocą pól pod parabolami.

W porównaniu z metodą trapezów zwykle daje większą dokładność, ponieważ zamiast prostymi odcinkami przybliża funkcję wielomianami drugiego stopnia.

Dla przedziału:

$$[a, b]$$

dzielonego na:

$$n$$

podprzedziałów, gdzie:

$$n$$

jest parzyste, definiujemy:

$$h = \frac{b - a}{n}$$

oraz:

$$x_i = a + ih$$

dla:

$$i = 0, 1, \dots, n$$

Wzór Simpsona:

$$S_n = \frac{h}{3} [f(x_0) + 4(f(x_1) + f(x_3) + \dots + f(x_{n-1})) + 2(f(x_2) + f(x_4) + \dots + f(x_{n-2})) + f(x_n)]$$

Warunek

Liczba podprzedziałów:

$$n$$

musi być parzysta.

Przykład w Pythonie

```
def f(x):  
    return x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
if n % 2 != 0:  
    print("n musi być parzyste")  
else:  
    h = (b - a) / n  
  
    suma_nieparzyste = 0.0  
    suma_parzyste = 0.0  
  
    for i in range(1, n):  
        x = a + i * h  
  
        if i % 2 == 1:  
            suma_nieparzyste = suma_nieparzyste + f(x)  
        else:  
            suma_parzyste = suma_parzyste + f(x)  
  
    calka = (h / 3) * (  
        f(a)  
        + 4 * suma_nieparzyste  
        + 2 * suma_parzyste  
        + f(b)  
    )  
  
    print("Metoda Simpsona =", calka)
```

PYTHON

15.1. Błąd metody Simpsona

Błąd przybliżenia metodą Simpsona wynosi:

$$\epsilon = |f^{(4)}(\xi)| \frac{(b-a)h^4}{180}$$

gdzie:

$$f^{(4)}(\xi)$$

to czwarta pochodna funkcji:

$$f$$

w punkcie:

$$\xi \in (a, b)$$

oraz:

$$h = \frac{b-a}{n}$$

Wzór Simpsona jest dokładny dla wielomianów stopnia co najwyżej:

$$3$$

```
def f(x):  
    return x * x * x  
  
a = 0.0  
b = 1.0  
n = 4  
  
if n % 2 != 0:  
    print("n musi być parzyste")  
else:  
    h = (b - a) / n  
  
    suma_nieparzyste = 0.0  
    suma_parzyste = 0.0  
  
    for i in range(1, n):  
        x = a + i * h  
  
        if i % 2 == 1:  
            suma_nieparzyste = suma_nieparzyste + f(x)  
        else:  
            suma_parzyste = suma_parzyste + f(x)  
  
    calka = (h / 3) * (  
        f(a)  
        + 4 * suma_nieparzyste  
        + 2 * suma_parzyste  
        + f(b)  
    )  
  
    dokladna = 1 / 4  
  
    print("Metoda Simpsona =", calka)  
    print("Wartość dokładna =", dokladna)  
    print("Błąd =", abs(calka - dokladna))
```

15.2. Przykład dla funkcji z wykładu

Dla funkcji:

$$f(x) = -0.1x^2 + 2x$$

na przedziale:

$$[0, 15]$$

można zastosować metodę Simpsona, jeżeli liczba podprzedziałów jest parzysta.

Dla przykładu weźmy:

$$n = 10$$

Wtedy:

$$h = \frac{15 - 0}{10} = 1.5$$

Przykład w Pythonie

PYTHON

```
def f(x):  
    return -0.1 * x * x + 2 * x  
  
a = 0.0  
b = 15.0  
n = 10  
  
if n % 2 != 0:  
    print("n musi być parzyste")  
else:  
    h = (b - a) / n  
  
    suma_nieparzyste = 0.0  
    suma_parzyste = 0.0  
  
    for i in range(1, n):  
        x = a + i * h  
  
        if i % 2 == 1:  
            suma_nieparzyste = suma_nieparzyste + f(x)  
        else:  
            suma_parzyste = suma_parzyste + f(x)  
  
    calka = (h / 3) * (  
        f(a)  
        + 4 * suma_nieparzyste  
        + 2 * suma_parzyste  
        + f(b)  
    )  
  
    print("Metoda Simpsona =", calka)  
    print("Dokładny wynik z wykładu =", 112.5)  
    print("Błąd =", abs(calka - 112.5))
```

16. Porównanie metod z wykładu

Metoda	Idea	Co przybliża pole?	Uwagi
Metoda prostokątów	zastępuje funkcję stałą wartością na podprzedziale	prostokąty	najprostsza metoda
Metoda trapezów	zastępuje funkcję prostą przez dwa punkty	trapezy	dokładna dla wielomianów stopnia co najwyżej 1
Metoda Simpsona	przybliża funkcję parabolami	parabole	wymaga parzystego n , dokładna dla wielomianów stopnia co najwyżej 3

17. Najważniejsze rzeczy do zapamiętania na kolosa

17.1. Całka oznaczona

$$\int_a^b f(x)dx = F(b) - F(a)$$

gdzie:

$$F'(x) = f(x)$$

17.2. Cel całkowania numerycznego

Całkowanie numeryczne służy do przybliżonego obliczania całek oznaczonych, szczególnie wtedy, gdy rozwiązanie analityczne jest trudne albo niemożliwe.

17.3. Błąd całkowania numerycznego

$$\text{błąd} = |I - I_{\text{przybl}}|$$

17.4. Rozwinięcie funkcji e^{-x^2}

$$e^{-x^2} = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{n!}$$

17.5. Całka z rozwinięcia funkcji e^{-x^2}

$$\int e^{-x^2} dx \approx \sum_{n=0}^m \frac{(-1)^n x^{2n+1}}{(2n+1)n!}$$

17.6. Podział przedziału

Dla równoodległych węzłów:

$$h = \frac{b-a}{n}$$

oraz:

$$x_i = a + ih$$

17.7. Kwadratury interpolacyjne

W kwadraturach interpolacyjnych funkcję:

$$f(x)$$

zastępuje się wielomianem interpolacyjnym:

$$W(x)$$

i liczy:

$$\int_{x^i}^{x_{i+1}} f(x) dx = \int_{x^i}^{x_{i+1}} W(x) dx$$

17.8. Metoda prostokątów

$$\int_a^b f(x) dx = h \sum_{i=0}^{n-1} y_i$$

gdzie najczęściej:

$$y_i = f\left(x_i + \frac{h}{2}\right)$$

17.9. Metoda trapezów

$$\int_a^b f(x)dx = h \left[\frac{1}{2}(y_0 + y_n) + \sum_{i=1}^{n-1} y_i \right]$$

17.10. Błąd metody trapezów

$$\delta = \frac{1}{12} |f''(\xi)|(b-a)^3$$

17.11. Metoda Simpsona

$$S_n = \frac{h}{3} [f(x_0) + 4(f(x_1) + f(x_3) + \dots + f(x_{n-1})) + 2(f(x_2) + f(x_4) + \dots + f(x_{n-2})) + f(x_n)]$$

gdzie:

$$n$$

musi być parzyste.

17.12. Błąd metody Simpsona

$$\epsilon = \frac{|f^{(4)}(\xi)|(b-a)h^4}{180}$$

Wykład 10: Miejsca zerowe wielomianów (lab 12)

1. Wprowadzenie

Poszukiwanie miejsc zerowych wielomianów jest ważnym problemem w matematyce stosowanej, fizyce i inżynierii.

Miejsce zerowe wielomianu to taka wartość argumentu, dla której wielomian przyjmuje wartość zero.

Czyli dla wielomianu:

$$P(z)$$

szukamy takiego:

$$z_0$$

że:

$$P(z_0) = 0$$

Miejsca zerowe są ważne między innymi w:

- analizie równań różniczkowych,
- optymalizacji,
- naukach inżynierskich,
- modelowaniu matematycznym.

Główne zagadnienia wykładu

W wykładzie omówiono:

1. algorytm Hornera,
2. wpływ zaburzeń współczynników,
3. metodę Laguerre'a,
4. obniżanie stopnia wielomianu, czyli deflację,
5. wygładzanie znalezionych miejsc zerowych.

Przykład

Dla wielomianu:

$$P(z) = z^2 - 4$$

miejskami zerowymi są:

$$z_1 = -2$$

oraz:

$$z_2 = 2$$

bo:

$$P(-2) = 0$$

oraz:

$$P(2) = 0$$

Przykład w Pythonie

PYTHON

```
def P(z):  
    return z * z - 4  
  
punkty = [-2, 0, 2]  
  
for z in punkty:  
    wartosc = P(z)  
  
    print("z =", z)  
    print("P(z) =", wartosc)  
  
    if wartosc == 0:  
        print("To jest miejsce zerowe")  
    else:  
        print("To nie jest miejsce zerowe")
```

2. Ogólna postać wielomianu

Wielomian stopnia n można zapisać jako:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

Równanie wielomianowe ma postać:

$$P_n(z) = 0$$

czyli:

$$a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0 = 0$$

gdzie:

- $a_n, a_{n-1}, \dots, a_0$
— współczynniki wielomianu,
- z
— zmienna,
- n
— stopień wielomianu.

Przykład

Dla wielomianu:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

mamy:

$$a_3 = 3$$

$$a_2 = 2$$

$$a_1 = -1$$

$$a_0 = 5$$

Przykład w Pythonie

```
# Współczynniki wielomianu:  
# P(z) = 3z^3 + 2z^2 - z + 5  
  
wspolczynniki = [3, 2, -1, 5]  
  
for i in range(len(wspolczynniki)):  
    print("współczynnik", i, "=", wspolczynniki[i])
```

PYTHON

3. Podstawowe Twierdzenie Algebry

Podstawowe Twierdzenie Algebry mówi, że wielomian stopnia:

$$n$$

ma na płaszczyźnie zespolonej dokładnie:

$$n$$

pierwiastków, przy czym pierwiastki wielokrotne liczy się z ich krotnościami.

Czyli jeżeli:

$$P_n(z)$$

jest wielomianem stopnia:

$$n$$

to na płaszczyźnie zespolonej ma dokładnie:

$$n$$

miejsc zerowych.

Ważna uwaga

W przypadku rzeczywistym wielomian stopnia n może nie mieć żadnych pierwiastków rzeczywistych.

Przykład:

$$P(x) = x^2 + 1$$

nie ma pierwiastków rzeczywistych, ale ma pierwiastki zespolone:

$$z_1 = i$$

oraz:

$$z_2 = -i$$

Każdy wielomian rzeczywisty stopnia nieparzystego ma przynajmniej jeden pierwiastek rzeczywisty.

Wynika to z tego, że dla wielomianu rzeczywistego stopnia nieparzystego granice niewłaściwe mają różne znaki, a wielomian jako funkcja ciągła ma własność Darboux.

Charakter twierdzenia

Podstawowe Twierdzenie Algebry nie jest konstruktywne.

Oznacza to, że mówi, ile pierwiastków istnieje, ale nie podaje sposobu ich znajdowania.

Przykład w Pythonie

```
def P(x):  
    return x * x + 1  
  
punkty = [-2, -1, 0, 1, 2]  
  
for x in punkty:  
    print("x =", x, "P(x) =", P(x))  
  
print("Dla liczb rzeczywistych nie widać miejsca zerowego")  
print("Pierwiastki są zespolone: i oraz -i")
```

PYTHON

4. Schemat Hornera

4.1. Znaczenie schematu Hornera

Schemat Hornera to efektywny sposób obliczania wartości wielomianu w danym punkcie.

Można go też wykorzystywać przy obliczaniu pochodnych wielomianu.

Zaletą schematu Hornera jest zmniejszenie liczby działań arytmetycznych.

Dla wielomianu:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

zamiast liczyć potęgi:

$$z^n, z^{n-1}, \dots, z^2$$

przekształcamy wielomian do postaci zagnieżdżonej.

4.2. Postać Hornera

Wielomian:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

można zapisać jako:

$$P_n(z) = (((a_n z + a_{n-1})z + a_{n-2})z + \dots + a_1)z + a_0$$

Dzięki temu do obliczenia wartości wielomianu potrzeba tylko:

$$n$$

mnożeń i:

$$n$$

dodawania.

Przykład

Dla:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

postać Hornera to:

$$P(z) = ((3z + 2)z - 1)z + 5$$

4.3. Algorytm Hornera

Dane:

- współczynniki wielomianu:

$$a_n, a_{n-1}, \dots, a_0$$

- punkt:

$$z$$

Algorytm:

- Ustaw:

$$p = a_n$$

- Dla:

$$i = n - 1, n - 2, \dots, 0$$

wykonuj:

$$p = p \cdot z + a_i$$

- Zwróć:

$$p$$

czyli wartość:

$$P_n(z)$$

Przykład w Pythonie

PYTHON

```
def horner(wspolczynniki, z):  
    p = wspolczynniki[0]  
  
    for i in range(1, len(wspolczynniki)):  
        p = p * z + wspolczynniki[i]  
  
    return p  
  
# P(z) = 3z^3 + 2z^2 - z + 5  
wspolczynniki = [3, 2, -1, 5]  
  
z = 2  
  
wartosc = horner(wspolczynniki, z)  
  
print("P(z) =", wartosc)
```

Wynik:

$$P(z) = 35$$

4.4. Przykład schematu Hornera z wykładu

Rozważamy wielomian:

$$P(z) = 3z^3 + 2z^2 - z + 5$$

i chcemy obliczyć jego wartość dla:

$$z = 2$$

Kroki:

$$p \leftarrow 3$$

Następnie:

$$p \leftarrow p \cdot 2 + 2 = 3 \cdot 2 + 2 = 8$$

Dalej:

$$p \leftarrow p \cdot 2 - 1 = 8 \cdot 2 - 1 = 15$$

Dalej:

$$p \leftarrow p \cdot 2 + 5 = 15 \cdot 2 + 5 = 35$$

Ostatecznie:

$$P(2) = 35$$

Przykład w Pythonie z wypisaniem kroków

```
wspolczynniki = [3, 2, -1, 5]
z = 2

p = wspolczynniki[0]

print("Start p =", p)

for i in range(1, len(wspolczynniki)):
    p = p * z + wspolczynniki[i]
    print("Po kroku", i, "p =", p)

print("Wynik końcowy:", p)
```

PYTHON

4.5. Obliczanie pochodnej wielomianu

Dla wielomianu:

$$P(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

pochodna ma postać:

$$P'(z) = n a_n z^{n-1} + (n-1) a_{n-1} z^{n-2} + \dots + a_1$$

Drugą pochodną można zapisać jako:

$$P''(z) = n(n-1) a_n z^{n-2} + (n-1)(n-2) a_{n-1} z^{n-3} + \dots$$

W metodzie Laguerre'a potrzebne są wartości:

$$P(z)$$

$$P'(z)$$

oraz:

$$P''(z)$$

w danym punkcie.

Przykład w Pythonie — współczynniki pochodnej

```
# P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10
wspolczynniki = [1, 5, 13, 19, 10]

n = len(wspolczynniki) - 1

pochodna = []

for i in range(len(wspolczynniki) - 1):
    stopien = n - i
    pochodna.append(wspolczynniki[i] * stopien)

print("Współczynniki P'(z):")
print(pochodna)
```

PYTHON

5. Wpływ zaburzeń współczynników na miejsca zerowe

5.1. Problem zaburzeń współczynników

W praktycznych obliczeniach współczynniki wielomianów rzadko są znane dokładnie.

Często są wynikiem wcześniejszych obliczeń, więc mogą być obarczone błędami.

To oznacza, że zamiast dokładnych współczynników:

$$a_k$$

znamy wartości przybliżone:

$$\tilde{a}_k = a_k + \delta_k$$

gdzie:

$$|\delta_k| \ll 1$$

czyli zaburzenie współczynnika jest małe.

Pytanie wykładu brzmi:

jaki jest wpływ błędów współczynników na wartości znalezionych numerycznie miejsc zerowych?

Odpowiedź: Błędy współczynników wielomianu mogą powodować błędy w obliczonych miejscach zerowych. Wpływ ten jest szczególnie duży, gdy pierwiastek jest wielokrotny albo gdy wartości $p'(x)$ w pobliżu pierwiastka są małe. Wtedy mała zmiana współczynników może spowodować dużą zmianę miejsc zerowych. Dlatego znajdowanie miejsc zerowych wielomianów, szczególnie wysokiego stopnia, może być zadaniem źle uwarunkowanym numerycznie.

5.2. Wielomian zaburzony

Dokładny wielomian ma postać:

$$P_n(z) = a_n z^n + a_{n-1} z^{n-1} + \dots + a_1 z + a_0$$

Niech:

$$z_0$$

będzie jego dokładnym pierwiastkiem.

Zamiast dokładnych współczynników znamy:

$$\tilde{a}_k = a_k + \delta_k$$

Wtedy otrzymujemy wielomian zaburzony:

$$\tilde{P}_n(z) = \tilde{a}_n z^n + \tilde{a}_{n-1} z^{n-1} + \dots + \tilde{a}_1 z + \tilde{a}_0$$

Zakładamy, że pierwiastek też się zaburza:

$$\tilde{z}_0 = z_0 + \varepsilon$$

gdzie:

$$|\varepsilon| \ll 1$$

5.3. Przybliżenie dwumianowe

W wyprowadzeniu korzystamy z przybliżenia:

$$(\tilde{z}_0 - \varepsilon)^k = \sum_{l=0}^k \binom{k}{l} \tilde{z}_0^{k-l} (-1)^l \varepsilon^l$$

Dla bardzo małego:

$$\varepsilon$$

zaniedbujemy wyższe potęgi:

$$\varepsilon^2, \varepsilon^3, \dots$$

i dostajemy:

$$(\tilde{z}_0 - \varepsilon)^k \approx \tilde{z}_0^k - k\tilde{z}_0^{k-1}\varepsilon$$

Zaniedbuje się też iloczyny:

$$\delta_k \varepsilon$$

ponieważ są bardzo małe.

5.4. Oszacowanie zaburzenia miejsca zerowego

Ostatecznie w wykładzie otrzymano oszacowanie wpływu zaburzeń współczynników na zaburzenie miejsca zerowego:

$$|\varepsilon| \approx \frac{|\sum_{k=0}^n \delta_k \tilde{z}_0^k|}{|\tilde{P}'_n(\tilde{z}_0)|}$$

Ten wzór mówi, że zaburzenie pierwiastka zależy od:

- zaburzeń współczynników:

$$\delta_k$$

- wartości pierwiastka:

$$\tilde{z}_0$$

- wartości pochodnej wielomianu w pobliżu pierwiastka:

$$\tilde{P}'_n(\tilde{z}_0)$$

Ważny wniosek

Jeżeli:

$$|\tilde{P}'_n(\tilde{z}_0)|$$

jest małe, to nawet małe zaburzenia współczynników mogą dać duże zaburzenie pierwiastka.

Przykład w Pythonie

PYTHON

```
# Prosty przykład obliczenia oszacowania:  
# |epsilon| ≈ |delta * z^k| / |P'(z)|  
  
delta = 10 ** (-7)  
z = -20  
k = 19  
  
pochodna = 1.0  
  
licznik = abs(delta * (z ** k))  
epsilon = licznik / abs(pochodna)  
  
print("Oszacowany licznik =", licznik)  
print("Oszacowane epsilon =", epsilon)
```

6. Przykład Wilkinsona

6.1. Wielomian Wilkinsona

W wykładzie podano przykład Wilkinsona:

$$W(z) = (z + 1)(z + 2) \dots (z + 20)$$

Jego miejsca zerowe to liczby całkowite ujemne:

$$-1, -2, \dots, -20$$

czyli:

$$\begin{aligned} z_1 &= -1 \\ z_2 &= -2 \\ &\dots \\ z_{20} &= -20 \end{aligned}$$

6.2. Zaburzenie jednego współczynnika

Zakładamy, że zaburzamy tylko jeden współczynnik:

$$\delta_{19} = 2^{-23} \approx 10^{-7}$$

a pozostałe zaburzenia są równe zero:

$$\delta_{k \neq 19} = 0$$

Pytanie: jak zmieni się położenie miejsca zerowego:

$$z_0 = -20$$

?

W wykładzie obliczono:

$$W'(-20) = -19!$$

Oszacowanie daje:

$$|\varepsilon| \approx 10^{-7} \cdot \frac{20^{19}}{19!} \approx 4.4$$

Wniosek

Zaburzenie miejsca zerowego jest o siedem rzędów wielkości większe niż zaburzenie pojedynczego współczynnika.

W rzeczywistości miejsca zerowe tak zaburzonego wielomianu mogą stać się zespolone.

To pokazuje, że zagadnienie znajdowania miejsc zerowych wielomianów może być źle uwarunkowane.

Przykład w Pythonie

PYTHON

```
import math

delta = 10 ** (-7)

licznik = 1.0

for i in range(19):
    licznik = licznik * 20

mianownik = 1.0

for i in range(1, 20):
    mianownik = mianownik * i

epsilon = delta * licznik / mianownik

print("Oszacowanie zaburzenia epsilon =", epsilon)
```

7. Zaburzenia wielokrotnych miejsc zerowych

Oszacowanie:

$$|\varepsilon| \approx \frac{|\sum_{k=0}^n \delta_k \tilde{z}_0^k|}{|\tilde{P}'_n(\tilde{z}_0)|}$$

załamuje się dla miejsc zerowych o krotności większej od jeden.

Dzieje się tak, ponieważ dla pierwiastka wielokrotnego znikają także pochodne wielomianu.

To oznacza, że wielokrotne miejsce zerowe może zmienić się bardzo mocno po niewielkim zaburzeniu współczynników.

7.1. Przykład z wykładu

Wykład podaje wielomian:

$$Q(x) = 39205740x^6 - 147747493x^5 + 173235338x^4 + 2869080x^3 - 158495872x^2 + 118949888x - 280$$

W postaci iloczynowej:

$$Q(x) = 17^3 \cdot 19 \cdot 20 \cdot 21 \left(x + \frac{20}{21}\right) \left(x - \frac{16}{17}\right)^3 \left(x - \frac{18}{19}\right) \left(x - \frac{19}{20}\right)$$

W postaci iloczynowej miejsca zerowe są łatwe do odczytania.

Mamy między innymi potrójne miejsce zerowe:

$$x = \frac{16}{17}$$

Jednak numeryczne znalezienie miejsc zerowych z postaci ogólnej może być bardzo trudne.

Ważna uwaga

Mała zmiana wielomianu, np. zwiększenie lub zmniejszenie wyrazu wolnego o **1**, może spowodować przesunięcie potrójnych miejsc zerowych z osi rzeczywistej na płaszczyznę zespoloną.

W praktyce czasami trudno stwierdzić, czy grupa bliskich miejsc zerowych oznacza różne pierwiastki, czy rozszczepione miejsce wielokrotne.

Przykład w Pythonie

```
# Pokazujemy wartości miejsc zerowych z postaci iloczynowej.
```

PYTHON

```
x1 = -20 / 21
x2 = 16 / 17
x3 = 18 / 19
x4 = 19 / 20

print("Pierwiastek 1 =", x1)
print("Pierwiastek potrójny =", x2)
print("Pierwiastek 3 =", x3)
print("Pierwiastek 4 =", x4)
```

8. Poszukiwanie miejsc zerowych wielomianów

W kontekście poszukiwania miejsc zerowych wielomianów potrzebne są dwie rzeczy:

1. specjalistyczna metoda numeryczna do wyznaczania pierwiastków wielomianów,
2. skuteczna strategia postępowania.

W wykładzie jako metodę specjalistyczną podano:

metodę Laguerre'a.

Strategia obejmuje:

- obniżanie stopnia wielomianu, czyli deflację,

- wygładzanie znalezionych miejsc zerowych za pomocą pierwotnego wielomianu.
-

9. Metoda Laguerre'a

9.1. Wzór iteracyjny

Niech:

$$P_n(z)$$

będzie wielomianem stopnia:

$$n$$

Metoda Laguerre'a jest określona iteracją:

$$z_{i+1} = z_i = \frac{nP_n(z_i)}{P'_n(z_i) \pm \sqrt{(n-1)((n-1)(P'_n(z_i))^2 - nP_n(z_i)P''_n(z_i))}}$$

Znak w mianowniku wybiera się tak, aby maksymalizować wartość bezwzględną mianownika.

9.2. Postać algorytmiczna metody Laguerre'a

Po przekształceniach definiujemy:

$$G = \frac{P'(z)}{P(z)}$$

oraz:

$$H = G^2 - \frac{P''(z)}{P(z)}$$

Następnie obliczamy:

$$a = \frac{n}{G \pm \sqrt{(n-1)(nH - G^2)}}$$

Znak wybieramy tak, aby mianownik miał większy moduł.

Nowe przybliżenie:

$$z_{new} = z_{old} - a$$

Iteracje kończymy, gdy:

$$|a| < \varepsilon$$

9.3. Sens wyboru znaku

W mianowniku są dwie możliwości:

$$G + \sqrt{(n-1)(nH - G^2)}$$

oraz:

$$G - \sqrt{(n-1)(nH - G^2)}$$

Wybieramy tę, która ma większą wartość bezwzględną.

Dzięki temu unikamy dzielenia przez bardzo małą liczbę.

9.4. Przykład w Pythonie — jedna iteracja Laguerre’a

PYTHON

```
import cmath

def horner(wspolczynniki, z):
    p = wspolczynniki[0]

    for i in range(1, len(wspolczynniki)):
        p = p * z + wspolczynniki[i]

    return p

def pochodna_wspolczynniki(wspolczynniki):
    n = len(wspolczynniki) - 1
    wynik = []

    for i in range(len(wspolczynniki) - 1):
        stopien = n - i
        wynik.append(wspolczynniki[i] * stopien)

    return wynik

#  $P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$ 
P_wsp = [1, 5, 13, 19, 10]

P1_wsp = pochodna_wspolczynniki(P_wsp)
P2_wsp = pochodna_wspolczynniki(P1_wsp)

z = 0 + 0j
n = len(P_wsp) - 1

Pz = horner(P_wsp, z)
P1z = horner(P1_wsp, z)
P2z = horner(P2_wsp, z)

G = P1z / Pz
H = G * G - P2z / Pz

pierwiastek = cmath.sqrt((n - 1) * (n * H - G * G))

mianownik1 = G + pierwiastek
mianownik2 = G - pierwiastek

if abs(mianownik1) > abs(mianownik2):
    mianownik = mianownik1
else:
    mianownik = mianownik2

a = n / mianownik
z_new = z - a
```

```
print("G =", G)
print("H =", H)
print("a =", a)
print("z_new =", z_new)
```

10. Zbieżność metody Laguerre'a

Jeżeli wszystkie pierwiastki wielomianu są pojedyncze i rzeczywiste, metoda jest zbieżna sześciennie dla dowolnego rzeczywistego przybliżenia początkowego.

Jeżeli:

$$|z_i - \bar{z}| < \epsilon \ll 1$$

to:

$$|z_{i+1} - \bar{z}| \sim \epsilon^3$$

gdzie:

$$\bar{z}$$

jest poszukiwanym pierwiastkiem.

W ogólności metoda jest zbieżna sześciennie do wszystkich pojedynczych pierwiastków, zarówno rzeczywistych, jak i zespolonych.

Ograniczenia

Metoda Laguerre'a:

- jest zbieżna liniowo do pierwiastków wielokrotnych,
- może nie zbiegać w rzadkich przypadkach,
- w przypadku stagnacji można wykonać jeden albo dwa kroki metody Newtona, a potem wrócić do metody Laguerre'a.

Zalety

Metoda Laguerre'a jest preferowana do wyszukiwania pierwiastków wielomianów:

- rzeczywistych,
- zespolonych.

Może prowadzić do zespolonych miejsc zerowych nawet wtedy, gdy startujemy z rzeczywistego punktu początkowego.

11. Deflacja — obniżanie stopnia wielomianu

11.1. Idea deflacji

Podczas szukania pierwiastków może się zdarzyć, że różne próby będą zbiegały do tego samego, już znalezionego pierwiastka.

Aby tego uniknąć, stosuje się **deflację**, czyli obniżanie stopnia wielomianu.

Jeżeli znaleźliśmy pierwiastek:

$$z_1$$

wielomianu:

$$P_n(z)$$

to możemy zapisać:

$$P_n(z) = (z - z_1)P_{n-1}(z)$$

Następnie szukamy kolejnego pierwiastka już dla wielomianu:

$$P_{n-1}(z)$$

11.2. Wygładzanie miejsc zerowych

Drobne zaburzenia współczynników mogą znacząco wpłynąć na znalezione pierwiastki.

Dlatego stosuje się **wygładzanie**.

Polega ono na tym, że znalezione przybliżone miejsce zerowe wykorzystuje się jako punkt startowy dla metody Laguerre'a, ale stosowanej ponownie do pełnego, pierwotnego wielomianu:

$$P_n(z)$$

Dzięki temu można poprawić dokładność znalezionego pierwiastka.

11.3. Kontynuacja faktoryzacji

Po znalezieniu i wygładzeniu kolejnego pierwiastka:

$$z_2$$

wykonujemy dalszą deflację:

$$P_{n-1}(z) = (z - z_2)P_{n-2}(z)$$

Wtedy:

$$P_n(z) = (z - z_1)(z - z_2)P_{n-2}(z)$$

Procedurę powtarzamy aż do wielomianu stopnia 2.

Dla wielomianu stopnia 2 można użyć wzorów dokładnych.

12. Deflacja wielomianu — wzory

Zakładamy, że:

$$z_0$$

jest pierwiastkiem wielomianu:

$$P_n(z)$$

Wtedy:

$$(z - z_0) \cdot P_{n-1}(z) = P_n(z)$$

Niech:

$$P_{n-1}(z) = b_{n-1}z^{n-1} + b_{n-2}z^{n-2} + \dots + b_1z + b_0$$

Wtedy:

$$(z - z_0)(b_{n-1}z^{n-1} + b_{n-2}z^{n-2} + \dots + b_1z + b_0) = P_n(z)$$

Porównując współczynniki, dostajemy:

$$\begin{aligned} b_{n-1} &= a_n \\ -z_0b_{n-1} + b_{n-2} &= a_{n-1} \\ &\dots \\ -z_0b_1 + b_0 &= a_1 \\ -z_0b_0 &= a_0 \end{aligned}$$

Układ można rozwiązać podstawieniem w przód.

12.1. Macierzowy zapis układu deflacji

Układ równań można zapisać macierzowo:

$$\begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ -z_0 & 1 & 0 & \dots & 0 \\ 0 & -z_0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & -z_0 & 1 \end{bmatrix} \begin{bmatrix} b_{n-1} \\ b_{n-2} \\ b_{n-3} \\ \vdots \\ b_0 \end{bmatrix} = \begin{bmatrix} a_n \\ a_{n-1} \\ a_{n-2} \\ \vdots \\ a_1 \end{bmatrix}$$

Przykład w Pythonie — deflacja przez pierwiastek

PYTHON

```
def deflacja(wspolczynniki, z0):
    # Współczynniki są w kolejności:
    # [a_n, a_{n-1}, ..., a_0]

    b = []

    # Pierwszy współczynnik po deflacji
    b.append(wspolczynniki[0])

    for i in range(1, len(wspolczynniki) - 1):
        nowy = wspolczynniki[i] + z0 * b[i - 1]
        b.append(nowy)

    return b

# P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10
wspolczynniki = [1, 5, 13, 19, 10]

# Pierwiastek z1 = -1, więc dzielimy przez z - (-1) = z + 1
z0 = -1

nowy_wielomian = deflacja(wspolczynniki, z0)

print("Współczynniki po deflacji:")
print(nowy_wielomian)
```

Wynik:

```
Współczynniki po deflacji:
[1, 4, 9, 10]
```

13. Strategia postępowania przy szukaniu wszystkich pierwiastków

Założmy, że znaleźliśmy już miejsca zerowe:

$$z_1, z_2, \dots, z_k$$

wielomianu:

$$P_n(z)$$

Po deflacji mamy wielomian:

$$P_{n-k}(z)$$

Dalsze kroki:

1. Rozpoczynamy z dowolnym przybliżeniem i stosujemy metodę Laguerre'a do znalezienia kolejnego pierwiastka:

$$\tilde{z}_{k+1}$$

wielomianu:

$$P_{n-k}(z)$$

2. W celu wygładzenia miejsca zerowego używamy:

$$\tilde{z}_{k+1}$$

jako punktu początkowego dla metody Laguerre'a na pełnym wielomianie:

$$P_n(z)$$

3. Otrzymujemy dokładniejsze miejsce zerowe:

$$z_{k+1}$$

4. Wykonujemy deflację:

$$P_{n-k}(z) = (z - z_{k+1})P_{n-k-1}(z)$$

5. Powtarzamy procedurę aż do wielomianu stopnia 2.

14. Końcowe uwagi o pierwiastkach zespolonych

Jeżeli pierwotny wielomian:

$$P_n(z)$$

ma rzeczywiste współczynniki, to jego pierwiastki są:

- rzeczywiste albo
- tworzą sprzężone pary zespolone.

Jeżeli znajdziemy pierwiastek zespolony:

$$z_k = x_k + iy_k$$

to pierwiastkiem jest też:

$$z_{k+1} = x_k - iy_k$$

czyli pierwiastek sprzężony.

Jeżeli wielomian ma całkowite współczynniki, warto najpierw sprawdzić, czy posiada pierwiastki wymierne, zanim przejdziemy do metod numerycznych.

Przykład w Pythonie

```
z = complex(-1, 2)

sprzezony = z.conjugate()

print("Pierwiastek z =", z)
print("Pierwiastek sprzężony =", sprzezony)
```

PYTHON

15. Przykład z wykładu

15.1. Wielomian

W wykładzie rozważano wielomian:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

Można go rozłożyć:

$$P(z) = (z + 1)(z + 2)(z^2 + 2z + 5)$$

Zatem dokładne pierwiastki to:

$$z_1 = -1$$

$$z_2 = -2$$

$$z_3 = -1 + 2i$$

$$z_4 = -1 - 2i$$

W praktyce często znamy tylko postać rozwiniętą:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

i pierwiastki trzeba wyznaczyć numerycznie.

15.2. Pochodne wielomianu

Dla:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

mamy:

$$P'(z) = 4z^3 + 15z^2 + 26z + 19$$

oraz:

$$P''(z) = 12z^2 + 30z + 26$$

Przyjmujemy:

$$z_0 = 0$$

oraz:

$$n = 4$$

Obliczamy:

$$P(0) = 10$$

$$P'(0) = 19$$

$$P''(0) = 26$$

Zatem:

$$G = \frac{19}{10} = 1.9$$

oraz:

$$H = 1.9^2 - \frac{26}{10}$$

czyli:

$$H = 3.61 - 2.6 = 1.01$$

15.3. Pierwszy krok metody Laguerre'a

Mamy:

$$G = 1.9$$

$$H = 1.01$$

$$n = 4$$

Liczymy wyrażenie pod pierwiastkiem:

$$(n-1)(nH - G^2) = 3(4 \cdot 1.01 - 1.9^2)$$

czyli:

$$3(4.04 - 3.61) = 1.29$$

Stąd:

$$\sqrt{(n-1)(nH - G^2)} \approx 1.13578$$

Porównujemy dwa mianowniki:

$$G + 1.13578 \approx 3.03578$$

oraz:

$$G - 1.13578 \approx 0.76422$$

Wybieramy większy moduł:

$$G + 1.13578$$

Następnie:

$$a = \frac{4}{3.03578} \approx 1.31762$$

Nowe przybliżenie:

$$z_1 = z_0 - a$$

czyli:

$$z_1 = 0 - 1.31762 = -1.31762$$

15.4. Zbieżność do pierwszego pierwiastka

Kolejne iteracje prowadzą do pierwiastka bliskiego:

$$-1$$

Tabela z wykładu:

k	z_k	a_k	$z_{k+1} = z_k - a_k$
0	0	1.31762	-1.31762
1	-1.31762	-0.30310	-1.01451
2	-1.01451	-0.01451	-1.00000029
3	-1.00000029	$-2.91 \cdot 10^{-7}$	-1.00000000

Dla dokładności:

$$\varepsilon = 10^{-6}$$

otrzymujemy:

$$z_1 \approx -1$$

15.5. Deflacja po znalezieniu pierwszego pierwiastka

Po znalezieniu:

$$z_1 \approx -1$$

obniżamy stopień wielomianu:

$$P(z) = (z - z_1)P_3(z)$$

Ponieważ:

$$z_1 = -1$$

dzielimy przez:

$$z - z_1 = z + 1$$

Dostajemy:

$$P(z) = (z + 1)(z^3 + 4z^2 + 9z + 10)$$

czyli:

$$P_3(z) = z^3 + 4z^2 + 9z + 10$$

Teraz metodę Laguerre'a stosujemy do:

$$P_3(z)$$

15.6. Drugi pierwiastek i liczby zespolone

Dla wielomianu po deflacji:

$$P_3(z) = z^3 + 4z^2 + 9z + 10$$

można ponownie zacząć od:

$$z_0 = 0$$

Metoda Laguerre'a może przejść do przybliżeń zespolonych nawet wtedy, gdy punkt startowy jest rzeczywisty.

Przykładowe iteracje z wykładu:

$$0 \rightarrow -1.13924 + 1.58101i \rightarrow -0.99832 + 1.99860i \rightarrow -1 + 2i$$

Otrzymujemy:

$$z_2 \approx -1 + 2i$$

Ponieważ współczynniki wielomianu są rzeczywiste, drugim pierwiastkiem z pary jest:

$$z_3 \approx -1 - 2i$$

15.7. Deflacja po parze pierwiastków zespolonych

Znaleźliśmy parę:

$$z_2 = -1 + 2i$$

oraz:

$$z_3 = -1 - 2i$$

Odpowiada jej czynnik kwadratowy:

$$(z - z_2)(z - z_3)$$

Podstawiamy:

$$(z + 1 - 2i)(z + 1 + 2i)$$

Korzystamy ze wzoru:

$$(a - bi)(a + bi) = a^2 + b^2$$

Dostajemy:

$$(z + 1)^2 + 4$$

czyli:

$$z^2 + 2z + 5$$

Po kolejnej deflacji:

$$P_3(z) = (z^2 + 2z + 5)(z + 2)$$

Zostaje ostatni pierwiastek:

$$z_4 = -2$$

15.8. Wynik końcowy

Dla wielomianu:

$$P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10$$

otrzymujemy pierwiastki:

$$z_1 = -1$$

$$z_2 = -2$$

$$z_3 = -1 + 2i$$

$$z_4 = -1 - 2i$$

Przykład w Pythonie — sprawdzenie pierwiastków

```
def P(z):  
    return z ** 4 + 5 * z ** 3 + 13 * z ** 2 + 19 * z + 10  
  
pierwiastki = [  
    -1,  
    -2,  
    complex(-1, 2),  
    complex(-1, -2)  
]  
  
for z in pierwiastki:  
    print("z =", z)  
    print("P(z) =", P(z))
```

PYTHON

16. Pełniejszy przykład w Pythonie — metoda Laguerre'a

Poniżej jest przykład programu znajdującego jeden pierwiastek metodą Laguerre'a.

```
import cmath

def horner(wspolczynniki, z):
    p = wspolczynniki[0]

    for i in range(1, len(wspolczynniki)):
        p = p * z + wspolczynniki[i]

    return p

def pochodna_wspolczynniki(wspolczynniki):
    n = len(wspolczynniki) - 1
    wynik = []

    for i in range(len(wspolczynniki) - 1):
        stopien = n - i
        wynik.append(wspolczynniki[i] * stopien)

    return wynik

def laguerre_jeden_pierwiastek(wspolczynniki, z, epsilon, maks_iteracji):
    n = len(wspolczynniki) - 1

    P1 = pochodna_wspolczynniki(wspolczynniki)
    P2 = pochodna_wspolczynniki(P1)

    for iteracja in range(maks_iteracji):
        Pz = horner(wspolczynniki, z)

        if abs(Pz) < epsilon:
            return z

        P1z = horner(P1, z)
        P2z = horner(P2, z)

        G = P1z / Pz
        H = G * G - P2z / Pz

        pierwiastek = cmath.sqrt((n - 1) * (n * H - G * G))

        mianownik1 = G + pierwiastek
        mianownik2 = G - pierwiastek

        if abs(mianownik1) > abs(mianownik2):
            mianownik = mianownik1
        else:
            mianownik = mianownik2

        if mianownik == 0:
            print("Mianownik równy zero")
```

```

        return z

    a = n / mianownik
    z = z - a

    if abs(a) < epsilon:
        return z

    return z

# P(z) = z^4 + 5z^3 + 13z^2 + 19z + 10
wspolczynniki = [1, 5, 13, 19, 10]

z_start = 0 + 0j
epsilon = 0.000001
maks_iteracji = 100

pierwiastek = laguerre_jeden_pierwiastek(
    wspolczynniki,
    z_start,
    epsilon,
    maks_iteracji
)

print("Znaleziony pierwiastek:", pierwiastek)

```

17. Wnioski z wykładu

Aby obliczyć wszystkie pierwiastki wielomianu, warto zastosować następującą strategię:

1. Wczytać współczynniki wielomianu:

$$[a_n, a_{n-1}, \dots, a_0]$$

2. Obliczać:

$$P(z)$$

$$P'(z)$$

oraz:

$$P''(z)$$

schematem Hornera.

3. Dla aktualnego wielomianu stosować metodę Laguerre'a:

$$z_{new} = z_{old} - a$$

4. Po znalezieniu pierwiastka wykonać wygładzanie, czyli poprawienie go na pełnym, pierwotnym wielomianie.
5. Wykonać deflację i powtarzać obliczenia aż do wielomianu stopnia 2.
6. Dla wielomianu stopnia 2 można użyć wzorów dokładnych.

18. Najważniejsze rzeczy do zapamiętania na kolosa

18.1. Miejsce zerowe wielomianu

Miejsce zerowe to taka wartość:

$$z_0$$

że:

$$P(z_0) = 0$$

18.2. Podstawowe Twierdzenie Algebry

Wielomian stopnia:

$$n$$

ma na płaszczyźnie zespolonej dokładnie:

$$n$$

pierwiastków, licząc krotności.

18.3. Schemat Hornera

Wielomian zapisujemy jako:

$$P_n(z) = (((a_n z + a_{n-1})z + a_{n-2})z + \dots + a_1)z + a_0$$

Algorytm:

$$p \leftarrow a_n$$

$$p \leftarrow pz + a_i$$

18.4. Zaburzenie współczynników

Jeżeli współczynniki są zaburzone:

$$\tilde{a}_k = a_k + \delta_k$$

to pierwiastek też może się zmienić:

$$\tilde{z}_0 = z_0 + \varepsilon$$

Oszacowanie:

$$|\varepsilon| \approx \frac{|\sum_{k=0}^n \delta_k \tilde{z}_0^k|}{|\tilde{P}'_n(\tilde{z}_0)|}$$

18.5. Przykład Wilkinsona

Wielomian:

$$W(z) = (z + 1)(z + 2) \dots (z + 20)$$

ma pierwiastki:

$$-1, -2, \dots, -20$$

Małe zaburzenie jednego współczynnika może spowodować dużą zmianę pierwiastków.

18.6. Pierwiastki wielokrotne

Dla pierwiastków wielokrotnych problem jest szczególnie trudny, bo pochodne wielomianu mogą zniknąć.

Małe zaburzenie współczynników może rozszczepić pierwiastek wielokrotny.

18.7. Metoda Laguerre'a

Definiujemy:

$$G = \frac{P'(z)}{P(z)}$$

oraz:

$$H = G^2 - \frac{P''(z)}{P(z)}$$

Następnie:

$$a = \frac{n}{G \pm \sqrt{(n-1)(nH - G^2)}}$$

i:

$$z_{new} = z_{old} - a$$

Kończymy, gdy:

$$|a| < \varepsilon$$

18.8. Deflacja

Jeżeli znaleźliśmy pierwiastek:

$$z_1$$

to zapisujemy:

$$P_n(z) = (z - z_1)P_{n-1}(z)$$

i dalej szukamy pierwiastków wielomianu niższego stopnia.

18.9. Wygładzanie

Wygładzanie polega na poprawieniu znalezionej pierwiastka przez ponowne zastosowanie metody Laguerre'a do pełnego, pierwotnego wielomianu.

18.10. Pierwiastki zespolone

Jeżeli wielomian ma rzeczywiste współczynniki i ma pierwiastek:

$$z = x + iy$$

to ma też pierwiastek sprzężony:

$$\bar{z} = x - iy$$

Wykład 11: Generatory liczb pseudolosowych (lab13)

1. Losowość

W naturze, technice, ekonomii i życiu społecznym często spotykamy zjawiska, które wydają się losowe.

Losowość może wynikać z:

- braku pełnych informacji o zjawisku,
- błędów obserwacji,
- ograniczeń technicznych w dostępie do danych,
- złożoności zjawiska,
- właściwości fizycznych badanego procesu.

Jeżeli zjawisko jest bardzo złożone, to jego dokładne modelowanie deterministyczne może być niemożliwe albo niepraktyczne.

Przykład

Rzut kostką wydaje się losowy, ponieważ trudno dokładnie przewidzieć wynik. W rzeczywistości wynik zależy od wielu czynników fizycznych, np. siły rzutu, kąta, powierzchni i obrotu kostki.

Przykład w Pythonie

PYTHON

```
# Prosty przykład pokazujący możliwe wyniki rzutu kostką.  
# Nie używamy tutaj generatora losowego, tylko wypisujemy możliwe wartości.  
  
wyniki = [1, 2, 3, 4, 5, 6]  
  
for wynik in wyniki:  
    print("Możliwy wynik rzutu kostką:", wynik)
```

2. Losowość w matematyce i kryptografii

Losowość pojawia się także w matematyce.

Przykładem jest rozmieszczenie liczb pierwszych wśród liczb naturalnych. Można badać średnią częstość ich występowania, ale dokładne rozmieszczenie liczb pierwszych jest trudne do przewidzenia.

Liczby losowe są stosowane między innymi w:

- statystycznych badaniach reprezentatywnych,
- kontroli jakości,
- badaniach rynkowych,
- naukach eksperymentalnych,
- metodach Monte Carlo,
- optymalizacji,
- kryptografii.

W kryptografii liczby losowe są bardzo ważne, ponieważ mogą służyć jako klucze w szyfrach i zwiększać bezpieczeństwo przesyłanych informacji.

Przykład

Jeżeli klucz szyfrujący jest przewidywalny, to szyfr może być łatwy do złamania. Dlatego w kryptografii potrzebne są liczby trudne do przewidzenia.

3. Losowość w symulacjach i systemach komunikacyjnych

Symulacje komputerowe używają liczb losowych do naśladowania rzeczywistych procesów.

Liczby losowe są używane wtedy, gdy w modelu występują czynniki losowe albo gdy zjawisko jest zbyt złożone, aby badać je dokładnie metodami analitycznymi.

Liczby losowe są używane także w:

- grach komputerowych,
- symulatorach treningowych,
- grach strategicznych,
- systemach komunikacyjnych,
- sieciach komputerowych.

W grach komputerowych liczby losowe tworzą złudzenie realizmu, np. przez losowe zachowanie przeciwników albo losowe zdarzenia.

4. Ciąg losowy

Ciąg liczbowy nazywamy losowym, jeżeli nie istnieje krótszy algorytm opisujący ten ciąg niż sam ciąg.

Oznacza to, że:

- nie można znaleźć reguły pozwalającej odtworzyć ciąg,
- nie można przewidzieć kolejnego elementu na podstawie poprzednich,
- nie można opisać ciągu krócej niż przez wypisanie wszystkich jego elementów.

Sens definicji

Jeżeli ciąg ma prostą regułę, to nie jest naprawdę losowy.

Na przykład ciąg:

1, 2, 3, 4, 5, 6, ...

nie jest losowy, bo łatwo opisać go regułą:

$$a_n = n$$

Przykład w Pythonie

```
# Ciąg, który nie jest losowy, bo ma prostą regułę: a_n = n.  
  
for n in range(1, 11):  
    print(n)
```

PYTHON

5. Ciągi pseudolosowe

Ciąg pseudolosowy to ciąg, który jest generowany według określonej reguły, ale wygląda tak, jakby był losowy.

W przeciwieństwie do prawdziwie losowych, liczby pseudolosowe są generowane przez algorytmy deterministyczne.

To znaczy, że jeżeli znamy:

- algorytm,
- parametry,
- wartość początkową,

to możemy odtworzyć cały ciąg.

Ważne cechy ciągów pseudolosowych

Dobry ciąg pseudolosowy powinien:

- wyglądać jak losowy,
- mieć dobre własności statystyczne,
- być trudny do przewidzenia,
- dobrze symulować losowe zachowania.

Przykład

Jeżeli generator zawsze zaczyna od tego samego ziarna, to za każdym uruchomieniem może wygenerować ten sam ciąg.

Przykład w Pythonie

PYTHON

```
# Prosty deterministyczny ciąg pseudolosowy.  
# Zawsze dla tego samego X0 dostaniemy ten sam wynik.  
  
X = 3  
M = 10  
  
for i in range(10):  
    X = (2 * X + 1) % M  
    print(X)
```

6. Przykłady generatorów liczb pseudolosowych

W wykładzie wymieniono przykłady generatorów:

1. **LCG**, czyli liniowy generator kongruencyjny.
2. **Mersenne Twister**, znany z bardzo długiego okresu i wysokiej jakości ciągów.
3. **CSPRNG**, czyli kryptograficznie bezpieczne generatory pseudolosowe.

Krótkie porównanie

Generator	Cechy
LCG	prosty, szybki, historycznie ważny
Mersenne Twister	bardzo długi okres, popularny w bibliotekach
CSPRNG	trudny do przewidzenia, stosowany w kryptografii

7. Historyczne metody otrzymywania liczb losowych

Od dawna istniało zapotrzebowanie na liczby losowe, szczególnie w badaniach statystycznych.

Jednymi z pierwszych źródeł liczb losowych były tablice liczb losowych.

7.1. Wczesne tablice liczb losowych

Przykłady historyczne z wykładu:

1. W **1927** roku L.H. Tippet opublikował pierwszą tablicę losowych cyfr, składającą się z **41600** cyfr pochodzących z danych spisu powszechnego w Wielkiej Brytanii.

2. W 1939 roku R.A. Fisher i F. Yates wydali tablicę 15000 losowych cyfr zaczerpniętych z cyfr od 15 do 19 z tablic logarytmicznych.
3. Kendall, Babington i Smith w tym samym roku zaprezentowali tablicę 100000 cyfr losowych uzyskanych za pomocą „elektrycznej ruletki”.
4. W 1951 roku w Polsce GUS opracował własną tablicę liczb losowych.
5. W 1955 roku RAND Corporation stworzyła tablicę miliona cyfr losowych.

Wada tablic liczb losowych

Tablice miały ograniczoną długość, dlatego potrzebne stały się algorytmy generujące nowe ciągi liczb.

Przykład w Pythonie

PYTHON

```
# Przykładowa "tablica" cyfr.  
# W praktyce historyczne tablice były dużo większe.  
  
tablica = [4, 1, 9, 0, 2, 8, 7, 3, 5, 6]  
  
for cyfra in tablica:  
    print(cyfra)
```

7.2. Generowanie ciągu na podstawie tablicy cyfr

W wykładzie opisano przykładowy sposób korzystania z tablicy cyfr losowych:

1. Wybieramy losową pięciocyfrową liczbę z tablicy.
2. Modyfikujemy pierwszą cyfrę liczby modulo 2.
3. Tak zmieniona liczba pięciocyfrowa wskazuje numer wiersza w tablicy.
4. Zredukowana dwucyfrowa końcówka liczby modulo 50 wskazuje numer kolumny.
5. Od wybranej pozycji w tablicy zaczynamy tworzenie losowego ciągu.

Przykład w Pythonie

Losowanie z tablicy jednej liczby:

```
import random

tablica = [53827, 12345, 98765, 24680, 13579]

# Losujemy jedną liczbę z tablicy
liczba = random.choice(tablica)

print("Wylosowana liczba =", liczba)

# Pierwsza cyfra
pierwsza = liczba // 10000

# Reszta liczby po usunięciu pierwszej cyfry
reszta = liczba % 10000

# Modyfikacja pierwszej cyfry modulo 2
pierwsza_mod = pierwsza % 2

# Nowy numer wiersza
wiersz = pierwsza_mod * 10000 + reszta

# Dwucyfrowa końcówka
koncowka = liczba % 100

# Numer kolumny modulo 50
kolumna = koncowka % 50

print("wiersz =", wiersz)
print("kolumna =", kolumna)
```

Losowanie z tablicy wszystkich indeksów:


```
import random

tablica = [53827, 12345, 98765, 24680, 13579]

# tworzymy listę indeksów: [0, 1, 2, 3, 4]
indeksy = list(range(len(tablica)))

# mieszamy indeksy losowo
random.shuffle(indeksy)

for indeks in indeksy:
    liczba = tablica[indeks]

    print("indeks =", indeks)
    print("liczba =", liczba)

    # Pierwsza cyfra
    pierwsza = liczba // 10000

    # Reszta liczby po usunięciu pierwszej cyfry
    reszta = liczba % 10000

    # Modyfikacja pierwszej cyfry modulo 2
    pierwsza_mod = pierwsza % 2

    # Nowy numer wiersza
    wiersz = pierwsza_mod * 10000 + reszta

    # Dwucyfrowa końcówka
    koncowka = liczba % 100

    # Numer kolumny modulo 50
    kolumna = koncowka % 50

    print("wiersz =", wiersz)
    print("kolumna =", kolumna)
    print()
```

```
liczba = 53827

# Pierwsza cyfra
pierwsza = liczba // 10000

# Reszta liczby po usunięciu pierwszej cyfry
reszta = liczba % 10000

# Modyfikacja pierwszej cyfry modulo 2
pierwsza_mod = pierwsza % 2

# Nowy numer wiersza
wiersz = pierwsza_mod * 10000 + reszta

# Dwucyfrowa końcówka
koncowka = liczba % 100

# Numer kolumny modulo 50
kolumna = koncowka % 50

print("wiersz =", wiersz)
print("kolumna =", kolumna)
```

8. Współczesne metody otrzymywania liczb losowych

Współczesne metody generacji liczb losowych dzielą się na:

1. **algorytmiczne**,
2. **fizyczne**.

8.1. Generatory algorytmiczne

Generatory algorytmiczne używają wzorów matematycznych.

Ich ważna cecha:

- dla tych samych parametrów i tego samego ziarna dają ten sam ciąg.

Dzięki temu są powtarzalne, co jest przydatne np. w testach i symulacjach.

8.2. Generatory fizyczne

Generatory fizyczne opierają się na mierzalnych parametrach procesów fizycznych, które zachodzą losowo.

Przykłady z wykładu:

- moneta,
- kostka do gry,
- ruletka,
- licznik Geigera,
- elektroniczne liczniki impulsów,
- urządzenia wykorzystujące szum diodowy.

Ważna uwaga

Każdy wygenerowany ciąg liczb losowych powinien być testowany przed użyciem.

W przypadku awarii urządzenia fizycznego wygenerowany ciąg może stracić własności losowości.

9. Generatory algorytmiczne — rozkład jednostajny

Podstawą algorytmicznego generowania liczb pseudolosowych jest otrzymanie ciągu liczb całkowitych:

$$X_i \in 0, 1, \dots, M - 1$$

dla:

$$i = 1, 2, \dots$$

Te liczby powinny możliwie dobrze imitować losowanie z rozkładu jednostajnego.

Następnie liczby całkowite przekształca się do przedziału:

$$[0, 1)$$

według wzoru:

$$R_i = \frac{X_i}{M}$$

Wtedy:

$$R_i \in [0, 1)$$

Otrzymany ciąg traktuje się jako dyskretne przybliżenie rozkładu jednostajnego na przedziale:

$$[0, 1)$$

Przykład

Jeżeli:

$$M = 10$$

oraz:

$$X_i = 7$$

to:

$$R_i = \frac{7}{10} = 0.7$$

Przykład w Pythonie

PYTHON

```
X = [0, 2, 5, 7, 9]
M = 10

R = []

for i in range(len(X)):
    R.append(X[i] / M)

for i in range(len(R)):
    print("X =", X[i], "R =", R[i])
```

10. Przeskalowanie wartości generatora

Założmy, że generator zwraca liczby całkowite:

$$X \in 0, 1, \dots, MAX$$

10.1. Wartość z przedziału $[0, 1)$

Wartość z przedziału:

$$[0, 1)$$

można otrzymać przez:

$$R = \frac{X}{MAX + 1}$$

10.2. Wartość całkowita z przedziału $0, 1, \dots, max$

Jeżeli:

$$max < MAX$$

to wartość całkowitą z przedziału:

$$0, 1, \dots, max$$

można otrzymać ze wzoru:

$$Y = \left\lfloor \frac{X}{MAX + 1} (max + 1) \right\rfloor$$

10.3. Wartość całkowita z przedziału $min, min + 1, \dots, max$

Wartość całkowitą z przedziału:

$$min, min + 1, \dots, max$$

można otrzymać jako:

$$Y = min + \left\lfloor \frac{X}{MAX + 1} (max - min + 1) \right\rfloor$$

Przykład w Pythonie

```

X = 37
MAX = 99

R = X / (MAX + 1)

min_wartosc = 10
max_wartosc = 20

Y = min_wartosc + int((X / (MAX + 1)) * (max_wartosc - min_wartosc + 1))

print("R =", R)
print("Y =", Y)

```

PYTHON

11. Uwaga praktyczna — operator modulo

W programach często spotyka się zapis:

$$Y = X \bmod (max + 1)$$

Taki zapis ma zwracać liczby z przedziału:

$$0, 1, \dots, max$$

Problem

Jeżeli liczba możliwych wartości generatora nie jest podzielna przez:

$$max + 1$$

to niektóre wyniki mogą pojawiać się częściej niż inne.

To nazywa się obciążeniem modulo.

Ważny wniosek

Do prostych ćwiczeń taki zapis bywa akceptowalny, ale w zastosowaniach wymagających dobrej jakości losowości lepiej stosować metody unikające obciążenia modulo.

Przykład w Pythonie

PYTHON

```
# Przykład pokazujący, że modulo może rozłożyć wyniki nierówno.
```

```
MAX = 9
```

```
max_wartosc = 5
```

```
liczniki = []
```

```
for i in range(max_wartosc + 1):  
    liczniki.append(0)
```

```
for X in range(MAX + 1):  
    Y = X % (max_wartosc + 1)  
    liczniki[Y] = liczniki[Y] + 1
```

```
for i in range(len(liczniki)):  
    print("wartość", i, "liczba wystąpień", liczniki[i])
```

12. Generatory kongruencyjne LCG

Najbardziej znanym sposobem generowania liczb pseudolosowych jest metoda opracowana przez Lehmera w 1951 roku.

Nazywa się ją liniowym generatorem kongruencyjnym:

LCG

czyli:

Linear Congruential Generator

12.1. Addytywny LCG

Addytywny generator LCG ma wzór:

$$X_{n+1} = (a \cdot X_n + c) \bmod M$$

gdzie:

-

X_n

— n -ta liczba pseudolosowa,

- a
— mnożnik,
- c
— parametr,
- M
— moduł generatora.

12.2. Multiplikatywny LCG

Multiplikatywny LCG ma wzór:

$$X_{n+1} = a \cdot X_n \bmod M$$

Jest to szczególny przypadek generatora LCG, w którym:

$$c = 0$$

Przykład w Pythonie — addytywny LCG

```
a = 4
c = 2
M = 9
X = 0

ile = 10

for i in range(ile):
    print("X_", i, "=", X)
    X = (a * X + c) % M
```

PYTHON

13. Okres generatora LCG

Dla generatora:

$$X_{n+1} = (aX_n + c) \bmod M$$

okres oznacza liczbę kolejnych wartości, po których ciąg zaczyna się powtarzać.

Maksymalny możliwy okres nie może być większy niż liczba różnych stanów, czyli:

$$M$$

13.1. Generator mieszany

Dla generatora mieszanego, czyli gdy:

$$c \neq 0$$

generator może mieć pełny okres równy:

$$M$$

jeżeli parametry spełniają odpowiednie warunki.

Jednym z warunków jest:

$$\text{nwd}(c, M) = 1$$

Dla:

$$M = 2^m$$

typowym warunkiem jest:

$$a \equiv 1 \pmod{4}$$

13.2. Generator multiplikatywny

Dla generatora multiplikatywnego:

$$c = 0$$

Jeżeli:

$$M = p$$

gdzie:

$$p$$

jest liczbą pierwszą, to dla niezerowego ziarna maksymalny okres wynosi:

$$p - 1$$

Osiąga się go wtedy, gdy:

$$a$$

jest pierwiastkiem pierwotnym modulo:

$$p$$

Przykład w Pythonie — sprawdzanie okresu

PYTHON

```
a = 4
c = 2
M = 9
X0 = 0

X = X0
okres = 0

while True:
    X = (a * X + c) % M
    okres = okres + 1

    if X == X0:
        break

print("Okres generatora =", okres)
```

14. Dobór parametrów LCG — warunki Hulla-Dobella

Dla generatora mieszanego:

$$X_{n+1} = (aX_n + c) \bmod m$$

pełny okres równy:

$$m$$

można uzyskać, gdy spełnione są warunki Hulla-Dobella.

Warunki pełnego okresu

1. Liczby:

$$c$$

oraz:

$$m$$

są względnie pierwsze:

$$\text{nwd}(c, m) = 1$$

2. Liczba:

$$a - 1$$

jest podzielna przez każdy czynnik pierwszy liczby:

$$m$$

3. Jeżeli:

$$m$$

jest podzielne przez:

$$4$$

to:

$$a - 1$$

również jest podzielne przez:

$$4$$

Ważna uwaga

Spełnienie tych warunków gwarantuje maksymalną długość cyklu, ale nie oznacza jeszcze, że generator ma bardzo dobre własności statystyczne.

14.1. Przykład doboru parametrów LCG

Rozważamy generator:

$$X_{n+1} = (aX_n + c) \bmod m$$

Parametry:

$$m = 16$$

$$a = 5$$

$$c = 3$$

Sprawdzamy warunki:

$$\text{nwd}(3, 16) = 1$$

2. Czynnikiem pierwszym liczby 16 jest 2, a:

$$a - 1 = 5 - 1 = 4$$

jest podzielne przez 2.

3. Ponieważ 16 jest podzielne przez 4, sprawdzamy, czy:

$$a - 1 = 4$$

jest podzielne przez 4.

Jest.

Generator może więc osiągnąć pełny okres:

$$m = 16$$

```
def nwd(a, b):
    while b != 0:
        reszta = a % b
        a = b
        b = reszta

    return a

m = 16
a = 5
c = 3

warunek_1 = nwd(c, m) == 1
warunek_2 = (a - 1) % 2 == 0
warunek_3 = (a - 1) % 4 == 0

print("Warunek 1:", warunek_1)
print("Warunek 2:", warunek_2)
print("Warunek 3:", warunek_3)

if warunek_1 and warunek_2 and warunek_3:
    print("Generator może mieć pełny okres")
else:
    print("Warunki nie są spełnione")
```

15. Jak sprawdzić okres generatora?

Aby sprawdzić okres generatora LCG, można zapamiętywać wygenerowane wartości aż do momentu powtórzenia stanu początkowego.

Idea algorytmu

1. Ustal ziarno:

$$X_0$$

2. Generuj kolejne wartości:

$$X_{n+1} = (aX_n + c) \bmod m$$

3. Zliczaj wygenerowane wartości.
4. Zakończ, gdy ponownie pojawi się:

$$X_0$$

Jeżeli generator mieszany ma pełny okres, to przed powrotem do:

$$X_0$$

powinien wygenerować:

$$m$$

różnych stanów.

Przykład w Pythonie

```
PYTHON

a = 5
c = 3
m = 16
X0 = 0

X = X0
okres = 0

while True:
    X = (a * X + c) % m
    okres = okres + 1

    if X == X0:
        break

print("Okres =", okres)
```

16. Wady generatorów liniowych

Generator liniowy może mieć krótki cykl, jeżeli parametry są źle dobrane.

Wykład podaje przykład, że dla pewnych wartości można otrzymać sekwencję:

$$0, 1, 20, 0, 1, 20, \dots$$

czyli bardzo krótki cykl.

Główna wada LCG

Główną wadą generatorów liniowych jest ich przewidywalność.

Punkty w przestrzeni wielowymiarowej mogą układać się na ograniczonej liczbie:

- prostych,
- płaszczyzn,
- hiperpłaszczyzn.

Jest to słabość wykrywana między innymi testem widmowym.

Przykład w Pythonie

PYTHON

```
# Przykład generatora o krótkim cyklu.
```

```
a = 1
```

```
c = 1
```

```
M = 3
```

```
X = 0
```

```
for i in range(10):
```

```
    print(X)
```

```
    X = (a * X + c) % M
```

17. Popularne generatory LCG

W wykładzie podano tabelę wybranych generatorów.

Nazwa	M	a	c
Numerical Recipes	2^{32}	1664525	1013904223
Borland C/C++	2^{32}	22695477	1
GNU Compiler Collection	2^{32}	69069	5
ANSI C	2^{32}	1103515245	12345
Borland Delphi, Virtual Pascal	2^{32}	134775813	1
Microsoft Visual/Quick C/C++	2^{32}	214013	2531011
ANSIC	2^{31}	1103515245	12345
MINSTD	$2^{31} - 1$	16807	0

```
# Parametry MINSTD

M = 2 ** 31 - 1
a = 16807
c = 0

print("M =", M)
print("a =", a)
print("c =", c)
```

18. Przykład generatora kongruencyjnego mieszanego

Parametry generatora z wykładu:

$$a = 4$$

$$c = 2$$

$$M = 9$$

$$X_0 = 0$$

Kolejne wartości obliczamy ze wzoru:

$$X_n = (4 \cdot X_{n-1} + 2) \bmod 9$$

dla:

$$n = 1, 2, \dots$$

Kolejne wartości:

$$X_0 = 0$$

$$X_1 = (4 \cdot 0 + 2) \bmod 9 = 2$$

$$X_2 = (4 \cdot 2 + 2) \bmod 9 = 1$$

$$X_3 = (4 \cdot 1 + 2) \bmod 9 = 6$$

$$X_4 = (4 \cdot 6 + 2) \bmod 9 = 8$$

$$X_5 = (4 \cdot 8 + 2) \bmod 9 = 7$$

$$X_6 = (4 \cdot 7 + 2) \bmod 9 = 3$$

$$X_7 = (4 \cdot 3 + 2) \bmod 9 = 5$$

$$X_8 = (4 \cdot 5 + 2) \bmod 9 = 4$$

$$X_9 = (4 \cdot 4 + 2) \bmod 9 = 0$$

Po wartości:

$$X_9 = 0$$

ciąg zaczyna się powtarzać.

Okres generatora wynosi:

9

Przykład w Pythonie

PYTHON

```
a = 4
c = 2
M = 9
X0 = 0

X_poprzednie = X0

print("X_0 =", X0)

n = 1

while True:
    X_n = (a * X_poprzednie + c) % M

    print(f"X_{n} = ({a} * {X_poprzednie} + {c}) mod {M} = {X_n}")

    if X_n == X0:
        print("Ciąg wrócił do wartości początkowej.")
        print("Okres generatora wynosi:", n)
        break

    X_poprzednie = X_n
    n += 1
```

PYTHON

```
a = 4
c = 2
M = 9
X = 0

for n in range(19):
    print("X_", n, "=", X)
    X = (a * X + c) % M
```

19. Wizualizacja jakości generatora

Jednym z prostych sposobów oceny generatora jest tworzenie punktów z kolejnych wartości ciągu.

Można tworzyć pary:

$$(X_0, X_1), (X_2, X_3), (X_4, X_5), \dots$$

albo pary sąsiednie:

$$(X_0, X_1), (X_1, X_2), (X_2, X_3), \dots$$

Aby narysować punkty w kwadracie jednostkowym, skalujemy wartości:

$$R_n = \frac{X_n}{m}$$

Następnie rysujemy punkty:

$$(R_0, R_1), (R_2, R_3), (R_4, R_5), \dots$$

Interpretacja

Jeżeli punkty tworzą wyraźne linie, pasy albo regularne wzory, generator może mieć słabe własności statystyczne.

Przykład w Pythonie

PYTHON

```
# Przykład tworzenia punktów z kolejnych wartości generatora.
```

```
a = 4  
c = 2  
m = 9  
X = 0
```

```
liczby = []
```

```
for i in range(20):  
    liczby.append(X)  
    X = (a * X + c) % m
```

```
punkty = []
```

```
for i in range(0, len(liczby) - 1, 2):  
    R1 = liczby[i] / m  
    R2 = liczby[i + 1] / m  
    punkty.append([R1, R2])
```

```
for punkt in punkty:  
    print(punkt)
```

20. Generator Lehmera

Generator Lehmera jest odmianą generatora LCG.

Jest to generator multiplikatywny:

$$X_{k+1} = a \cdot X_k \bmod M$$

gdzie:

- M
— liczba pierwsza albo potęga liczby pierwszej,
- a
— element mający wysoki rząd modulo M ,
- X_0
— ziarno względnie pierwsze z M .

Generator Lehmera jest też nazywany generatorem Parka-Millera.

20.1. MINSTD

W 1988 roku Park i Miller zaproponowali parametry:

$$M = 2^{31} - 1$$

oraz:

$$a = 7^5 = 16807$$

Ten generator znany jest jako:

$$MINSTD$$

Później zaproponowano również mnożnik:

$$a = 48271$$

Maksymalny okres

Jeżeli:

$$M$$

jest liczbą pierwszą i:

$$a$$

jest pierwiastkiem pierwotnym, to maksymalny okres generatora Lehmera wynosi:

$$M - 1$$

Przykład w Pythonie

PYTHON

```
M = 2 ** 31 - 1
a = 16807
X = 1

for i in range(10):
    X = (a * X) % M
    print(X)
```

21. Uogólniony generator liniowy

Uogólnienie generatora liniowego polega na wykorzystaniu kilku poprzednich wartości ciągu.

Wzór:

$$X_n = (a_1 X_{n-1} + a_2 X_{n-2} + \dots + a_k X_{n-k} + b) \bmod M$$

Przy odpowiednim doborze stałych:

$$a_1, \dots, a_k, b < M$$

generator może osiągnąć maksymalny okres:

$$M$$

Ważna uwaga

Uogólniony generator liniowy mimo większej złożoności nadal nie nadaje się do zastosowań kryptograficznych.

Przykład w Pythonie

PYTHON

```
M = 17
a1 = 2
a2 = 3
b = 1

X = [4, 7]

for n in range(2, 10):
    nowy = (a1 * X[n - 1] + a2 * X[n - 2] + b) % M
    X.append(nowy)

for i in range(len(X)):
    print("X_", i, "=", X[i])
```

22. Generator Fibonacciego

Generator Fibonacciego jest odmianą uogólnionych generatorów liniowych.

Opiera się na ciągu Fibonacciego, ale obliczenia wykonuje się modulo:

$$m$$

Wzór:

$$X_n = (X_{n-1} + X_{n-2}) \bmod m$$

dla:

$$n \geq 2$$

Do działania potrzebne są dwie wartości początkowe:

$$X_0$$

oraz:

$$X_1$$

Wada

Generator Fibonacciego może mieć korelacje między kolejnymi wyrazami ciągu.

To oznacza, że wartości mogą mieć poprawny rozkład, ale nie muszą być wystarczająco niezależne.

Przykład w Pythonie

```
m = 17

X = [7, 16]

for n in range(2, 12):
    nowy = (X[n - 1] + X[n - 2]) % m
    X.append(nowy)

for i in range(len(X)):
    print("X_", i, "=", X[i])
```

PYTHON

23. Lagged Fibonacci Generator — LFG

Aby zmniejszyć proste zależności między kolejnymi wyrazami, generator Fibonacciego można uogólnić do postaci:

$$X_n = (X_{n-p} + X_{n-q}) \bmod m$$

gdzie:

$$n \geq p$$

oraz:

$$p > q \geq 1$$

Liczby:

$$p$$

oraz:

$$q$$

oznaczają opóźnienia generatora.

Do rozpoczęcia generowania potrzebne są wartości początkowe:

$$X_0, X_1, \dots, X_{p-1}$$

Generator można modyfikować przez zastąpienie dodawania inną operacją, np.:

- odejmowaniem,
- mnożeniem,
- operacją XOR.

Ogólniej:

$$X_n = (X_{n-p} \diamond X_{n-q}) \bmod m$$

gdzie:

$$\diamond$$

oznacza wybraną operację.

Przykład z wykładu

Rozważamy generator:

$$X_n = (X_{n-p} + X_{n-q}) \bmod m$$

gdzie:

$$m = 17$$

$$p = 3$$

$$q = 1$$

Wartości początkowe:

$$X_0 = 7$$

$$X_1 = 16$$

$$X_2 = 5$$

Kolejne wartości:

$$X_3 = (X_0 + X_2) \bmod 17 = (7 + 5) \bmod 17 = 12$$

$$X_4 = (X_1 + X_3) \bmod 17 = (16 + 12) \bmod 17 = 11$$

$$X_5 = (X_2 + X_4) \bmod 17 = (5 + 11) \bmod 17 = 16$$

Kolejny ciąg zaczyna się:

7, 16, 5, 12, 11, 16, 11, 5, 4, 15, 3, 7, ...

Przykład w Pythonie

PYTHON

```
m = 17
p = 3
q = 1

X = [7, 16, 5]

stan_poczatkowy = X.copy()

print("X_0 =", X[0])
print("X_1 =", X[1])
print("X_2 =", X[2])

n = p

while True:
    nowy = (X[n - p] + X[n - q]) % m
    X.append(nowy)

    print("X_", n, "=", nowy)

    # sprawdzamy, czy ostatnie p wartości są takie jak początkowe
    if X[-p:] == stan_poczatkowy:
        print("Ciąg wrócił do stanu początkowego.")
        print("Okres generatora wynosi:", n - p + 1)
        break

    n += 1
```

W generatorze opóźnionym stan generatora tworzy kilka ostatnich wartości, a nie tylko jedna wartość. Dlatego okres należy wykrywać przez powrót całego stanu początkowego, np. `[x0, x1, x2]`, a nie tylko przez pojawienie się samego `x0`.

```

m = 17
p = 3
q = 1

X = [7, 16, 5]

ile = 12

for n in range(p, ile):
    nowy = (X[n - p] + X[n - q]) % m
    X.append(nowy)

for i in range(len(X)):
    print("X_", i, "=", X[i])

```

24. Zaawansowane uogólnienia generatora Fibonacciego

Generator Fibonacciego można rozbudować, używając większej liczby poprzednich wartości.

Wzór:

$$X_n = (X_{n-p_1} \diamond X_{n-p_2} \diamond \dots \diamond X_{n-p_k}) \bmod m$$

gdzie:

$$n \geq p_k$$

oraz:

$$p_k > p_{k-1} > \dots > p_1 \geq 1$$

Symbole:

$$p_1, p_2, \dots, p_k$$

oznaczają opóźnienia.

Symbol:

$$\diamond$$

oznacza wybraną operację, np.:

- dodawanie,
- odejmowanie,
- mnożenie,
- XOR.

Przykładami generatorów tego typu są generatory Marsagli, np. **Marsa-LFIB4**, oraz generator Ziffa.

Przykład w Pythonie

PYTHON

```
# Przykład z trzema opóźnieniami i dodawaniem.

m = 31

X = [3, 7, 11, 19]

for n in range(4, 12):
    nowy = (X[n - 1] + X[n - 2] + X[n - 4]) % m
    X.append(nowy)

for i in range(len(X)):
    print("X_", i, "=", X[i])
```

25. Kwadratowy generator kongruencyjny

Aby uniknąć prostej zależności liniowej między kolejnymi wartościami ciągu, można użyć zależności kwadratowej.

Wzór:

$$X_{n+1} = (aX_n^2 + bX_n + c) \bmod m$$

Parametry:

$$a, b, c, m$$

oraz wartość początkowa:

$$X_0$$

muszą być dobrane tak, aby generator miał możliwie długi okres i dobre własności statystyczne.

Dla odpowiednich parametrów maksymalny okres może być równy:

$$m$$

```
a = 2
b = 3
c = 1
m = 17

X = 5

for n in range(10):
    print("X_", n, "=", X)
    X = (a * X * X + b * X + c) % m
```

26. Generator wykorzystujący wielomiany permutacyjne

Generator może wykorzystywać wielomian:

$$g(x) = \sum_{k=0}^r a_k x^k$$

gdzie:

$$a_k \in 0, 1, \dots, m-1$$

Wielomian:

$$g(x)$$

jest wielomianem permutacyjnym modulo:

$$m$$

jeżeli funkcja:

$$x \mapsto g(x) \bmod m$$

przestawia elementy zbioru:

$$0, 1, \dots, m-1$$

To oznacza, że każdy element zbioru pojawia się dokładnie raz jako wynik działania funkcji.

Ważna uwaga

Generator tego typu może ograniczać proste zależności liniowe z klasycznych generatorów kongruencyjnych, ale nadal jest algorytmem deterministycznym.

Nie należy automatycznie traktować go jako generatora kryptograficznie bezpiecznego.

Przykład w Pythonie

PYTHON

```
m = 5

# współczynniki wielomianu:
# g(x) = a0*x^0 + a1*x^1
# g(x) = 1 + 1*x
a = [1, 1]

wartosci = []

for x in range(m):
    suma = 0

    for k in range(len(a)):
        suma += a[k] * (x ** k)

    g = suma % m
    wartosci.append(g)

print("Wartości funkcji modulo m:")

for wartosc in wartosci:
    print(wartosc)

if len(set(wartosci)) == m:
    print("g(x) jest wielomianem permutacyjnym modulo", m)
else:
    print("g(x) nie jest wielomianem permutacyjnym modulo", m)
```

Prostrzy przykład:

PYTHON

```
# Sprawdzamy wartości przykładowej funkcji g(x) = x + 1 modulo m.

m = 5

wartosci = []

for x in range(m):
    g = (x + 1) % m
    wartosci.append(g)

print("Wartości funkcji modulo m:")

for wartosc in wartosci:
    print(wartosc)
```

27. Generator inwersyjny

Generator inwersyjny wykorzystuje odwrotność modulo liczby pierwszej.

Wzór:

$$X_{n+1} = \begin{cases} (aX_n^{-1} + b) \bmod p, & X_n \neq 0 \\ b, & X_n = 0 \end{cases}$$

gdzie:

$$p$$

jest liczbą pierwszą.

Maksymalny okres generatora, przy odpowiednich wartościach:

$$a$$

oraz:

$$b$$

może wynosić:

$$p - 1$$

```
def odwrotnosc_modulo(x, p):
    # Szukamy takiego y, że (x*y) mod p = 1.
    for y in range(1, p):
        if (x * y) % p == 1:
            return y

    return None

p = 17
a = 3
b = 5

X0 = 4
X = X0

n = 0

while True:
    print("X_", n, "=", X)

    if X != 0:
        odw = odwrotnosc_modulo(X, p)
        X = (a * odw + b) % p
    else:
        X = b

    n += 1

    if X == X0:
        print("Ciąg wrócił do wartości początkowej.")
        print("Okres generatora wynosi:", n)
        break
```

28. Nowoczesne generatory do symulacji numerycznych

Współczesne generatory stosowane w symulacjach nie są zwykle prostymi generatorami LCG.

Celem jest uzyskanie:

- długiego okresu,
- dobrych własności statystycznych,
- wydajnej implementacji.

Przykładowe rodziny generatorów:

1. **Mersenne Twister** — bardzo długi okres, popularny w bibliotekach, ale ma duży stan i nie jest bezpieczny kryptograficznie.
2. **PCG** — łączy prosty mechanizm kongruencyjny z permutacją bitów wyjściowych.
3. **xoshiro/xoroshiro** — szybkie generatory oparte na operacjach XOR, przesunięciach i rotacjach bitowych.

Ważna uwaga

Trzeba odróżniać generator do symulacji od generatora do kryptografii.

29. Generator PCG — idea

Generator PCG łączy dwie idee:

1. prostą rekurencję kongruencyjną dla stanu wewnętrznego,
2. dodatkową permutację bitów przed zwróceniem wyniku.

Schemat działania:

$$X_{n+1} = (aX_n + c) \bmod 2^m$$

a następnie:

$$R_n = \text{permute}(X_n)$$

Dlaczego to pomaga?

Sama rekurencja kongruencyjna może mieć widoczne zależności liniowe.

Permutacja bitów wyjściowych utrudnia pojawianie się prostych wzorców w obserwowanym ciągu.

Ważna uwaga

PCG jest dobrym przykładem generatora do symulacji, ale nie jest standardowym generatorem kryptograficznym.

Przykład w Pythonie

PYTHON

```
# Bardzo uproszczony przykład idei generatora PCG:
# 1. najpierw aktualizujemy stan za pomocą generatora kongruencyjnego,
# 2. potem wykonujemy prostą permutację bitów przed zwróceniem wyniku.

m = 16
modul = 2 ** m

a = 1103515245
c = 12345

X = 7

for n in range(5):
    # Rekurencja kongruencyjna:
    #  $X_{n+1} = (a * X_n + c) \bmod 2^m$ 
    X = (a * X + c) % modul

    # Uproszczona permutacja bitów:
    # przesunięcie bitowe w prawo i operacja XOR
    R = X ^ (X >> 5)

    print("stan =", X, "wynik =", R)
```

Dodatkowe

mniej uproszczony przykład PCG, bliższy prawdziwej wersji **PCG32**.

Tutaj są dwa etapy:

1. aktualizacja stanu przez LCG:

$$X_{n+1} = (aX_n + c) \bmod 2^{64}$$

2. permutacja bitów, czyli przekształcenie stanu na wynik 32-bitowy.

```

# Mniej uproszczony przykład generatora PCG32.
# Generator ma 64-bitowy stan wewnętrzny,
# a zwraca 32-bitową liczbę pseudolosową.

MASK_64 = (1 << 64) - 1
MASK_32 = (1 << 32) - 1

def rotacja_w_prawo(x, r):
    # Funkcja wykonuje rotację bitów w prawo dla liczby 32-bitowej.
    # To znaczy, że bity "wypchnięte" z prawej strony wracają z lewej strony.
    return ((x >> r) | (x << ((-r) & 31))) & MASK_32

class PCG32:
    def __init__(self, seed=42, seq=54):
        # seed to wartość początkowa generatora.
        # seq pozwala tworzyć różne niezależne ciągi.
        self.state = 0

        # Stała mnożnika używana w PCG.
        self.multiplier = 6364136223846793005

        # Wartość increment musi być nieparzysta.
        # Dlatego bierzemy seq << 1 i dodajemy 1.
        self.increment = ((seq << 1) | 1) & MASK_64

        # Inicjalizacja zgodna z ideą PCG.
        self.random()
        self.state = (self.state + seed) & MASK_64
        self.random()

    def random(self):
        # Zapamiętujemy stary stan.
        oldstate = self.state

        # Aktualizacja stanu:
        #  $X_{n+1} = (a * X_n + c) \bmod 2^{64}$ 
        self.state = (oldstate * self.multiplier + self.increment) & MASK_64

        # Permutacja bitów.
        # Najpierw mieszamy bity starego stanu przez XOR i przesunięcie.
        xorshifted = (((oldstate >> 18) ^ oldstate) >> 27) & MASK_32

        # Liczymy wartość rotacji na podstawie starszych bitów stanu.
        rot = oldstate >> 59

        # Zwracamy wynik po rotacji bitów.
        return rotacja_w_prawo(xorshifted, rot)

```

Przykład użycia

```
generator = PCG32(seed=7, seq=3)
```

```
for i in range(10):  
    liczba = generator.random()  
    print("R_", i, "=", liczba)
```

Najważniejsze linie odpowiadają slajdowi:

```
self.state = (oldstate * self.multiplier + self.increment) & MASK_64
```

PYTHON

To jest rekurencja kongruencyjna:

$$X_{n+1} = (aX_n + c) \bmod 2^{64}$$

A to jest permutacja bitów:

```
xorshifted = (((oldstate >> 18) ^ oldstate) >> 27) & MASK_32  
rot = oldstate >> 59  
return rotacja_w_prawo(xorshifted, rot)
```

PYTHON

Czyli generator nie zwraca bezpośrednio `state`, tylko najpierw miesza jego bity. Dzięki temu wynik ma lepsze własności niż zwykły generator kongruencyjny.

W generatorze PCG stan wewnętrzny jest aktualizowany prostą rekurencją kongruencyjną, ale wynik nie jest zwracany bezpośrednio. Przed zwróceniem wykonywana jest permutacja bitów, np. przesunięcia, XOR oraz rotacja. Dzięki temu ogranicza się widoczne zależności liniowe typowe dla klasycznych generatorów LCG. nadal nie jest to standardowy generator kryptograficzny.

30. Generatory kryptograficznie bezpieczne

W kryptografii nie wystarcza dobry rozkład statystyczny.

Generator powinien być odporny na przewidywanie kolejnych bitów nawet wtedy, gdy przeciwnik zna część wcześniejszych wyników.

W praktyce stosuje się generatory typu:

- `CSPRNG`,
- `DRBG`.

Przykłady:

- `Hash DRBG`,

- HMAC DRBG,
- CTR DRBG.

Ważna zasada praktyczna

Do symulacji numerycznych można używać szybkich generatorów statystycznych.

Do kluczy, soli, tokenów i haseł należy używać generatorów kryptograficznych dostarczanych przez system operacyjny albo sprawdzone biblioteki kryptograficzne.

Przykład w Pythonie

```
# Przykład pokazujący rozróżnienie zastosowań.

zastosowanie = "kryptografia"

if zastosowanie == "symulacja":
    print("Można użyć generatora statystycznego")
elif zastosowanie == "kryptografia":
    print("Należy użyć generatora kryptograficznego")
else:
    print("Trzeba dobrać generator do zastosowania")
```

PYTHON

31. Metody testowania generatorów

Aby ocenić generator albo wygenerowany ciąg bitów, stosuje się testy statystyczne.

Testy powinny potwierdzać:

- równomierny rozkład ciągu bitów,
- losowość rozkładu,
- niezależność kolejnych bitów.

W literaturze występują różne testy:

- ogólne, dotyczące rozkładów liczb całkowitych,
- specyficzne, dotyczące ciągów binarnych.

Przykład w Pythonie

PYTHON

```
bity = [1, 0, 1, 1, 0, 0, 1, 0]

liczba_jedynek = 0

for bit in bity:
    if bit == 1:
        liczba_jedynek = liczba_jedynek + 1

print("Liczba jedynek =", liczba_jedynek)
```

32. Historyczne testy FIPS dla ciągów bitowych

W praktycznej weryfikacji generatorów stosowano proste testy statystyczne dla próbek długości:

20000

bitów.

Przykładowe testy:

1. test monobitowy,
2. test pokerowy,
3. test serii,
4. test długich serii.

Obecnie w zastosowaniach kryptograficznych większe znaczenie mają standardy dotyczące konstrukcji generatorów, źródeł entropii i monitorowania działania generatora.

33. Test monobitowy

Test monobitowy sprawdza, czy liczba jedynek w ciągu bitów mieści się w granicach statystycznie prawdopodobnych dla losowego ciągu.

Dla ciągu długości:

20000

liczymy:

X = liczba jedynek w ciągu 20000 bitów

Kryterium akceptacji z wykładu:

$$9725 < X < 10275$$

Jeżeli liczba jedynek spełnia ten warunek, ciąg jest uznawany za zgodny z oczekiwaniami dla rozkładu równomiernego.

Przykład w Pythonie

PYTHON

```
# Przykład dla krótszego ciągu, żeby pokazać ideę.
```

```
bity = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]
```

```
X = 0
```

```
for bit in bity:
    if bit == 1:
        X = X + 1
```

```
print("Liczba jedynek =", X)
```

34. Test pokerowy

Test pokerowy analizuje segmenty czterobitowe w ciągu bitów.

Procedura dla ciągu 20000 bitów:

1. Dzielimy ciąg na 5000 segmentów czterobitowych.
2. Liczymy częstość wystąpienia każdego z 16 możliwych segmentów:

0000, 0001, ..., 1111

3. Obliczamy statystykę:

$$X = \frac{16}{5000} \left(\sum_{i=0}^{15} [f(i)]^2 \right) - 5000$$

gdzie:

$$f(i)$$

to liczba wystąpień i -tego segmentu czterobitowego.

4. Akceptowalny zakres z wykładu:

$$2.16 < X < 46.17$$

Przykład w Pythonie

PYTHON

```
# Przykład dla krótkiego ciągu bitów.
# Dzielimy go na bloki 4-bitowe i zliczamy wystąpienia.

bity = "00010010111100011111"

liczniki = []

for i in range(16):
    liczniki.append(0)

for i in range(0, len(bity), 4):
    blok = bity[i:i + 4]

    if len(blok) == 4:
        wartosc = int(blok, 2)
        liczniki[wartosc] = liczniki[wartosc] + 1

print("Liczności bloków:")

for i in range(16):
    print(format(i, "04b"), "=", liczniki[i])

# liczba pełnych bloków 4-bitowych
N = len(bity) // 4

suma_kwadratow = 0

for i in range(16):
    suma_kwadratow += liczniki[i] ** 2

X = (16 / N) * suma_kwadratow - N

print("Statystyka X =", X)
```

uwaga: dla tego krótkiego ciągu **nie powinno się stosować zakresu:**

$$2.16 < X < 46.17$$

pełniejszy przykład zgodny ze slajdem wygląda tak:

```

import random

# Generujemy ciąg 20000 bitów
bity = ""

for i in range(20000):
    bity += str(random.randint(0, 1))

liczniki = []

for i in range(16):
    liczniki.append(0)

# Dzielimy ciąg na bloki 4-bitowe
for i in range(0, len(bity), 4):
    blok = bity[i:i + 4]

    wartosc = int(blok, 2)
    liczniki[wartosc] = liczniki[wartosc] + 1

print("Liczności bloków:")

for i in range(16):
    print(format(i, "04b"), "=", liczniki[i])

# Liczba bloków
N = 5000

suma_kwadratow = 0

for i in range(16):
    suma_kwadratow += liczniki[i] ** 2

X = (16 / N) * suma_kwadratow - N

print("Statystyka X =", X)

if 2.16 < X < 46.17:
    print("Test pokerowy zaliczony")
else:
    print("Test pokerowy niezaliczony")

```

35. Test serii

Serią nazywamy ciąg kolejnych bitów o tej samej wartości.

Przykład:

jest serią jedynek długości 3.

Test serii sprawdza, czy liczba serii różnej długości jest zgodna z oczekiwaniami dla losowego ciągu bitów.

W teście analizuje się długości serii:

- 1,
- 2,
- 3,
- 4,
- 5,
- 6 i więcej.

Akceptowalne zakresy z wykładu:

Długość serii	Zakres
1	2343 do 2657
2	1135 do 1365
3	542 do 708
4	251 do 373
5	111 do 201
6 i więcej	111 do 201

```
bity = "111001000111100"

serie = []

aktualny_bit = bity[0]
dlugosc = 1

for i in range(1, len(bity)):
    if bity[i] == aktualny_bit:
        dlugosc = dlugosc + 1
    else:
        serie.append([aktualny_bit, dlugosc])
        aktualny_bit = bity[i]
        dlugosc = 1

serie.append([aktualny_bit, dlugosc])

for seria in serie:
    print("bit =", seria[0], "długość serii =", seria[1])
```

36. Różne rodzaje testów statystycznych

W literaturze można znaleźć różne grupy testów:

1. **Testy losowości** — sprawdzają, czy ciąg wyników może być traktowany jako ciąg zmiennych losowych.
2. **Testy zgodności** — sprawdzają, czy obserwacje mają ten sam rozkład prawdopodobieństwa.
3. **Testy normalności** — sprawdzają, czy dane pochodzą z rozkładu normalnego.
4. **Testy dotyczące parametrów rozkładu** — potwierdzają wiarygodność estymowanych parametrów.
5. **Testy niezależności** — sprawdzają, czy zmienne są niezależne.

37. Kryteria dobrego generatora

Dobry generator powinien spełniać kilka kryteriów.

37.1. Okres

Okres powinien być długi.

Idealny generator miałby okres równy albo zbliżony do maksymalnej możliwej liczby unikatowych stanów.

Długi okres oznacza, że ciąg wartości nie powtarza się przez bardzo długi czas.

37.2. Równomierność

Równomierność oznacza, że każda możliwa wartość w zakresie generatora powinna pojawiać się z podobnym prawdopodobieństwem.

Równomierność można oceniać testami statystycznymi, np. testem chi-kwadrat.

37.3. Nieprzewidywalność

Nieprzewidywalność oznacza, że nie można efektywnie przewidzieć kolejnych wartości na podstawie skończonej liczby wcześniejszych wartości.

Można ją oceniać przez analizę korelacji i wzorców w ciągu.

Przykładem testu nieprzewidywalności jest test następnego bitu.

Przykład w Pythonie

```
okres_dlugi = True
rozklad_rowny = True
nieprzewidywalny = False

if okres_dlugi and rozklad_rowny and nieprzewidywalny:
    print("Generator spełnia główne kryteria")
else:
    print("Generator wymaga dalszej oceny")
```

PYTHON

38. Najważniejsze rzeczy do zapamiętania na kolosa

38.1. Ciąg losowy

Ciąg losowy to taki ciąg, którego nie da się opisać krótszym algorytmem niż sam ciąg.

38.2. Ciąg pseudolosowy

Ciąg pseudolosowy jest generowany deterministycznie, ale wygląda jak losowy.

38.3. Ziarno

Ziarno, czyli **seed**, to wartość początkowa generatora:

$$X_0$$

Dla tego samego ziarna generator algorytmiczny daje ten sam ciąg.

38.4. Skalowanie do przedziału $[0, 1)$

Jeżeli:

$$X_i \in 0, 1, \dots, M - 1$$

to:

$$R_i = \frac{X_i}{M}$$

38.5. Skalowanie do przedziału całkowitego

Dla:

$$X \in 0, 1, \dots, MAX$$

oraz wartości od **min** do **max**:

$$Y = min + \left\lfloor \frac{X}{MAX + 1} (max - min + 1) \right\rfloor$$

38.6. Problem modulo

Zapis:

$$Y = X \bmod (max + 1)$$

może powodować nierównomierny rozkład, jeżeli liczba stanów generatora nie dzieli się przez:

$$max + 1$$

38.7. Addytywny LCG

$$X_{n+1} = (aX_n + c) \bmod M$$

38.8. Multiplikatywny LCG

$$X_{n+1} = aX_n \bmod M$$

czyli przypadek:

$$c = 0$$

38.9. Okres generatora

Okres to liczba kolejnych wartości, po których ciąg zaczyna się powtarzać.

38.10. Warunki Hulla-Dobella

Dla generatora mieszanego pełny okres można uzyskać, gdy:

$$\text{nwd}(c, m) = 1$$

$$a - 1$$

jest podzielne przez każdy czynnik pierwszy:

$$m$$

oraz jeżeli:

$$4 \mid m$$

to:

$$4 \mid (a - 1)$$

38.11. Generator Lehmera

$$X_{k+1} = aX_k \bmod M$$

Dla liczby pierwszej:

$$M$$

i pierwiastka pierwotnego:

$$a$$

maksymalny okres wynosi:

$$M - 1$$

38.12. Generator Fibonacciego

$$X_n = (X_{n-1} + X_{n-2}) \bmod m$$

38.13. Lagged Fibonacci Generator

$$X_n = (X_{n-p} + X_{n-q}) \bmod m$$

gdzie:

$$p > q \geq 1$$

38.14. Kwadratowy generator kongruencyjny

$$X_{n+1} = (aX_n^2 + bX_n + c) \bmod m$$

38.15. Generator inwersyjny

$$X_{n+1} = \begin{cases} (aX_n^{-1} + b) \bmod p, & X_n \neq 0 \\ b, & X_n = 0 \end{cases}$$

38.16. Test monobitowy

Dla 20000 bitów liczymy liczbę jedynek:

$$X$$

Warunek akceptacji:

$$9725 < X < 10275$$

38.17. Test pokerowy

Dzielimy 20000 bitów na 5000 bloków czterobitowych i liczymy statystykę:

$$X = \frac{16}{5000} \left(\sum_{i=0}^{15} f(i)^2 \right) - 5000$$

Zakres akceptacji:

$$2.16 < X < 46.17$$

38.18. Test serii

Test serii analizuje długości kolejnych ciągów takich samych bitów.

38.19. Kryteria dobrego generatora

Dobry generator powinien mieć:

- długi okres,
 - równomierny rozkład,
 - nieprzewidywalność,
 - możliwie małe korelacje między wartościami.
-

Wykład 12: Metody Monte Carlo (lab 14)

1. Wprowadzenie do metody Monte Carlo

Metoda Monte Carlo jest metodą numeryczną opartą na losowym próbkowaniu przestrzeni rozwiązań.

Stosuje się ją do rozwiązywania problemów, które są zbyt złożone, aby rozwiązać je klasycznymi metodami analitycznymi.

Metoda Monte Carlo polega na tym, że zamiast liczyć dokładne rozwiązanie, wykonujemy wiele losowych prób, a następnie uśredniamy lub analizujemy otrzymane wyniki.

Geneza metody

Nazwa **Monte Carlo** została użyta w latach 40. XX wieku przez naukowców z Instytutu Los Alamos.

Metoda była wtedy wykorzystywana przy projektach jądrowych, między innymi do symulacji losowego zachowania neutronów w substancjach rozszczepialnych.

Z metodą Monte Carlo związani byli między innymi:

- Stanisław Ulam,
- John von Neumann,
- Enrico Fermi.

Przykład intuicyjny

Jeżeli trudno dokładnie obliczyć pole skomplikowanego obszaru, można losować punkty w prostokącie obejmującym ten obszar i sprawdzać, ile z nich trafiło do środka badanego obszaru.

Im więcej punktów wylosujemy, tym zwykle lepsze będzie przybliżenie.

Przykład w Pythonie

PYTHON

```
# Prosty przykład idei Monte Carlo:  
# zamiast dokładnie analizować cały obszar, wykonujemy wiele prób.  
  
liczba_prob = 10  
  
for i in range(liczba_prob):  
    print("Wykonujemy próbę numer:", i + 1)
```

2. Zastosowania metody Monte Carlo

Metoda Monte Carlo znalazła zastosowanie w wielu dziedzinach nauki i techniki.

Z wykładu:

- fizyka,
- finanse,
- zarządzanie projektami,
- zarządzanie ryzykiem,
- nauki społeczne,
- symulacje procesów fizycznych,
- symulacje procesów matematycznych,
- symulacje procesów ekonomicznych,
- symulacje procesów inżynierskich.

W finansach metoda Monte Carlo może pomagać ocenić, przy jakim czasie trwania projektu lub przy jakiej wysokości budżetu osiąga się określony poziom ryzyka.

3. Ogólny algorytm metody Monte Carlo

Metoda Monte Carlo opiera się na czterech podstawowych krokach.

1. Definicja przestrzeni możliwych danych wejściowych.

2. Losowe generowanie danych wejściowych z tej przestrzeni.
3. Wykonanie obliczeń probabilistycznych na podstawie wylosowanych danych.
4. Agregacja wyników, aby uzyskać ostateczne rozwiązanie.

Schemat

Można to zapisać ogólnie:

$$wynik \approx \frac{1}{N} \sum_{i=1}^N wynik_i$$

gdzie:

- N
— liczba prób,
- $wynik_i$
— wynik pojedynczej próby.

Przykład w Pythonie

Przykład agregacji wyników bez używania sum().

```
wyniki = [2, 4, 3, 5, 6]
```

```
suma = 0
```

```
for i in range(len(wyniki)):
    suma = suma + wyniki[i]
```

```
srednia = suma / len(wyniki)
```

```
print("Średnia z wyników =", srednia)
```

PYTHON

4. Zmienna losowa

Zmienna losowa to wynik obserwacji procesu losowego, który można przedstawić za pomocą wartości.

Wynik może być:

- liczbowy, np. wynik rzutu kostką,
- binarny, np. wynik rzutu monetą,
- symboliczny, np. karta z talii.

Zdarzenia losowe mogą generować ciągi zmiennych losowych:

$$X_1, X_2, X_3, \dots$$

Każdej wartości można przypisać prawdopodobieństwo:

$$p_1, p_2, p_3, \dots$$

Prawdopodobieństwo można rozumieć jako granicę częstości wystąpień danej wartości w bardzo długiej serii prób:

$$\lim_{N \rightarrow \infty} \frac{n_i}{N} = p_i$$

gdzie:

- n_i
— liczba wystąpień danego wyniku,
- N
— liczba wszystkich prób,
- p_i
— prawdopodobieństwo danego wyniku.

Przykład

Jeżeli rzucamy uczciwą kostką bardzo wiele razy, to częstość wypadania jedynki powinna zbliżać się do:

$$\frac{1}{6}$$

Przykład w Pythonie

PYTHON

```
# Przykład obliczania częstości wystąpienia wartości 1.

wyniki = [1, 2, 1, 4, 5, 1, 3, 6, 1, 2]

liczba_jedynek = 0

for i in range(len(wyniki)):
    if wyniki[i] == 1:
        liczba_jedynek = liczba_jedynek + 1

czestosc = liczba_jedynek / len(wyniki)

print("Liczba jedynek =", liczba_jedynek)
print("Częstość jedynek =", czestosc)
```

```
# Przykład obliczania częstości występowania orła w rzutach monetą.

wyniki = ["orzeł", "reszka", "orzeł", "orzeł", "reszka", "orzeł", "reszka",
"reszka"]

liczba_orlow = 0

for i in range(len(wyniki)):
    if wyniki[i] == "orzeł":
        liczba_orlow = liczba_orlow + 1

czestosc = liczba_orlow / len(wyniki)

print("Liczba orłów =", liczba_orlow)
print("Liczba wszystkich rzutów =", len(wyniki))
print("Częstość orłów =", czestosc)
```

Dla uczciwej monety przy bardzo dużej liczbie rzutów częstość orła powinna zbliżać się do:

$$1/2 = 0.5$$

```
# Przykład obliczania częstości występowania asa w losowaniach kart z talii.

wyniki = ["as", "król", "dama", "as", "10", "walet", "as", "9", "król", "2"]

liczba_asow = 0

for i in range(len(wyniki)):
    if wyniki[i] == "as":
        liczba_asow = liczba_asow + 1

czestosc = liczba_asow / len(wyniki)

print("Liczba asów =", liczba_asow)
print("Liczba wszystkich losowań =", len(wyniki))
print("Częstość asów =", czestosc)
```

W talii 52 kart są 4 asy, więc prawdopodobieństwo wylosowania asa wynosi:

$$4/52 = 1/13 \approx 0.0769$$

5. Rozkład zmiennej losowej dyskretnej

Zmienna losowa dyskretna przyjmuje skończoną lub przeliczalną liczbę wartości.

Każdej wartości można przypisać określone prawdopodobieństwo.

Rozkład prawdopodobieństwa to lista możliwych wartości zmiennej losowej i odpowiadających im prawdopodobieństw.

Przykład — rzut uczciwą kostką

Dla rzutu uczciwą kostką:

$$P(X = x) = \frac{1}{6}$$

dla:

$$x = 1, 2, 3, 4, 5, 6$$

Własności rozkładu prawdopodobieństwa

Prawdopodobieństwa są nieujemne:

$$P(X = x) \geq 0$$

Suma prawdopodobieństw wszystkich wartości wynosi:

$$\sum_x P(X = x) = 1$$

Przykład w Pythonie

```
wartosci = [1, 2, 3, 4, 5, 6]
prawdopodobienstwa = [1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6]

suma_p = 0

for i in range(len(prawdopodobienstwa)):
    suma_p = suma_p + prawdopodobienstwa[i]

for i in range(len(wartosci)):
    print("P(X =", wartosci[i], ") =", prawdopodobienstwa[i])

print("Suma prawdopodobieństw =", suma_p)
```

PYTHON

6. Moment zmiennej losowej

Moment n-tego rzędu zmiennej losowej jest średnią wartością jej n -tej potęgi.

Dla zmiennej dyskretnej:

$$E(X^n) = \sum_{i=1}^n x_i^n p(x_i)$$

W tym wzorze:

- x_i
— możliwa wartość zmiennej,
- $p(x_i)$
— prawdopodobieństwo tej wartości.

Przykład

Dla rzutu kostką moment drugiego rzędu to:

$$E(X^2) = 1^2 \cdot \frac{1}{6} + 2^2 \cdot \frac{1}{6} + \dots + 6^2 \cdot \frac{1}{6}$$

Przykład w Pythonie

PYTHON

```
wartosci = [1, 2, 3, 4, 5, 6]
prawdopodobienstwa = [1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6]

r = 2

moment = 0

for i in range(len(wartosci)):
    potega = 1

    for j in range(r):
        potega = potega * wartosci[i]

    moment = moment + potega * prawdopodobienstwa[i]

print("Moment rzędu", r, "=", moment)
```

7. Wartość oczekiwana

Wartość oczekiwana jest pierwszym momentem rozkładu.

Oznacza się ją jako:

$$E(X) = \mu$$

Dla zmiennej losowej dyskretnej:

$$E(X) = \sum_{i=1}^n x_i p(x_i)$$

Wartość oczekiwana jest średnią teoretyczną zmiennej losowej.

Przykład — rzut kostką

Dla uczciwej kostki:

$$E(X) = 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6}$$

czyli:

$$E(X) = 3.5$$

Przykład w Pythonie

```
wartosci = [1, 2, 3, 4, 5, 6]
prawdopodobienstwa = [1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6]

wartosc_oczekiwana = 0

for i in range(len(wartosci)):
    wartosc_oczekiwana = wartosc_oczekiwana + wartosci[i] * prawdopodobienstwa[i]

print("E(X) =", wartosc_oczekiwana)
```

PYTHON

8. Wariancja i odchylenie standardowe

Wariancja mierzy rozproszenie zmiennej losowej wokół średniej.

Dla zmiennej losowej:

$$Var(X) = E[(X - E(X))^2]$$

Można też zapisać:

$$Var(X) = E(X^2) - [E(X)]^2$$

Dla zmiennej dyskretnej:

$$Var(X) = \sum_{i=1}^n (x_i - \mu)^2 p(x_i)$$

Odchylenie standardowe:

$$\sigma = \sqrt{Var(X)}$$

Przykład w Pythonie

PYTHON

```
wartosci = [1, 2, 3, 4, 5, 6]
prawdopodobienstwa = [1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6, 1 / 6]

mu = 0

for i in range(len(wartosci)):
    mu = mu + wartosci[i] * prawdopodobienstwa[i]

wariancja = 0

for i in range(len(wartosci)):
    roznica = wartosci[i] - mu
    wariancja = wariancja + roznica * roznica * prawdopodobienstwa[i]

sigma = wariancja ** 0.5

print("Wartość oczekiwana =", mu)
print("Wariancja =", wariancja)
print("Odchylenie standardowe =", sigma)
```

9. Dystrybuanta

Dystrybuanta zmiennej losowej:

$$X$$

oznaczana jako:

$$F(x)$$

określa prawdopodobieństwo, że zmienna losowa przyjmie wartość nie większą niż:

$$x$$

czyli:

$$F(x) = P(X \leq x)$$

Dla zmiennej dyskretnej:

$$F(x) = \sum_{x_i \leq x} p(x_i)$$

Przykład — rozkład Bernoulliego

W rozkładzie Bernoulliego zmienna losowa przyjmuje:

- wartość **1** z prawdopodobieństwem:

$$p$$

- wartość 0 z prawdopodobieństwem:

$$1 - p$$

Wartość oczekiwana wynosi:

$$p$$

Wariancja wynosi:

$$p(1 - p)$$

Przykład w Pythonie

PYTHON

```
wartosci = [0, 1]
p = 0.3
prawdopodobienstwa = [1 - p, p]

x = 0

F = 0

for i in range(len(wartosci)):
    if wartosci[i] <= x:
        F = F + prawdopodobienstwa[i]

print("F(", x, ") =", F)
```

10. Prawa wielkich liczb

Prawa wielkich liczb opisują, jak wyniki prób losowych zbliżają się do wartości teoretycznych, gdy liczba prób staje się bardzo duża.

Wspólna cecha tych praw dotyczy asymptotycznego zachowania wyrażen typu:

$$\frac{X_1 + X_2 + \dots + X_n - a_n}{b_n}$$

W wykładzie przypomniano:

1. Prawo Wielkich Liczb Bernoulliego.
2. Mocne Prawo Wielkich Liczb Kołmogorowa.

Sens

Im więcej prób wykonujemy, tym średni wynik powinien być bliższy wartości oczekiwanej.

Przykład w Pythonie

PYTHON

```
# Prosta symulacja średniej z wyników.  
# Używamy przygotowanego ciągu wyników zamiast gotowego generatora.  
  
wyniki = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]  
  
suma = 0  
  
for i in range(len(wyniki)):  
    suma = suma + wyniki[i]  
  
srednia = suma / len(wyniki)  
  
print("Średnia =", srednia)
```

10.1. Prawo Wielkich Liczb Bernoulliego

Jeżeli:

$$S_n$$

jest liczbą sukcesów w schemacie Bernoulliego z prawdopodobieństwem sukcesu:

$$p$$

to dla każdego:

$$\epsilon > 0$$

zachodzi:

$$P\left(\left|\frac{S_n}{n} - p\right| \leq \epsilon\right) \rightarrow 1$$

gdy:

$$n \rightarrow \infty$$

Przykłady z wykładu

Dla dużej liczby rzutów uczciwą monetą liczba orłów stabilizuje się wokół:

$$0.5$$

Dla dużej liczby rzutów uczciwą kostką liczba jedynek powinna wynosić około:

$$\frac{1}{6}$$

wszystkich rzutów.

Przykład w Pythonie

PYTHON

```
# Przykład dla ciągu wyników 0/1.  
# 1 oznacza sukces, 0 oznacza porażkę.  
  
wyniki = [1, 0, 1, 1, 0, 1, 0, 1, 1, 0]  
  
sukcesy = 0  
  
for i in range(len(wyniki)):  
    if wyniki[i] == 1:  
        sukcesy = sukcesy + 1  
  
czestosc = sukcesy / len(wyniki)  
  
print("Liczba sukcesów =", sukcesy)  
print("Częstość sukcesów =", czestosc)
```

10.2. Mocne Prawo Wielkich Liczb Kołmogorowa

Jeżeli:

$$X_1, X_2, \dots, X_n$$

są niezależnymi zmiennymi losowymi o jednakowym rozkładzie i wartości oczekiwanej:

$$\mu$$

to średnia wyników z coraz większej liczby prób będzie coraz bliższa wartości oczekiwanej.

Intuicyjnie:

$$\frac{X_1 + X_2 + \dots + X_n}{n} \rightarrow \mu$$

dla:

$$n \rightarrow \infty$$

Znaczenie dla Monte Carlo

Prawa wielkich liczb uzasadniają metodę Monte Carlo, ponieważ pokazują, że średnia z wielu losowych prób zbiega do wartości teoretycznej.

Przykład w Pythonie

PYTHON

```
wyniki = [2, 4, 3, 5, 6, 4, 3, 5]

suma = 0

for i in range(len(wyniki)):
    suma = suma + wyniki[i]
    srednia_czesciowa = suma / (i + 1)
    print("Po", i + 1, "próbach średnia =", srednia_czesciowa)
```

11. Znaczenie praw wielkich liczb w metodach Monte Carlo

Prawa wielkich liczb są teoretycznym uzasadnieniem metod Monte Carlo.

Pokazują, że średnia z wielu prób będzie zbiegać do oczekiwanej wartości teoretycznej.

Dzięki nim można:

- przewidywać błąd,
- kontrolować błąd w symulacjach,
- stosować symulacje do problemów trudnych analitycznie.

Przykład

Jeżeli w metodzie Monte Carlo obliczamy całkę jako średnią wartości funkcji w losowych punktach, to prawa wielkich liczb mówią, że przy dużej liczbie punktów ta średnia powinna zbliżać się do właściwej wartości.

Przykład w Pythonie

PYTHON

```
wartosci_funkcji = [1.0, 1.2, 1.5, 1.7, 2.0]

suma = 0

for i in range(len(wartosci_funkcji)):
    suma = suma + wartosci_funkcji[i]

srednia = suma / len(wartosci_funkcji)

print("Średnia z próbek =", srednia)
```

12. Tradycyjne metody obliczania pola powierzchni

Tradycyjne metody obliczania pola opierają się na całkach.

Dla funkcji:

$$f(x)$$

pole pod wykresem na przedziale:

$$[a, b]$$

można zapisać jako:

$$I = \int_a^b f(x), dx$$

Problem pojawia się wtedy, gdy funkcja jest skomplikowana i obliczenie całki jest trudne lub niewykonalne w rozsądnym czasie.

Pytanie z wykładu

Czy istnieje sposób umożliwiający stosunkowo dokładne określenie pola dowolnego obszaru na płaszczyźnie w rozsądnym czasie, z minimalnym błędem?

Odpowiedzią jest metoda Monte Carlo.

Przykład w Pythonie

PYTHON

Przykład funkcji, której pole pod wykresem chcemy obliczyć tradycyjnie, czyli przez całkę.

```
def f(x):  
    return x * x + 1
```

Przedział całkowania

a = 1

b = 2

print("Funkcja: f(x) = x^2 + 1")

print("Przedział: [", a, ",", b, "]")

Dla funkcji f(x) = x^2 + 1 funkcja pierwotna ma postać:

F(x) = x^3 / 3 + x

```
def F(x):  
    return x**3 / 3 + x
```

Pole pod wykresem liczymy ze wzoru:

I = F(b) - F(a)

pole = F(b) - F(a)

print("Pole pod wykresem:")

print("I =", pole)

Wynik:

Funkcja: f(x) = x^2 + 1

Przedział: [1 , 2]

Pole pod wykresem: I = 3.3333333333333335

13. Algorytm Monte Carlo do obliczania pola

Algorytm obliczania pola metodą Monte Carlo:

1. Ograniczamy badany obszar do obszaru o znanym polu.
2. Losujemy niezależne próbki z ograniczonego obszaru.
3. Zliczamy próbki znajdujące się wewnątrz nieznanego obszaru.
4. Obliczamy stosunek liczby próbek wewnątrz obszaru do liczby wszystkich próbek.

5. Mnożymy ten stosunek przez pole obszaru ograniczającego.

Jeżeli:

$$k$$

oznacza liczbę punktów trafionych w badany obszar, a:

$$N$$

liczbę wszystkich punktów, to:

$$P_{obszaru} \approx \frac{k}{N} P_{ograniczający}$$

Przykład w Pythonie

PYTHON

```
# Prosty przykład:  
# mamy 10 prób, z czego 4 punkty trafiły do obszaru.  
# Pole prostokąta ograniczającego wynosi 20.  
  
N = 10  
k = 4  
pole_ograniczajace = 20  
  
pole = (k / N) * pole_ograniczajace  
  
print("Przybliżone pole =", pole)
```

14. Przykład — obliczanie liczby π

Rozważamy koło o promieniu:

$$r = 1$$

wpisane w kwadrat o boku długości:

$$2$$

Pole koła:

$$P_{kola} = \pi r^2 = \pi$$

Pole kwadratu:

$$P_{kwadratu} = 2 \cdot 2 = 4$$

Stosunek pól:

$$\frac{P_{kola}}{P_{kwadratu}} = \frac{\pi}{4}$$

Jeżeli losujemy punkty równomiernie w kwadracie, to:

$$\frac{\text{liczba punktów w kole}}{\text{liczba wszystkich punktów}} \approx \frac{\pi}{4}$$

Stąd:

$$\pi \approx 4 \cdot \frac{\text{liczba punktów w kole}}{\text{liczba wszystkich punktów}}$$

Przykład w Pythonie

PYTHON

```
# Obliczanie pi metodą Monte Carlo.
# Używamy prostego LCG zamiast gotowego random.random().

def nastepna_liczba(X): # funkcja generuje kolejną liczbę pseudolosową na
    podstawie poprzedniej wartości X
    a = 1103515245 # mnożnik generatora LCG
    c = 12345 # przyrost generatora LCG
    m = 2 ** 31 # moduł generatora LCG
    return (a * X + c) % m # zwracamy kolejną wartość według wzoru  $X_{n+1} = (a \cdot X_n + c) \bmod m$ 

def losuj_0_1(X): # funkcja zwraca nowy stan generatora oraz liczbę z przedziału
    [0, 1)
    X = nastepna_liczba(X) # generujemy kolejną liczbę pseudolosową
    r = X / (2 ** 31) # skalujemy liczbę do przedziału [0, 1)
    return X, r # zwracamy nowy stan generatora oraz przeskalowaną liczbę

N = 10000 # liczba losowanych punktów
X = 7 # wartość początkowa generatora, czyli ziarno

trafienia = 0 # licznik punktów, które trafiły do koła jednostkowego

for i in range(N): # wykonujemy N prób losowania punktów
    X, rx = losuj_0_1(X) # losujemy pierwszą współrzędną z przedziału [0, 1)
    X, ry = losuj_0_1(X) # losujemy drugą współrzędną z przedziału [0, 1)

    # Przeskalowanie z [0,1) do [-1,1)
    x = -1 + 2 * rx # przekształcamy rx na współrzędną x z przedziału [-1, 1)
    y = -1 + 2 * ry # przekształcamy ry na współrzędną y z przedziału [-1, 1)

    if x * x + y * y <= 1: # sprawdzamy, czy punkt (x, y) leży wewnątrz koła
        jednostkowego
        trafienia = trafienia + 1 # jeśli punkt leży w kole, zwiększamy licznik
        trafień

pi_przyblizone = 4 * trafienia / N # obliczamy przybliżenie pi ze stosunku pól
koła i kwadratu

print("Liczba trafień =", trafienia) # wypisujemy liczbę punktów, które trafiły do
koła
print("Przybliżenie pi =", pi_przyblizone) # wypisujemy otrzymane przybliżenie
liczby pi
```

15. Metoda Crude Monte Carlo

Crude Monte Carlo to najprostsza wersja całkowania metodą Monte Carlo.

Chcemy obliczyć całkę:

$$I = \int_a^b f(x) dx$$

Losujemy:

$$N$$

punktów:

$$x_1, x_2, \dots, x_N \sim U(a, b)$$

Przybliżenie całki:

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

Idea

Średnia z losowych próbek przybliża wartość oczekiwaną.

Im więcej punktów, tym zwykle lepsze przybliżenie.

Metoda działa również dla dużej liczby wymiarów.

Przykład w Pythonie

PYTHON

```
# Crude Monte Carlo dla całki z  $f(x)=x^2$  na przedziale  $[1,2]$ .
# Dokładna wartość z wykładu to  $7/3 = 2.333333...$ 

def nastepna_liczba(X): # funkcja generuje następną liczbę pseudolosową metodą LCG
    a = 1103515245 # mnożnik generatora LCG
    c = 12345 # przyrost generatora LCG
    m = 2 ** 31 # moduł generatora LCG
    return (a * X + c) % m # zwracamy następną wartość ze wzoru  $X_{n+1} = (a \cdot X_n + c) \bmod m$ 

def losuj_0_1(X): # funkcja zwraca nowy stan generatora oraz liczbę z przedziału  $[0,1]$ 
    X = nastepna_liczba(X) # generujemy kolejną liczbę pseudolosową
    r = X / (2 ** 31) # skalujemy wynik do przedziału  $[0,1]$ 
    return X, r # zwracamy nowy stan generatora oraz wylosowaną wartość r

def f(x): # definiujemy funkcję podcałkową
    return x * x # zwracamy wartość  $f(x)=x^2$ 

a = 1.0 # początek przedziału całkowania
b = 2.0 # koniec przedziału całkowania
N = 300 # liczba losowanych punktów

X = 5 # wartość początkowa generatora, czyli ziarno

suma = 0.0 # zmienna przechowująca sumę wartości funkcji w losowych punktach

for i in range(N): # wykonujemy N losowań
    X, r = losuj_0_1(X) # losujemy liczbę r z przedziału  $[0,1]$ 

    x = a + (b - a) * r # przeskalowujemy r z  $[0,1]$  na punkt x z przedziału  $[a,b]$ 
    suma = suma + f(x) # dodajemy wartość funkcji w wylosowanym punkcie do sumy

I = ((b - a) / N) * suma # obliczamy przybliżenie całki metodą Crude Monte Carlo

print("Przybliżona całka =", I) # wypisujemy wartość przybliżoną
print("Wartość dokładna =", 7 / 3) # wypisujemy wartość dokładną całki z  $x^2$  na  $[1,2]$ 
print("Błąd =", abs(I - 7 / 3)) # wypisujemy błąd bezwzględny
```

16. Dlaczego metoda Crude Monte Carlo działa?

Dla zmiennej losowej:

$$X \sim U(a, b)$$

wartość oczekiwana funkcji:

$$f(X)$$

wynosi:

$$E[f(X)] = \frac{1}{b-a} \int_a^b f(x) dx$$

Stąd:

$$\int_a^b f(x) dx = (b-a)E[f(X)]$$

Z prawa wielkich liczb:

$$\frac{1}{N} \sum_{i=1}^N f(x_i) \rightarrow E[f(X)]$$

gdy:

$$N \rightarrow \infty$$

Dlatego Monte Carlo sprowadza całkowanie do obliczania średniej.

Przykład w Pythonie

PYTHON

```
# Pokazujemy samą ideę:
# najpierw liczymy średnią z wartości f(x_i),
# potem mnożymy przez długość przedziału.

wartosci_f = [1.2, 1.5, 2.0, 3.1]

suma = 0.0

for i in range(len(wartosci_f)):
    suma = suma + wartosci_f[i]

srednia = suma / len(wartosci_f)

a = 1
b = 2

calka = (b - a) * srednia

print("Średnia =", srednia)
print("Przybliżona całka =", calka)
```

17. Dokładność metody Monte Carlo

Błąd średni metody Monte Carlo maleje jak:

$$O\left(\frac{1}{\sqrt{N}}\right)$$

gdzie:

$$N$$

jest liczbą próbek.

Wniosek

Aby zmniejszyć błąd 10 razy, trzeba zwiększyć liczbę próbek 100 razy.

Metoda Monte Carlo jest prosta, ale jej zbieżność jest stosunkowo powolna.

Mimo to metoda jest popularna dla problemów wielowymiarowych.

Przykład w Pythonie

```
# Porównanie liczby próbek potrzebnej do zmniejszenia błędu.
```

PYTHON

```
N1 = 100  
N2 = 10000
```

```
bład1 = 1 / (N1 ** 0.5)  
bład2 = 1 / (N2 ** 0.5)
```

```
print("Dla N =", N1, "błąd proporcjonalny do", bład1)  
print("Dla N =", N2, "błąd proporcjonalny do", bład2)  
print("Stosunek błędów =", bład1 / bład2)
```

18. Monte Carlo a metody klasyczne

W wykładzie porównano metody klasyczne i Monte Carlo.

Cecha	Metody klasyczne	Monte Carlo
1 wymiar	bardzo dokładne	słabsze
wiele wymiarów	bardzo kosztowne	działa dobrze
deterministyczność	tak	nie
łatwość implementacji	średnia	bardzo duża
równoległość	ograniczona	bardzo dobra

Monte Carlo jest szczególnie użyteczne w:

- grafice komputerowej,
 - AI,
 - uczeniu maszynowym,
 - symulacjach fizycznych,
 - analizie ryzyka.
-

19. Metoda akceptacji–odrzućenia

Metoda akceptacji–odrzućenia jest jedną z podstawowych metod Monte Carlo.

Idea:

1. Losujemy punkty z prostego obszaru.
2. Sprawdzamy, które punkty spełniają warunek.
3. Na podstawie proporcji punktów szacujemy pole lub całkę.

Dla całki:

$$I = \int_a^b f(x) dx$$

losujemy punkty z prostokąta:

$$[a, b] \times [0, M]$$

gdzie:

$$f(x) \leq M$$

dla każdego:

$$x \in [a, b]$$

Przykład w Pythonie

PYTHON

```
# Metoda akceptacji-odrzućenia dla całki z funkcji  $f(x)=x^2$  na przedziale  $[0,1]$ .
# Losujemy punkty z prostokąta  $[a,b] \times [0,M]$ .
# Jeżeli punkt leży pod wykresem funkcji, to go akceptujemy.

def nastepna_liczba(X): # funkcja generuje następną liczbę pseudolosową metodą LCG
    a = 1103515245 # mnożnik generatora LCG
    c = 12345 # przyrost generatora LCG
    m = 2 ** 31 # moduł generatora LCG
    return (a * X + c) % m # zwracamy kolejną wartość ze wzoru  $X_{n+1} = (a \cdot X_n + c) \bmod m$ 

def losuj_0_1(X): # funkcja zwraca nowy stan generatora oraz liczbę z przedziału  $[0,1]$ 
    X = nastepna_liczba(X) # generujemy kolejną liczbę pseudolosową
    r = X / (2 ** 31) # skalujemy liczbę do przedziału  $[0,1]$ 
    return X, r # zwracamy nowy stan generatora i wylosowaną liczbę

def f(x): # definiujemy funkcję podcałkową
    return x * x # zwracamy wartość  $f(x)=x^2$ 

a = 0.0 # początek przedziału całkowania
b = 1.0 # koniec przedziału całkowania
M = 1.0 # górne ograniczenie funkcji na przedziale  $[0,1]$ 
N = 10000 # liczba losowanych punktów

X = 7 # ziarno generatora pseudolosowego

trafienia = 0 # licznik punktów zaakceptowanych, czyli leżących pod wykresem funkcji

for i in range(N): # wykonujemy N losowań punktów
    X, rx = losuj_0_1(X) # losujemy liczbę do współrzędnej x
    X, ry = losuj_0_1(X) # losujemy liczbę do współrzędnej y

    x = a + (b - a) * rx # przeskalowujemy rx z  $[0,1]$  na przedział  $[a,b]$ 
    y = M * ry # przeskalowujemy ry z  $[0,1]$  na przedział  $[0,M]$ 

    if y <= f(x): # sprawdzamy, czy punkt (x,y) leży pod wykresem funkcji
        trafienia = trafienia + 1 # jeśli tak, zwiększamy liczbę zaakceptowanych punktów

pole_prostokata = (b - a) * M # pole prostokąta, z którego losujemy punkty

calka = pole_prostokata * trafienia / N # przybliżenie całki jako proporcja trafień razy pole prostokąta
```

```
print("Liczba wszystkich punktów =", N) # wypisujemy liczbę wszystkich prób
print("Liczba zaakceptowanych punktów =", trafienia) # wypisujemy liczbę punktów
pod wykresem
print("Przybliżona całka =", calka) # wypisujemy przybliżoną wartość całki
print("Wartość dokładna =", 1 / 3) # wypisujemy dokładną wartość całki
print("Błąd =", abs(calka - 1 / 3)) # wypisujemy błąd bezwzględny
```

19.1. Idea geometryczna metody akceptacji–odrzućenia

Pole prostokąta:

$$P_{prost} = (b - a)M$$

Losujemy:

$$N$$

punktów równomiernie w prostokącie.

Niech:

$$k$$

oznacza liczbę punktów pod wykresem funkcji:

$$y = f(x)$$

Wtedy:

$$\frac{k}{N} \approx \frac{\text{pole pod wykresem}}{\text{pole prostokąta}}$$

czyli:

$$I \approx \frac{k}{N}(b - a)M$$

```
# Przykład geometrycznej idei metody akceptacji-odrzućenia.
# Zakładamy, że wylosowano N punktów w prostokącie
# i że k z nich znalazło się pod wykresem funkcji.

N = 100 # liczba wszystkich wylosowanych punktów
k = 23  # liczba punktów, które znalazły się pod wykresem funkcji

a = 0 # początek przedziału całkowania
b = 1 # koniec przedziału całkowania
M = 1 # wysokość prostokąta, czyli górne ograniczenie funkcji

pole_prostokata = (b - a) * M # obliczamy pole prostokąta, z którego losowano
punkty

proporcja = k / N # obliczamy, jaka część punktów trafiła pod wykres funkcji

I = proporcja * pole_prostokata # przybliżamy pole pod wykresem, czyli wartość
całki

print("Pole prostokąta =", pole_prostokata) # wypisujemy pole całego prostokąta
print("Proporcja trafień =", proporcja) # wypisujemy stosunek punktów pod wykresem
do wszystkich punktów
print("Przybliżona całka =", I) # wypisujemy przybliżoną wartość całki
```

19.2. Algorytm akceptacji–odrzućenia

1. Losujemy:

$$x \sim U(a, b)$$

2. Losujemy:

$$y \sim U(0, M)$$

3. Jeżeli:

$$y \leq f(x)$$

to punkt akceptujemy.

4. W przeciwnym przypadku punkt odrzucamy.

Po wykonaniu:

$$N$$

losowań:

$$I \approx \frac{k}{N}(b - a)M$$

gdzie:

- k
— liczba zaakceptowanych punktów,
- N
— liczba wszystkich punktów.

Przykład w Pythonie

PYTHON

```
# Metoda akceptacji-odrzućenia dla całki z  $x^5$  na  $[0,1]$ .

def nastepna_liczba(X): # funkcja generuje następną liczbę pseudolosową metodą LCG
    a = 1103515245 # mnożnik generatora LCG
    c = 12345 # przyrost generatora LCG
    m = 2 ** 31 # moduł generatora LCG
    return (a * X + c) % m # zwracamy nową wartość ze wzoru  $X_{n+1} = (a * X_n + c) \bmod m$ 

def losuj_0_1(X): # funkcja zwraca nowy stan generatora oraz liczbę z przedziału [0,1)
    X = nastepna_liczba(X) # generujemy następną liczbę pseudolosową
    r = X / (2 ** 31) # skalujemy wynik do przedziału [0,1)
    return X, r # zwracamy nowy stan generatora i wylosowaną liczbę

def f(x): # definiujemy funkcję podcałkową
    return x * x * x * x * x # zwracamy wartość  $f(x)=x^5$ 

a = 0.0 # początek przedziału całkowania
b = 1.0 # koniec przedziału całkowania
M = 1.0 # maksymalna wysokość prostokąta, bo  $x^5 \leq 1$  na  $[0,1]$ 
N = 10000 # liczba losowanych punktów

X = 11 # wartość początkowa generatora, czyli ziarno
k = 0 # licznik punktów zaakceptowanych, czyli leżących pod wykresem funkcji

for i in range(N): # wykonujemy N prób losowania punktów
    X, rx = losuj_0_1(X) # losujemy liczbę do wyznaczenia współrzędnej x
    X, ry = losuj_0_1(X) # losujemy liczbę do wyznaczenia współrzędnej y

    x = a + (b - a) * rx # skalujemy rx z [0,1) na przedział [a,b]
    y = M * ry # skalujemy ry z [0,1) na przedział [0,M]

    if y <= f(x): # sprawdzamy, czy punkt (x,y) znajduje się pod wykresem funkcji
        k = k + 1 # jeśli punkt jest pod wykresem, zwiększamy licznik zaakceptowanych punktów

I = (k / N) * (b - a) * M # przybliżamy całkę jako proporcję trafień pomnożoną przez pole prostokąta

print("Liczba zaakceptowanych punktów =", k) # wypisujemy liczbę punktów pod wykresem
print("Przybliżona całka =", I) # wypisujemy przybliżoną wartość całki
print("Wartość dokładna =", 1 / 6) # wypisujemy dokładną wartość całki
print("Błąd =", abs(I - 1 / 6)) # wypisanie błędu
```

20. Przykład metody akceptacji–odrzućenia

Rozważamy całkę:

$$I = \int_0^1 x^5 dx$$

Wartość dokładna:

$$I = \frac{1}{6} \approx 0.1667$$

Losujemy punkty z kwadratu:

$$[0, 1] \times [0, 1]$$

Punkt akceptujemy, gdy:

$$y \leq x^5$$

Ponieważ:

$$(b - a)M = 1$$

to przybliżenie wynosi:

$$I \approx \frac{k}{N}$$

W przykładzie z wykładu dla:

$$N = 10000$$

otrzymano:

$$I_{approx} = 0.1669$$

a wartość dokładna to:

$$I_{exact} = \frac{1}{6} \approx 0.1667$$

Przykład w Pythonie

PYTHON

```
# Ten sam przykład, ale z mniejszą liczbą próbek dla krótszego działania.

def f(x): # definiujemy funkcję podcałkową
    return x * x * x * x * x # zwracamy wartość  $f(x)=x^5$ 

punkty = [ # lista ręcznie wybranych punktów w prostokącie  $[0,1] \times [0,1]$ 
    [0.1, 0.2], # pierwszy punkt:  $x=0.1, y=0.2$ 
    [0.5, 0.01], # drugi punkt:  $x=0.5, y=0.01$ 
    [0.8, 0.3], # trzeci punkt:  $x=0.8, y=0.3$ 
    [0.9, 0.7] # czwarty punkt:  $x=0.9, y=0.7$ 
]

zaakceptowane = 0 # licznik punktów, które znajdują się pod wykresem funkcji

for punkt in punkty: # przechodzimy po wszystkich punktach z listy
    x = punkt[0] # pobieramy współrzędną x punktu
    y = punkt[1] # pobieramy współrzędną y punktu

    if y <= f(x): # sprawdzamy, czy punkt leży pod wykresem funkcji  $y=f(x)$ 
        zaakceptowane = zaakceptowane + 1 # jeśli tak, zwiększamy licznik
zaakceptowanych punktów
    print("Punkt", punkt, "zaakceptowany") # wypisujemy informację, że punkt
został zaakceptowany
    else: # jeśli punkt leży nad wykresem funkcji
        print("Punkt", punkt, "odrzucony") # wypisujemy informację, że punkt
został odrzucony

I = zaakceptowane / len(punkty) # przybliżamy całkę jako stosunek punktów
zaakceptowanych do wszystkich punktów

print("Przybliżenie całki =", I) # wypisujemy przybliżoną wartość całki
```

21. Efektywność metody akceptacji–odrzućenia

Metoda działa najlepiej, gdy prostokąt dobrze dopasowuje się do wykresu funkcji.

Jeżeli prostokąt jest znacznie większy od pola pod wykresem, większość punktów będzie odrzucana.

Wtedy:

- metoda staje się wolniejsza,
- potrzeba więcej losowań,
- efektywność spada.

Dlatego ważne jest:

- dobranie możliwie małego:

$$M$$

- ograniczenie liczby odrzucanych punktów.

Przykład w Pythonie

PYTHON

```
N = 10000 # liczba wszystkich wylosowanych punktów

zaakceptowane = 2000 # liczba punktów, które znalazły się pod wykresem funkcji

udzial = zaakceptowane / N # obliczamy udział zaakceptowanych punktów

procent = udzial * 100 # zamieniamy udział na procenty

print("Udział zaakceptowanych punktów =", udzial) # wypisujemy udział, np. 0.2
print("Procent zaakceptowanych punktów =", procent, "%") # wypisujemy procent,
np. 20%

if udzial < 0.2: # sprawdzamy, czy zaakceptowano mniej niż 20% punktów
    print("Metoda może być mało efektywna") # dużo punktów zostało odrzuconych
else:
    print("Efektywność jest lepsza") # zaakceptowano wystarczająco dużo punktów
```

22. Model Isinga

Model Isinga to matematyczny model statystyczny stosowany w fizyce, szczególnie w teorii faz magnetycznych.

Model opisuje zachowanie spinów w sieci krystalicznej.

Każdy spin może przyjąć jedną z dwóch wartości:

$$+1$$

albo:

$$-1$$

Model Isinga jest prosty, ale pozwala badać bogate zachowanie fazowe.

Przykład w Pythonie

PYTHON

```
# Przykładowa sieć spinów jednowymiarowa.  
  
spiny = [1, -1, 1, 1, -1]  
  
for i in range(len(spiny)):  
    print("Spin", i, "=", spiny[i])
```

22.1. Definicja modelu Isinga

Rozważamy sieć o:

$$N$$

spinach.

Każdy spin:

$$s_i$$

dla:

$$i = 1, 2, \dots, N$$

przyjmuje wartości:

$$\pm 1$$

Energia układu jest opisana przez Hamiltonian:

$$H = -J \sum_{\langle i, j \rangle} s_i s_j - h \sum_i s_i$$

gdzie:

- J
— stała sprzężenia między najbliższymi sąsiadami,
- h
— zewnętrzne pole magnetyczne,
- $\langle i, j \rangle$
— sumowanie po najbliższych sąsiadach.

Przykład w Pythonie

PYTHON

```
# Energia prostego jednowymiarowego modelu Isinga bez pola magnetycznego.

spiny = [1, -1, 1, 1]
J = 1
h = 0

energia = 0

for i in range(len(spiny) - 1):
    energia = energia - J * spiny[i] * spiny[i + 1]

suma_spinow = 0

for i in range(len(spiny)):
    suma_spinow = suma_spinow + spiny[i]

energia = energia - h * suma_spinow

print("Energia =", energia)
```

22.2. Symulacje Monte Carlo w modelu Isinga

Metoda Monte Carlo jest stosowana do badania modelu Isinga, szczególnie przy analizie przejść fazowych.

Wykorzystuje się między innymi algorytm Metropolis'a.

Algorytm Metropolis'a

1. Losowo wybierz spin.
2. Oblicz zmianę energii:

$$\Delta E$$

po odwróceniu spinu.

3. Jeżeli:

$$\Delta E \leq 0$$

odwróć spin.

4. W przeciwnym razie odwróć spin z prawdopodobieństwem:

$$e^{-\Delta E/kT}$$

gdzie:

- k
— stała Boltzmannna,
- T
— temperatura.

Przykład w Pythonie

PYTHON

```
import math
import random

# Uproszczony przykład decyzji w algorytmie Metropolis'a.
# Zakładamy, że znamy zmianę energii DeltaE.

DeltaE = 2.0 # zmiana energii po proponowanym odwróceniu spinu
k = 1.0      # dla uproszczenia przyjmujemy k = 1
T = 3.0      # temperatura układu

if DeltaE <= 0:
    # Jeśli energia maleje albo się nie zmienia,
    # to zmianę zawsze akceptujemy.
    print("Akceptujemy zmianę, bo energia nie rośnie")

else:
    # Jeśli energia rośnie, obliczamy prawdopodobieństwo akceptacji.
    P = math.exp(-DeltaE / (k * T))

    # Losujemy liczbę z przedziału [0,1).
    r = random.random()

    print("Prawdopodobieństwo akceptacji =", P)
    print("Wylosowana liczba =", r)

    if r < P:
        print("Akceptujemy zmianę mimo wzrostu energii")
    else:
        print("Odrzucamy zmianę")
```

Najważniejszy warunek

W kodzie decyzja sprowadza się do dwóch przypadków:

PYTHON

```
if DeltaE <= 0:
    akceptujemy zmianę
else:
    akceptujemy zmianę tylko z prawdopodobieństwem exp(-DeltaE / (k*T))
```

Czyli algorytm Metropolis'a pozwala czasem przyjmować gorsze stany, aby symulacja mogła lepiej badać przestrzeń możliwych konfiguracji.

23. Zastosowania modelu Isinga

Model Isinga może być używany do:

- analizy magnetyzmu w materiałach,
- badania zjawisk krytycznych,
- badania przejść fazowych.

Ze względu na prostotę i możliwość skalowania znajduje zastosowanie także w:

- informatyce,
 - biologii,
 - ekonomii,
 - analizie systemów decyzyjnych,
 - sieciach neuronowych.
-

24. Symulowane wyżarzanie

Symulowane wyżarzanie to technika optymalizacji, która naśladuje proces wyżarzania w metalurgii.

Metoda jest użyteczna do znajdowania minimum globalnego funkcji kosztu, szczególnie wtedy, gdy przestrzeń rozwiązań jest duża lub skomplikowana.

Idea

Algorytm zaczyna od wysokiej temperatury.

Potem temperatura stopniowo maleje.

Wysoka temperatura pozwala akceptować czasem gorsze rozwiązania, co pomaga uniknąć utknięcia w minimum lokalnym.

Przykład w Pythonie

PYTHON

```
temperatura = 1000

for i in range(5):
    print("Iteracja:", i, "temperatura =", temperatura)
    temperatura = temperatura * 0.5
```

24.1. Podstawy symulowanego wyżarzania

W każdym kroku algorytm próbuje zastąpić aktualne rozwiązanie nowym rozwiązaniem z sąsiedztwa.

Nowe rozwiązanie może być nawet gorsze.

Prawdopodobieństwo akceptacji gorszego rozwiązania zależy od różnicy kosztów i temperatury:

$$P(\Delta E) = e^{-\Delta E/T}$$

gdzie:

- ΔE
— różnica kosztów,
- T
— aktualna temperatura.

Przykład w Pythonie

PYTHON

```
# Uproszczona logika:
# jeśli nowe rozwiązanie jest lepsze, akceptujemy je od razu.

koszt_stary = 10
koszt_nowy = 7

DeltaE = koszt_nowy - koszt_stary

if DeltaE < 0:
    print("Nowe rozwiązanie jest lepsze, akceptujemy")
else:
    print("Nowe rozwiązanie jest gorsze, akceptacja zależy od temperatury")
```

24.2. Algorytm symulowanego wyżarzania

1. Ustal początkowe rozwiązanie i początkową temperaturę.
2. Losowo wybierz nowe rozwiązanie z sąsiedztwa aktualnego rozwiązania.
3. Oblicz zmianę funkcji kosztu:

$$\Delta E$$

4. Jeżeli:

$$\Delta E < 0$$

zaakceptuj nowe rozwiązanie.

5. Jeżeli:

$$\Delta E \geq 0$$

zaakceptuj nowe rozwiązanie z prawdopodobieństwem:

$$e^{-\Delta E/T}$$

6. Zmniejsz temperaturę i powtórz proces.
7. Zakończ, gdy temperatura osiągnie minimalny poziom albo gdy wykonano określoną liczbę iteracji.

Przykład w Pythonie

PYTHON

```
# Prosty schemat bez losowania prawdopodobieństwa.  
# Pokazuje najważniejsze zmienne algorytmu.
```

```
x = 10  
T = 1000  
T_min = 1  
  
def f(x):  
    return x * x - 4 * x + 4  
  
while T > T_min:  
    x_nowy = x - 1  
  
    DeltaE = f(x_nowy) - f(x)  
  
    if DeltaE < 0:  
        x = x_nowy  
  
    print("x =", x, "f(x) =", f(x), "T =", T)  
  
    T = T * 0.5
```

24.3. Przykład symulowanego wyżarzania

Z wykładu:

Minimalizujemy funkcję:

$$f(x) = x^2 - 4x + 4$$

Zaczynamy od punktu:

$$x = 10$$

oraz temperatury:

$$T = 1000$$

Po kolejnych iteracjach i obniżaniu temperatury algorytm powinien osiągnąć minimum funkcji blisko:

$$x = 2$$

Jest to minimum globalne, ponieważ:

$$f(x) = x^2 - 4x + 4 = (x - 2)^2$$

Przykład w Pythonie

PYTHON

```
import math # importujemy math, żeby użyć funkcji exp()
import random # importujemy random, żeby losować nowe rozwiązania i decyzję
akceptacji

# Szukamy minimum funkcji  $f(x) = x^2 - 4x + 4$ 
# Ta funkcja ma minimum dla  $x = 2$ 

def f(x): # definiujemy funkcję kosztu
    return x * x - 4 * x + 4 # zwracamy wartość funkcji f(x)

x = 10 # początkowe rozwiązanie
T = 1000.0 # początkowa temperatura
T_min = 1.0 # minimalna temperatura, przy której kończymy algorytm
alfa = 0.5 # współczynnik chłodzenia, czyli tempo zmniejszania temperatury

najlepsze_x = x # zapamiętujemy najlepsze znalezione rozwiązanie
najlepsza_wartosc = f(x) # zapamiętujemy wartość funkcji dla najlepszego
rozwiązania

while T > T_min: # wykonujemy algorytm, dopóki temperatura jest większa od
minimalnej
    krok = random.choice([-1, 1]) # losujemy kierunek zmiany: w lewo albo w prawo

    x_nowy = x + krok # tworzymy nowe rozwiązanie z sąsiedztwa aktualnego
rozwiązania

    DeltaE = f(x_nowy) - f(x) # obliczamy zmianę funkcji kosztu

    if DeltaE < 0: # jeśli nowe rozwiązanie jest lepsze
        x = x_nowy # akceptujemy nowe rozwiązanie

    else: # jeśli nowe rozwiązanie jest gorsze albo takie samo
        prawdopodobienstwo = math.exp(-DeltaE / T) # obliczamy prawdopodobieństwo
akceptacji

        r = random.random() # losujemy liczbę z przedziału [0, 1)

        if r < prawdopodobienstwo: # jeśli wylosowana liczba jest mniejsza od
prawdopodobieństwa
            x = x_nowy # akceptujemy gorsze rozwiązanie

        if f(x) < najlepsza_wartosc: # sprawdzamy, czy aktualne rozwiązanie jest
najlepsze do tej pory
            najlepsze_x = x # zapisujemy najlepsze x
            najlepsza_wartosc = f(x) # zapisujemy najlepszą wartość funkcji

    print("x =", x, "f(x) =", f(x), "T =", T) # wypisujemy aktualny stan
algorytmu
```

```
T = T * alfa # zmniejszamy temperaturę
```

```
print("Najlepsze znalezione rozwiązanie:")  
print("x =", najlepsze_x)  
print("f(x) =", najlepsza_wartosc)
```

inny przykład:

```
def f(x):  
    return x * x - 4 * x + 4  
  
punkty = [10, 8, 6, 4, 2]  
  
for x in punkty:  
    print("x =", x, "f(x) =", f(x))
```

PYTHON

25. Powiązania między symulowanym wyżarzaniem a metodami Monte Carlo

Symulowane wyżarzanie jest powiązane z metodami Monte Carlo, ponieważ także korzysta z losowości.

Z wykładu najważniejsze powiązania:

1. Obie metody bazują na losowym doborze próbek lub rozwiązań.
2. Symulowane wyżarzanie akceptuje czasem gorsze rozwiązania z pewnym prawdopodobieństwem.
3. Proces chłodzenia przypomina dostosowywanie parametrów w niektórych algorytmach Monte Carlo.
4. Obie metody opierają się na prawdopodobieństwie i statystyce.
5. Obie metody szukają równowagi między eksploracją a eksploatacją.

Przykład w Pythonie

PYTHON

```
# Eksploracja: sprawdzamy nowe rozwiązania.
# Eksploatacja: zostajemy przy dobrym rozwiązaniu.

rozwiązania = [10, 8, 5, 3, 2]

def koszt(x):
    return x * x - 4 * x + 4

najlepsze = rozwiązania[0]

for i in range(1, len(rozwiązania)):
    if koszt(rozwiązania[i]) < koszt(najlepsze):
        najlepsze = rozwiązania[i]

print("Najlepsze znalezione rozwiązanie =", najlepsze)
print("Koszt =", koszt(najlepsze))
```

26. Monte Carlo i sztuczna inteligencja

Współczesna sztuczna inteligencja często operuje na:

- prawdopodobieństwach,
- niepewności,
- wielu możliwych scenariuszach.

Metody Monte Carlo są używane między innymi w:

- Bayesian Machine Learning,
- probabilistycznych modelach AI,
- uczeniu przez wzmacnianie,
- algorytmach MCMC.

Monte Carlo pozwala zastąpić trudne obliczenia losowym próbkowaniem i uśrednianiem wyników.

27. Monte Carlo w statystyce bayesowskiej

W statystyce bayesowskiej interesuje nas rozkład:

$$P(\theta|D)$$

czyli rozkład parametrów:

θ

po zaobserwowaniu danych:

D

Z twierdzenia Bayesa:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

Problem polega na tym, że:

$P(D)$

często jest bardzo trudne do obliczenia.

Monte Carlo może wtedy:

- losować wiele możliwych parametrów,
- sprawdzać zgodność z danymi,
- przybliżać rozkład posterior.

Przykład w Pythonie

PYTHON

```
# Uproszczony przykład:
# sprawdzamy kilka kandydatów parametru theta.

theta = [0.1, 0.3, 0.5, 0.7]
zgodnosc = [0.2, 0.6, 0.9, 0.4]

najlepszy_indeks = 0

for i in range(1, len(theta)):
    if zgodnosc[i] > zgodnosc[najlepszy_indeks]:
        najlepszy_indeks = i

print("Najlepszy kandydat theta =", theta[najlepszy_indeks])
```

inny przykład:

```

# Monte Carlo w statystyce bayesowskiej.
# Szacujemy prawdopodobieństwo wyrzucenia orła dla monety.

import random # importujemy random, żeby losować kandydatów parametru theta

only = 7 # liczba zaobserwowanych orłów
reszki = 3 # liczba zaobserwowanych reszek

N = 10000 # liczba losowanych kandydatów theta

suma_wag = 0.0 # suma wag, czyli suma zgodności kandydatów z danymi
suma_theta_wazona = 0.0 # suma theta pomnożonych przez ich wagi

najlepsze_theta = 0.0 # zmienna przechowująca najlepszego kandydata theta
najlepsza_waga = 0.0 # zmienna przechowująca największą znalezioną wagę

for i in range(N): # wykonujemy N losowań możliwych wartości theta
    theta = random.random() # losujemy theta z przedziału [0,1)

    # Liczymy zgodność theta z danymi.
    # Jeśli mamy 7 orłów i 3 reszki, to prawdopodobieństwo danych wynosi:
    #  $\theta^7 * (1-\theta)^3$ 
    waga = (theta ** only) * ((1 - theta) ** reszki)

    suma_wag = suma_wag + waga # dodajemy wagę do sumy wszystkich wag

    suma_theta_wazona = suma_theta_wazona + theta * waga # dodajemy theta
    pomnożone przez wagę

    if waga > najlepsza_waga: # sprawdzamy, czy aktualna waga jest największa
        najlepsza_waga = waga # zapamiętujemy największą wagę
        najlepsze_theta = theta # zapamiętujemy theta najlepiej pasujące do danych

theta_srednie = suma_theta_wazona / suma_wag # obliczamy przybliżoną średnią
rozkładu posterior

print("Dane:") # wypisujemy nagłówek
print("orły =", only) # wypisujemy liczbę orłów
print("reszki =", reszki) # wypisujemy liczbę reszek

print("Najlepszy kandydat theta =", najlepsze_theta) # wypisujemy theta o
największej zgodności z danymi
print("Średnie theta z rozkładu posterior ≈", theta_srednie) # wypisujemy
przybliżoną średnią posterior

```

28. MCMC — Markov Chain Monte Carlo

MCMC to rodzina algorytmów służących do generowania próbek z trudnych rozkładów prawdopodobieństwa.

MCMC łączy dwie idee:

Markov Chain

Kolejna próbka zależy od poprzedniej.

Tworzy się tak zwany łańcuch Markowa.

Monte Carlo

Wykorzystujemy:

- losowanie,
- decyzje probabilistyczne,
- dużą liczbę próbek.

Idea

Algorytm wykonuje losowy spacer po przestrzeni rozwiązań i częściej odwiedza obszary bardziej prawdopodobne.

Przykład w Pythonie

PYTHON

```
# Prosty przykład MCMC metodą Metropolis'a.
# Chcemy generować próbki z rozkładu podobnego do normalnego:
#  $p(x) \sim \exp(-x^2 / 2)$ 

import math # importujemy math, żeby użyć funkcji exp()
import random # importujemy random, żeby wykonywać losowania

def gestosc(x): # funkcja opisuje rozkład, z którego chcemy próbować
    return math.exp(-x * x / 2) # zwracamy wartość  $\exp(-x^2/2)$ 

stan = 0.0 # początkowy stan łańcucha Markowa

krok = 1.0 # maksymalna wielkość losowej zmiany stanu

liczba_iteracji = 1000 # liczba kroków algorytmu MCMC

probki = [] # lista, w której będziemy zapisywać wygenerowane próbki

for i in range(liczba_iteracji): # wykonujemy kolejne kroki algorytmu
    propozycja = stan + random.uniform(-krok, krok) # losujemy nowy kandydat w
    pobliżu aktualnego stanu

    prawdopodobienstwo_akceptacji = gestosc(propozycja) / gestosc(stan) #
    porównujemy, jak dobry jest nowy stan względem starego

    if prawdopodobienstwo_akceptacji >= 1: # jeśli nowy stan jest bardziej
    prawdopodobny
        stan = propozycja # akceptujemy nowy stan

    else: # jeśli nowy stan jest mniej prawdopodobny
        r = random.random() # losujemy liczbę z przedziału [0,1)

        if r < prawdopodobienstwo_akceptacji: # czasem akceptujemy gorszy stan
            stan = propozycja # akceptujemy propozycję mimo mniejszego
            prawdopodobieństwa

    probki.append(stan) # zapisujemy aktualny stan jako próbkę

print("Pierwsze 20 próbek:") # wypisujemy opis wyniku
print(probki[:20]) # wypisujemy pierwsze 20 wygenerowanych próbek

srednia = sum(probki) / len(probki) # liczymy średnią z próbek

print("Średnia z próbek =", srednia) # wypisujemy średnią
```

28.1. Przykłady algorytmów MCMC

Wykład wymienia:

1. Metropolis–Hastings

- proponujemy nowy losowy punkt,
- akceptujemy go z pewnym prawdopodobieństwem,
- algorytm eksploruje przestrzeń parametrów.

2. Gibbs Sampling

- losujemy parametry pojedynczo,
- upraszcza obliczenia w modelach wielowymiarowych.

3. Particle Filters

- przechowują wiele możliwych hipotez,
- stosowane są np. w robotyce i pojazdach autonomicznych.

29. Monte Carlo i Reinforcement Learning

RL, czyli **Reinforcement Learning**, oznacza uczenie przez wzmocnianie.

Agent:

- wykonuje akcje,
- obserwuje skutki,
- otrzymuje nagrody lub kary,
- uczy się najlepszej strategii.

Monte Carlo w RL:

- symuluje wiele losowych epizodów,
- oblicza średnią nagrodę,
- pomaga ocenić jakość strategii.

Zastosowania:

- gry komputerowe,
- robotyka,
- autonomiczne systemy AI.

Przykład w Pythonie

PYTHON

```
# Monte Carlo w Reinforcement Learning.
# Symulujemy kilka epizodów prostego agenta.
# Agent startuje w punkcie 0 i chce dojść do punktu 5.
# Za dojście do celu dostaje nagrodę +10.
# Za każdy zwykły krok dostaje karę -1.

import random # importujemy random, żeby agent mógł losowo wybierać akcje

def wykonaj_epizod(): # funkcja symuluje jeden epizod
    pozycja = 0 # agent zaczyna w pozycji 0
    cel = 5 # celem agenta jest dojście do pozycji 5
    suma_nagrod = 0 # tutaj zapisujemy łączną nagrodę z epizodu
    maks_krokow = 20 # ograniczamy długość epizodu, żeby nie trwał nieskończenie

    for krok in range(maks_krokow): # wykonujemy kolejne kroki epizodu
        akcja = random.choice([-1, 1]) # agent losuje akcję: -1 oznacza ruch w
        lewo, 1 oznacza ruch w prawo

        pozycja = pozycja + akcja # aktualizujemy pozycję agenta

        if pozycja < 0: # sprawdzamy, czy agent wyszedł poza lewą granicę
            pozycja = 0 # jeśli tak, zatrzymujemy go na pozycji 0

        suma_nagrod = suma_nagrod - 1 # za każdy krok agent dostaje karę -1

        if pozycja == cel: # sprawdzamy, czy agent dotarł do celu
            suma_nagrod = suma_nagrod + 10 # za dojście do celu agent dostaje
            nagrodę +10
            break # kończymy epizod, bo cel został osiągnięty

    return suma_nagrod # zwracamy łączną nagrodę z jednego epizodu

liczba_epizodow = 10 # liczba symulowanych epizodów

nagrody = [] # lista na wyniki kolejnych epizodów

for i in range(liczba_epizodow): # uruchamiamy wiele epizodów
    wynik = wykonaj_epizod() # wykonujemy jeden epizod i zapisujemy jego wynik
    nagrody.append(wynik) # dodajemy wynik epizodu do listy

suma = 0 # zmienna na sumę nagród ze wszystkich epizodów

for i in range(len(nagrody)): # przechodzimy po wszystkich wynikach
    suma = suma + nagrody[i] # dodajemy wynik epizodu do sumy

srednia_nagroda = suma / len(nagrody) # liczymy średnią nagrodę z epizodów
```

```
print("Nagrody z epizodów:", nagrody) # wypisujemy wyniki pojedynczych epizodów
print("Średnia nagroda =", srednia_nagroda) # wypisujemy średnią nagrodę
```

30. Problemy praktyczne Monte Carlo

Najczęstsze problemy metody Monte Carlo:

- słaby generator liczb pseudolosowych,
- zbyt mała liczba próbek,
- wysoka wariancja,
- bardzo wolna zbieżność,
- błędna interpretacja wyników.

W praktyce należy:

- wykonywać wiele niezależnych eksperymentów,
- analizować odchylenie standardowe,
- kontrolować jakość generatora.

Przykład w Pythonie

PYTHON

```
# Przykład analizy kilku wyników eksperymentów Monte Carlo.
# Nie patrzymy tylko na jeden wynik, ale sprawdzamy średnią i odchylenie
standardowe.

wyniki_eksperymentow = [2.1, 2.4, 2.3, 2.2, 2.5] # wyniki kilku niezależnych
eksperymentów Monte Carlo

suma = 0 # zmienna na sumę wyników

for i in range(len(wyniki_eksperymentow)): # przechodzimy po wszystkich wynikach
eksperymentów
    suma = suma + wyniki_eksperymentow[i] # dodajemy aktualny wynik do sumy

srednia = suma / len(wyniki_eksperymentow) # obliczamy średnią wartość wyników

suma_kwadratow = 0 # zmienna na sumę kwadratów odchyłeń od średniej

for i in range(len(wyniki_eksperymentow)): # ponownie przechodzimy po wszystkich
wynikach
    roznica = wyniki_eksperymentow[i] - srednia # liczymy różnicę między wynikiem
a średnią
    suma_kwadratow = suma_kwadratow + roznica * roznica # dodajemy kwadrat tej
różnicy

wariancja = suma_kwadratow / len(wyniki_eksperymentow) # obliczamy wariancję
wyników

odchylenie = wariancja ** 0.5 # pierwiastkujemy wariancję, żeby dostać odchylenie
standardowe

print("Wyniki eksperymentów =", wyniki_eksperymentow) # wypisujemy wszystkie
wyniki
print("Średnia =", srednia) # wypisujemy średni wynik
print("Odchylenie standardowe =", odchylenie) # wypisujemy rozrzut wyników
```

31. Monte Carlo na GPU

Metoda Monte Carlo bardzo dobrze nadaje się do równoległości.

Każda próbka:

- może być liczona niezależnie,
- nie wymaga komunikacji z innymi próbkami.

Dlatego Monte Carlo często wykorzystuje:

- GPU,
- CUDA,
- OpenCL,
- obliczenia rozproszone.

To jest jedna z przyczyn popularności tej metody.

Przykład w Pythonie

PYTHON

```
# Przykład pokazujący, dlaczego Monte Carlo dobrze nadaje się na GPU.
# Każda próbka jest niezależna od pozostałych.
# Tutaj przybliżamy liczbę pi metodą Monte Carlo.

import random # importujemy random, żeby losować punkty

def jedna_probka(): # funkcja wykonuje jedną niezależną próbkę Monte Carlo
    x = random.uniform(-1, 1) # losujemy współrzędną x z przedziału [-1, 1]
    y = random.uniform(-1, 1) # losujemy współrzędną y z przedziału [-1, 1]

    if x * x + y * y <= 1: # sprawdzamy, czy punkt leży w kole jednostkowym
        return 1 # zwracamy 1, jeśli punkt trafił do koła
    else: # jeśli punkt nie leży w kole
        return 0 # zwracamy 0

N = 10000 # liczba wszystkich próbek

trafienia = 0 # licznik punktów, które trafiły do koła

for i in range(N): # wykonujemy N niezależnych próbek
    trafienia = trafienia + jedna_probka() # dodajemy wynik jednej próbki do
    liczby trafień

pi_przyblizone = 4 * trafienia / N # obliczamy przybliżenie pi

print("Liczba próbek =", N) # wypisujemy liczbę próbek
print("Liczba trafień =", trafienia) # wypisujemy liczbę trafień w koło
print("Przybliżenie pi =", pi_przyblizone) # wypisujemy przybliżenie liczby pi
```

32. Dokładność i poprawność wyników Monte Carlo

Dokładność metod Monte Carlo zależy od liczby symulacji.

Większa liczba prób zwykle prowadzi do bardziej precyzyjnych wyników.

Błąd statystyczny jest związany z odchyleniem standardowym i zwykle zmniejsza się jak:

$$\frac{1}{\sqrt{N}}$$

gdzie:

$$N$$

to liczba prób.

Dokładność można poprawić technikami redukcji wariancji, takimi jak:

- próbkowanie ważone,
- stratyfikacja,
- zmienne kontrolne.

W praktyce dokładność jest ograniczona także przez jakość generatora liczb pseudolosowych.

Przykład w Pythonie

```
PYTHON
# Pokazujemy, jak teoretyczny błąd Monte Carlo zmniejsza się wraz z liczbą prób.
# Błąd jest proporcjonalny do 1 / sqrt(N).

liczby_prob = [100, 1000, 10000, 100000] # różne liczby prób N

for N in liczby_prob: # przechodzimy po kolejnych wartościach N
    blad = 1 / (N ** 0.5) # obliczamy wartość proporcjonalną do błędu: 1/sqrt(N)

    print("Dla N =", N) # wypisujemy liczbę prób
    print("Błąd proporcjonalny do =", blad) # wypisujemy oszacowanie zależności
    blad
    print()
```

32.1. Poprawność wyników

Poprawność wyników zależy od:

- założeń modelu,
- dobrania parametrów symulacji,
- poprawnej implementacji,
- zgodności modelu z rzeczywistym problemem.

Trzeba uważać na błędy systematyczne wynikające z niewłaściwego modelowania lub błędów w implementacji.

Poprawność można sprawdzać przez porównanie wyników z:

- innymi metodami numerycznymi,
- danymi eksperymentalnymi.

Przykład w Pythonie

PYTHON

```
# Przykład sprawdzenia poprawności wyniku Monte Carlo.
# Porównujemy wynik Monte Carlo z wynikiem otrzymanym inną metodą.

wynik_monte_carlo = 2.34 # wynik uzyskany metodą Monte Carlo

wynik_innej_metody = 2.33 # wynik uzyskany inną metodą, np. metodą numeryczną albo
analityczną

roznica = abs(wynik_monte_carlo - wynik_innej_metody) # obliczamy różnicę
bezwzględną między wynikami

print("Wynik Monte Carlo =", wynik_monte_carlo) # wypisujemy wynik Monte Carlo
print("Wynik innej metody =", wynik_innej_metody) # wypisujemy wynik porównawczy
print("Różnica wyników =", roznica) # wypisujemy różnicę między wynikami

if roznica < 0.05: # sprawdzamy, czy różnica mieści się w przyjętej tolerancji
    print("Wyniki są podobne") # jeśli różnica jest mała, wynik można uznać za
    zgodny
else: # jeśli różnica jest zbyt duża
    print("Trzeba sprawdzić model lub implementację") # duża różnica może oznaczać
    błąd w modelu albo kodzie
```

33. Zalety metody Monte Carlo

Zalety metody Monte Carlo z wykładu:

1. Umożliwia rozwiązywanie skomplikowanych problemów.
 2. Jest prostą alternatywą dla skomplikowanych rozwiązań analitycznych.
 3. Wzrost mocy obliczeniowej sprzętu komputerowego zwiększa jej efektywność.
 4. Uwalnia użytkownika od konieczności zrozumienia bardzo skomplikowanej teorii matematycznej.
 5. Jest elastyczna i można ją dopasować do różnych problemów.
-

34. Wady metody Monte Carlo

Wady metody Monte Carlo z wykładu:

1. Eksperymenty są ograniczone do skończonej liczby prób.
 2. Wyniki zawsze są przybliżeniami.
 3. Jakość wyników zależy od jakości generatora liczb pseudolosowych.
 4. Rozwiązania mogą być obarczone niepewnością statystyczną.
-

35. Najważniejsze rzeczy do zapamiętania na kolosa

35.1. Definicja Monte Carlo

Metoda Monte Carlo polega na użyciu losowego próbkowania do przybliżonego rozwiązania problemu.

35.2. Ogólny algorytm Monte Carlo

1. Definiujemy przestrzeń danych wejściowych.
 2. Losujemy dane wejściowe.
 3. Wykonujemy obliczenia probabilistyczne.
 4. Agregujemy wyniki.
-

35.3. Zmienna losowa

Zmienna losowa to wynik procesu losowego reprezentowany wartością liczbową, binarną lub symboliczną.

35.4. Rozkład prawdopodobieństwa

Dla zmiennej dyskretnej rozkład prawdopodobieństwa przypisuje każdej możliwej wartości jej prawdopodobieństwo.

35.5. Wartość oczekiwana

$$E(X) = \sum_{i=1}^n x_i p(x_i)$$

35.6. Wariancja

$$Var(X) = E[(X - E(X))^2]$$

oraz:

$$Var(X) = E(X^2) - [E(X)]^2$$

35.7. Dystrybuanta

$$F(x) = P(X \leq x)$$

35.8. Prawo Wielkich Liczb Bernoulliego

$$P\left(\left|\frac{S_n}{n} - p\right| \leq \epsilon\right) \rightarrow 1$$

gdym:

$$n \rightarrow \infty$$

35.9. Znaczenie praw wielkich liczb

Prawa wielkich liczb uzasadniają Monte Carlo, ponieważ średnia z wielu prób zbiega do wartości oczekiwanej.

35.10. Pole metodą Monte Carlo

$$P_{\text{obszaru}} \approx \frac{k}{N} P_{\text{ograniczający}}$$

35.11. Obliczanie liczby π

$$\pi \approx 4 \cdot \frac{\text{liczba punktów w kole}}{\text{liczba wszystkich punktów}}$$

35.12. Crude Monte Carlo

$$I = \int_a^b f(x) dx$$

oraz:

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i)$$

35.13. Dlaczego Crude MC działa?

Dla:

$$X \sim U(a, b)$$

mamy:

$$E[f(X)] = \frac{1}{b-a} \int_a^b f(x) dx$$

czyli:

$$\int_a^b f(x) dx = (b-a)E[f(X)]$$

35.14. Błąd Monte Carlo

Błąd średni maleje jak:

$$O\left(\frac{1}{\sqrt{N}}\right)$$

35.15. Metoda akceptacji–odrzućcenia

Losujemy punkty z prostokąta:

$$[a, b] \times [0, M]$$

Akceptujemy punkt, gdy:

$$y \leq f(x)$$

Wtedy:

$$I \approx \frac{k}{N}(b-a)M$$

35.16. Model Isinga

Hamiltonian modelu Isinga:

$$H = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i$$

35.17. Algorytm Metropolis

Jeżeli:

$$\Delta E \leq 0$$

zmianę akceptujemy.

Jeżeli:

$$\Delta E > 0$$

zmianę akceptujemy z prawdopodobieństwem:

$$e^{-\Delta E/kT}$$

35.18. Symulowane wyżarzanie

Symulowane wyżarzanie naśladuje proces chłodzenia materiału i służy do optymalizacji.

Prawdopodobieństwo akceptacji gorszego rozwiązania:

$$P(\Delta E) = e^{-\Delta E/T}$$

35.19. Monte Carlo w AI

Metody Monte Carlo są używane w:

- statystyce bayesowskiej,
 - MCMC,
 - uczeniu przez wzmacnianie,
 - modelach probabilistycznych.
-

35.20. MCMC

MCMC generuje próbki z trudnych rozkładów prawdopodobieństwa za pomocą łańcucha Markowa i losowania.

35.21. Problemy praktyczne Monte Carlo

Najczęstsze problemy:

- zbyt mała liczba próbek,
 - wysoka wariancja,
 - słaby generator liczb pseudolosowych,
 - wolna zbieżność,
 - błędna interpretacja wyników.
-

35.22. Zalety Monte Carlo

Metoda jest:

- prosta,
 - elastyczna,
 - dobra dla problemów złożonych,
 - dobrze równoległa.
-

35.23. Wady Monte Carlo

Metoda daje wyniki przybliżone i wymaga dużej liczby prób.

Całki - obliczanie

Domyślna funkcja do liczenia całki **numerycznie metodą prostokątów ze środkiem przedziału**:

PYTHON

```
def calka(f, a, b, n=1000):  
    h = (b - a) / n  
  
    suma = 0.0  
  
    for i in range(n):  
        x_srodek = a + (i + 0.5) * h  
        suma += f(x_srodek)  
  
    return h * suma
```

Przykład użycia:

PYTHON

```
def f(x):  
    return x * x  
  
wynik = calka(f, 1, 2)  
  
print("Przybliżona całka =", wynik)  
print("Wartość dokładna =", 7 / 3)
```

Wersja z komentarzami:

PYTHON

```
def calka(f, a, b, n=1000): # funkcja przybliża całkę z f na przedziale [a,b]  
    h = (b - a) / n # szerokość jednego małego przedziału  
  
    suma = 0.0 # suma wartości funkcji w środkach przedziałów  
  
    for i in range(n): # wykonujemy obliczenia dla n prostokątów  
        x_srodek = a + (i + 0.5) * h # środek i-tego przedziału  
        suma += f(x_srodek) # dodajemy wartość funkcji w środku przedziału  
  
    return h * suma # zwracamy przybliżone pole pod wykresem
```

Jeśli masz **całkę bez zakresów**, czyli całkę nieoznaczoną:

$$\int f(x) dx$$

to nie liczysz liczby, tylko szukasz **funkcji pierwotnej**.

Przykład:

$$\int x^2 dx = \frac{x^3}{3} + C$$

W Pythonie bez bibliotek symbolicznych nie da się tego ogólnie policzyć dla dowolnej funkcji tak jak na kartce. Możesz tylko ręcznie zapisać funkcję pierwotną:

```
def F(x):  
    return x**3 / 3
```

PYTHON

A jeśli potem dostaniesz przedział, np. 1, 2, to liczysz:

```
a = 1  
b = 2  
  
wynik = F(b) - F(a)  
  
print(wynik)
```

PYTHON

Czyli:

całka bez zakresów → funkcja pierwotna + C
całka z zakresami → konkretna liczba

Dla całki nieoznaczonej:

$$\int f(x) dx$$

wynikiem jest funkcja pierwotna:

$$F(x) + C$$

gdzie:

$$F'(x) = f(x)$$

Przykład:

$$\int x^2 dx = \frac{x^3}{3} + C$$

Jeżeli chcesz liczyć takie całki symbolicznie w Pythonie, można użyć **sympy**:

```
import sympy as sp  
  
x = sp.Symbol("x")  
  
wynik = sp.integrate(x**2, x)  
  
print(wynik)
```

PYTHON

Wynik:

$x^{**3/3}$